

DISEÑO FUNCIONAL

App control versiones y diagnostico de dependencias MS ACCESS y SQL SERVER

Versión 1.0

Fecha: 20/04/2025

Autor: Julien Bott Louet

Índice

1. Presentación breve del app	Páginas 5 - 11
2. Aclaración previa sobre el contenido del diseño funcional	Páginas 12 - 13
3. Agradecimientos	Páginas 14 - 15
4. Módulos del app	Páginas 16 - 36
Descripción y contenido de los módulos	Páginas 17 - 19
APP_1_GUI.py	
Objetos definidos en el módulo	Página 20
Diagnostico dependencias de objetos	Páginas 21 - 22
Muestra de las clases propias en la GUI	Página 23
APP_2_GENERAL	
Objetos definidos en el módulo	Página 24
Diccionarios de configuración	Páginas 25 - 27
Diagnostico dependencias de objetos	Páginas 28 - 31
APP_3_BACK_END_MS_ACCESS	
Objetos definidos en el módulo	Página 32
Diccionarios de configuración	Página 33
Diagnostico dependencias de objetos	Página 34
APP_3_BACK_END_SQL_SERVER	
Objetos definidos en el módulo	Página 35
Diagnostico dependencias de objetos	Página 36

Índice

5. Configuración de pantalla al abrir el app	Páginas 37 - 39
6. Proceso control de versiones	Páginas 40 - 83
Cálculo del proceso	Páginas 41 - 58
Localizar cambios y realizar migraciones de scripts	Páginas 59 - 75
Realizar MERGE en bases de datos físicas	Páginas 76 - 83
7. Proceso diagnostico dependencias objetos en MS ACCESS	Páginas 84 - 89
8. Proceso diagnostico dependencias objetos en SQL SERVER	Páginas 90 - 96
9. Limitaciones del app	Páginas 97 - 102
10. Futuros desarrollos	Páginas 103 - 112

Índice

11. ANEXOS

RUTINAS DE IMPORTACIÓN

ANEXO_01: def_proceso_access_1_import

Páginas 114 - 131

ANEXO_02: def_proceso_sql_server_1_import

Páginas 132 - 142

RUTINAS/ FUNCIONES DE CALCULO CONTROL DE VERSIONES

ANEXO_03: func_dicc_control_versiones

Páginas 143 - 155

ANEXO_04: def_proceso_access_2_control_versiones

Páginas 156 - 168

ANEXO_05: def_proceso_sql_server_2_control_versiones

Páginas 169 - 175

ANEXO_06: def_proceso_merge_realizar_cambios

Páginas 176 - 194

ANEXO_07: def_merge_bbdd_fisicas

Páginas 195 - 234

RUTINAS / FUNCIONES DE DIAGNOSTICO DE DEPENDENCIAS

ANEXO_08: func_access_bbdd_codigo_calculos

Páginas 235 - 274

ANEXO_09: func_access_bbdd_codigo_calculos_varios

Páginas 275 - 328

ANEXO_10: func_diagnostico_access_tablas_codigo_preservar

Páginas 329 - 344

ANEXO_11: def_proceso_access_2_diagnostico

Páginas 345 - 364

ANEXO_12: def_proceso_sql_server_2_diagnostico

Páginas 365 - 379

Presentación breve del app

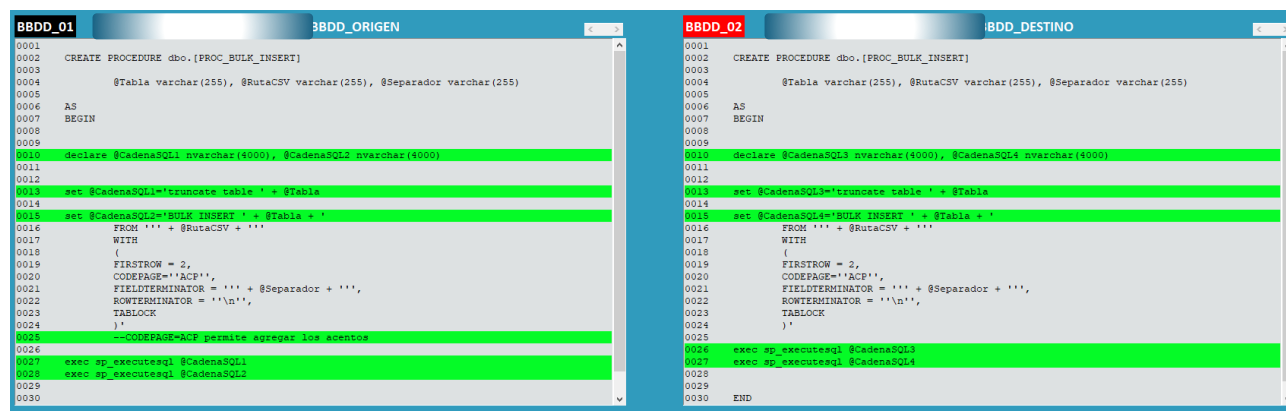
Presentación breve del app

El app permite realizar permite calcular 2 procesos en entornos de bases de datos MS ACCESS y/o SQL SERVER:

- **Control de versiones** de scripts de objetos entre 2 bases de datos de un mismo entorno (MS ACCESS o SQL SERVER)
- **Diagnostico de dependencias** para una base de datos MS ACCESS y para una lista (seleccionable) de bases de datos de un servidor SQL SERVER

Control de versiones

La funcionalidad permite localizar los cambios en el código de scripts de objetos y marcarlas de un color. Los nuevos objetos entre una base de datos y otra también se localizan.



```
BBDD_01 3BDD_ORIGEN
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009     declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0010
0011
0012
0013     set @CadenaSQL1='truncate table ' + @Tabla
0014
0015     set @CadenaSQL2='BULK INSERT ' + @Tabla + '
0016 FROM ''' + @RutaCSV + '''
0017 WITH
0018 (
0019     FIRSTROW = 2,
0020     CODEPAGE='ACP'',
0021     FIELDTERMINATOR = ''' + @Separador + ''',
0022     ROWTERMINATOR = '''\n'',
0023     TABLOCK
0024 )'
0025     --CODEPAGE=ACP permite agregar los acentos
0026
0027     exec sp_executesql @CadenaSQL1
0028     exec sp_executesql @CadenaSQL2
0029
0030
0031
0032
0033
0034
0035
0036
0037
0038
0039
0040
0041
0042
0043
0044
0045
0046
0047
0048
0049
0050
0051
0052
0053
0054
0055
0056
0057
0058
0059
0060
0061
0062
0063
0064
0065
0066
0067
0068
0069
0070
0071
0072
0073
0074
0075
0076
0077
0078
0079
0080
0081
0082
0083
0084
0085
0086
0087
0088
0089
0090
0091
0092
0093
0094
0095
0096
0097
0098
0099
0100
0101
0102
0103
0104
0105
0106
0107
0108
0109
0110
0111
0112
0113
0114
0115
0116
0117
0118
0119
0120
0121
0122
0123
0124
0125
0126
0127
0128
0129
0130
0131
0132
0133
0134
0135
0136
0137
0138
0139
0140
0141
0142
0143
0144
0145
0146
0147
0148
0149
0150
0151
0152
0153
0154
0155
0156
0157
0158
0159
0160
0161
0162
0163
0164
0165
0166
0167
0168
0169
0170
0171
0172
0173
0174
0175
0176
0177
0178
0179
0180
0181
0182
0183
0184
0185
0186
0187
0188
0189
0190
0191
0192
0193
0194
0195
0196
0197
0198
0199
0200
0201
0202
0203
0204
0205
0206
0207
0208
0209
0210
0211
0212
0213
0214
0215
0216
0217
0218
0219
0220
0221
0222
0223
0224
0225
0226
0227
0228
0229
0230
0231
0232
0233
0234
0235
0236
0237
0238
0239
0240
0241
0242
0243
0244
0245
0246
0247
0248
0249
0250
0251
0252
0253
0254
0255
0256
0257
0258
0259
0260
0261
0262
0263
0264
0265
0266
0267
0268
0269
0270
0271
0272
0273
0274
0275
0276
0277
0278
0279
0280
0281
0282
0283
0284
0285
0286
0287
0288
0289
0290
0291
0292
0293
0294
0295
0296
0297
0298
0299
0300
0301
0302
0303
0304
0305
0306
0307
0308
0309
0310
0311
0312
0313
0314
0315
0316
0317
0318
0319
0320
0321
0322
0323
0324
0325
0326
0327
0328
0329
0330
0331
0332
0333
0334
0335
0336
0337
0338
0339
0340
0341
0342
0343
0344
0345
0346
0347
0348
0349
0350
0351
0352
0353
0354
0355
0356
0357
0358
0359
0360
0361
0362
0363
0364
0365
0366
0367
0368
0369
0370
0371
0372
0373
0374
0375
0376
0377
0378
0379
0380
0381
0382
0383
0384
0385
0386
0387
0388
0389
0390
0391
0392
0393
0394
0395
0396
0397
0398
0399
0400
0401
0402
0403
0404
0405
0406
0407
0408
0409
0410
0411
0412
0413
0414
0415
0416
0417
0418
0419
0420
0421
0422
0423
0424
0425
0426
0427
0428
0429
0430
0431
0432
0433
0434
0435
0436
0437
0438
0439
0440
0441
0442
0443
0444
0445
0446
0447
0448
0449
0450
0451
0452
0453
0454
0455
0456
0457
0458
0459
0460
0461
0462
0463
0464
0465
0466
0467
0468
0469
0470
0471
0472
0473
0474
0475
0476
0477
0478
0479
0480
0481
0482
0483
0484
0485
0486
0487
0488
0489
0490
0491
0492
0493
0494
0495
0496
0497
0498
0499
0500
0501
0502
0503
0504
0505
0506
0507
0508
0509
0510
0511
0512
0513
0514
0515
0516
0517
0518
0519
0520
0521
0522
0523
0524
0525
0526
0527
0528
0529
0530
0531
0532
0533
0534
0535
0536
0537
0538
0539
0540
0541
0542
0543
0544
0545
0546
0547
0548
0549
0550
0551
0552
0553
0554
0555
0556
0557
0558
0559
0560
0561
0562
0563
0564
0565
0566
0567
0568
0569
0570
0571
0572
0573
0574
0575
0576
0577
0578
0579
0580
0581
0582
0583
0584
0585
0586
0587
0588
0589
0590
0591
0592
0593
0594
0595
0596
0597
0598
0599
0600
0601
0602
0603
0604
0605
0606
0607
0608
0609
0610
0611
0612
0613
0614
0615
0616
0617
0618
0619
0620
0621
0622
0623
0624
0625
0626
0627
0628
0629
0630
0631
0632
0633
0634
0635
0636
0637
0638
0639
0640
0641
0642
0643
0644
0645
0646
0647
0648
0649
0650
0651
0652
0653
0654
0655
0656
0657
0658
0659
0660
0661
0662
0663
0664
0665
0666
0667
0668
0669
0670
0671
0672
0673
0674
0675
0676
0677
0678
0679
0680
0681
0682
0683
0684
0685
0686
0687
0688
0689
0690
0691
0692
0693
0694
0695
0696
0697
0698
0699
0700
0701
0702
0703
0704
0705
0706
0707
0708
0709
0710
0711
0712
0713
0714
0715
0716
0717
0718
0719
0720
0721
0722
0723
0724
0725
0726
0727
0728
0729
0730
0731
0732
0733
0734
0735
0736
0737
0738
0739
0740
0741
0742
0743
0744
0745
0746
0747
0748
0749
0750
0751
0752
0753
0754
0755
0756
0757
0758
0759
0760
0761
0762
0763
0764
0765
0766
0767
0768
0769
0770
0771
0772
0773
0774
0775
0776
0777
0778
0779
0780
0781
0782
0783
0784
0785
0786
0787
0788
0789
0790
0791
0792
0793
0794
0795
0796
0797
0798
0799
0800
0801
0802
0803
0804
0805
0806
0807
0808
0809
0810
0811
0812
0813
0814
0815
0816
0817
0818
0819
0820
0821
0822
0823
0824
0825
0826
0827
0828
0829
0830
0831
0832
0833
0834
0835
0836
0837
0838
0839
0840
0841
0842
0843
0844
0845
0846
0847
0848
0849
0850
0851
0852
0853
0854
0855
0856
0857
0858
0859
0860
0861
0862
0863
0864
0865
0866
0867
0868
0869
0870
0871
0872
0873
0874
0875
0876
0877
0878
0879
0880
0881
0882
0883
0884
0885
0886
0887
0888
0889
0890
0891
0892
0893
0894
0895
0896
0897
0898
0899
0900
0901
0902
0903
0904
0905
0906
0907
0908
0909
0910
0911
0912
0913
0914
0915
0916
0917
0918
0919
0920
0921
0922
0923
0924
0925
0926
0927
0928
0929
0930
0931
0932
0933
0934
0935
0936
0937
0938
0939
0940
0941
0942
0943
0944
0945
0946
0947
0948
0949
0950
0951
0952
0953
0954
0955
0956
0957
0958
0959
0960
0961
0962
0963
0964
0965
0966
0967
0968
0969
0970
0971
0972
0973
0974
0975
0976
0977
0978
0979
0980
0981
0982
0983
0984
0985
0986
0987
0988
0989
0990
0991
0992
0993
0994
0995
0996
0997
0998
0999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356

```

Presentación breve del app

Control de versiones

Aplica para los objetos siguientes:

ENTORNO	TIPO OBJETO
MS ACCESS	Tablas locales
	Vinculos ODBC o hacia otras fuentes externas
	Variables públicas VBA
	Rutinas y funciones VBA
SQL SERVER	Tablas
	Views
	Stored procedures
	Funciones (Scalar Function, Inline Table-Valued Function, Table-Valued Function - Multi-Statement y Aggregate Function)

El app permite al usuario navegar por los distintos objetos con cambios localizados con un simple click.

Tipo Objeto

MS ACCESS: rutinas VBA

Tipo Concepto

Objetos ya existentes en las 2 BBDD's

VER

EXCEL

TIPO BBDD	TIPO OBJETO	REPOSITORIO	OBJETO	BBDD_01	BBDD_02
MS_ACCESS	Rutinas VBA	Mod_00_Config	Config_Usuario	2	3
MS_ACCESS	Rutinas VBA	Mod_01_Extraccion	Func_AccessCampID	4	2

BBDD_01 PRUEBA_CONTROL_VERSIONES_1.accdB

```
0001 Sub Config_Usuario()  
0002 'se hace mediante conexionEXT y no import pq la tabla contiene los password de dichas DBS  
0003  
0004 Dim MiConexion As New ADODB.Connection  
0005 Dim MiConexionEXT As New ADODB.Connection  
0006 Dim MiRecordset1 As New ADODB.Recordset  
0007 Dim MiRecordset2 As New ADODB.Recordset  
0008 Dim MiRecordset3 As New ADODB.Recordset  
0009 Dim MiSQL1 As String, MiSQL2 As String  
0010  
0011 Set MiConexion = CurrentProject.Connection  
0012  
0013  
0014  
0015 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre FROM IF_M0_ConfigUsuario"  
0016  
0017 MiRecordset1.Open MiSQL1, MiConexion, 1, 2  
0018 MiRecordset1.MoveFirst  
0019  
0020 Public IPUsuario = MiRecordset1(0)  
0021 Public Usuario_Nom = MiRecordset1(1)  
0022 Public RutaODB = CurrentProject.Path & "\BBDD\ODB.accdB"  
0023  
0024 MiRecordset1.Close  
0025 Set MiRecordset1 = Nothing  
0026  
0027  
0028  
0029 MiConexion.Close  
0030 Set MiConexion = Nothing
```

BBDD_02 PRUEBA_CONTROL_VERSIONES_2.accdB

```
0001 Sub Config_Usuario()  
0002 'se hace mediante conexionEXT y no import pq la tabla contiene los password de dichas DBS  
0003  
0004 Dim MiConexion As New ADODB.Connection  
0005 Dim MiConexionEXT As New ADODB.Connection  
0006 Dim MiRecordset1 As New ADODB.Recordset  
0007 Dim MiRecordset2 As New ADODB.Recordset  
0008 Dim MiRecordset3 As New ADODB.Recordset  
0009 Dim MiSQL1 As String, MiSQL2 As String  
0010  
0011 Set MiConexion = CurrentProject.Connection  
0012  
0013  
0014  
0015 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre, Usuario_Telefono FROM IF_M0_ConfigUsuario"  
0016  
0017 MiRecordset1.Open MiSQL1, MiConexion, 1, 2  
0018 MiRecordset1.MoveFirst  
0019  
0020 Public IPUsuario_Nom = MiRecordset1(0)  
0021 Public Usuario_Nom = MiRecordset1(1)  
0022 Public Usuario_Telefono = MiRecordset1(2)  
0023  
0024 Public RutaODB = CurrentProject.Path & "\BBDD\ODB.accdB"  
0025  
0026 MiRecordset1.Close  
0027 Set MiRecordset1 = Nothing  
0028  
0029  
0030 MiConexion.Close
```

Presentación breve del app

Control de versiones

El usuario tiene la posibilidad de realizar migraciones de líneas de código de un script a otro, pasar a otro objeto realizar los cambios oportunos y así sucesivamente. También puede revertir cambios. Todos los cambios se almacenan en la memoria RAM del PC.

MERGE

LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD Acción

Lineas Origen

Lineas Destino

ACCIÓN MERGE

Lineas Origen 15 16

Lineas Destino 14

ACCIÓN

BBDD_01 PRUEBA_CONTROL_VERSIONES_1.accdB

```
0001 Sub Config_Usuario()  
0002 'se hace mediante conexionEXT y no import pq la tabla contiene los password de dichas DBS  
0003  
0004 Dim MiConexion As New ADODB.Connection  
0005 Dim MiConexionEXT As New ADODB.Connection  
0006 Dim MiRecordset1 As New ADODB.Recordset  
0007 Dim MiRecordset2 As New ADODB.Recordset  
0008 Dim MiRecordset3 As New ADODB.Recordset  
0009 Dim MiSQL1 As String, MiSQL2 As String  
0010  
0011  
0012 Set MiConexion = CurrentProject.Connection  
0013  
0014  
0015 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre FROM [T_00_ConfigUsuario]"  
0016  
0017 MiRecordset1.Open MiSQL1, MiConexion, 1, 2  
0018 MiRecordset1.MoveFirst  
0019  
0020 Public IDUsuario = MiRecordset1(i)  
0021 Public Usuario_Nom = MiRecordset1(i)  
0022 Public RutaCDB = CurrentProject.Path & "\BBDD\DBS.accdB"  
0023  
0024 MiRecordset1.Close  
0025 Set MiRecordset1 = Nothing  
0026  
0027  
0028  
0029 MiConexion.Close  
0030 Set MiConexion = Nothing
```

BBDD_02 PRUEBA_CONTROL_VERSIONES_2.accdB

```
0001 Sub Config_Usuario()  
0002 'se hace mediante conexionEXT y no import pq la tabla contiene los password de dichas DBS  
0003  
0004 Dim MiConexion As New ADODB.Connection  
0005 Dim MiConexionEXT As New ADODB.Connection  
0006 Dim MiRecordset1 As New ADODB.Recordset  
0007 Dim MiRecordset2 As New ADODB.Recordset  
0008 Dim MiRecordset3 As New ADODB.Recordset  
0009 Dim MiSQL1 As String, MiSQL2 As String  
0010  
0011  
0012 Set MiConexion = CurrentProject.Connection  
0013  
0014  
0015 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre, Usuario_Telefono FROM [T_00_ConfigUsuario]"  
0016  
0017 MiRecordset1.Open MiSQL1, MiConexion, 1, 2  
0018 MiRecordset1.MoveFirst  
0019  
0020 Public IDUsuario_NOM = MiRecordset1(i)  
0021 Public Usuario_Nom = MiRecordset1(i)  
0022 Public Usuario_Telefono = MiRecordset1(i)  
0023 Public RutaCDB = CurrentProject.Path & "\BBDD\DBS.accdB"  
0024  
0025 MiRecordset1.Close  
0026 Set MiRecordset1 = Nothing  
0027  
0028  
0029 MiConexion.Close  
0030
```

BBDD_01 PRUEBA_CONTROL_VERSIONES_1.accdB

```
0001 Sub Config_Usuario()  
0002 'se hace mediante conexionEXT y no import pq la tabla contiene los password de dichas DBS  
0003  
0004 Dim MiConexion As New ADODB.Connection  
0005 Dim MiConexionEXT As New ADODB.Connection  
0006 Dim MiRecordset1 As New ADODB.Recordset  
0007 Dim MiRecordset2 As New ADODB.Recordset  
0008 Dim MiRecordset3 As New ADODB.Recordset  
0009 Dim MiSQL1 As String, MiSQL2 As String  
0010  
0011  
0012 Set MiConexion = CurrentProject.Connection  
0013  
0014  
0015 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre FROM [T_00_ConfigUsuario]"  
0016  
0017 MiRecordset1.Open MiSQL1, MiConexion, 1, 2  
0018 MiRecordset1.MoveFirst
```

BBDD_02 PRUEBA_CONTROL_VERSIONES_2.accdB

```
0001 Sub Config_Usuario()  
0002 'se hace mediante conexionEXT y no import pq la tabla contiene los password de dichas DBS  
0003  
0004 Dim MiConexion As New ADODB.Connection  
0005 Dim MiConexionEXT As New ADODB.Connection  
0006 Dim MiRecordset1 As New ADODB.Recordset  
0007 Dim MiRecordset2 As New ADODB.Recordset  
0008 Dim MiRecordset3 As New ADODB.Recordset  
0009 Dim MiSQL1 As String, MiSQL2 As String  
0010  
0011  
0012 Set MiConexion = CurrentProject.Connection  
0013  
0014  
0015 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre FROM [T_00_ConfigUsuario]"  
0016  
0017 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre, Usuario_Telefono FROM [T_00_ConfigUsuario]"  
0018
```

Presentación breve del app

Control de versiones

Tras realizar todos los cambios en los objetos que juzgue conveniente, el usuario tiene la posibilidad de realizar los cambios en la base de datos física y documentar los cambios en ficheros .txt.



Presentación breve del app

Diagnostico de dependencias

El app permite para cada objeto que conforman una base de datos (caso de MS ACCESS) y para cada objeto de 1 o varias bases de datos de un servidor SQL SERVER localizar en que rutinas y/o funciones se usan.

Asimismo, localiza los objetos sin dependencias con otros objetos (en el caso de SQL SERVER es especialmente útil para localizar aquellas tablas que son estáticas o que se cargan mediante procesos de ETL sean app Python, VBA u otros).

Por último, para cada objeto listado aporta información complementaria como pueden ser el tipo de objeto, los parámetros de una rutina, el tipo de variable pública etc etc.

Matizar, sin entrar en detalles, que la **búsqueda de dependencias** entre objetos se limita a **código activo** (no a objetos nombrados en líneas o bloques de líneas de comentarios). Asimismo, esta búsqueda no se extralimita a **buscar objetos en rutinas aplicando el método “contiene”**. El criterio de búsqueda **es mucho más afinado** según las particularidades de los lenguajes VBA y T-SQL (esto se documenta en los Anexos del presente documento relacionados con las rutinas de diagnostico de dependencias).

El output final es un fichero Excel para que sea más cómodo y flexible para el usuario realizar sus análisis.

Presentación breve del app

Diagnostico de dependencias

MS_ACCESS

LISTADO

A	B	C	D	E	F	G	H	I	J	K	L
TIPO OBJETO	TIPO MODULO	NOMBRE MODULO	NOMBRE OBJETO	TIPO OBJETO 2	TIPO DECLARACION RUTINA	PARAMETROS RUTINA	VARIABLE PUBLICA TIPO DATO	VARIABLE PUBLICA ES CONSTANTE	VARIABLE PUBLICA VALOR CONSTANTE	VARIABLE DEFINIDA POR EL USUARIO (SUB-VARIABLES)	CONNECTING STRING VINCULOS
RUTINAS_VBA	Estandar	Mod_00_Config	Func_ParametrosDDBI	FUNCION	PUBLICA	xOpcion As String					
VARIABLES_VBA	Estandar	AAA	Enum_Prueba_1	VARIABLE PUBLICA DEFINIDA POR USUARIO			Enum	-		SubVar_1, SubVar_2 (1), SubVar_3	

DEPENDENCIAS

A	B	C	D	E	F	G
TIPO OBJETO	TIPO MODULO	NOMBRE MODU	NOMBRE OBJETO	SE USA EN TIPO MODULO	SE USA EN NOMBRE MODU	SE USA EN NOMBRE RUTINA
RUTINAS_VBA	Estandar	Mod_00_Config	ActivarPanelNavegacion	Formulario/Reporte	Form_F_Inicio	BotonPANEL_Click
RUTINAS_VBA	Estandar	Mod_00_Config	Config_Administrador	UserForm	UF_Administrador	BotonCONFIG_Click

SIN DEPENDENCIAS

A	B	C	D
TIPO OBJETO	TIPO MODULO	NOMBRE MODULO	NOMBRE OBJETO
RUTINAS_VBA	Estandar	Mod_00_Config	Config_UbicacionDDBB
RUTINAS_VBA	Estandar	Mod_00_Funciones	Func_StringSQLFecha

SQL SERVER

LISTADO

A	B	C	D	E
BBDD	ESQUEMA	TIPO_OBJETO	NOMBRE_OBJETO	PARAMETROS
BBDD_DESTINO	dbo	TABLAS	T_PRUEBA_1	
BBDD_DESTINO	dbo	STORED_PROCEDURES	PROC_BULK_INSERT	@Tabla VARCHAR(255), @RutaCSV VARCHAR(255), @Separador VARCHAR(255)

DEPENDENCIAS

A	B	C	D	E	F	G	H
BBDD	ESQUEMA	TIPO_OBJE	NOMBRE_OBJETO	SE_USA_EN_BBDD	SE_USA_EN_ESQUEMA	SE_USA_EN_TIPO_OBJET	SE_USA_EN_NOMBRE_OBJETO
BBDD_PRUEBA_1	INPUTS	TABLAS	T_CONFIG_CheckDates	BBDD_PRUEBA_1	INPUTS	STORED_PROCEDURES	PROC_Import_CheckDates
BBDD_PRUEBA_1	INPUTS	TABLAS	T_CONFIG_CheckDates	BBDD_PRUEBA_1	INPUTS	STORED_PROCEDURES	VIEWS_FormatoVIEW

SIN DEPENDENCIAS

A	B	C	D
BBDD	ESQUEMA	TIPO_OBJETO	NOMBRE_OBJETO
BBDD_DESTINO	dbo	STORED_PROCEDURES	PROC_BULK_INSERT
BBDD_DESTINO	dbo	TABLAS	ErrorLog

Aclaración previa sobre el contenido del diseño funcional

Aclaración previa sobre el contenido del diseño funcional

El presente diseño funcional sirve de guía para entender la **arquitectura** del app y como interactúan las variables globales y rutinas / funciones que componen los distintos módulos Python (apartados 4 a 8).

El código está documentado tanto a nivel de variable global como a nivel de rutina / función.

Para las rutinas / funciones, el código se documenta brevemente al principio de cada una explicando lo que hace y luego se documenta dentro del código por bloques de instrucciones. En el caso de las funciones, también se comenta cuales son sus resultados.

Algunas rutinas / funciones no se documentan en este documento, tan solo se mencionan cuando se usan en los procesos. Con los comentarios en el código, considero para estos casos que la explicación es más que suficiente.

No obstante, otras rutinas / funciones **necesitan una explicación más detallada y en algunos casos con ejemplos** para realmente entender su alcance (ir a los **ANEXOS** del presente documento).

El app tiene ciertas **limitaciones** que se detallan en el apartado 9 del presente documento.

También se listan, en el apartado 10, una lista de **(posibles) futuros desarrollos**.

Agradecimientos

Agradecimientos

Antes de entrar a fondo con el diseño funcional, quiero mostrar mi agradecimiento a:

Aunque la arquitectura y la mayoría del código es fruto de mi iniciativa personal y de lo aprendido en los últimos años, quiero dar en primer lugar mi agradecimiento a **ChatGPT (versión 3.5)**, con quien me he apoyado en contadas ocasiones (cuando no sabes algo, lo consultas en internet, así se aprende hoy en día). Su ayuda me ha permitido:

- Descubrir ciertos widgets para la GUI y resolver bugs al encapsularla en clases.
- Entender la librería diffliib, que no conocía, y profundizar en las librerías re y win32com.client, que ya conocía, pero no para los usos específicos requeridos en esta app.
- Superar un stopper concreto en la función que localiza bloques de código T-SQL comentados, especialmente cuando hay /* y */ anidados los unos dentro de los otros.
- Consultas varias sobre tablas de sistema T-SQL que habría podido solventar buscando a la “antigua” en Google.
- Confeccionar el repositorio Github y redactar el contrato de MIT Licence.

También agradezco profundamente la **música de ambiente tipo Blade Runner**, que me ha acompañado durante sesiones nocturnas de trabajo entre semana o de día durante los fines de semana (sí, soy un friki de ciencia ficción especialmente rollo cyberpunk).

Y por último, y no menos importante, **mi maquina de café**.

Módulos del app

Descripción y contenido de los módulos

El app consta de 4 módulos Python:

APP_1_GUI

Es el módulo desde el cual se ejecuta el app. Contiene todas las clases propias para poder ejecutar la interfaz de usuario (en adelante la GUI). Tan solo en APP_1_GUI se usan clases propias.

APP_2_GENERAL

Es el módulo que contiene variables globales que se usan en varios módulos. Contiene asimismo rutinas / funciones de carácter general que aplican tanto a los cálculos de procesos referentes a bases de datos MS Access y SQL Server.

APP_3_BACK_END_ACCESS

Es el módulo que contiene variables globales y rutinas / funciones que aplican solo al cálculo de los procesos relacionados con bases de datos MS Access.

APP_4_BACK_END_SQL_SERVER

Es el módulo que contiene variables globales y rutinas / funciones que aplican solo al cálculo de los procesos relacionados con bases de datos SQL Server.

Descripción y contenido de los módulos

Las rutinas y funciones que se usan en los distintos módulos para mayor legibilidad del código se declaran todas de la forma siguiente:

- Rutinas → empiezan siempre por **def_** (salvo algunas que se usan en la clase `gui_widgets` en el módulo de la GUI)

```
def def_proceso_access_1_import(proceso_id, opcion_bbdd, path_bbdd):
```

- Funciones → empiezan siempre por **func_** (en el comentario general de la función se especifica cual es su return)

```
def func_access_df_bbdd_codigo_calculos_varios(opcion, df_bbdd_codigo_param):
```

Las variables globales declaradas dentro de rutinas o funciones vienen todas precedidas de **global_**.

```
def def_calc_global(proceso_selecc_id, ruta_destino_fichero_logs, **kwargs):  
    #rutina para ejecutar los threads segun el proceso escogido  
  
    global global_proceso_en_ejecucion  
    global global_msg_errores_proceso_access  
    global global_msg_errores_proceso_sql_server
```

Descripción y contenido de los módulos

Las librerías Python usadas en cada módulo son:

APP_1_GUI

```
1 import pandas as pd
2 import os
3 import re
4 import tkinter as tk
5 from tkinter import messagebox, filedialog as fd, ttk, scrolledtext
6 from threading import Thread
7
8 import APP_2_GENERAL as mod_gen
9 import APP_3_BACK_END_MS_ACCESS as mod_access
10 import APP_3_BACK_END_SQL_SERVER as mod_sql_server
```

APP_2_GENERAL

```
1 import pandas as pd
2 import pyodbc
3 import os
4 import sys
5 import re
6 import pathlib
7 import traceback
8 import datetime as dt
9 import xlwings as xw
10 import difflib
11 from threading import Thread
12 import win32com.client
13 import subprocess
14 import time
15
16 import APP_3_BACK_END_MS_ACCESS as mod_access
17 import APP_3_BACK_END_SQL_SERVER as mod_sql_server
```

APP_3_BACK_END_ACCESS

```
1 import pandas as pd
2 import os
3 import re
4 import shutil
5 import traceback
6 import win32com.client
7 import xlwings as xw
8 import warnings
9 import datetime as dt
10
11 import APP_2_GENERAL as mod_gen
```

APP_4_BACK_END_SQL_SERVER

```
1 import pandas as pd
2 import traceback
3 import pyodbc
4 import os
5 import re
6 import shutil
7 import xlwings as xw
8 import datetime as dt
9 import warnings
10
11 import APP_2_GENERAL as mod_gen
```

Módulo APP_1_GUI

Las variables globales y rutinas que componen el módulo son los siguientes:

TIPO OBJETO	NOMBRE OBJETO	CLASE DONDE SE UBICA	SE DOCUMENTA
Variable global	bg_GUI_inicio		No
	bg_GUI_control_versiones		No
	bg_GUI_control_versiones_merge		No
	bg_GUI_merge_bbdd_fisica		No
	bg_GUI_merge_bbdd_fisica_errores		No
	bg_GUI_scrolledtext_scripts		No
	bg_GUI_lineas_control_versiones_cambios_localizados		No
	bg_GUI_lineas_control_versiones_cambios_agregados		No
	bg_GUI_lineas_control_versiones_cambios_eliminados		No
	resolucion_pantalla_recomendada		No
Rutina	config_lock_unlock		No
	combobox_update_lista_valores		No
	def_combobox_bind_rutina_con_param		No
	combobox_bind_select	gui_widgets	No
	scrolledtext_delete		No
	scrolledtext_insert		No
	entry_bind		No
	def_GUI_threads		No
	def_GUI_combobox_proceso		No
	def_GUI_access_add_clear		No
	def_GUI_combobox_sql_server_servidor_1	gui_ventana_inicio	No
	def_GUI_combobox_sql_server_servidor_2		No
	def_GUI_boton_check		No
	def_GUI_conexion_servidor_sql_server	gui_sql_server_authentication	No
	def_GUI_sql_server_diagnostico_threads		No
	def_GUI_sql_server_diagnostico_listbox_all_none	gui_diagnostico_bbdd_sql_server	No
	def_GUI_sql_server_diagnostico_boton_check		No
	def_control_versiones_cambio_tipo_objeto		No
	def_control_versiones_click_boton_seleccion		No
	def_control_versiones_click_boton_excel		No
	def_control_versiones_update_subform_objetos		No
	def_control_versiones_update_subform_objetos_click_item		No
	def_control_versiones_rellenar_scripts_scrolledtext	gui_ventana_control_versiones	No
	def_GUI_proceso_merge_lineas_exit		No
	def_proceso_merge_combobox_bbdd_selecc		No
	def_click_proceso_merge_boton_merge		No
	def_click_proceso_merge_boton_cambios_en_bbdd		No
	def_merge_bbdd_fisicas_click_boton_seleccion		No
	def_merge_bbdd_fisicas_update_subform_objetos		No
	def_merge_bbdd_fisicas_update_subform_objetos_click_item	gui_ventana_merge_bbdd_fisicas	No
	def_merge_bbdd_fisicas_rellenar_scripts_scrolledtext		No
	def_merge_bbdd_fisicas_click_boton_realizar		No

No se documentan en el presente documento ningún objeto pero si se les menciona en la explicación de los procesos. Refiriéndose al código del módulo, considero que es suficiente con los comentarios ahí informados.

Módulo APP_1_GUI

El diagnostico de dependencias de los objetos del módulo es el siguiente:

Variables globales

TIPO OBJETO	NOMBRE CLASE	NOMBRE OBJETO	EN QUE MODULO SE USA	EN QUE CLASE SE USA	DONDE SE USA EN LA CLASE	EN QUE RUTINA / FUNCION SE USA
Variable global		bg_GUI_inicio	APP_1_GUI APP_1_GUI APP_1_GUI	gui_ventana_inicio gui_sql_server_authentication gui_diagnostico_bbdd_sql_server	Constructor Constructor Constructor	
		bg_GUI_control_versiones	APP_1_GUI	gui_ventana_control_versiones	Constructor	
		bg_GUI_control_versiones_merge	APP_1_GUI	gui_ventana_control_versiones	Constructor	
		bg_GUI_merge_bbdd_fisica	APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Constructor	
		bg_GUI_scrolledtext_scripts	APP_1_GUI APP_1_GUI	gui_ventana_control_versiones gui_ventana_merge_bbdd_fisicas	Constructor Constructor	
		bg_GUI_lineas_control_versiones_cambios_localizados	APP_1_GUI APP_1_GUI	gui_ventana_control_versiones gui_ventana_merge_bbdd_fisicas	Constructor Constructor	
		bg_GUI_lineas_control_versiones_cambios_agregados	APP_1_GUI APP_1_GUI	gui_ventana_control_versiones gui_ventana_merge_bbdd_fisicas	Constructor Constructor	
		bg_GUI_lineas_control_versiones_cambios_eliminados	APP_1_GUI APP_1_GUI	gui_ventana_control_versiones gui_ventana_merge_bbdd_fisicas	Constructor Constructor	
		resolucion_pantalla_recomendada	APP_1_GUI	gui_ventana_inicio	Constructor	

Módulo APP_1_GUI

El diagnostico de dependencias de los objetos del módulo es el siguiente:

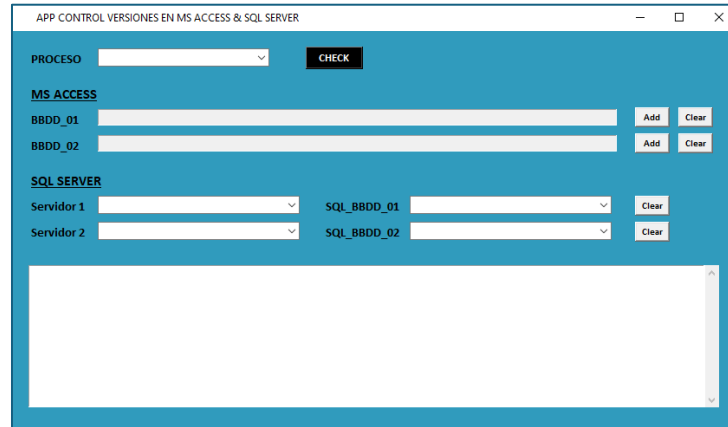
Rutinas

TIPO OBJETO	NOMBRE CLASE	NOMBRE OBJETO	EN QUE MODULO SE USA	EN QUE CLASE SE USA	DONDE SE USA EN LA CLASE	EN QUE Rutina / FUNCION SE USA
Rutina	gui_widgets	config_lock_unlock	APP_1_GUI	gui_ventana_inicio	Constructor	
				gui_ventana_inicio	Rutina integrada	def_GUI_combobox_proceso
				gui_sql_server_authentication	Constructor	
		combobox_update_lista_valores		gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_1
				gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_2
	gui_ventana_inicio	scrolledtext_delete		gui_ventana_control_versiones	Rutina integrada	def_proceso_merge_combobox_bbdd_selecc
		scrolledtext_insert		gui_ventana_inicio	Rutina integrada	def_GUI_combobox_proceso
		entry_bind		gui_ventana_inicio	Rutina integrada	def_GUI_combobox_proceso
				gui_ventana_control_versiones	Constructor	
		def_GUI_threads		gui_ventana_inicio	Constructor	
	gui_sql_server_authentication	def_GUI_combobox_proceso		gui_ventana_inicio	Constructor	
		def_GUI_access_add_clear		gui_ventana_inicio	Constructor	
		def_GUI_combobox_sql_server_servidor_1		gui_ventana_inicio	Constructor	
		def_GUI_combobox_sql_server_servidor_2		gui_ventana_inicio	Constructor	
		def_GUI_boton_check		gui_ventana_inicio	Rutina integrada	def_GUI_threads
	gui_diagnostico_bbdd_sql_server	def_GUI_conexion_servidor_sql_server		gui_sql_server_authentication	Constructor	
		def_GUI_sql_server_diagnostico_threads		gui_diagnostico_bbdd_sql_server	Constructor	
		def_GUI_sql_server_diagnostico_listbox_all_none		gui_diagnostico_bbdd_sql_server	Constructor	
		def_GUI_sql_server_diagnostico_boton_check		gui_diagnostico_bbdd_sql_server	Rutina integrada	def_GUI_sql_server_diagnostico_threads
		def_control_versiones_cambio_tipo_objeto		gui_ventana_control_versiones	Constructor	
	gui_ventana_control_versiones	def_control_versiones_click_boton_seleccion		gui_ventana_control_versiones	Constructor	
		def_control_versiones_click_boton_excel		gui_ventana_control_versiones	Constructor	
		def_control_versiones_update_subform_objetos		gui_ventana_control_versiones	Constructor	
		def_control_versiones_update_subform_objetos_click_item		gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_seleccion
				gui_ventana_control_versiones	Rutina integrada	def_control_versiones_update_subform_objetos
		def_control_versiones_rellenar_scripts_scrolledtext		gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_seleccion
		def_GUI_proceso_merge_lineas_exit		gui_ventana_control_versiones	Rutina integrada	def_control_versiones_update_subform_objetos
		def_proceso_merge_combobox_bbdd_selecc		gui_ventana_control_versiones	Rutina integrada	def_control_versiones_update_subform_objetos_click_item
		def_click_proceso_merge_boton_merge		gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_merge
		def_click_proceso_merge_boton_cambios_en_bbdd		gui_ventana_control_versiones	Constructor	
	gui_ventana_merge_bbdd_fisicas	def_merge_bbdd_fisicas_click_boton_seleccion		gui_ventana_control_versiones	Constructor	
				gui_ventana_merge_bbdd_fisicas	Constructor	
		def_merge_bbdd_fisicas_update_subform_objetos		gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_seleccion
		def_merge_bbdd_fisicas_update_subform_objetos_click_item		gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos
		def_merge_bbdd_fisicas_rellenar_scripts_scrolledtext		gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos
		def_merge_bbdd_fisicas_click_boton_realizar		gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos_click_item
				gui_ventana_merge_bbdd_fisicas	Constructor	

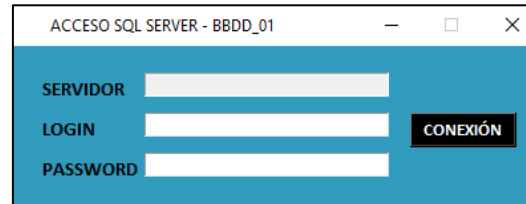
Módulo APP_1_GUI

Las clases propias enumeradas en los listados anteriores son las que permiten generar estas interfaces:

gui_ventana_inicio



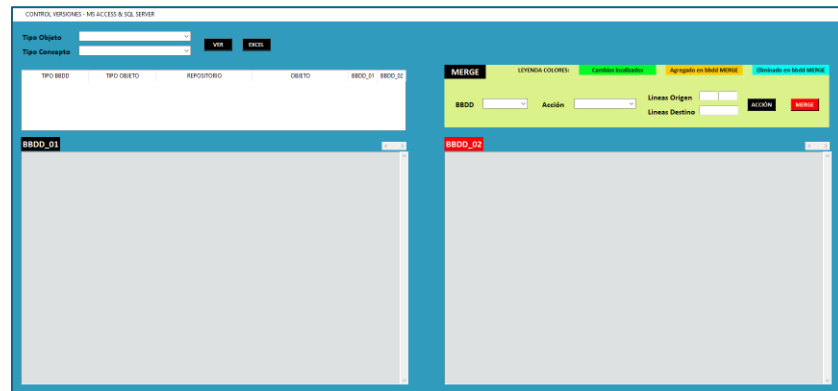
gui_sql_server_authentication



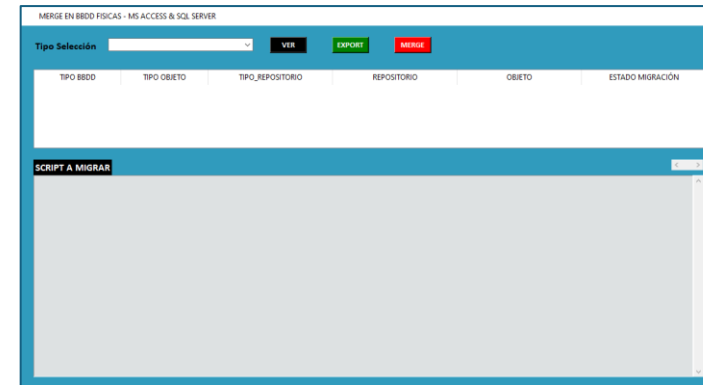
gui_diagnostico_bbdd_sql_server



gui_ventana_control_versiones



gui_ventana_merge_bbdd_fisicas



Módulo APP_2_GENERAL

Las variables globales y rutinas que componen el módulo son los siguientes:

TIPO OBJETO	NOMBRE OBJETO	SE DOCUMENTA
Variable global	nombre_app	No
	nombre_control_versiones	No
	nombre_merge_bbdd_fisicas	No
	global_proceso_en_ejecucion	No
	lista_GUI_proceso_merge_tipo_accion_bbdd_1	No
	lista_GUI_proceso_merge_tipo_accion_bbdd_2	No
	lista_acciones_migrar_quitar	No
	lista_acciones_revertir	No
	label_merge_access_bbdd_fisica_en_manual	No
	lista_headers_df_codigo_control_versiones_1	No
	lista_headers_df_codigo_control_versiones_2	No
	dicc_procesos	No
	dicc_codigos_bbdd	No
	dicc_control_versiones_tipo_objeto	No
	dicc_control_versiones_tipo_concepto	No
	lista_GUI_seleccion_tipo_objeto_access	No
	lista_GUI_seleccion_tipo_objeto_sql_server	No
	lista_GUI_seleccion_tipo_concepto	No
	lista_GUI_seleccion_bbdd	No
	dicc_errores_procesos	No
Función	func_procesos_errores	No
	func_dicc_control_versiones_tipo_objeto_buscar_en_dicc	No
	func_se_puede_ejecutar_proceso	No
	func_dicc_control_versiones	ANEXO_03
	func_control_versiones_dicc_proceso_merge_anteriores	No
Rutina	def_generacion_logs	No
	def_calc_global	Apartados 6-7-8
	def_control_versiones_export_excel	No
	def_proceso_merge_realizar_cambios	ANEXO_06
	def_merge_access_ajustes_manuales	No
	def_merge_bbdd_fisica_lista_objetos	No
	def_merge_bbdd_fisicas_export	No
	def_merge_bbdd_fisicas	ANEXO_07

Las variables marcadas en **AMARILLO**, correspondientes a **diccionarios de configuración**, se comentan en las páginas siguientes. Los objetos que no se documentan, con referirse al código del módulo es suficiente con los comentarios ahí informados.

Módulo APP_2_GENERAL

Diccionarios de configuración

dicc_procesos

KEY 1	KEY 2
PROCESO_01	PROCESO
	COMENTARIO
PROCESO_02	PROCESO
	COMENTARIO
PROCESO_03	PROCESO
	COMENTARIO

dicc_codigos_bbdd

KEY 1	KEY 2	KEY 3
BBDD_01	MS_ACCESS	PATH_BBDD
		LISTA_LIBRERIAS_DLL
		DF_CODIGO_CALCULADO_TRAS_IMPORT
		CONTROL_VERSIONES_LISTA_DICC_TABLAS_LOCALES
		CONTROL_VERSIONES_LISTA_DICC_VINCULOS_ODBC
		CONTROL_VERSIONES_LISTA_DICC_VINCULOS_OTROS
		CONTROL_VERSIONES_LISTA_DICC Rutinas
		CONTROL_VERSIONES_LISTA_DICC VARIABLES
		DIAGNOSTICO_LISTA_DICC_TABLAS_LOCALES
		DIAGNOSTICO_LISTA_DICC_VINCULOS_ODBC
		DIAGNOSTICO_LISTA_DICC_VINCULOS_OTROS
		DIAGNOSTICO_LISTA_DICC Rutinas
		DIAGNOSTICO_LISTA_DICC VARIABLES
		DIAGNOSTICO_DF_LISTADO_TABLAS_Y_VINCULOS
		DIAGNOSTICO_DF_LISTADO Rutinas
		DIAGNOSTICO_DF_LISTADO VARIABLES
		DIAGNOSTICO_DF_CODIGO_TABLAS_Y_VINCULOS
		DIAGNOSTICO_DF_CODIGO Rutinas
		DIAGNOSTICO_DF_CODIGO VARIABLES
	SQL_SERVER	SERVIDOR
		BBDD
		CONNECTING_STRING
		LISTA_DICC_OBJETOS
		LISTA_BBDD_SERVIDOR

KEY 1	KEY 2	KEY 3
BBDD_02	MS_ACCESS	PATH_BBDD
		LISTA_LIBRERIAS_DLL
		DF_CODIGO_CALCULADO_TRAS_IMPORT
		CONTROL_VERSIONES_LISTA_DICC_TABLAS_LOCALES
		CONTROL_VERSIONES_LISTA_DICC_VINCULOS_ODBC
		CONTROL_VERSIONES_LISTA_DICC_VINCULOS_OTROS
		CONTROL_VERSIONES_LISTA_DICC Rutinas
		CONTROL_VERSIONES_LISTA_DICC VARIABLES
		DIAGNOSTICO_LISTA_DICC_TABLAS_LOCALES
		DIAGNOSTICO_LISTA_DICC_VINCULOS_ODBC
		DIAGNOSTICO_LISTA_DICC_VINCULOS_OTROS
		DIAGNOSTICO_LISTA_DICC Rutinas
		DIAGNOSTICO_LISTA_DICC VARIABLES
		DIAGNOSTICO_DF_LISTADO_TABLAS_Y_VINCULOS
		DIAGNOSTICO_DF_LISTADO Rutinas
		DIAGNOSTICO_DF_LISTADO VARIABLES
		DIAGNOSTICO_DF_CODIGO_TABLAS_Y_VINCULOS
		DIAGNOSTICO_DF_CODIGO Rutinas
		DIAGNOSTICO_DF_CODIGO VARIABLES
	SQL_SERVER	SERVIDOR
		BBDD
		CONNECTING_STRING
		LISTA_DICC_OBJETOS
		LISTA_BBDD_SERVIDOR

Los nombres de las distintas keys **no se han de modificar** puesto que en el código del app se usan estos nombres en múltiples ocasiones.

Los valores informados para las keys marcadas en **ROJO** **son configurables** (pero han de ser string).

Módulo APP_2_GENERAL

Diccionarios de configuración

dicc_control_versiones_tipo_objeto								
KEY 1	KEY 2	KEY 3	KEY 4	KEY 1	KEY 2	KEY 3	KEY 4	
MS_ACCESS	TIPO_OBJETO	TODOS	COMBOBOX_CONTROL_VERSIONES	SQL_SERVER	TIPO_OBJETO	TODOS	COMBOBOX_CONTROL_VERSIONES	
			TIPO_OBJETO_SUBFORM				TIPO_OBJETO_SUBFORM	
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES				LISTA_DICC_OBJETOS_CONTROL_VERSIONES	
		TABLA_LOCAL	COMBOBOX_CONTROL_VERSIONES			TABLAS	COMBOBOX_CONTROL_VERSIONES	
			TIPO_OBJETO_SUBFORM				TIPO_OBJETO_SUBFORM	
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES				LISTA_DICC_OBJETOS_CONTROL_VERSIONES	
		VINCULO_ODBC	COMBOBOX_CONTROL_VERSIONES			VIEWS	LISTA_OBJECT_ID	
			TIPO_OBJETO_SUBFORM				COMBOBOX_CONTROL_VERSIONES	
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES				TIPO_OBJETO_SUBFORM	
		VINCULO_OTRO	COMBOBOX_CONTROL_VERSIONES			STORED_PROCEDURES	LISTA_DICC_OBJETOS_CONTROL_VERSIONES	
			TIPO_OBJETO_SUBFORM				LISTA_OBJECT_ID	
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES			FUNCIONES	COMBOBOX_CONTROL_VERSIONES	
	MERGE_BBDD_FISICA	RUTINAS_VBA	COMBOBOX_CONTROL_VERSIONES				TIPO_OBJETO_SUBFORM	
			TIPO_OBJETO_SUBFORM				LISTA_DICC_OBJETOS_CONTROL_VERSIONES	
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES				LISTA_OBJECT_ID	
		VARIABLES_VBA	COMBOBOX_CONTROL_VERSIONES			OBJETOS_A_MIGRAR	COMBOBOX_GUI	
			TIPO_OBJETO_SUBFORM				LISTA_DICC_OBJETOS	
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES			MERGE_BBDD_FISICA		
		AJUSTES_MANUALES	COMBOBOX_GUI		MERGE_BBDD_FISICA			
			LISTA_DICC_OBJETOS					
			COMBOBOX_GUI					
		OBJETOS_A_MIGRAR	LISTA_DICC_OBJETOS		LISTA_DICC_ERRORES_MIGRACION			

Los nombres de las distintas keys **no se han de modificar** puesto que en el código del app se usan estos nombres en múltiples ocasiones.

Los valores informados para las keys marcadas en **ROJO** **son configurables** (pero han de ser string).

Módulo APP_2_GENERAL

Diccionarios de configuración

dicc_control_versiones_tipo_concepto

KEY 1
YA_EXISTE
SOLO_EN_BBDD_01
SOLO_EN_BBDD_02

dicc_errores_procesos

KEY 1	KEY 2	KEY 3
PROCESO_01	MS_ACCESS	LISTA_DICC_ERRORES_IMPORTACION_BBDD_01
		LISTA_DICC_ERRORES_IMPORTACION_BBDD_02
		LISTA_DICC_ERRORES_CALCULO
PROCESO_01	SQL_SERVER	LISTA_DICC_ERRORES_IMPORTACION_BBDD_01
		LISTA_DICC_ERRORES_IMPORTACION_BBDD_02
		LISTA_DICC_ERRORES_CALCULO
PROCESO_02	MS_ACCESS	LISTA_DICC_ERRORES_IMPORTACION_BBDD_01
		LISTA_DICC_ERRORES_CALCULO
PROCESO_03	SQL_SERVER	LISTA_DICC_ERRORES_IMPORTACION_BBDD_01
		LISTA_DICC_ERRORES_CALCULO

Los nombres de las distintas keys no se han de modificar puesto que en el código del app se usan estos nombres en múltiples ocasiones.

Los valores informados para las keys marcadas en **ROJO** **son configurables** (pero han de ser string).

Módulo APP_2_GENERAL

El diagnostico de dependencias de las variables globales es el siguiente:

Variables globales (1/3)

TIPO OBJETO	NOMBRE OBJETO	EN QUE MODULO SE USA	EN QUE CLASE SE USA	DONDE SE USA EN LA CLASE	EN QUE Rutina / FUNCION SE USA
Variable global	nombre_app	APP_1_GUI	gui_ventana_inicio	Constructor	
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_threads
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_access_add_clear
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_1
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_2
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_boton_check
		APP_1_GUI	gui_sql_server_authentication	Rutina integrada	def_GUI_conexion_servidor_sql_server
		APP_1_GUI	gui diagnostico_bbdd_sql_server	Constructor	
		APP_1_GUI	gui diagnostico_bbdd_sql_server	Rutina integrada	def_GUI_sql_server diagnostico_threads
		APP_1_GUI	gui diagnostico_bbdd_sql_server	Rutina integrada	def_GUI_sql_server diagnostico_boton_check
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_seleccion
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_excel
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_GUI_proceso_merge_lineas_exit
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_proceso_merge_combobox_bbdd_selecc
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_merge
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_cambios_en_bbdd
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_seleccion
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_export
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_realizar
	nombre_control_versiones	APP_1_GUI	gui_ventana_control_versiones	Constructor	
	nombre_merge_bbdd_fisicas	APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Constructor	
	lista_acciones_migrar_quitar	APP_2_GENERAL			def_proceso_merge_realizar_cambios
	lista_acciones_revertir	APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_merge
		APP_2_GENERAL			def_proceso_merge_realizar_cambios
	global_proceso_en_ejecucion	APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_threads
		APP_1_GUI	gui diagnostico_bbdd_sql_server	Rutina integrada	def_GUI_sql_server diagnostico_threads
		APP_2_GENERAL			def_calc_global
	lista_GUI_proceso_merge_tipo_accion_bbdd_1	APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_proceso_merge_combobox_bbdd_selecc
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_merge
		APP_2_GENERAL			def_proceso_merge_realizar_cambios
	lista_GUI_proceso_merge_tipo_accion_bbdd_2	APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_proceso_merge_combobox_bbdd_selecc
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_merge
		APP_2_GENERAL			def_proceso_merge_realizar_cambios
	label_merge_access_bbdd_fisica_en_manual	APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos_click_item
		APP_2_GENERAL			def_merge_access_ajustes_manuales
	dicc_procesos	APP_1_GUI	gui_ventana_inicio	Constructor	
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_proceso
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_boton_check
		APP_2_GENERAL			def_generacion_logs

Módulo APP_2_GENERAL

El diagnostico de dependencias de las variables globales es el siguiente:

Variables globales (2/3)

TIPO OBJETO	NOMBRE OBJETO	EN QUE MODULO SE USA	EN QUE CLASE SE USA	DONDE SE USA EN LA CLASE	EN QUE RUTINA / FUNCION SE USA
Variable global	dicc_codigos_bbdd	APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_access_add_clear
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_1
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_2
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_boton_check
		APP_1_GUI	gui_sql_server_authentication	Rutina integrada	def_GUI_conexion_servidor_sql_server
		APP_1_GUI	gui_ventana_control_versiones	Constructor	
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_cambio_tipo_objeto
		APP_2_GENERAL			def_generacion_logs
		APP_2_GENERAL			func_se_puede_ejecutar_proceso
		APP_2_GENERAL			def_calc_global
		APP_2_GENERAL			def_control_versiones_export_excel
		APP_2_GENERAL			def_merge_access_ajustes_manuales
		APP_2_GENERAL			def_merge_bbdd_fisicas
		APP_3_BACK_END_ACCESS			def_proceso_access_1_import
		APP_3_BACK_END_ACCESS			def_proceso_access_2_control_versiones
		APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
		APP_3_BACK_END_SQL_SERVER			def_sql_server_servidor_permisos
		APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_1_import
		APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_control_versiones
	dicc_control_versiones_tipo_concepto	APP_1_GUI	gui_ventana_control_versiones	Constructor	
		APP_2_GENERAL			func_df_control_versiones
		APP_2_GENERAL			def_control_versiones_export_excel
		APP_3_BACK_END_ACCESS			def_proceso_access_2_control_versiones
		APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_control_versiones
	lista_GUI_seleccion_tipo_objeto_access	APP_1_GUI	gui_ventana_control_versiones	Constructor	
	lista_GUI_seleccion_tipo_objeto_sql_server	APP_1_GUI	gui_ventana_control_versiones	Constructor	
	lista_GUI_seleccion_tipo_concepto	APP_1_GUI	gui_ventana_control_versiones	Constructor	
	lista_GUI_seleccion_bbdd	APP_1_GUI	gui_ventana_control_versiones	Constructor	
	dicc_errores_procesos	APP_2_GENERAL			func_procesos_errores
		APP_2_GENERAL			def_generacion_logs
		APP_2_GENERAL			def_calc_global
		APP_3_BACK_END_ACCESS			def_proceso_access_1_import
		APP_3_BACK_END_ACCESS			def_proceso_access_2_control_versiones
		APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
		APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_1_import
		APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_control_versiones
		APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_diagnostico

Módulo APP_2_GENERAL

El diagnostico de dependencias de las variables globales es el siguiente:

Variables globales (3/3)

TIPO OBJETO	NOMBRE OBJETO	EN QUE MODULO SE USA	EN QUE CLASE SE USA	DONDE SE USA EN LA CLASE	EN QUE RUTINA / FUNCION SE USA
Variable global	dicc_control_versiones_tipo_objeto	APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_cambio_tipo_objeto
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_seleccion
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_excel
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_update_subform_objetos
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_update_subform_objetos_click_item
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_merge
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_cambios_en_bbdd
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Constructor	
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_seleccion
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos_click_item
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_export
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_realizar
		APP_2_GENERAL			def_generacion_logs
		APP_2_GENERAL			func_dicc_control_versiones_tipo_objeto_buscar_en_dicc
		APP_2_GENERAL			func_df_control_versiones
		APP_2_GENERAL			def_calc_global
		APP_2_GENERAL			def_proceso_merge_realizar_cambios
		APP_2_GENERAL			def_merge_access_ajustes_manuales
		APP_2_GENERAL			def_merge_bbdd_fisica_lista_objetos
		APP_2_GENERAL			def_merge_bbdd_fisicas_export
		APP_2_GENERAL			def_merge_bbdd_fisicas
		APP_3_BACK_END_ACCESS			def_proceso_access_2_control_versiones
		APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_control_versiones

Módulo APP_2_GENERAL

El diagnostico de dependencias de las rutinas / funciones es el siguiente:

Rutinas / funciones

TIPO OBJETO	NOMBRE OBJETO	EN QUE MODULO SE USA	EN QUE CLASE SE USA	DONDE SE USA EN LA CLASE	EN QUE Rutina / FUNCION SE USA
Función	func_procesos_errores	APP_2_GENERAL			def_calc_global
	func_dicc_control_versiones_tipo_objeto_buscar_en_dicc	APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_cambio_tipo_objeto
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_seleccion
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_excel
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_update_subform_objetos
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_update_subform_objetos_click_item
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_merge
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_cambios_en_bbdd
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Constructor	
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_seleccion
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos_click_item
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_export
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_realizar
		APP_2_GENERAL			def_merge_bbdd_fisicas
	func_se_puede_ejecutar_proceso	APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_boton_check
		APP_1_GUI	gui_ventana_control_versiones	Constructor	
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Constructor	
	func_control_versiones_dicc_proceso_merge_anteriores	APP_2_GENERAL			def_calc_global
		APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_update_subform_objetos_click_item
	func_dicc_control_versiones	APP_2_GENERAL			def_proceso_merge_realizar_cambios
		APP_3_BACK_END_ACCESS			def_proceso_access_2_control_versiones
		APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_control_versiones
Rutina	def_calc_global	APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_boton_check
		APP_1_GUI	gui_diagnostico_bbdd_sql_server	Rutina integrada	def_GUI_sql_server_diagnostico_boton_check
	def_control_versiones_export_excel	APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_control_versiones_click_boton_excel
	def_proceso_merge_realizar_cambios	APP_1_GUI	gui_ventana_control_versiones	Rutina integrada	def_click_proceso_merge_boton_merge
	def_merge_access_ajustes_manuales	APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Constructor	
	def_merge_bbdd_fisica_lista_objetos	APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Constructor	
	def_merge_bbdd_fisicas_export	APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_export
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Constructor	
	def_merge_bbdd_fisicas	APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_seleccion
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_update_subform_objetos_click_item
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_export
		APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_realizar
	def_generacion_logs	APP_1_GUI	gui_ventana_merge_bbdd_fisicas	Rutina integrada	def_merge_bbdd_fisicas_click_boton_realizar
		APP_2_GENERAL			def_calc_global

Módulo APP_3_BACK_END_MS_ACCESS

Las variables globales y rutinas que componen el módulo son los siguientes:

TIPO OBJETO	NOMBRE OBJETO	SE DOCUMENTA
Variable global	query_access_vinculos	No
	num_max_parametros_rutina_vba_access	No
	indicador_comentario_vba_access	No
	indicador_linea_codigo_vba_truncada_a_linea_siguiente	No
	indicador_variable_vba_declaracion_tipo_dato	No
	indicador_variable_signo_igual	No
	lista_GUI_askopenfilename_ms_access	No
	dicc_objetos_vba	No
	dicc_diagnostico_tablas_access_donde_buscar	No
	lista_headers_df_access_vinculos	No
	lista_headers_nuevas_columnas_rutinas	No
	lista_headers_nuevas_columnas_variables_publicas	No
	lista_headers_df_bbdd_codigo	No
	lista_headers_df_diagnostico_donde_buscar_campos_usados	No
	lista_headers_df_diagnostico_donde_buscar_tablas	No
	lista_headers_df_diagnostico_donde_buscar_rutinas_y_variables	No
	lista_headers_df_diagnostico_donde_buscar_variables_definidas_por_usuario	No
	lista_headers_df_listado	No
	lista_headers_df_dependencias	No
	lista_headers_df_sin_dependencias	No
	lista_headers_df_check_manual_tablas	No

TIPO OBJETO	NOMBRE OBJETO	SE DOCUMENTA
Función	func_codigo_vba_quitar_comentarios	No
	func_vba_string_quitar_encapsulado_entre_caracteres	No
	func_diagnostico_access_tablas_df_create_table	No
	func_access_bbdd_codigo_calculos	ANEXO_08
	func_access_bbdd_codigo_calculos_varios	ANEXO_09
Rutina	func_diagnostico_access_tablas_codigo_preservar	ANEXO_10
	def_proceso_access_1_import	ANEXO_01
	def_proceso_access_2_control_versiones	ANEXO_04
	def_proceso_access_2_diagnostico	ANEXO_11

Las variables marcadas en **AMARILLO**, correspondientes a **diccionarios de configuración**, se comentan en la página siguiente. Los objetos que no se documentan, con referirse al código del módulo es suficiente con los comentarios ahí informados.

Módulo

APP_3_BACK_END_MS_ACCESS

Diccionarios de configuración

dicc_objetos_vba		
KEY 1	KEY 2	KEY 3
VARIABLE_PUBLICA_NORMAL	STRING_PARA_LISTADO_EXCEL	
	LISTA_DECLARACION_INICIO_NO_ES_CONSTANTE	
	LISTA_DECLARACION_INICIO_SI_ES_CONSTANTE	
VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO	STRING_PARA_LISTADO_EXCEL	
	LISTA_DECLARACION_INICIO	
	LISTA_DECLARACION_FIN	
RUTINAS	RUTINA_PRIVADA	TIPO_RUTINA
		TIPO_DECLARACION
		LISTA_DECLARACION_INICIO
		LISTA_DECLARACION_FIN
	RUTINA_PUBLICA	TIPO_RUTINA
		TIPO_DECLARACION
		LISTA_DECLARACION_INICIO
		LISTA_DECLARACION_FIN
	FUNCION_PRIVADA	TIPO_RUTINA
		TIPO_DECLARACION
		LISTA_DECLARACION_INICIO
		LISTA_DECLARACION_FIN
	FUNCION_PUBLICA	TIPO_RUTINA
		TIPO_DECLARACION
		LISTA_DECLARACION_INICIO
		LISTA_DECLARACION_FIN
RUTINAS_VARIABLES_LOCALES	LISTA_DECLARACION_INICIO_NO_ES_CONSTANTE	
	LISTA_DECLARACION_INICIO_SI_ES_CONSTANTE	

dicc_diagnostico_tablas_access_donde_buscar	
KEY 1	
LISTA_TEXTOS_SIMILARES_SENTENCIAS_SQL	
LISTA_TEXTOS_INSTRUCCIONES_VBA_MANIP_TABLAS	

los valores de las keys en AZUL son configurables pero han de ser **LISTAS**, ver en el código del módulo los comentarios del diccionario para saber como se han de configurar

los valores de las keys en ROJO son configurables pero han de ser **string**

Los nombres de las distintas keys no se han de modificar puesto que en el código del app se usan estos nombres en múltiples ocasiones.

Los valores informados para las keys marcadas en **AZUL** y **ROJO** son configurables.

Módulo

APP_3_BACK_END_MS_ACCESS

El diagnostico de dependencias de las **variables globales** y **rutinas / funciones** es el siguiente:

TIPO OBJETO	NOMBRE OBJETO	EN QUE MODULO SE USA	EN QUE CLASE SE USA	DONDE SE USA EN LA CLASE	EN QUE RUTINA / FUNCION SE USA
Variable global	query_access_vinculos	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	num_max_parametros_rutina_vba_access	APP_3_BACK_END_ACCESS			func_access_df_bbdd_codigo_calculos_varios
	indicador_comentario_vba_access	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	indicador_linea_codigo_vba_truncada_a_linea_siguiente	APP_3_BACK_END_ACCESS			func_diagnostico_access_rutinas_calculos_variables
	indicador_variable_vba_declaracion_tipo_dato	APP_3_BACK_END_ACCESS			func_access_df_bbdd_codigo_calculos_varios
	indicador_variable_signo_igual	APP_3_BACK_END_ACCESS			func_access_df_bbdd_codigo_calculos_varios
	lista_GUI_askopenfilename_ms_access	APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_access_add_clear
	dicc_objetos_vba	APP_3_BACK_END_ACCESS			func_access_df_bbdd_codigo_calculos_varios
	dicc_diagnostico_tablas_access_donde_buscar	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	lista_headers_df_access_vinculos	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	lista_headers_nuevas_columnas_rutinas	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	lista_headers_nuevas_columnas_variables_publicas	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	lista_headers_df_bbdd_codigo	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	lista_headers_df_diagnostico_donde_buscar_campos_usados	APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
	lista_headers_df_diagnostico_donde_buscar_tablas	APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
	lista_headers_df_diagnostico_donde_buscar_rutinas_y_variables	APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
	lista_headers_df_diagnostico_donde_buscar_variables_definidas_por_usuario	APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
	lista_headers_df_listado	APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
	lista_headers_df_dependencias	APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
	lista_headers_df_sin_dependencias	APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
Función	lista_headers_df_check_manual_tablas	APP_3_BACK_END_ACCESS			def_proceso_access_2_diagnostico
	func_codigo_vba_quitar_comentarios	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	func_vba_string_quitar_encapsulado_entre_caracteres	APP_3_BACK_END_ACCESS			func_access_df_bbdd_codigo_calculos_varios
	func_vba_string_quitar_encapsulado_entre_caracteres	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	func_diagnostico_access_tablas_df_create_table	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	func_access_bbdd_codigo_calculos	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	func_access_bbdd_codigo_calculos_varios	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
Rutina	func_access_bbdd_codigo_calculos_varios	APP_3_BACK_END_ACCESS			func_access_df_bbdd_codigo_calculos
	func_diagnostico_access_tablas_codigo_preservar	APP_3_BACK_END_ACCESS			def_proceso_access_1_import
	def_proceso_access_1_import	APP_2_GENERAL			def_calc_global
	def_proceso_access_2_control_versiones	APP_2_GENERAL			def_calc_global
	def_proceso_access_2_diagnostico	APP_2_GENERAL			def_calc_global

Módulo APP_3_BACK_END_SQL_SERVER

Las variables globales y rutinas que componen el módulo son los siguientes:

TIPO OBJETO	NOMBRE OBJETO	SE DOCUMENTA
Variable global	driver_sql_server	No
	conn_str_sql_server_windows_authentication	No
	conn_str_sql_server_login_password_authentication	No
	conn_str_sql_server	No
	codigo_error_pyodbc_connexion_por_windows_authentication	No
	lista_GUI_sql_server_servidor	No
	lista_GUI_diagnostico_combobox_sql_server	No
	query_sql_server_bbdd_permisos	No
	query_lista_bbdd_sql_server	No
	query_objetos_sql_server	No
	query_objetos_sql_server_parametros	No
	query_campos_tablas_sql_server	No
	query_helptext_sql_server	No
	query_drop_if_exists_objeto	No
	df_encabezado_descarga_codigo_objetos	No
	indicador_sql_server_comentario_linea	No
	indicador_sql_server_comentario_bloque_linea_ini	No
	indicador_sql_server_comentario_bloque_linea_fin	No
	lista_headers_campos_df_codigo_objetos	No
	lista_headers_sql_server_diagnostico_df_bbdd_selecc	No
	lista_headers_sql_server_diagnostico_df_listado_objetos	No
	lista_headers_sql_server_diagnostico_df_dependencias	No
	lista_headers_sql_server_diagnostico_df_sin_dependencias	No
Función	func_sql_server_tipo_conexion_servidor	No
	func_sql_server_df_codigo_objeto	No
Rutina	def_sql_server_servidor_permisos	No
	def_proceso_sql_server_1_import	ANEXO_02
	def_proceso_sql_server_2_control_versiones	ANEXO_05
	def_proceso_sql_server_2_diagnostico	ANEXO_12

Los objetos que no se documentan, con referirse al código del módulo es suficiente con los comentarios ahí informados.

Módulo APP_3_BACK_END_SQL_SERVER

El diagnostico de dependencias de las **variables globales** y **rutinas / funciones** es el siguiente:

TIPO OBJETO	NOMBRE OBJETO	EN QUE MODULO SE USA	EN QUE CLASE SE USA	DONDE SE USA EN LA CLASE	EN QUE RUTINA / FUNCION SE USA
Variable global	conn_str_sql_server_windows_authentication	APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_1
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_2
		APP_3_BACK_END_SQL_SERVER			func_sql_server_tipo_conexion_servidor
	conn_str_sql_server_login_password_authentication	APP_1_GUI	gui_sql_server_authentication	Rutina integrada	def_GUI_conexion_servidor_sql_server
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_1
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_2
	conn_str_sql_server	APP_1_GUI	gui_sql_server_authentication	Rutina integrada	def_GUI_conexion_servidor_sql_server
		APP_3_BACK_END_SQL_SERVER			func_sql_server_tipo_conexion_servidor
		APP_3_BACK_END_SQL_SERVER			func_sql_server_tipo_conexion_servidor
	codigo_error_pyodbc_conexion_por_windows_authentication	APP_3_BACK_END_SQL_SERVER			func_sql_server_tipo_conexion_servidor
	lista_GUI_sql_server_servidor	APP_1_GUI	gui_ventana_inicio	Constructor	
	lista_GUI_diagnostico_combobox_sql_server	APP_1_GUI	gui_diagnostico_bbdd_sql_server	Constructor	
	query_sql_server_bbdd_permisos	APP_3_BACK_END_SQL_SERVER			def_sql_server_servidor_permisos
	query_lista_bbdd_sql_server	APP_3_BACK_END_SQL_SERVER			def_sql_server_servidor_permisos
	query_objetos_sql_server	APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_1_import
	query_objetos_sql_server_parametros	APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_1_import
	query_campos_tablas_sql_server	APP_3_BACK_END_SQL_SERVER			func_sql_server_df_codigo_objeto
	query_helptext_sql_server	APP_3_BACK_END_SQL_SERVER			func_sql_server_df_codigo_objeto
	query_drop_if_exists_objeto	APP_2_GENERAL			def_merge_bbdd_fisicas
	df_encabezado_descarga_codigo_objetos	APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_diagnostico
	indicador_sql_server_comentario_linea	APP_3_BACK_END_SQL_SERVER			func_sql_server_df_codigo_objeto
	indicador_sql_server_comentario_bloque_linea_ini	APP_3_BACK_END_SQL_SERVER			func_sql_server_df_codigo_objeto
	indicador_sql_server_comentario_bloque_linea_fin	APP_3_BACK_END_SQL_SERVER			func_sql_server_df_codigo_objeto
	lista_headers_campos_df_codigo_objetos	APP_3_BACK_END_SQL_SERVER			func_sql_server_df_codigo_objeto
	lista_headers_sql_server_diagnostico_df_bbdd_selecc	APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_diagnostico
	lista_headers_sql_server_diagnostico_df_listado_objetos	APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_diagnostico
	lista_headers_sql_server_diagnostico_df_dependencias	APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_diagnostico
	lista_headers_sql_server_diagnostico_df_sin_dependencias	APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_2_diagnostico
Función	func_sql_server_tipo_conexion_servidor	APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_1
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_2
	func_sql_server_df_codigo_objeto	APP_3_BACK_END_SQL_SERVER			def_proceso_sql_server_1_import
Rutina	def_sql_server_servidor_permisos	APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_1
		APP_1_GUI	gui_ventana_inicio	Rutina integrada	def_GUI_combobox_sql_server_servidor_2
		APP_1_GUI	gui_diagnostico_bbdd_sql_server	Constructor	
	def_proceso_sql_server_1_import	APP_2_GENERAL			def_calc_global
	def_proceso_sql_server_2_control_versiones	APP_2_GENERAL			def_calc_global
	def_proceso_sql_server_2_diagnostico	APP_2_GENERAL			def_calc_global

Configuración de pantalla al abrir el app

Configuración de pantalla al abrir el app

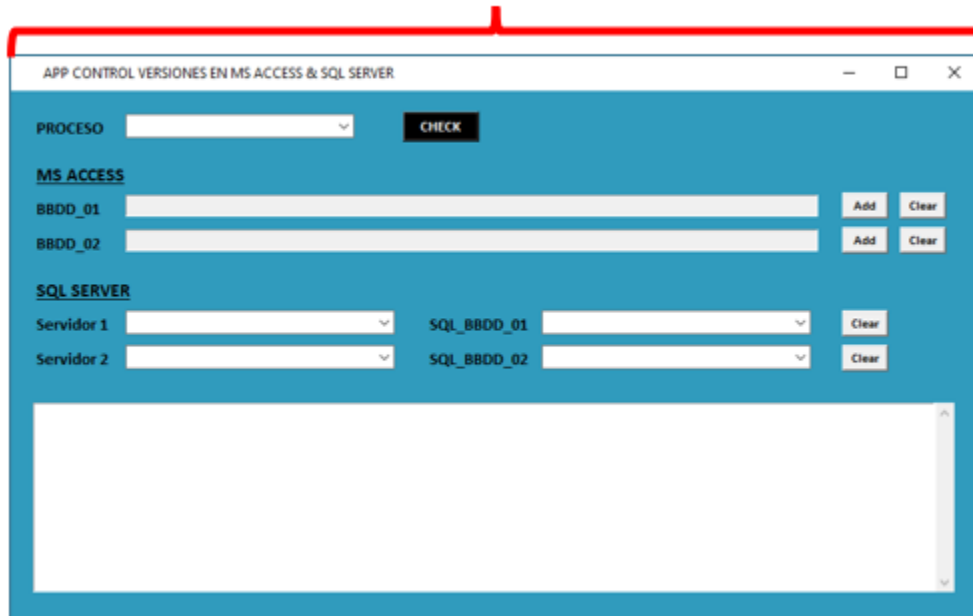
El app se desarrolló con una resolución de pantalla de **1920x1080**

La GUI del app se desarrolló usando la librería nativa de Python **tkinter** y todos los widgets (labels, combobox, textbox, botones etc etc) se han colocado en la GUI con el método **place** (en vez de usar los métodos **pack** o **grid**).

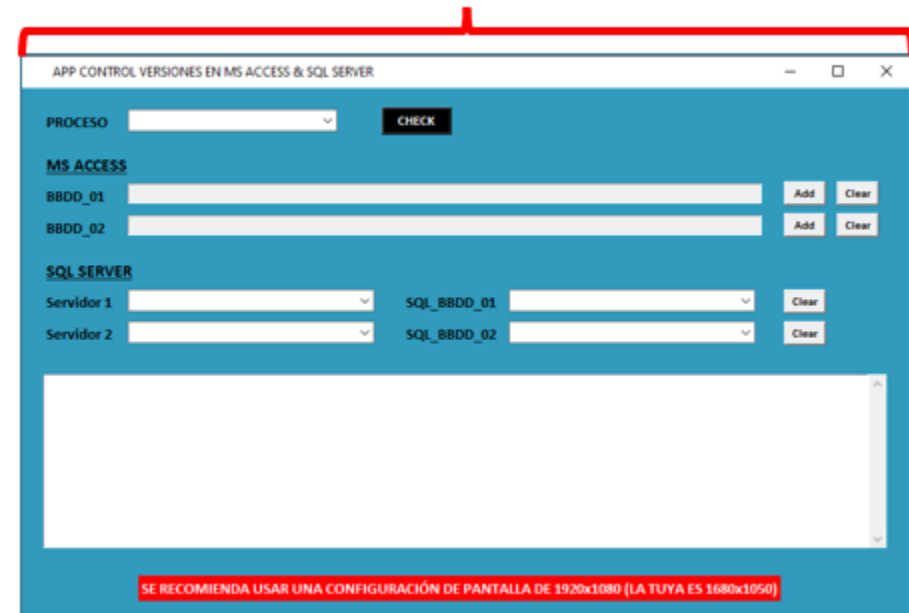
Por lo que, en caso una resolución de pantalla distinta de la indicada, los widgets de la GUI pueden verse **desplazados**.

Al abrir el app, se avisa de ello

ES LA RESOLUCIÓN RECOMENDADA



NO ES LA RESOLUCIÓN RECOMENDADA



Configuración de pantalla al abrir el app

A nivel de código, esto se realiza en el módulo APP_1_GUI.

Módulo APP_1_GUI

```
33 #resolucion pantalla --> se recomienda que sea de 1920x1080
34 #sino es posible que la GUI tenga los widgets descolocados debido al uso para colocarlos del metodo place en vez de usar pack o grid
35 resolucion_pantalla_recomendada = "1920x1080"
```

Módulo APP_1_GUI

```
213 class gui_ventana_inicio:
214     #clase que permite generar la GUI de inicio con sus widgets y rutinas asociadas
215
216     def __init__(self, master, **kwargs):
217
218         resolucion_pantalla = str(master.winfo_screenwidth()) + "x" + str(master.winfo_screenheight()) #se calcula es_resolucion_pantalla_recomendada para saber si informar en la GUI
219         es_resolucion_pantalla_recomendada = "SI" if resolucion_pantalla == resolucion_pantalla_recomendada else "NO" #que la resolucion de la pantalla no es la recomendada
220
221
222         self.master = master
223         self.master.title(mod_gen.nombre_app)
224         self.master.configure(bg = bg_GUI_inicio)
225
226         if es_resolucion_pantalla_recomendada == "SI":
227             self.master.geometry("840x460")
228         else:
229             self.master.geometry("840x500")
230
```

Módulo APP_1_GUI

```
361 if es_resolucion_pantalla_recomendada == "NO":
362     string_label_warning_resolucion_pantalla = "SE RECOMIENDA USAR UNA CONFIGURACIÓN DE PANTALLA DE " + resolucion_pantalla_recomendada + " (LA TUYA ES " + resolucion_pantalla + ")"
363
364     self.label_warning_resolucion_pantalla = (gui_widgets(master, tipo_widget = "label", text = string_label_warning_resolucion_pantalla, bg = "red", fg = "white",
365     font = ("Calibri", 11, "bold"), place = (110, 460)))
```

Proceso control de versiones

CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

Nada más abrir el app se ejecuta la clase **gui_ventana_inicio**.

APP CONTROL VERSIONES EN MS ACCESS & SQL SERVER

PROCESO **Control de versiones** **CHECK**

MS ACCESS

BBDD_01

BBDD_02

SQL SERVER

Servidor1

Servidor2

SQL_BBDD_01

SQL_BBDD_02

El control de versiones puede hacerse según 3 opciones:

1. Solamente entre 2 bbdd MS Access,
2. Solamente entre 2 bbdd SQL Server.
3. Sobre 2 bbdd MS Access y 2 bbdd SQL Server.

MS ACCESS:

--> las 2 bbdd deben tener el código VBA desbloqueado.

--> las 2 bbdd deben tener la macro AutoExec desactivada.

CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

1

Selección del proceso

Al seleccionar el proceso, se actualiza el scrolledtext con el descriptivo de lo que hace el proceso. Se realiza mediante la rutina **def_GUI_combobox_proceso** (módulo APP_1_GUI) y usando el diccionario **dicc_procesos** (módulo APP_2_GENERAL).

2

Selección bases de datos MS ACCESS

Al pulsar los botones **Add** en la GUI, tanto para la BBDD_01 como para la BBDD_02, permite guardar en memoria las ubicaciones de las bases de datos.

Se realiza mediante la rutina **def_GUI_access_add_clear** (módulo APP_1_GUI) que permite mostrar en la GUI las ubicaciones seleccionadas y guardarlas en el diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) en PATH_BBDD (subkey_3 asociada a la subkey_2 llamada MS_ACCESS) .

3

Borrar bases de datos MS ACCESS

Al pulsar los botones **Clean** en la GUI, se borran las ubicaciones de las bases de datos en la GUI y se reestablece a None el valor de la subkey_3 PATH_BBDD del diccionario **dicc_codigos_bbdd**, mediante la rutina **def_GUI_access_add_clear** (módulo APP_1_GUI).

CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

4

Selección servidores SQL SERVER

Los combobox de selección de servidores se alimentan de la lista de servidores configurados en la lista **lista_GUI_sql_server_servidor** (módulo APP_3_BACK_END_SQL_SERVER).

Al seleccionar un servidor, según sea el servidor 1 o 2 se ejecuta la rutina **def_GUI_combobox_sql_server_servidor_1** o **def_GUI_combobox_sql_server_servidor_2** (módulo APP_1_GUI).

En estas rutinas se comprueba mediante la función **func_sql_server_tipo_conexion_servidor** (módulo APP_3_BACK_END_SQL_SERVER) el tipo de conexión a SQL SERVER:

- si la conexión es por Windows Authentication, se actualiza el diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) en CONNECTING STRING (subkey_3 asociada a la subkey_2 llamada SQL SERVER) usando la variable **conn_str_sql_server_windows_authentication** (módulo APP_3_BACK_END_SQL_SERVER).
- Si la conexión anterior por Windows Authentication falla (bloque try except) por el número de error 28000 asociado a la librería Python **pyodbc** (**codigo_error_pyodbc_connexion_por_windows_authentication**, módulo APP_3_BACK_END_SQL_SERVER) se ejecuta la clase propia **gui_sql_server_authentication** que permite probar la conexión por SQL Server Authentication y poder registrar el login y password por parte del usuario en la connecting string que se almacena en la subkey_3 (CONNECTING STRING) mencionada anteriormente.

CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

4

Selección servidores SQL SERVER

Módulo APP_3_BACK_END_SQL_SERVER

```
217 def func_sql_server_tipo_conexion_servidor(servidor):
218     #funcion que permite determinar el tipo de conexión a SQL Server (WINDOWS_AUTHENTICATION o SQL_SERVER_AUTHENTICATION)
219
220     try:
221         tipo_connecting_string = None
222         conn_str = conn_str_sql_server_windows_authentication.replace("REEMPLAZA_SERVIDOR", servidor).replace("REEMPLAZA_BBDD", "master")
223         MiConexion = pyodbc.connect(conn_str)
224
225     except pyodbc.Error as Err:
226         if Err.args[0] == codigo_error_pyodbc_conexion_por_windows_authentication:
227             tipo_connecting_string = "SQL_SERVER_AUTHENTICATION"
228         else:
229             tipo_connecting_string = None
230
231     pass
232
233     else:
234         tipo_connecting_string = "WINDOWS_AUTHENTICATION"
235         MiConexion.close()
236
237     return tipo_connecting_string
```


CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

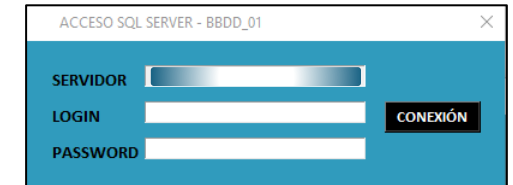
4

Selección servidores SQL SERVER

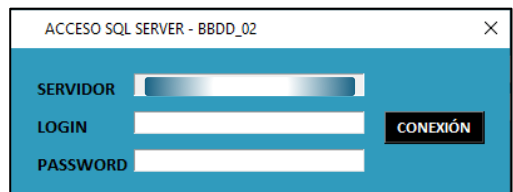
Módulo APP_1_GUI



```
469 if mod_sql_server.func_sql_server_tipo_conexion_servidor(servidor_selecc) == "WINDOWS_AUTHENTICATION":
470
471     mod_gen.dicc_codigos_bbdd["BBDD_01"]["SQL_SERVER"]["CONNECTING_STRING"] = mod_sql_server.conn_str_sql_server_windows_authentication#aquí es BBDD_01
472
473     #se localizan los permisos
474     mod_sql_server.def_sql_server_servidor_permisos("BBDD_01", servidor_selecc)
475
476     if mod_sql_server.global_acceso_servidor_selecc == "NO":
477         messagebox.showerror(title = mod_gen.nombre_app, message = "No tienes acceso al servidor seleccionado.")
478
479     else:
480         if not isinstance(mod_sql_server.global_servidor_bbdd_permisos_acceso_codigo, list):
481             messagebox.showerror(title = mod_gen.nombre_app, message = "No tienes permiso de acceso al código de los objetos de ninguna de las bbdd del servidor seleccionado.")
482
483         else:
484             self.combobox_sql_server_bbdd_1.combobox_update_lista_valores(combobox_lista_valores = mod_sql_server.global_servidor_bbdd_permisos_acceso_codigo)
485
486
487 elif mod_sql_server.func_sql_server_tipo_conexion_servidor(servidor_selecc) == "SQL_SERVER_AUTHENTICATION":
488
489     self.toplevel_sql_server_authentication = tk.Toplevel(self.master)
490     self.toplevel_sql_server_authentication.transient(self.master)
491     self.toplevel_sql_server_authentication.grab_set()
492
493     call_gui_sql_server_authentication = (gui_sql_server_authentication(self.toplevel_sql_server_authentication,
494                                     combobox_sql_server_bbdd = self.combobox_sql_server_bbdd_1,
495                                     opcion_bbdd = "BBDD_01", servidor_sql_server = servidor_selecc))#aquí es BBDD_01
```



```
509 if mod_sql_server.func_sql_server_tipo_conexion_servidor(servidor_selecc) == "WINDOWS_AUTHENTICATION":
510
511     mod_gen.dicc_codigos_bbdd["BBDD_02"]["SQL_SERVER"]["CONNECTING_STRING"] = mod_sql_server.conn_str_sql_server_windows_authentication#aquí es BBDD_02
512
513     #se localizan los permisos
514     mod_sql_server.def_sql_server_servidor_permisos("BBDD_02", servidor_selecc)
515
516     if mod_sql_server.global_acceso_servidor_selecc == "NO":
517         messagebox.showerror(title = mod_gen.nombre_app, message = "No tienes acceso al servidor seleccionado.")
518
519     else:
520         if not isinstance(mod_sql_server.global_servidor_bbdd_permisos_acceso_codigo, list):
521             messagebox.showerror(title = mod_gen.nombre_app, message = "No tienes permiso de acceso al código de los objetos de ninguna de las bbdd del servidor seleccionado.")
522
523         else:
524             self.combobox_sql_server_bbdd_2.combobox_update_lista_valores(combobox_lista_valores = mod_sql_server.global_servidor_bbdd_permisos_acceso_codigo)
525
526
527 elif mod_sql_server.func_sql_server_tipo_conexion_servidor(servidor_selecc) == "SQL_SERVER_AUTHENTICATION":
528
529     self.toplevel_sql_server_authentication = tk.Toplevel(self.master)
530     self.toplevel_sql_server_authentication.transient(self.master)
531     self.toplevel_sql_server_authentication.grab_set()
532
533     call_gui_sql_server_authentication = (gui_sql_server_authentication(self.toplevel_sql_server_authentication,
534                                     combobox_sql_server_bbdd = self.combobox_sql_server_bbdd_2,
535                                     opcion_bbdd = "BBDD_02", servidor_sql_server = servidor_selecc))#aquí es BBDD_02
```



Módulo APP_1_GUI

```
758 elif len(login_selecc) != 0 and len(password_selecc) != 0:
759
760     conn_string = mod_sql_server.conn_str_sql_server_login_password_authentication.replace("REEMPLAZA_LOGIN", login_selecc).replace("REEMPLAZA_PASSWORD", password_selecc)
761     mod_gen.dicc_codigos_bbdd[self.opcion_bbdd]["SQL_SERVER"]["CONNECTING_STRING"] = conn_string
```

CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

4

Selección servidores SQL SERVER

Una vez registrada la connecting string en el diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL), se realiza la conexión al servidor seleccionado en el combobox de la GUI, mediante la rutina **def_sql_server_servidor_permisos** (módulo APP_3_BACK_END_SQL_SERVER).

Se define en esta rutina una variable global llamada **global_acceso_servidor_selecc** que devuelve **SI** o **NO**, según haya conexión al servidor, y poder así usarla para generar un warning en la GUI si no hay conexión.

```
400      global global_acceso_servidor_selecc
```

APP_3_BACK_END_SQL_SERVER

```
410      try:#se chequea si el usuario puede acceder al servidor
411          conn_str = mod_gen.dicc_codigos_bbdd[opcion_bbdd]["SQL_SERVER"]["CONNECTING_STRING"].replace("REEMPLAZA_SERVIDOR", path_servidor).replace("REEMPLAZA_BBDD", "master")
412          MiConexion_1 = pyodbc.connect(conn_str)
413
414      except:#en caso de que no se informa global_acceso_servidor_selecc = NO 413
415          global_acceso_servidor_selecc = "NO"
416          pass
417
418      else:
419          global_acceso_servidor_selecc = "SI"
```

CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

4

Selección servidores SQL SERVER

Se define también en esta rutina otra variable global llamada **global_servidor_bbdd_permisos_acceso_codigo** que permite, en el caso de que haya conexión al servidor, listar en una lista todas las bases de datos del mismo donde el usuario tiene permisos de acceso al código T-SQL de los objetos.

```
401 global global_servidor_bbdd_permisos_acceso_codigo
```

En caso de que haya conexión al servidor se listan en todas las bases de datos del mismo (excluyendo las de sistema) en la variable local **lista_bbdd_servidor**. Se usa para ello la query **query_lista_bbdd_sql_server** (módulo APP_3_BACK_END_SQL_SERVER).

APP_3_BACK_END_SQL_SERVER

```
418 else:
419     global_acceso_servidor_selecc = "SI"
420
421     #se listan todas las bbdd del servidor seleccionado (excluyendo las de master, tempdb, model y msdb)
422     query_temp = query_lista_bbdd_sql_server
423     df_temp = pd.read_sql_query(query_temp, MiConexion_1)
424     MiConexion_1.close()
425
426     lista_bbdd_servidor = [df_temp.iloc[i, 0] for i in df_temp.index if len(df_temp) != 0 else []]
```

CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

4

Selección servidores SQL SERVER

Se define la variable global **global_servidor_bbdd_permisos_acceso_codigo** como una lista. Mediante la ejecución de un bucle sobre la lista encapsulada en la variable local **lista_bbdd_servidor**, en cada iteración:

- se crea conexión a la base de datos de la iteración
- se ejecuta la query **query_sql_server_bbdd_permisos** (módulo APP_3_BACK_END_SQL_SERVER)
- Se encapsula el resultado del paso anterior en un dataframe
- Si el dataframe no esta vacío se agrega la base de datos a **global_servidor_bbdd_permisos_acceso_codigo**

APP_3_BACK_END_SQL_SERVER

```
430 global_servidor_bbdd_permisos_acceso_codigo = []
431 if len(lista_bbdd_servidor) != 0:
432     for bbdd in lista_bbdd_servidor:
433         try:#se conecta a la bbdd de la iteracion
434             conn_str = mod_gen.dicc_codigos_bbdd[opcion_bbdd][["SQL_SERVER"]]["CONNECTING_STRING"].replace("REEMPLAZA_SERVIDOR", path_servidor).replace("REEMPLAZA_BBDD", bbdd)
435             MiConexion_2 = pyodbc.connect(conn_str)
436         except:
437             pass
438         else:
439             #si se establece la conexion a la bbdd y si la query query_sql_server_bbdd_permisos filtrada por permisos_sql_acceso_codigos
440             #devuelve un df no vacio se agrega la bbdd a global_servidor_bbdd_permisos_acceso_codigo
441             query_permiso = query_sql_server_bbdd_permisos
442             df_permiso = pd.read_sql_query(query_permiso, MiConexion_2)
443             if len(df_permiso) != 0:
444                 global_servidor_bbdd_permisos_acceso_codigo.append(bbdd)
445             MiConexion_2.close()
```

CONTROL DE VERSIONES

Cálculo del proceso

Configuración previa al cálculo del proceso de control de versiones

5 Selección bases de datos SQL SERVER

La selección de las bases de datos no tiene ninguna rutina Python asociada.

6 Borrar bases de datos SQL SERVER

Al pulsar los botones **Clean** en la GUI, se borran los Servidores y las bases de datos en la GUI y se reestablece a None los valores de las subkey_3 SERVIDOR, BBDD y CONNECTING STRING del diccionario **dicc_codigos_bbdd**, mediante la rutina **def_GUI_access_add_clear** (módulo APP_1_GUI).

APP_1_GUI

```
441 elif opcion == "CLEAR_SQL_SERVER_BBDD_01":
442     self.strvar_sql_server_servidor_1.set("")
443     self.strvar_sql_server_bbdd_1.set("")
444     mod_gen.dicc_codigos_bbdd[bbdd]["SQL_SERVER"]["SERVIDOR"] = None
445     mod_gen.dicc_codigos_bbdd[bbdd]["SQL_SERVER"]["BBDD"] = None
446     mod_gen.dicc_codigos_bbdd[bbdd]["SQL_SERVER"]["CONNECTING_STRING"] = None
447
448 elif opcion == "CLEAR_SQL_SERVER_BBDD_02":
449     self.strvar_sql_server_servidor_2.set("")
450     self.strvar_sql_server_bbdd_2.set("")
451     mod_gen.dicc_codigos_bbdd[bbdd]["SQL_SERVER"]["SERVIDOR"] = None
452     mod_gen.dicc_codigos_bbdd[bbdd]["SQL_SERVER"]["BBDD"] = None
453     mod_gen.dicc_codigos_bbdd[bbdd]["SQL_SERVER"]["CONNECTING_STRING"] = None
```

CONTROL DE VERSIONES

Cálculo del proceso

Ejecución del proceso de cálculo de CONTROL DE VERSIONES

7

Botón CHECK

Al pulsar en el botón **CHECK**, ejecuta la rutina **def_GUI_threads** (módulo APP_1_GUI) que permite iniciar un thread sobre la rutina **def_GUI_boton_check** (módulo APP_1_GUI).

Se ejecuta un thread para poder iniciar la variable global **global_proceso_en_ejecucion** (módulo APP_2_GENERAL) con el valor de **SI**. Esto permite dentro de la rutina **def_GUI_threads** generar un warning cuando el usuario intenta ejecutar otro proceso con el botón **CHECK** cuando otro proceso está todavía en curso de ejecución.

APP_1_GUI

```
382     if mod_gen.global_proceso_en_ejecucion == "SI":
383         messagebox.showerror(title = mod_gen.nombre_app, message = "Espera a que acabe el proceso actualmente en ejecución.")
384
385     else:
386         Thread(target = self.def_GUI_boton_check).start()
```

En el caso de que no haya ningún proceso en curso, la rutina **def_GUI_boton_check** comprueba en 1er lugar si el proceso de control de versiones se puede calcular tanto para MS ACCESS como para SQL SERVER mediante la función **func_se_puede_ejecutar_proceso** (módulo APP_2_GENERAL).

APP_1_GUI

```
569     #se comprueba si se pueden realizar los procesos
570     check_control_versiones_access = mod_gen.func_se_puede_ejecutar_proceso("CONTROL_VERSIONES", "MS_ACCESS")
571     check_control_versiones_sql_server = mod_gen.func_se_puede_ejecutar_proceso("CONTROL_VERSIONES", "SQL_SERVER")
```

CONTROL DE VERSIONES

Cálculo del proceso

Ejecución del proceso de cálculo de CONTROL DE VERSIONES

7

Botón CHECK

La función **func_se_puede_ejecutar_proceso** funciona de forma distinta según el tipo de bases de datos:

- **MS ACCESS:** comprueba que los valores almacenados en la subkey_3 PATH_BBDD del diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) están informados tanto para las key_1 (BBDD_01 y BBDD_02) y que los rutas son distintas. En caso de no ser rutas de bases de datos distintas sale un warning avisando que el proceso no se calculara.
- **SQL SERVER:** realiza la misma comprobación que en MS ACCESS pero para las subkeys_3 SERVIDOR y BBDD del diccionario **dicc_codigos_bbdd**. En caso de que Servidor + BBDD no sean distintos para la key_1 (BBDD_01 y BBDD_02) sale un warning avisando que el proceso no se calculara.

En caso de que las bases de datos MS ACCESS y/o SQL SERVER están correctamente configuradas para poder ejecutar el proceso de cálculo de control de versiones, sale un mensaje que informa al usuario que indique una ruta en su PC donde almacenar los posibles errores de ejecución de los scripts Python (referente a la generación de logs de errores, ver más adelante en el diseño funcional).

Al indicar esta ruta se inicia el proceso mediante la rutina **def_calc_global** (módulo APP_2_GENERAL).

APP 1 GUI

```
615     mensaje = "SELECCIONA DONDE QUIERES GUARDAR LOS LOGS DE ERRORES (si los hubiese):"
616     ruta_destino_logs = fd.askdirectory(parent = root, title = mensaje)
617
618     self.master.config(cursor = "wait")
619     mod_gen.def_calc_global(proceso_selecc_id, ruta_destino_logs)
620     self.master.config(cursor = "")
```

CONTROL DE VERSIONES

Cálculo del proceso

Ejecución del proceso de cálculo de CONTROL DE VERSIONES

7

Botón CHECK

La rutina **def_calc_global** realiza las tareas siguientes:

- Vacía los valores de algunas de las subkeys de los diccionarios **dicc_codigos_bbdd**, **dicc_control_versiones_tipo_objeto** y **dicc_errores_procesos** (módulo APP_2_GENERAL).
- Cierra todas las aplicaciones MS ACCESS abiertas en caso de que el proceso de control de versiones afecte a este tipo de bases de datos.
- Inicializa la variable global **global_proceso_en_ejecución** (ver anteriormente) a **SI**.
- Según que se pueda ejecutar el proceso sobre MS ACCESS y/o SQL SERVER crea una lista de threads sobre las rutinas y ejecuta el hilo de threads haciendo un bucle sobre esta lista:
 1. **def_proceso_access_1_import** (módulo APP_3_BACK_END_MS_ACCESS) para BBDD_01 y BBDD_02.
 2. **def_proceso_sql_server_1_import** (módulo APP_3_BACK_END_SQL_SERVER) para BBDD_01 y BBDD_02

CONTROL DE VERSIONES

Cálculo del proceso

Ejecución del proceso de cálculo de CONTROL DE VERSIONES

7

Botón CHECK

APP_2_GENERAL

```
1041 lista_threads = []
1042 if func_se_puede_ejecutar_proceso("CONTROL_VERSIONES", "MS_ACCESS") == "SI":
1043
1044     thread = Thread(target = mod_access.def_proceso_access_1_import, args = (proceso_selecc_id, "BBDD_01", access_path_bbdd_1))
1045     lista_threads.append(thread)
1046
1047     thread = Thread(target = mod_access.def_proceso_access_1_import, args = (proceso_selecc_id, "BBDD_02", access_path_bbdd_2))
1048     lista_threads.append(thread)
1049
1050
1051 if func_se_puede_ejecutar_proceso("CONTROL_VERSIONES", "SQL_SERVER") == "SI":
1052
1053     thread = Thread(target = mod_sql_server.def_proceso_sql_server_1_import, args = (proceso_selecc_id, "BBDD_01", sql_server_servidor_1, sql_server_bbdd_lista_1))
1054     lista_threads.append(thread)
1055
1056     thread = Thread(target = mod_sql_server.def_proceso_sql_server_1_import, args = (proceso_selecc_id, "BBDD_02", sql_server_servidor_2, sql_server_bbdd_lista_2))
1057     lista_threads.append(thread)
1058
1059
1060 for thread in lista_threads:
1061     thread.start()
1062
1063 for thread in lista_threads:
1064     thread.join()
```

Las rutinas **def_proceso_access_1_import** y **def_proceso_sql_server_1_import** se detallan más en profundidad en los anexos **ANEXO_01** y **ANEXO_02** de este documento respectivamente.

CONTROL DE VERSIONES

Cálculo del proceso

Ejecución del proceso de cálculo de CONTROL DE VERSIONES

7

Botón CHECK

Al finalizar la ejecución del hilo de threads se realiza la acción previa de cerrar todas las bases de datos MS ACCESS abiertas (en caso de que de que el proceso se haya ejecutado para MS ACCESS).

APP_2_GENERAL

```
1068     if func_se_puede_ejecutar_proceso("CONTROL_VERSIONES", "MS_ACCESS") == "SI":
1069         try:
1070             subprocess.Popen("taskkill /f /im msaccess.exe")
1071         except:
1072             pass
```

Esto se hace debido a que el método usado para abrir una base de datos MS Access e importar su código VBA (librería Python **win32com.client**), cuando se ejecuta el mismo proceso en un hilo de threads, cierra las bases de datos, pero deja la aplicación MS Access abierta.

APP_3_BACK_END_MS_ACCESS

```
2154     access_app = win32com.client.Dispatch("Access.Application")
2155     access_app.OpenCurrentDatabase(path_bbdd)
```

APP_3_BACK_END_MS_ACCESS

```
2301     access_app.CloseCurrentDatabase()
2302     access_app.Quit()
2303     access_app = None
```

Si se han localizado errores, se pasa a crear los logs de errores en formato .txt en la ruta que el usuario indico tras pulsar el botón **CHECK** mediante la rutina **def generacion_logs** (módulo APP_2_GENERAL).

```
1122     def generacion_logs("PROCESOS_LOGS_ERRORES_IMPORT_BBDD", tipo_bbdd, ruta_destino_fichero_logs, proceso_id = proceso_selecc_id, opcion_bbdd = "BBDD_01")
1123     def generacion_logs("PROCESOS_LOGS_ERRORES_IMPORT_BBDD", tipo_bbdd, ruta_destino_fichero_logs, proceso_id = proceso_selecc_id, opcion_bbdd = "BBDD_02")
```

Esta rutina se ejecuta tan solo si se localizan en las subkeys_3 (LISTA DICC ERRORES BBDD 01 y LISTA DICC ERRORES BBDD 02) del diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) listas de diccionarios y no valores None.

CONTROL DE VERSIONES

Cálculo del proceso

Ejecución del proceso de cálculo de CONTROL DE VERSIONES

7

Botón CHECK

A continuación, si no se han localizado **errores** en el proceso de importación, se ejecutan las rutinas **def_proceso_access_2_control_versiones** y/o **def_proceso_sql_server_2_control_versiones** (módulo APP_2_GENERAL) según el tipo de bases de datos por los que se ha optado por calcular el proceso de control de versiones.

La ejecución de estas rutinas no se hace por hilo de threads sino realizando un bucle sobre la key_1 del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL).

APP_2_GENERAL

```
1085     for tipo_bbdd in dicc_control_versiones_tipo_objeto.keys():
1086
1087         tipo_bbdd_msg = tipo_bbdd.replace("_", " ")
1088
1089         if func_se_puede_ejecutar_proceso("CONTROL_VERSIONES", tipo_bbdd) == "SI":
1090
1091             if func_procesos_errores(proceso_selecc_id, tipo_bbdd, "IMPORT") == "SIN_ERROR":
1092
1093                 if tipo_bbdd == "MS_ACCESS":
1094                     mod_access.def_proceso_access_2_control_versiones()
1095
1096                 elif tipo_bbdd == "SQL_SERVER":
1097                     mod_sql_server.def_proceso_sql_server_2_control_versiones()
```

Las rutinas **def_proceso_access_2_control_versiones** y **def_proceso_sql_server_2_control_versiones** se detallan más en profundidad en los anexos **ANEXO_04** y **ANEXO_05** respectivamente.

CONTROL DE VERSIONES

Cálculo del proceso

Ejecución del proceso de cálculo de CONTROL DE VERSIONES

7

Botón CHECK

Si las rutinas **def_proceso_access_2_control_versiones** y/o **def_proceso_sql_server_2_control_versiones** han generado errores se pasa a crear los logs de errores en formato .txt en la ruta que el usuario indico tras pulsar el botón **CHECK** mediante la rutina **def_generacion_logs** (módulo APP_2_GENERAL).

APP_2_GENERAL

```
1100 if func_procesos_errores(proceso_selecc_id, tipo_bbdd, "CALCULO") == "ERROR":
1101     mensaje1 = "Control versiones " + tipo_bbdd_msg + "\n\nEl proceso de importacion de los datos se ha realizado correctamente
1102     mensaje2 = "pero el proceso de calculo del control de versiones ha generado errores por lo que el proceso se ha cancelado (
1103
1104     if tipo_bbdd == "MS_ACCESS":
1105         global_msg_errores_proceso_access = mensaje1 + mensaje2
1106
1107     elif tipo_bbdd == "SQL_SERVER":
1108         global_msg_errores_proceso_sql_server = mensaje1 + mensaje2
1109
1110     #se generan los logs en ficheros .txt
1111     def_generacion_logs("PROCESOS_LOGS_ERRORES_CALCULO", tipo_bbdd, ruta_destino_fichero_logs, proceso_id = proceso_selecc_id)
```

Al igual que en el proceso de importación, esta rutina se ejecuta tan solo si se localizan en las subkeys_3 (LISTA DICC ERRORES CALCULO) del diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) listas de diccionarios y no valores None

APP_2_GENERAL

```
1129 global_proceso_en_ejecucion = "NO"
```

Al finalizar el proceso, la variable global **global_proceso_en_ejecucion** se reestablece a **NO** para que el usuario pueda volver a ejecutar procesos en la GUI de inicio.

CONTROL DE VERSIONES

Cálculo del proceso

Ejecución del proceso de cálculo de CONTROL DE VERSIONES

7

Botón CHECK

Antes de finalizar y pasar a comentar el proceso de **Localizar cambios y realizar migraciones de scripts**, aclarar como se registra el flujo de errores en las rutinas:

- `def_proceso_access_1_import`
- `def_proceso_sql_server_1_import`
- `def_proceso_access_2_control_versiones`
- `def_proceso_sql_server_2_control_versiones`

Todas nada más iniciarse el código tienen un bloque try except configurado. En el bloque **Except**, cada error que se localice se encapsula en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL). donde se indica el tipo de base de datos, el módulo Python donde esta la rutina, el nombre de la rutina, la línea de código en el módulo y el descriptivo del error que se ha generado.

Cada diccionario, según que la rutina sea de importación o de calculo del proceso de control de versiones, se agrega a una lista de diccionarios acumulativa con todos los errores surgidos durante la ejecución y se guarda en las subkeys_3 del diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL).

El bloque **except** finaliza con la instrucción **pass** porque interesa ver todos los errores que puedan surgir durante la ejecución del proceso y no cancelar este último al 1er error que localice.

CONTROL DE VERSIONES

Cálculo del proceso

7

Botón CHECK

APP_3_BACK_END_MS_ACCESS

```
2555 except Exception as Err:
2556
2557     traceback_error = traceback.extract_tb(Err.__traceback__)
2558     modulo_python = os.path.basename(traceback_error[0].filename)
2559     rutina_python = traceback_error[0].name
2560     linea_error = traceback_error[0].lineno
2561
2562     lista_diccErrores_migracion = mod_gen.diccErrores_procesos[proceso_id][["MS_ACCESS"]][["LISTA_DICC_ERRORES_IMPORTACION_" + opcion_bbdd]]
2563
2564     diccErrores_temp = {"TIPO_BBDD": "MS_ACCESS"
2565                        , "MODULO_PYTHON": modulo_python
2566                        , "RUTINA_PYTHON": rutina_python
2567                        , "LINEA_ERROR": linea_error
2568                        , "ERRORES": str(Err)
2569                        }
2570
2571     if isinstance(lista_diccErrores_migracion, list):
2572         lista_diccErrores_migracion.append(diccErrores_temp)
2573     else:
2574         lista_diccErrores_migracion = [diccErrores_temp]
2575
2576     mod_gen.diccErrores_procesos[proceso_id][["MS_ACCESS"]][["LISTA_DICC_ERRORES_IMPORTACION_" + opcion_bbdd]] = lista_diccErrores_migracion
2577     del diccErrores_temp
2578     del lista_diccErrores_migracion
2579     pass #es pass para que se localicen todos los posibles errores en el proceso de import
```

Se muestra tan solo el pantallazo del bloque **except** de la rutina **def_proceso_access_1_import** porque funciona exactamente igual para las 4 rutinas mencionadas en la página anterior.

Tan solo cambia el nombre de la subkey_3 del diccionario **diccErrores_procesos** en el cual se recupera la lista de errores anteriores para agregarle el error en curso y volver a guardar la lista de diccionarios en el valor de esta subkey_3.

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

Si las subkeys_3 del diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) tienen por valor None y no son LISTAS se procede a abrir la clase propia **gui_ventana_control_versiones**.

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

1

Selección del tipo de objeto y tipo de selección

La lista de opciones del combobox de tipo de objeto varían según que se haya optado por el control de versiones en MS ACCESS y /o SQL SERVER.

Esta lista se calcula en base al valor tomado por la subkey_4 (COMBOBOX_CONTROL_VERSIONES) del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL) asociada a las subkey_2 (TIPO OBJETO).

Nada más abrir el app, desde el módulo APP_2_GENERAL se crean las listas:

- **lista_GUI_seleccion_tipo_objeto_access**
- **lista_GUI_seleccion_tipo_objeto_sql_server**

APP_2_GENERAL

```
398 lista_GUI_seleccion_tipo_objeto_access = []
399 for key in dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"].keys():
400     lista_GUI_seleccion_tipo_objeto_access.append(dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"][key]["COMBOBOX_CONTROL_VERSIONES"])
401
402
403 lista_GUI_seleccion_tipo_objeto_sql_server = []
404 for key in dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"].keys():
405     lista_GUI_seleccion_tipo_objeto_sql_server.append(dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"][key]["COMBOBOX_CONTROL_VERSIONES"])
```


CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

1

Selección del tipo de objeto y tipo de selección

Se usa la función **func_se_puede_ejecutar_proceso** (módulo APP_2_GENERAL) para saber si el control de versiones aplica a MS ACCESS y/o SQL SERVER y según el resultado se crea combinación de las 2 listas de la página anterior para crear la lista de opciones del combobox de selección de tipo de objeto en la GUI.

Esto se realiza en el módulo APP_1_GUI:

APP_1_GUI

```
962 lista_temp = []
963 if hacer_control_versiones_access == "SI" and hacer_control_versiones_sql_server == "SI":
964     lista_temp = mod_gen.lista_GUI_seleccion_tipo_objeto_access + mod_gen.lista_GUI_seleccion_tipo_objeto_sql_server
965
966 elif hacer_control_versiones_access == "SI" and hacer_control_versiones_sql_server == "NO":
967     lista_temp = mod_gen.lista_GUI_seleccion_tipo_objeto_access
968
969 elif hacer_control_versiones_access == "NO" and hacer_control_versiones_sql_server == "SI":
970     lista_temp = mod_gen.lista_GUI_seleccion_tipo_objeto_sql_server
971
972
973
974 self.combobox_tipo_objeto = (gui_widgets(master, tipo_widget = "combobox", textvariable = self.strvar_combobox_tipo_objeto, width = 30, font = ("Calibri", 10, "bold"),
975                                     combobox_lista_valores = lista_temp, combobox_tipo_acceso_valores = "readonly", combobox_bind_mousewheel = True,
976                                     place = (140, 20)))
```

La lista de opciones del combobox tipo de concepto se calcula mediante bucle sobre los valores de la key del diccionario **dicc_control_versiones_tipo_concepto** (módulo APP_2_GENERAL).

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

1

Selección del tipo de objeto y tipo de selección

La lista de opciones del combobox de tipo de concepto en la GUI se calcula en base a los valores tomados por la key_1 del diccionario **dicc_control_versiones_tipo_concepto** (módulo APP_2_GENERAL).

2

Botón EXCEL

Según el tipo de objeto seleccionado, el botón **EXCEL** permite descargar en un fichero Excel un listado de todos los objetos con cambios de script de una base de datos a otra según el tipo de base de datos (MS ACCESS o SQL SERVER) asociado al tipo de objeto seleccionado.

Esto se hace con la rutina **def_control_versiones_click_boton_excel** (módulo APP_1_GUI) la cual permite ejecutar la rutina **def_control_versiones_export_excel** (módulo APP_2_GENERAL).

```
APP_1_GUI
1305 path_xls = fd.askdirectory(parent = self.master, title = "INDICA DONDE QUIERES GUARDAR EL FICHERO EXCEL:")
1306
1307 self.master.config(cursor = "wait")
1308 mod_gen.def_control_versiones_export_excel(tipo_bbdd, lista_control_versiones_selecc, path_xls)
1309 self.master.config(cursor = "")
1310
1311 del lista_control_versiones_selecc
1312
1313 messagebox.showinfo(mod_gen.nombre_app, message = "Excel generado.")
```

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

3

Botón VER

Tras seleccionar el tipo de objeto y tipo de concepto, al pulsar el botón **VER** se actualiza el sub-formulario que lista todos los objetos localizados con cambios en los scripts de una base de datos a otra (BBDD_01 y BBDD_02).

El botón **VER** se asocia con la rutina **def_control_versiones_click_boton_seleccion** (módulo APP_1_GUI) la cual al ejecutar la rutina **def_control_versiones_update_subform_objetos** (módulo APP_1_GUI) actualiza el sub-formulario.

APP_1_GUI

```
1250     if not isinstance(lista_control_versiones_segun_tipo_objeto_selecc, list):
1251         messagebox.showerror(title = mod_gen.nombre_app, message = "No se han localizado objetos con cambios.")
1252
1253     else:
1254         self.script_bbdd_01.config(state = tk.NORMAL)
1255         self.script_bbdd_01.delete(1.0, tk.END)
1256         self.script_bbdd_01.config(state = tk.DISABLED)
1257
1258         self.script_bbdd_02.config(state = tk.NORMAL)
1259         self.script_bbdd_02.delete(1.0, tk.END)
1260         self.script_bbdd_02.config(state = tk.DISABLED)
1261
1262         self.def_control_versiones_update_subform_objetos(tipo_objeto_selecc, tipo_concepto_selecc, lista_control_versiones_segun_tipo_objeto_selecc)
1263
1264         self.strvar_proceso_merge_bbdd_origen.set("")
1265         self.strvar_proceso_merge_bbdd_lineas_origen_selecc_1.set("")
1266         self.strvar_proceso_merge_bbdd_lineas_origen_selecc_2.set("")
1267         self.strvar_proceso_merge_bbdd_lineas_destino_selecc.set("")
1268         self.strvar_combobox_merge_accion.set("")
```

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

4

Actualización de los scripts de BBDD_01 y BBDD_02 en la GUI al clicar sobre un item del sub-formulario

AL clicar en cualquier item del sub-formulario, mediante la rutina **def_control_versiones_update_subform_objetos_click_item** (módulo APP_1_GUI) se realizan 2 acciones:

- almacenar el tipo de objeto, el repositorio y el nombre de objeto en variables globales definidas en esta rutina para poder reutilizarlos cuando se realice el proceso de migrar líneas de código de un script a otro entre las bases de datos BBDD_01 y BBDD_02.

APP_1_GUI

```
1361     global global_tipo_objeto_subform
1362     global global_repositorio_subform
1363     global global_nombre_objeto_subform
1364
1365
1366     tipo_objeto_selecc = self.strvar_combobox_tipo_objeto.get()
1367
1368     item_selecc = self.subform_control_versiones_objetos.focus()
1369
1370     if item_selecc:
1371         item_values = self.subform_control_versiones_objetos.item(item_selecc, 'values')
1372
1373         global_tipo_objeto_subform = item_values[1]
1374         global_repositorio_subform = item_values[2]
1375         global_nombre_objeto_subform = item_values[3]
```

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

4

Actualización de los scripts de BBDD_01 y BBDD_02 en la GUI al clicar sobre un item del sub-formulario

- Recuperar los scripts de BBDD_01 y BBDD_02 del objeto seleccionado para poder actualizarlos en la GUI con los colores correspondientes en las líneas de código. **LEYENDA COLORES:** Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

Pueden darse 2 casos al clicar sobre el objeto en el sub-formulario:

Caso 1: es la 1era vez que se cliqua sobre el objeto y por lo tanto se ha de informar en la GUI los códigos originales que resultan del cálculo del proceso de control de versiones. Estos scripts tan solo aparecen con los cambios localizados de un script a otro, marcados del color siguiente **Cambios localizados**

Caso 2: se realizaron anteriormente, en la misma sesión, movimientos de código de un script a otro, se cliquo en otro objeto y se vuelve al clicar sobre el objeto. Aquí se han de tener en cuenta los cambios realizados anteriormente por el usuario de tal forma que aparezcan las líneas de código marcadas con estos colores **Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE**

Estos colores son **configurables**:

APP_1_GUI

```
27  #colores para los scripts en el control de versiones
28  bg_GUI_lineas_control_versiones_cambios_localizados = "#05FB27"
29  bg_GUI_lineas_control_versiones_cambios_agregados = "#FBCB05"
30  bg_GUI_lineas_control_versiones_cambios_eliminados = "#05FBF0"
```

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

4

Actualización de los scripts de BBDD_01 y BBDD_02 en la GUI al clicar sobre un item del sub-formulario

La localización de los scripts de BBDD_01 y BBDD_02 según que sean del caso 1 o 2 (originales o modificados respectivamente) se realiza mediante la función **func_control_versiones_dicc_proceso_merge_anteriores** (módulo APP_2_GENERAL). La actualización en la GUI de los scripts con los colores se hace mediante la rutina **def_control_versiones_rellenar_scripts_scrolledtext** (módulo APP_1_GUI).

APP_1_GUI

```
1378 #se recuperan los df codigos actuales de BBDD 01 y BBDD 02
1379 dicc_proceso_merge_anteriores = mod_gen.func_control_versiones_dicc_proceso_merge_anteriores(tipo_objeto_selecc, global_tipo_objeto_subform, global_repositorio_subform, global_nombre_objeto_subform)
1380
1381 df_codigo_actual_1 = dicc_proceso_merge_anteriores["DF_CODIGO_ACTUAL_1"]
1382 df_codigo_actual_2 = dicc_proceso_merge_anteriores["DF_CODIGO_ACTUAL_2"]
1383
1384 #se rellenan los scrolledtext
1385
1386 self.def_control_versiones_rellenar_scripts_scrolledtext(df_codigo_actual_1, df_codigo_actual_2)
```

La función **func_control_versiones_dicc_proceso_merge_anteriores** devuelve un diccionario con varias keys entre las cuales figuran DF_CODIGO_ACTUAL_1 y DF_CODIGO_ACTUAL_2:

- **Caso 1** (pagina anterior): los dataframes de código originales resultantes del proceso de cálculo del proceso de control de versiones.
- **Caso 2** (pagina anterior): los dataframes de código con los cambios realizados por el usuario.

```
df_codigo_actual_1 = dicc_proceso_merge_anteriores["DF_CODIGO_ACTUAL_1"]
df_codigo_actual_2 = dicc_proceso_merge_anteriores["DF_CODIGO_ACTUAL_2"]
```

La función **func_control_versiones_dicc_proceso_merge_anteriores** no se detalla en los anexos de este documento, con referirse directamente al código y con los comentarios ahí incorporados considero que es más que suficiente para entender su funcionamiento.

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

4

Actualización de los scripts de BBDD_01 y BBDD_02 en la GUI al clicar sobre un item del sub-formulario

El proceso de marcar las líneas de código de los scripts en estos colores Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE se hace mediante el método **tag_configure** de la librería Python Tkinter que permite según un valor tomado marcar las líneas de un color u otro.

APP_1_GUI

```
1753 self.script_migrar.tag_configure("CAMBIOS_LOCALIZADOS", background = bg_GUI_lineas_control_versiones_cambios_localizados)
1754 self.script_migrar.tag_configure("AGREGADO", background = bg_GUI_lineas_control_versiones_cambios_agregados)
1755 self.script_migrar.tag_configure("ELIMINADO", background = bg_GUI_lineas_control_versiones_cambios_eliminados)
```

En la rutina **def_control_versiones_update_subform_objetos_click_item** (módulo APP_1_GUI), los tags se aplican mediante estas sentencias:

APP_1_GUI

```
1421 self.script_bbdd_01.insert(tk.END, df_codigo_bbdd_1.iloc[ind, ind_col_codigo_con_num_linea] + '\n', df_codigo_bbdd_1.iloc[ind, ind_col_control_cambios])
1458 self.script_bbdd_02.insert(tk.END, df_codigo_bbdd_2.iloc[ind, ind_col_codigo_con_num_linea] + '\n', df_codigo_bbdd_2.iloc[ind, ind_col_control_cambios])
```

Los dataframes de código que se calculan mediante la función **func_df_control_versiones** (módulo APP_2_GENERAL) vienen calculados con varias columnas. La última columna (CONTROL_CAMBIOS_ACTUAL) es donde se informan de los cambios localizados y también es donde se actualizan los cambios de un script a otro y es la que sirve de referencia en el método **insert** del pantallazo anterior.

NUM LINEA	CODIGO	CODIGO_CON_NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t	\r	0001\t\r	None
1	0002\t	CREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r	0002\tCREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r	None
2	0003\t	\r	0003\t\r	None
3	0004\t	\t@Tabla varchar(255), @RutaCSV varchar(255), ...	0004\t\t@Tabla varchar(255), @RutaCSV varchar(...	None
4	0005\t	\r	0005\t\r	None
5	0006\t	AS\r	0006\tAS\r	None
6	0007\t	BEGIN\r	0007\tBEGIN\r	None
7	0008\t	\r	0008\t\r	ELIMINADO
8	0009\t	\r	0009\t\r	ELIMINADO
9	0010\t	declare @CadenaSQL1 nvarchar(4000), @CadenaSQL...	0010\tdeclare @CadenaSQL1 nvarchar(4000), @Cad...	CAMBIOS_LOCALIZADOS
10	0011\t	declare @CadenaSQL3 nvarchar(4000), @CadenaSQL...	0011\tdeclare @CadenaSQL3 nvarchar(4000), @Cad...	CAMBIOS_LOCALIZADOS

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

4

Actualización de los scripts de BBDD_01 y BBDD_02 en la GUI al clicar sobre un item del sub-formulario

La última columna (CONTROL_CAMBIOS_ACTUAL) es la que permite rellenar las líneas de código con colores

La función **func_control_versiones_dicc_proceso_merge_anteriores** devuelve un diccionario con varias keys entre las cuales figuran DF_CODIGO_ACTUAL_1 y DF_CODIGO_ACTUAL_2:

- **Caso 1** (pagina anterior): los dataframes de código originales resultantes del proceso de cálculo del proceso de control de versiones.
- **Caso 2** (pagina anterior): los dataframes de código con los cambios realizados por el usuario.

	NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t	\r	0001\t\r	None	None
1	0002\t	CREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r	0002\tCREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r	None	None
2	0003\t	\r	0003\t\r	None	None
3	0004\t	\t@Tabla varchar(255), @RutaCSV varchar(255), ...	0004\t\t@Tabla varchar(255), @RutaCSV varchar(...	None	None
4	0005\t	\r	0005\t\r	None	None
5	0006\t	AS\r	0006\tAS\r	None	None
6	0007\t	BEGIN\r	0007\tBEGIN\r	None	None
7	0008\t	\r	0008\t\r	None	ELIMINADO
8	0009\t	\r	0009\t\r	None	ELIMINADO
9	0010\t	declare @CadenaSQL1 nvarchar(4000), @CadenaSQL...	0010\tdeclare @CadenaSQL1 nvarchar(4000), @Cad...	CAMBIOS_LOCALIZADOS	AGREGADO
10	0011\t	declare @CadenaSQL3 nvarchar(4000), @CadenaSQL...	0011\tdeclare @CadenaSQL3 nvarchar(4000), @Cad...	CAMBIOS_LOCALIZADOS	CAMBIOS_LOCALIZADOS

La función **func_control_versiones_dicc_proceso_merge_anteriores** no se detalla en los **Anexos**, con referirse directamente a los comentarios insertados dentro del código de la función considero que es suficiente para entender su funcionamiento.

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

5

Selección de la BBDD y del tipo de acción a realizar

La lista de opciones del combobox de selección de la base de datos (BBDD_01 y BBDD_02) se calculan en el módulo APP_2_GENERAL

APP_2_GENERAL

```
408 lista_GUI_seleccion_tipo_concepto = [dicc_control_versiones_tipo_concepto[key] for key in dicc_control_versiones_tipo_concepto.keys()]
409
410 lista_GUI_seleccion_bbdd = [key for key in dicc_codigos_bbdd.keys()]
```

Se usan tal cual en el módulo APP_1_GUI:

APP_1_GUI

```
989 self.combobox_tipo_concepto = (gui_widgets(master, tipo_widget = "combobox", textvariable = self.strvar_combobox_tipo_concepto, width = 30, font = ("Calibri", 10, "bold"),
990                                     combobox_lista_valores = mod_gen.lista_GUI_seleccion_tipo_concepto, combobox_tipo_acceso_valores = "readonly", combobox_bind_mousewheel = True,
991                                     place = (140, 50)))

1128 self.combobox_proceso_merge_bbdd_selecc = (gui_widgets(self.frame_merge, tipo_widget = "combobox", textvariable = self.strvar_proceso_merge_bbdd_origen, width = 10, justify = tk.CENTER,
1129                                                         font = ("Calibri", 10, "bold"), combobox_lista_valores = mod_gen.lista_GUI_seleccion_bbdd, combobox_tipo_acceso_valores = "readonly",
1130                                                         combobox_bind_mousewheel = True, combobox_bind_comboboxselected_rutina = self.def_proceso_merge_combobox_bbdd_selecc,
1131                                                         place = (80, 70)))
```

El combobox de selección de BBDD tiene asociada la rutina **def_proceso_merge_combobox_bbdd_selecc** (módulo APP_1_GUI) que permite actualizar el combobox de tipo de acción según el valor seleccionado en el combobox de selección de BBDD:

- Si opta por el valor **BBDD_01**, la lista de opciones es la de **lista_GUI_proceso_merge_tipo_accion_bbdd_1** (módulo APP_1_GENERAL).
- Si opta por el valor **BBDD_02**, la lista de opciones es la de **lista_GUI_proceso_merge_tipo_accion_bbdd_2** (módulo APP_1_GENERAL).

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

6

Líneas Origen y líneas Destino

Los textbox de líneas de Origen (desde y Hasta) y Líneas Destino tiene asociada la rutina **def_GUI_proceso_merge_lineas_exit** (módulo APP_1_GUI) que impide al usuario informar caracteres no numéricos. En caso de hacerlo, borra el contenido del textbox y genera un warning avisando de ello.

7

Botón ACCIÓN

El botón **ACCIÓN** se asocia con la rutina **def_click_proceso_merge_boton_merge** (módulo APP_1_GUI).

Se crea la variable **lineas_origen** para usarla como parámetro en la rutina **def_proceso_merge_realizar_cambios** (módulo APP_2_GENERAL). Se pueden dar varios casos según los valores de Líneas Origen (Desde) y Líneas Origen (Hasta):

- Desde y Hasta están los 2 informados. Si Desde es menor o igual a Hasta no se realizan cambios en la GUI, sino se permutan los valores en la GUI.
- Solo Desde esta informado. Se informa Hasta con el valor de Desde en la GUI.
- Solo Hasta esta informado. Se informa Desde con el valor de Hasta en la GUI.

Se concatenan los valores de Desde y Hasta separados por un guión “-” y se encapsula en la variable **lineas_origen**.

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

7

Botón ACCIÓN

APP_1_GUI

```
1539     lineas_origen = ""
1540     if len(lineas_origen_1) != 0 and len(lineas_origen_2) != 0:
1541         #si lineas origen (hasta) es menor que lineas origen (desde) se permutan los valores
1542         lineas_origen = str(int(lineas_origen_1)) + "-" + str(int(lineas_origen_2)) if int(lineas_origen_1) <= int(lineas_origen_2) else str(int(lineas_origen_2)) + "-" + str(int(lineas_origen_1))
1543         self.strvar_proceso_merge_bbdd_lineas_origen_selecc_1.set(lineas_origen.split("-")[0])
1544         self.strvar_proceso_merge_bbdd_lineas_origen_selecc_2.set(lineas_origen.split("-")[1])
1545
1546     elif len(lineas_origen_1) != 0 and len(lineas_origen_2) == 0:
1547         #si solo se informa lineas origen (desde) se completa lineas origen (hasta) con el mismo valor
1548         lineas_origen = str(int(lineas_origen_1)) + "-" + str(int(lineas_origen_1))
1549         self.strvar_proceso_merge_bbdd_lineas_origen_selecc_2.set(lineas_origen_1)
1550
1551     elif len(lineas_origen_1) == 0 and len(lineas_origen_2) != 0:
1552         #si solo se informa lineas origen (hasta) se completa lineas origen (desde) con el mismo valor
1553         lineas_origen = str(int(lineas_origen_2)) + "-" + str(int(lineas_origen_2))
```

A posteriori, se localiza si hubo datos con cambios anteriores para el objeto seleccionado. Para localizar el objeto seleccionado se usan los valores de las variables globales que se definen e informan al clicar sobre un objeto en el sub-formulario con la rutina **def_control_versiones_update_subform_objetos_click_item**, comentada en páginas anteriores.

```
1361     global global_tipo_objeto_subform
1362     global global_repositorio_subform
1363     global global_nombre_objeto_subform
```

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

7

Botón ACCIÓN

Mediante la función **func_control_versiones_dicc_proceso_merge_anteriores** (módulo APP_2_GENERAL) donde se la pasa por parámetros estas variables globales, se busca el valor de la key LISTA_DICC_MERGE_HECHOS en el diccionario que genera la función. Si es una lista es que hay cambios anteriores, en caso contrario (devuelve None) es que no los hay.

Esto permite, para las acciones de **revertir cambios**, generar un warning si no hay cambios anteriores localizados para el objeto seleccionado.

APP_1_GUI

```
1558     dicc_proceso_merge_anteriores = mod_gen.func_control_versiones_dicc_proceso_merge_anteriores(tipo_objeto_selecc, global_tipo_objeto_subform, global_repositorio_subform, global_nombre_objeto_subform)
1559     lista_merge_hechos = dicc_proceso_merge_anteriores["LISTA_DICC_MERGE_HECHOS"]

1580     check_accion_revertir = "OK"
1581     if tipo_accion in mod_gen.lista_acciones_revertir:#acciones de reversion
1582
1583         #se localiza si ya hay merge hechos anteriores para saber si se puede revertir cambios
1584         if isinstance(lista_merge_hechos, list):
1585             check_accion_revertir = "OK"
1586         else:
1587             check_accion_revertir = "KO"
1588
1589
1590     if check_accion_revertir == "KO":
1591         messagebox.showerror(title = mod_gen.nombre_app, message = "REVERTIR CAMBIOS:\n\nNo se realizaron merge anteriores.")
```

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

7

Botón ACCIÓN

Se ejecutan los cambios realizados por el usuario mediante la rutina **def_proceso_merge_realizar_cambios** (módulo APP_2_GENERAL).

Mediante la función **func_control_versiones_dicc_proceso_merge_anteriores**, comentada en la página anterior, se recuperan los dataframes de códigos con los últimos cambios realizados por el usuario (si no los hay todavía se recuperan los códigos originales).

La actualización en la GUI se hace mediante la rutina **def_control_versiones_rellenar_scripts_scrolledtext** (módulo APP_1_GUI), mencionada anteriormente.

Se guardan también los cambios en el diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL) en la subkey_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES) asociada a la subkey_2 (TIPO_OBJETO) para el tipo de bases de datos (key_1) relacionada con el objeto seleccionado.

APP_1_GUI

```
1600 mod_gen.def_proceso_merge_realizar_cambios(tipo_accion, tipo_objeto_selecc, global_tipo_objeto_subform, global_repositorio_subform, global_nombre_objeto_subform, lineas_origen, lineas_de
1601
1602
1603 #se recuperan los df codigos actuales tras los cambios de BRDD 01 v BRDD 02
1604 dicc_proceso_merge_anteriores = mod_gen.func_control_versiones_dicc_proceso_merge_anteriores(tipo_objeto_selecc, global_tipo_objeto_subform, global_repositorio_subform, global_nombre_obj
1605 df_codigo_actual_1 = dicc_proceso_merge_anteriores["DF_CODIGO_ACTUAL_1"]
1606 df_codigo_actual_2 = dicc_proceso_merge_anteriores["DF_CODIGO_ACTUAL_2"]
1607
1608
1609
1610 #se rellenan los scrolledtext
1611 self.def_control_versiones_rellenar_scripts_scrolledtext(df_codigo_actual_1, df_codigo_actual_2)
```

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

7

Botón ACCIÓN

La rutina **def_proceso_merge_realizar_cambios** mencionada en la página anterior, que se detalla más en profundidad en el anexo **ANEXO_06**, permite en resumen realizar lo siguiente.

LISTA_DICC_OBJETOS_CONTROL_VERSIONES contiene una lista de diccionarios que se crea cuando se calcula el proceso de control de versiones en la GUI de inicio.

Hay un diccionario para cada uno de los objetos localizados con cambios en el código de una base de datos a otra. Cada diccionario contiene una key llamada LISTA_DICC_MERGE_HECHOS que asimismo es una lista de diccionarios con todos los cambios hechos por el usuario.

Cada diccionario que se agrega a esta lista LISTA_DICC_MERGE_HECHOS contiene el dataframe de código de BBDD_01 y BBDD_02 tras el ultimo cambio cogiendo como base el del anterior cambio de tal forma que cada nuevo diccionario sea un acumulativo de los cambios anteriores.

Este procedimiento permite localizar rápidamente los scripts modificados actuales (ultimo diccionario de la lista) y permite la acción de revertir el último cambio (eliminando de la lista el último diccionario).

CONTROL DE VERSIONES

Localizar cambios y realizar migraciones de scripts

8

Botón MERGE

Este botón permite acceder a la GUI de **MERGE** en base de datos física (clase **gui_ventana_merge_bbdd_fisicas**, módulo APP_1_GUI), si se localiza al menos un cambio realizado por el usuario.

El botón tiene asociada la rutina **def_click_proceso_merge_boton_cambios_en_bbdd** (módulo APP_1_GUI) y mediante la función **func_dicc_control_versiones_tipo_objeto_buscar_en_dicc** (módulo APP_2_GENERAL).

Esta función permite rastrear en todas las subkey_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES) del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL) que diccionarios integrantes de esta lista LISTA_DICC_OBJETOS_CONTROL_VERSIONES tienen en la key LISTA_DICC_MERGE_HECHOS una lista de diccionarios asociada y no por valor None.

Si no localiza ningún cambio genera un warning avisando de ello, en caso contrario ejecuta la clase **gui_ventana_merge_bbdd_fisicas**.

APP_1_GUI

```
1653 self.toplevel_merge_bbdd_fisicas = tk.Toplevel(self.master)
1654 self.toplevel_merge_bbdd_fisicas.transient(self.master)
1655 self.toplevel_merge_bbdd_fisicas.grab_set()
1656
1657 call_gui_ventana_merge_bbdd_fisicas = gui_ventana_merge_bbdd_fisicas(self.toplevel_merge_bbdd_fisicas)
```

CONTROL DE VERSIONES

Realizar MERGE en bases de datos físicas

MERGE EN BBDD FISICAS - MS ACCESS & SQL SERVER

Tipo Selección

TIPO BBDD	TIPO OBJETO	TIPO_REPOSITORIO	REPOSITORIO	OBJETO	ESTADO MIGRACIÓN
3					

SCRIPT A MIGRAR

3

CONTROL DE VERSIONES

Realizar MERGE en bases de datos físicas

Antes de proceder a comentar los pasos 1 a 5 mostrados en el pantallazo de la página anterior, comentar que nada más ejecutar la clase **gui_ventana_merge_bbdd_fisicas** (módulo APP_1_GUI) en el constructor de la misma se ejecutan unos calculos previos necesarios.

APP 1 GUI

```
1683 #se calculan las listas de objetos donde realizar merge en bbdd fisica asociadas a cada opcion del combobox
1684 mod_gen.def_merge_bbdd_fisica_lista_objetos()
1685
1686
1687 #se calculan los ajustes manuales a realizar en access
1688 if mod_gen.func_se_puede_ejecutar_proceso("CONTROL_VERSIONES", "MS_ACCESS") == "SI":
1689     mod_gen.def_merge_access_ajustes_manuales()
```

def merge_bbdd_fisica_lista_objetos

La rutina **def_merge_bbdd_fisica_lista_objetos** (módulo APP_2_GENERAL) permite crear la lista de diccionarios con todos los objetos donde el usuario ha realizado cambios en los scripts de BBDD_02 (que es por defecto la base de datos donde se realiza el merge en base de datos fisica).

La generación de esta lista de diccionarios (en adelante lista_dicc_1) se realiza mediante bucle sobre el diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL) donde se buscan las listas de diccionarios (en adelante lista_dicc_2) informadas en las subkeys_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES).

En cada diccionario de estas listas **lista_dicc_2** se mira en la key LISTA_DICC_MERGE_HECHOS si contiene otra lista de diccionarios (en adelante lista_dicc_3) . En caso de que si, dentro el último diccionario de **lista_dicc_3** se recupera el dataframe de código de la key DF_CODIGO_ACTUAL_2 (el de BBDD_02) y se agrega el objeto a **lista_dicc_1**.

CONTROL DE VERSIONES

Realizar MERGE en bases de datos físicas

def merge_bbdd fisica lista objetos

APP_2_GENERAL

```
1896 for tipo_bbdd in dicc_control_versiones_tipo_objeto.keys():
1897
1898     for tipo_objeto in dicc_control_versiones_tipo_objeto[tipo_bbdd]["TIPO_OBJETO"].keys():
1899
1900         if tipo_objeto != "TODOS":
1901
1902             if isinstance(dicc_control_versiones_tipo_objeto[tipo_bbdd]["TIPO_OBJETO"][tipo_objeto]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"], list):
1903
1904                 for dicc in dicc_control_versiones_tipo_objeto[tipo_bbdd]["TIPO_OBJETO"][tipo_objeto]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"]:
1905
1906                     if isinstance(dicc["LISTA_DICC_MERGE_HECHOS"], list):
1907
1908                         #dentro de la lista dicc["LISTA_DICC_MERGE_HECHOS"] se recupera el df asociado a la key DF_CODIGO_ACTUAL_2 del ultimo diccionario
1909                         #es DF_CODIGO_ACTUAL_2 pq por defecto la BBD_02 es donde se realiza el merge
1910                         df_codigo = dicc["LISTA_DICC_MERGE_HECHOS"][-1]["DF_CODIGO_ACTUAL_2"]
1911
1912                         dicc_temp = {
1913                             "TIPO_BBDD": dicc["TIPO_BBDD"]
1914                             , "CHECK_OBJETO": dicc["CHECK_OBJETO"]
1915                             , "TIPO_OBJETO_SUBFORM": dicc["TIPO_OBJETO_SUBFORM"]
1916                             , "TIPO_REPOSITORIO": dicc["TIPO_REPOSITORIO"]
1917                             , "REPOSITORIO": dicc["REPOSITORIO"]
1918                             , "NOMBRE_OBJETO": dicc["NOMBRE_OBJETO"]
1919                             , "ESTADO_MIGRACION": "Pendiente migrar"
1920                             , "DF_CODIGO": df_codigo
1921                         }
```

La **lista_dicc_1** se agrega, al final de la rutina, en la subkey_4 (LISTA_DICC_OBJETOS) asociada a las subkeys_2 y 3 (MERGE_BBDD_FISICA y OBJETOS_A_MIGRAR respectivamente) del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL)

APP_2_GENERAL

```
1938 dicc_control_versiones_tipo_objeto["MS_ACCESS"]["MERGE_BBDD_FISICA"]["OBJETOS_A_MIGRAR"]["LISTA_DICC_OBJETOS"] = lista_objetos_access
1939 dicc_control_versiones_tipo_objeto["SQL_SERVER"]["MERGE_BBDD_FISICA"]["OBJETOS_A_MIGRAR"]["LISTA_DICC_OBJETOS"] = lista_objetos_sql_server
```

CONTROL DE VERSIONES

Realizar MERGE en bases de datos físicas

def merge_access_ajustes_manuales

La rutina **def_merge_access_ajustes_manuales** (módulo APP_2_GENERAL) permite localizar todos los ajustes manuales que tendra que realizar el usuario porque no se pueden hacer mediante programación en el app.

La lista de posibles ajustes manuales es:

- Activar librerías DLL que no lo estan en BBDD_02 y si en BBDD_01
- Replicar de BBDD_01 a BBDD_02 los formularios creados en BBDD_01 que no estan en BBDD_02
- Replicar de BBDD_01 a BBDD_02 los Userform VBA creados en BBDD_01 que no estan en BBDD_02

La rutina, si localiza estos casos, almacena los datos en una lista de diccionarios. Es una lista con **1 solo diccionario** en vez de ser un diccionario, esto es para que el app se ejecute igual en la GUI de merge en base de datos fisica, tanto para los ajustes manuales como los objetos donde el usuario ha realizado cambios, al clicar sobre un elemento del subformulario (ver más adelante).

APP_2_GENERAL

```
1868 lista_dicc_ajustes_manuales = [  
1869     {"TIPO_BBDD": "MS_ACCESS"  
1870     , "TIPO_OBJETO_SUBFORM": "----"  
1871     , "TIPO_REPOSITORIO": "----"  
1872     , "REPOSITORIO": "----"  
1873     , "NOMBRE_OBJETO": "----"  
1874     , "ESTADO_MIGRACION": label_merge_access_bbdd_fisica_en_manual  
1875     , "DF_CODIGO": df_temp  
1876     }  
1877 ]
```

Esta lista de 1 solo diccionario se agrega, al final de la rutina, en la subkey_4 (LISTA_DICC_OBJETOS) asociada a las subkeys_2 y 3 (MERGE_BBDD_FISICA y AJUSTES_MANUALES respectivamente) del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL).

APP_2_GENERAL

```
1879 dicc_control_versiones_tipo_objeto["MS_ACCESS"]["MERGE_BBDD_FISICA"]["AJUSTES_MANUALES"]["LISTA_DICC_OBJETOS"] = lista_dicc_ajustes_manuales
```

CONTROL DE VERSIONES

Realizar MERGE en bases de datos físicas

1

Tipo Selección

Mediante la función **func_dicc_control_versiones_tipo_objeto_buscar_en_dicc** (módulo APP_2_GENERAL) se crea la lista de opciones para el combobox de tipo de selección en la GUI de merge en base de datos física.

APP_1_GUI

```
1692 #se calcula la lista para el combobox de seleccion
1693 lista_combobox_seleccion = mod_gen.func_dicc_control_versiones_tipo_objeto_buscar_en_dicc("LISTA_COMBOBOX_MERGE_BBDD_FISICAS")
```

La función localiza en el diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL) en que tipo de bases de datos (MS ACCESS y/o SQL SERVER) el usuario ha realizado cambios en la GUI de control de versiones.

Para ello la búsqueda se realiza en las subkey_2 (MERGE_BBDD_FISICA):

- **MS ACCESS:** en las subkey_3 (AJUSTES_MANUALES) se agrega a esta lista de opciones el valor de la subkey_4 asociada (COMBOBOX_GUI) si el valor de la subkey_4 asociada (DF_AJUSTES) devuelve un dataframe y no None.
- **MS ACCESS:** en las subkey_3 (OBJETOS_A_MIGRAR) se agrega a esta lista de opciones el valor de la subkey_4 asociada (COMBOBOX_GUI) si el valor de la subkey_4 asociada (LISTA_DICC_OBJETOS) devuelve una lista y no None.
- **SQL SERVER:** en las subkey_3 (OBJETOS_A_MIGRAR) se agrega a esta lista de opciones el valor de la subkey_4 asociada (COMBOBOX_GUI) si el valor de la subkey_4 asociada (LISTA_DICC_OBJETOS) devuelve una lista y no None.

CONTROL DE VERSIONES

Realizar MERGE en bases de datos físicas

2

Botón VER

Tras seleccionar el tipo de selección, al pulsar el botón **VER** se actualiza el sub-formulario que lista todos los objetos localizados con cambios por el usuario en los scripts de **BBDD_02**.

Los ítems que se actualizan en el sub-formulario provienen de los valores de la subkey_4 (AJUSTES_MANUALES y OBJETOS_A_MIGRAR) del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL).

El botón **VER** se asocia con la rutina **def_merge_bbdd_fisicas_click_boton_seleccion** (módulo APP_1_GUI) la cual al ejecutar la rutina **def_merge_bbdd_fisicas_update_subform_objetos** (módulo APP_1_GUI) actualiza el sub-formulario.

APP_1_GUI

```
1776 opcion_proceso_merge = self.strvar_combobox_tipo_seleccion.get()
1777 lista_temp = mod_gen.func_dicc_control_versiones_tipo_objeto_buscar_en_dicc("LISTA_DICC_OBJETOS_MERGE_BBDD_FISICAS", opcion_gui_merge_bbdd_fisica = opcion_proceso_merge)
1778
1779
1780 if len(opcion_proceso_merge) == 0:
1781     messagebox.showerror(title = mod_gen.nombre_app, message = "El tipo de selección es obligatorio.")
1782 else:
1783
1784     self.script_migrar.config(state = tk.NORMAL)
1785     self.script_migrar.delete(1.0, tk.END)
1786     self.script_migrar.config(state = tk.DISABLED)
1787
1788     if isinstance(lista_temp, list):
1789
1790         self.def_merge_bbdd_fisicas_update_subform_objetos(opcion_proceso_merge)
1791
1792         num_objetos = len(lista_temp)
1793         messagebox.showinfo(title = mod_gen.nombre_app, message = str(num_objetos) + " objetos localizados.")
1794
1795     else:
1796         messagebox.showinfo(title = mod_gen.nombre_app, message = "0 objetos localizados.")
```

CONTROL DE VERSIONES

Realizar MERGE en bases de datos físicas

3

Actualización del script a migrar en la GUI al cliquar sobre un item del sub-formulario

Al cliquar en cualquier item del sub-formulario, mediante la rutina **def_control_versiones_update_subform_objetos_click_item** (módulo APP_1_GUI) se actualiza el script del objeto con los últimos cambios realizados por el usuario manteniendo los colores de las líneas donde el usuario ha realizado cambios.

La actualización del script en la GUI se hace mediante la rutina **def_merge_bbdd_fisicas_rellenar_scripts_scrolledtext** (módulo APP_1_GUI) y funciona de la misma forma que la rutina **def_control_versiones_rellenar_scripts_scrolledtext** comentada anteriormente en este documento.

Lo que permite marcar los colores en el script es el método **tag_configure** en el scrolledtext del script a migrar y el método **insert** dentro de la rutina **def_merge_bbdd_fisicas_rellenar_scripts_scrolledtext**.

APP_1_GUI

```
1753 self.script_migrar.tag_configure("CAMBIOS_LOCALIZADOS", background = bg_GUI_lineas_control_versiones_cambios_localizados)
1754 self.script_migrar.tag_configure("AGREGADO", background = bg_GUI_lineas_control_versiones_cambios_agregados)
1755 self.script_migrar.tag_configure("ELIMINADO", background = bg_GUI_lineas_control_versiones_cambios_eliminados)

1948 self.script_migrar.insert(tk.END, df_script.iloc[ind, ind_col_codigo_con_num_linea] + '\n', df_script.iloc[ind, ind_col_control_cambios])
```

CONTROL DE VERSIONES

Realizar MERGE en bases de datos físicas

4

Ejecución de los MERGE en base de datos física

El botón **MERGE** permite:

- ejecutar los merge en base de datos física según el tipo de selección informado (no se realiza esta acción para los ajustes manuales, si los hubiese, en MS ACCESS).
- generar, en una ruta que indique el usuario, los logs en formatos .txt de los objetos migrados correctamente y el de los posibles errores.
- Documentar el proceso mediante ficheros en formato .txt

El botón se asocia con la rutina **def_merge_bbdd_fisicas_click_boton_realizar** (módulo APP_1_GUI) la cual se asocia a su vez con las rutinas **def_merge_bbdd_fisicas** y **def_generacion_logs** (módulo APP_2_GENERAL).

```
APP_1_GUI
2010 lista_dicc_objetos_migrar_bbdd_fisica = [dicc for dicc in lista_dicc_objetos_migrar_bbdd_fisica if dicc["TIPO_BBDD"] == tipo_bbdd]
2011 mod_gen.def_merge_bbdd_fisicas(tipo_bbdd, lista_dicc_objetos_migrar_bbdd_fisica, ruta_export)
2012
2013
2014 #se ejecutan los logs (OK + errores) y crea los ficheros (se ejecuta sobre BBDD_02 que es por defecto donde se hace el merge)
2015 mod_gen.def_generacion_logs("MERGE_BBDD_FISICA_LOGS_OK", tipo_bbdd, ruta_export, opcion_bbdd = "BBDD_02")
2016 mod_gen.def_generacion_logs("MERGE_BBDD_FISICA_LOGS_ERRORES", tipo_bbdd, ruta_export, opcion_bbdd = "BBDD_02")
```

La rutina **def_merge_bbdd_fisicas** se explica más en profundidad en el anexo **ANEXO_07**.

Proceso diagnostico dependencias objetos en MS ACCESS

Proceso diagnostico dependencias objetos en MS ACCESS

El diagnostico de dependencias de objetos en MS ACCESS se configura por defecto en **BBDD_01** en la GUI de inicio.

APP CONTROL VERSIONES EN MS ACCESS & SQL SERVER

PROCESO **Diagnostico BBDD Access** **CHECK**

MS ACCESS

BBDD_01

BBDD_02

SQL SERVER

Servidor 1 SQL_BBDD_01

Servidor 2 SQL_BBDD_02

El diagnostico se realizara solo sobre la bbdd MS Access que tengas configurada para BBDD_01.

IMPORTANTE: la BBDD MS Access debe tener el código VBA desbloqueado y no debe tener una macro AutoExec activada.

Al finalizar el proceso, se generara un excel donde:

TABLAS (LISTADO): se listaran todas las tablas locales, vinculos ODBC y vinculos otros.

TABLAS (DEPENDENCIAS): se listaran para cada tabla en que modulos/rutinas se usan.

No se comentan los pasos 1, 2 y 3 porque funcionan exactamente igual que en el control de versiones (ver anteriormente).

Proceso diagnostico dependencias objetos en MS ACCESS

4

Botón CHECK

Al igual que en el control de versiones, al pulsar en el botón **CHECK**, ejecuta la rutina **def_GUI_threads** (módulo APP_1_GUI) que permite iniciar un thread sobre la rutina **def_GUI_boton_check** (módulo APP_1_GUI).

Se ejecuta un thread para poder iniciar la variable global **global_proceso_en_ejecución** (módulo APP_2_GENERAL) con el valor de **SI**. Esto permite dentro de la rutina **def_GUI_threads** generar un warning cuando el usuario intenta ejecutar otro proceso con el botón **CHECK** cuando otro proceso está todavía en curso de ejecución.

```
APP_1_GUI
382     if mod_gen.global_proceso_en_ejecucion == "SI":
383         messagebox.showerror(title = mod_gen.nombre_app, message = "Espera a que acabe el proceso actualmente en ejecución.")
384
385     else:
386         Thread(target = self.def_GUI_boton_check).start()
```

En el caso de que no haya ningún proceso en curso, la rutina **def_GUI_boton_check** comprueba en 1er lugar si el proceso de diagnostico en MS ACCESS se puede calcular mediante la función **func_se_puede_ejecutar_proceso** (módulo APP_2_GENERAL).

En caso de que se pueda ejecutar el proceso, solicita al usuario una ruta donde guardar el output final en Excel y posteriormente pasa a ejecutar la rutina **def_calc_global** (módulo APP_2_GENERAL).

Proceso diagnostico dependencias objetos en MS ACCESS

4

Botón CHECK

APP_1 GUI

```
647     mensaje = "SELECCIONA DONDE QUIERES GUARDAR EL EXCEL DE DIAGNOSTICO DE DEPENDENCIAS Y LOS LOGS DE ERRORES (si los hubiese):"
648     ruta_destino_output = fd.askdirectory(parent = root, title = mensaje)
649
650     self.master.config(cursor = "wait")
651     mod_gen.def_calc_global(proceso_selecc_id, ruta_destino_output, ruta_destino_excel_diagnostico_access = ruta_destino_output)
652     self.master.config(cursor = "")
653
654
655     if len(mod_gen.global_msg_errores_proceso_access) != 0:
656         messagebox.showerror(mod_gen.nombre_app, message = mod_gen.global_msg_errores_proceso_access)
657     else:
658         messagebox.showinfo(mod_gen.nombre_app, message = "Proceso finalizado.")
```

La rutina **def_calc_global**, a diferencia del proceso de control de versiones, no requiere de la ejecución de un hilo de threads puesto que el proceso tan solo afecta a una sola base de datos (BBDD_01).

4

Botón CHECK

La rutina **def_calc_global** realiza las tareas siguientes:

- Vacía los valores de algunas de las subkeys de los diccionarios **dicc_codigos_bbdd**, **dicc_control_versiones_tipo_objeto** y **dicc_errores_procesos** (módulo APP_2_GENERAL).
- Cierra todas las aplicaciones MS ACCESS abiertas.
- Inicializa la variable global **global_proceso_en_ejecución** a SI.
- Ejecuta la rutina **def_proceso_access_1_import** (módulo APP_3_BACK_END_MS_ACCESS) para BBDD_01.

Si se han localizado errores, se para el proceso y se pasa a generar los logs de errores en formato .txt en la ruta que el usuario indico tras pulsar el botón **CHECK** mediante la rutina **def_generacion_logs** (módulo APP_2_GENERAL).

Si no se han localizado errores, se ejecuta la rutina **def_proceso_access_2_diagnostico** (módulo APP_3_BACK_END_MS_ACCESS). En caso de que esta rutina genere errores, al igual que en el párrafo anterior, se generan los logs de errores en formato .txt nuevamente mediante la rutina **def_generacion_logs**.

Si la rutina **def_proceso_access_2_diagnostico** no da error pasa a generar el output en Excel con el diagnostico de dependencias de objetos.

Proceso diagnostico dependencias objetos en MS ACCESS

4

Botón CHECK

APP_2_GENERAL

```
1161 if func_se_puede_ejecutar_proceso("DIAGNOSTICO", "MS_ACCESS") == "SI":
1162
1163     if func_procesos_errores(proceso_selecc_id, "MS_ACCESS", "IMPORT") == "SIN ERROR":
1164         mod_access.def_proceso_access_2_diagnostico(ruta_destino_excel_diagnostico_access)
1165
1166         if func_procesos_errores(proceso_selecc_id, "MS_ACCESS", "CALCULO") == "ERROR":
1167
1168             mensaje1 = "Diagnostico MS ACCESS\n\nEl proceso de importacion de los datos se ha realizado correctamente"
1169             mensaje2 = "pero el proceso de calculo del control de versiones ha generado errores por lo que el proceso se ha cancelado (consulta los logs).\n\n"
1170             global_msg_errores_proceso_access = mensaje1 + mensaje2
1171
1172             def_generacion_logs("PROCESOS_LOGS_ERRORES_CALCULO", "MS_ACCESS", ruta_destino_fichero_logs, proceso_id = proceso_selecc_id)
1173
1174         else:
1175             global_msg_errores_proceso_access = "Diagnostico MS ACCESS\n\nSe han localizado errores en el proceso de importacion de los datos por lo que el proceso se
1176
1177             def_generacion_logs("PROCESOS_LOGS_ERRORES_IMPORT_BBDD", "MS_ACCESS", ruta_destino_fichero_logs, proceso_id = proceso_selecc_id, opcion_bbdd = "BBDD_01")
```

Las rutinas **def_proceso_access_1_import** y **def_proceso_access_2_diagnostico** se detallan más en profundidad en los anexos **ANEXO_01** y **ANEXO_02** respectivamente.

Proceso diagnostico dependencias objetos en SQL SERVER

Proceso diagnostico dependencias objetos en SQL SERVER

El diagnostico de dependencias de objetos en SQL SERVER se configura por defecto en **el Servidor 1** en la GUI de inicio.

No se comentan los pasos 1 y 2 porque funcionan exactamente igual que en el control de versiones (ver anteriormente).

Proceso diagnostico dependencias objetos en SQL SERVER

3

Botón CHECK

Tras seleccionar el proceso y el servidor SQL SERVER (servidor 1), el botón **CHECK** ejecuta la clase propia **gui_diagnostico_bbdd_sql_server** (módulo APP_1_GUI).

APP_1_GUI

```
662 elif proceso_selecc_id == "PROCESO_03":
663     #diagnostico dependencias sql server
664
665     if check_diagnostico_sql_server == "NO":
666         messagebox.showerror(mod_gen.nombre_app, message = "Tienes que seleccionar el servidor SQL SERVER de la BBDD_01.")
667
668     else:
669         mensaje = "Se abra una nueva ventana donde podras configurar el proceso.\n\nDeseas continuar?"
670         msg = messagebox.askokcancel(mod_gen.nombre_app, message = mensaje)
671
672         if msg == True:
673
674             self.toplevel_gui_diagnostico_bbdd_sql_server = tk.Toplevel(self.master)
675             self.toplevel_gui_diagnostico_bbdd_sql_server.transient(self.master)
676             self.toplevel_gui_diagnostico_bbdd_sql_server.grab_set()
677
678             call_gui_diagnostico_bbdd_sql_server = gui_diagnostico_bbdd_sql_server(self.toplevel_gui_diagnostico_bbdd_sql_server, servidor_sql_server = servidor_sql_server_1)
```


Proceso diagnostico dependencias objetos en SQL SERVER



Proceso diagnostico dependencias objetos en SQL SERVER

1

Actualización de las bases de datos en el listbox selección multiple

En el constructor de la clase **gui_diagnostico_bbdd_sql_server** se integra la rutina **def_sql_server_servidor_permisos** (módulo APP_3_BACK_END_SQL_SERVER) que se comento anteriormente en el apartado del cálculo del proceso de control de versiones.

Esta rutina tiene integrada una variable global **global_servidor_bbdd_permisos_acceso_código** que permite listar, para el servidor seleccionado, todas las bases de datos SQL SERVER donde el usuario tiene permisos de acceso al código T-SQL de los objetos.

Esta variable global es la que permite crear las opciones de bases de datos en el listbox de selección múltiple en la GUI.

APP_1_GUI

```
829 #se crea la lista de bbdd asociadas al servidor
830 mod_sql_server.def_sql_server_servidor_permisos("BBDD_01", servidor_sql_server)
831 mod_sql_server.global_servidor_bbdd_permisos_acceso_codigo
832
833
834
835 #se crean las opciones del listbox con las bbdd (con permisos de acceso al codigo) d
836 if isinstance(mod_sql_server.global_servidor_bbdd_permisos_acceso_codigo, list):
837
838     self.listbox_sql_server_diagnostico.delete(0, tk.END)
839     for value in mod_sql_server.global_servidor_bbdd_permisos_acceso_codigo:
840         self.listbox_sql_server_diagnostico.insert(tk.END, value)
```

Proceso diagnostico dependencias objetos en SQL SERVER

2

Seleccionar todo / des-seleccionar todo

El botón **All / None** tiene asociada la rutina **def_GUI_sql_server_diagnostico_listbox_all_none** (módulo APP_1_GUI).

3

Selección del proceso

La lista de opciones del combobox de selección del proceso provienen de la variable global **lista_GUI_diagnostico_combobox_sql_server** (módulo APP_3_BACK_END_SQL_SERVER).

4

Botón CHECK

Al pulsar en el botón **CHECK**, ejecuta la rutina **def_GUI_sql_server_diagnostico_threads** (módulo APP_1_GUI) que permite iniciar un thread sobre la rutina **def_GUI_sql_server_diagnostico_boton_check** (módulo APP_1_GUI).

Se ejecuta un thread para poder iniciar la variable global **global_proceso_en_ejecucion** (módulo APP_2_GENERAL) con el valor de **SI**. Esto permite dentro de la rutina **def_GUI_sql_server_diagnostico_threads** generar un warning cuando el usuario intenta ejecutar otro proceso con el botón **CHECK** cuando otro proceso está todavía en curso de ejecución.

En el caso de que no haya ningún proceso en curso, la rutina **def_GUI_sql_server_diagnostico_boton_check** pasa a ejecutar la rutina **def_calc_global** (módulo APP_2_GENERAL) para el proceso seleccionado en el paso 3.

Proceso diagnostico dependencias objetos en SQL SERVER

4

Botón CHECK

APP 1 GUI

```
855     if mod_gen.global_proceso_en_ejecucion == "SI":
856         messagebox.showerror(title = mod_gen.nombre_app, message = "Espera a que acabe el proceso actualmente en ejecución.")
857
858     else:
859         Thread(target = self.def_GUI_sql_server_diagnostico_boton_check, args = (servidor_sql_server,)).start()
```

```
892     #opcion diagnostico
893     if opcion_diagnostico_sql_server == mod_sql_server.lista_GUI_diagnostico_combobox_sql_server[0]:
894         mensaje_1 = "SELECCIONA DONDE QUIERES GUARDAR EL EXCEL DE DIAGNOSTICO DE DEPENDENCIAS Y LOS LOGS DE ERRORES (si los hubiese):"
895         mensaje_2 = "Diagnostico de dependencias SQL Server descargado en Excel en la ruta indicada."
896
897     #opcion descarga codigo
898     elif opcion_diagnostico_sql_server == mod_sql_server.lista_GUI_diagnostico_combobox_sql_server[1]:
899         mensaje_1 = "SELECCIONA DONDE QUIERES GUARDAR LOS CODIGOS T-SQL DE LOS OBJETOS Y LOS LOGS DE ERRORES (si los hubiese):"
900         mensaje_2 = "Códigos T-SQL de los objetos descargados en ficheros .sql en la ruta indicada."
901
902     ruta_destino_diagnostico_sql_server = fd.askdirectory(parent = root, title = mensaje_1)
903
904     self.master.config(cursor = "wait")
905     mod_gen.def_calc_global("PROCESO_03", ruta_destino_diagnostico_sql_server
906                             , opcion_diagnostico_sql_server = opcion_diagnostico_sql_server
907                             , ruta_destino_diagnostico_sql_server = ruta_destino_diagnostico_sql_server)
908
909     self.master.config(cursor = "")
```

Limitaciones del app

Limitaciones del app

El app tiene ciertas limitaciones que se enumeran a continuación:

- 1 El app se desarrollo en un PC con sistema operativo **Windows 10**. No se ha probado con otros sistemas operativos por lo que es posible que haya fallos.
- 2 Para **MS ACCESS**, no se han incluido los objetos de clase tanto en el control de versiones como en el diagnostico de dependencias. Para **SQL SERVER**, los tipos de objetos se han limitado a las tablas, views, stored procedures y funciones (Scalar Function, Inline Table-Valued Function, Table-Valued Function - Multi-Statement y Aggregate Function). En ambos entornos, para agregar más tipos de objetos el modus operandi seria:
 - agregarlos en la subkey_3 asociada a la key_1 (MS ACCESS o SQL SERVER) y a la subkey_2 (TIPO_OBJETO) del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL).

→ **MS ACCESS**: adaptar los valores de las subkeys_4 (COMBOBOX_CONTROL_VERSIONES y TIPO_OBJETO_SUBFORM).

```
282 , "TABLA_LOCAL":  
283     , "COMBOBOX_CONTROL_VERSIONES": "MS ACCESS: tablas locales"  
284     , "TIPO_OBJETO_SUBFORM": "Tablas Locales"  
285     , "LISTA_DICC_OBJETOS_CONTROL_VERSIONES": None  
286 }
```

Es IMPORTANTE que la subkey_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES) se establezca a None.

Limitaciones del app

2

→ **SQL SERVER:** adaptar los valores de las subkeys_4 (COMBOBOX_CONTROL_VERSIONES, TIPO_OBJETO_SUBFORM y LISTA_OBJECT_ID).

```
347 , "VIEWS":  
348 {  
349   "COMBOBOX_CONTROL_VERSIONES": "SQL SERVER: views"  
350   , "TIPO_OBJETO_SUBFORM": "Views"  
351   , "LISTA_DICC_OBJETOS_CONTROL_VERSIONES": None  
352   , "LISTA_OBJECT_ID": [{"VIEW", "V"}]  
353 }
```

Es IMPORTANTE que la subkey_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES) se establezca a None.

La lista almacenada en la subkey_4 (LISTA_OBJECT_ID) es lista de lista. Cada una de sus sublistas ha de contener el tipo de objeto en nomenclatura SQL SERVER (item 1) y el ID de objeto (item 2).

```
IF OBJECT_ID('dbo.V_PRUEBA' + 'V') IS NOT NULL  
BEGIN  
  DROP VIEW dbo.V_PRUEBA  
END
```

Esta lista de lista se usa en el control de versiones cuando se realiza el merge en base de datos física para eliminar los objetos que se van a actualizar, por si ya existen.

La sentencia DROP VIEW IF EXISTS funciona solo para versiones recientes de SQL SERVER por lo que la eliminación se hace con esta sentencia antigua, compatible con todas las versiones.

Limitaciones del app

- 2 • En caso de que los objetos sean de **SQL SERVER**, hay que adaptar la query **query_objetos_sql_server** (módulo APP_3_BACK_END_SQL_SERVER) agregando los nuevos objetos en la clausula CASE WHEN y la clausula WHERE con EXACTAMENTE los mismos nombres que en el diccionario **dicc_control_versiones_tipo_objeto**.

```
81 query_objetos_sql_server = """SELECT
82     'REEMPLAZA_BBDD' AS BBDD
83     , T2.name AS REPOSITORIO
84     , CASE WHEN T1.type = 'U' THEN 'TABLAS'
85           WHEN T1.type = 'V' THEN 'VIEWS'
86           WHEN T1.type = 'P' THEN 'STORED_PROCEDURES'
87           WHEN T1.type IN ('FN', 'IF', 'TF', 'AF') THEN 'FUNCIONES'
88     END AS TIPO OBJETO
89     , T1.name AS NOMBRE_OBJETO
90     , 'REEMPLAZA_BBDD' + '.' + T2.name + '.' + T1.name AS OBJETO_LONG
91     , T2.name + '.' + T1.name AS OBJETO_SHORT
92
93     FROM REEMPLAZA_BBDD.sys.objects T1
94     LEFT JOIN REEMPLAZA_BBDD.sys.schemas T2 on T1.schema_id = T2.schema_id
95
96     WHERE UPPER(T1.type) IN ('U', 'V', 'P', 'FN', 'IF', 'TF', 'AF')
97
98     ORDER BY T1.type, T2.name, T1.name
99 """
```

dicc_control_versiones_tipo_objeto

```
,"SQL_SERVER":
  ("TIPO_OBJETO":
    {"TODOS":
      , "TABLAS":
      , "VIEWS":
      , "STORED_PROCEDURES":
      , "FUNCIONES":
    }
  )
```

- Seleccionar “TABLA_LOCAL” (para MS_ACCESS) o “VIEWS” (para SQL SERVER), en ambos casos seleccionar también las comillas dobles de apertura y cierre) y **pulsar la hotkey Ctrl + F**. A partir de ahí hay que rastrear el código en los distintos módulos donde se usa para poder agregar líneas de código a los nuevos tipos de objeto. No se trata de hacer un simple copy/paste, en algunas partes de código, según el tipo de objeto, los cálculos se hacen de una forma u otra.

Limitaciones del app

3

El app usa 2 funciones para generar las sentencias CREATE TABLE asociadas al tipo de objeto tablas tanto para **MS ACCESS** como para **SQL SERVER**. La sentencia que se genera no incluye las primary keys (PK) ni los índices ni los constraint. Para adaptarlo habría que ir directamente a estas funciones y agregar las líneas de código convenientes:

- **func_diagnostico_access_tablas_df_create_table**
- **func_sql_server_df_codigo_objeto** (opción DF_CODIGO_ORIGINAL y parámetro kwargs tipo_objeto igual a TABLAS)

4

El diagnostico de dependencias en **MS ACCESS**, para la búsqueda de tablas o vínculos en el código VBA de las rutinas / funciones, hay un chequeo manual que debe realizar el usuario (ver en el presente documento cuando se menciona este check manual en los Anexos **ANEXO_01**, **ANEXO_09** y **ANEXO_11**). Cuando la tabla se usa dentro del valor de una variable local y luego esta variable local se usa en una sentencia tipo SQL o de manipulación de tablas, el app no puede validar que se trata de una dependencia. Es el usuario quien, desde el output final en fichero Excel, debe validar manualmente si se tratan de dependencias o no.

Si usa una tabla dentro del valor de una variable pública VBA y luego esta variable pública se usa en rutinas VBA en sentencias tipo SQL o de manipulación de tablas, **el app no contempla estos casos.**

5

El cálculo del diagnostico de dependencias para **SQL SERVER** se extralimita a objetos de bases de datos de un mismo servidor. No incluye las dependencias con objetos de servidores linkados (se incluye este punto en los futuros desarrollos del apartado 10 del presente documento).

6

En el diagnostico de dependencias, tanto en MS ACCESS como en SQL SERVER, otra limitación es cuando **una tabla tiene campos con exactamente el mismo nombre que una rutina o función**. El diagnostico podría dar resultados erróneos para estos casos (esta previsto incorporarse en los futuros desarrollos, ver apartado 10 del presente documento).

Limitaciones del app

7 En el control de versiones en **MS ACCESS**, hay ajustes manuales que debe realizar el usuario porque el app no lo automatiza (se comentan brevemente en las páginas anteriores cuando se comenta la rutina **def_merge_access_ajustes_manuales** del presente documento) y consisten en:

- Si la **BBDD_01** tiene librerías **DLL** que **BBDD_02** no tiene, el usuario ha de activarlas en el editor VBA de **BBDD_02**.
- Si en **BBDD_01** se han creado módulos asociados a Formularios o **UserForm VBA** y el usuario ha realizado migraciones hacia la **BBDD_02** de algunas de sus rutinas o funciones, **el usuario tendrá que crear estos formularios en BBDD_01** antes de realizar el merge en base de datos física.

8 Otra limitación es cuando **el usuario en BBDD_01 realiza cambios en los formularios**, agregando textbox, combobox, cambiando colores etc etc. El app no realiza estos ajustes, es al usuario realizarlos manualmente en **BBDD_02**.

9 En el control de versiones tanto para **MS ACCESS** como para **SQL SERVER**, si el usuario quita en un script de **BBDD_02** las sentencias de inicio o fin de declaración de rutinas o funciones o se quita la línea donde aparece el **CREATE TABLE** para las tablas, el merge en base de datos física generara errores en los logs. El app no tiene un mecanismo de warning que avise de ello.

10 En el control de versiones al realizar el merge en base de datos física para **SQL SERVER** se comenta en el **ANEXO_07** que los objetos se migran por un orden específico según la tipología de objetos (tablas → funciones → views → stored procedures) para evitar errores en los commit cuando por ejemplo una view que se migra depende de una función que también se migra. Si la función no se migra antes, el commit de la view daría error.

La limitación es cuando para un mismo tipo de objeto, por ejemplo las views, hay objetos a migrar que dependen los unos de los otros. El app, en la versión 1.0, no localiza cuales se han de crear primero para evitar errores en los commit.

Futuros desarrollos

Futuros desarrollos

Los (posibles) futuros desarrollos que tengo previsto implementar son



Desarrollos previstos



Ideas de desarrollos que en su debido momento valorare si implementar

1

El app usa diccionarios de configuración:

MODULO	DICCIONARIO
APP_2_GENERAL	<u>dicc_procesos</u>
	<u>dicc_codigos_bbdd</u>
	<u>dicc_control_versiones_tipo_objeto</u>
	<u>dicc_control_versiones_tipo_concepto</u>
	<u>dicc_errores_procesos</u>
APP_3_BACK_END_MS_ACCESS	<u>dicc_objetos_vba</u>
	<u>dicc_diagnostico_tablas_access_donde_buscar</u>

La idea es integrar los datos que componen estos diccionarios en tablas de una base de datos **SQLite** que serian configurables por el usuario.

En la GUI de inicio habría un botón que se conecta a esta base de datos SQLite y a partir de los datos de las tablas reconstruye los diccionarios mencionados. Esto es para que el usuario pueda tunear el app a su gusto, porque también habría en la GUI:

- un botón que se conecta a esta base de datos SQLite y descarga las tablas en un fichero Excel para que el usuario realice cambios
- un botón para importar el fichero Excel con los cambios hechos por el usuario a la base de datos SQLite

Con este desarrollo habría más **portabilidad** para el app. Tal como esta ahora, cada cambio en los diccionarios obliga a volver a compilar el app en archivo .exe. Con este desarrollo ya no seria necesario.

Futuros desarrollos

2

En el diagnostico de dependencias tanto para MS ACCESS como para SQL SERVER, agregar un rastreador de dependencias. Seria retomar lo hecho ya para el diagnostico de dependencias para presentarlo de una forma diferente.

Partiendo del script de un objeto seria en este mismo script localizar que otros objetos se usan (se marcaria como nivel 1), para cada objeto localizado en el script mencionado seria ir a sus scripts respectivos y repetir la operación (pasaría a ser el nivel 2) y así sucesivamente. Se añadiría en todos los casos el número y la línea de código para poder también poder ver los parámetros que toman las rutinas / funciones anidadas las unas dentro de las otras.

Este mapa-mundi de un objeto seria particularmente útil cuando tienes delante un **código “espagueti”** o para montar un **árbol genealógico** de un objeto.

La idea inicial es incorporar una nueva pestaña en el output en fichero Excel que permite descargar la V1 del app aunque me planteo montar también algo más visual dentro de la misma GUI. Esta por ver.

3

En el diagnostico de dependencias, tanto en MS ACCESS como en SQL SERVER, cuando **una tabla tiene campos con exactamente el mismo nombre que una rutina o función**, el diagnostico puede dar resultados erróneos para estos casos. La idea es incorporar en el fichero Excel una hoja adicional que funcionaria de forma similar la hoja “TABLAS (CHECK MANUAL)” en el diagnostico en MS ACCESS, aquí aplicaría también para SQL SERVER.

4

Localizar en SQL SERVER los stored procedures donde se generan las tablas

Se incluiría en el diagnostico de dependencias una hoja Excel que listaría para todas las tablas los stored procedures donde se realizan DROP TABLE, DELETE FROM, TRUNCATE TABLE, SELECT INTO FROM, INSERT INTO de estas mismas tablas. Se informaría también del número y línea de código.

Futuros desarrollos

5

Incluir en el diagnostico de dependencias en SQL las dependencias con objetos de servidores vinculados

El cálculo del diagnostico de dependencias para **SQL SERVER** se extralimita ahora mismo a objetos de bases de datos de un mismo servidor.

6

Mejoras tiempos de ejecución de los procesos

El app se desarrollo usando a modo de prueba bases de datos personales “pequeñas” tanto en MS ACCESS como en SQL SERVER por lo que los tiempos de ejecución son relativamente aceptables pero con bases de datos más grandes es una incógnita.

Hay unos cuantos bloques de código donde itero con bucles por índices en dataframes en vez de vectorizar el cálculo (métodos apply lambda). La adaptación esta prevista para el futuro. Tengo pendiente investigar una librería que tengo en la recamara desde hace tiempo y es **swifter**, igual por ahí puedo mejorar los tiempos de ejecución.

Futuros desarrollos

1

Usando la base de datos **SQLite** mencionada, crear un **sistema multi-usuario** (la base de datos SQLite debería estar en una unidad compartida) para que varios usuarios puedan consultar los cambios realizados por los demás. A nivel de merge en base de datos física, con SQL SERVER no debería haber problemas pero con MS ACCESS debo darle una vuelta más porque no es posible usar de forma compartida el mismo Access.

También crear un **histórico de cambios para el control de versiones** tanto en MS ACCESS y SQL SERVER con el fin de poder realizar consultas de cambios pasados.

2

Usando la base de datos **SQLite** mencionada, dar la posibilidad al usuario de poner el app en el idioma que quiera creando ahí una tabla por conceptos usados en la GUI del app y poner tantas columnas como de idiomas quiere el usuario (afectaría también a los diversos messagebox que aparecen en la GUI).

En el desarrollo 1, se comenta que el usuario tendría la posibilidad de descargar a un fichero Excel los datos almacenados en los diccionarios de configuración para luego poder importar este Excel a la base de datos SQLite. En el presente futuro desarrollo, se descargarían también los datos de la tabla de idiomas.

En la GUI de inicio habría un combobox donde la lista de opciones sería el header de cada columna de idioma de la tabla de idiomas en SQLite y otro botón para aplicar cambios. A nivel de código Python sería simplemente crear una función con los parámetros idioma y tipo_concepto y hacer llamadas a esta misma función en todos los messagebox y labels de la GUI.

Futuros desarrollos

3

Ampliar el diagnostico de dependencias

- otros proveedores SQL sean Oracle, MySQL, postgre-SQL etc etc.
- otras suites de MS Office (especialmente Excel).

En el caso de otros proveedores SQL, para quienes estén interesad@s en implementarlo sin esperar a que lo realice yo, el modus operandi tendría que ser este.

Instalar (pip install) e importar la librería que toque (en SQL SERVER, la que uso es **pyodbc** pero al incorporar otros proveedores SQL lo idóneo sería usar **sqlalchemy**).

```
3 import pyodbc
```

A modo recordatorio de lo que se comenta en los anexos del presente documento, lo que hace el app de forma muy resumida es recuperar los códigos T-SQL de cada objeto de las bases de datos seleccionadas (se obtienen mediante las tablas de sistema) y según el tipo de objeto (tablas por una parte y resto de objetos por otra), mediante la función **func_sql_server_df_codigo_objeto**, se recuperan los códigos de los objetos que se encapsulan en dataframes (librería pandas de Python):

TIPO OBJETO	DESCRIPCIÓN
Tablas	Para cada objeto del tipo tabla, mediante las tablas de sistema configuradas en la query query_campos_tablas_sql_server (módulo APP_3_BACK_END_SQL_SERVER), se recuperan los campos y tipos de datos asociados y al final del proceso se crea la sentencia CREATE TABLE que se almacena en un dataframe.
Resto objetos	Para cada objeto que no sea del tipo tabla, mediante la sentencia sp_helptext configurada en la query query_helptext_sql_server (módulo APP_3_BACK_END_SQL_SERVER) se almacena el código del objeto en una tabla temporal (las que vienen precedidas de #) que se encapsula en un dataframe.

Futuros desarrollos

3

Ampliar el diagnostico de dependencias

El listado de objetos Python (módulo APP_3_BACK_END_SQL_SERVER) que habría que adaptar es:

OBJETO PYTHON	DESCRIPCIÓN
driver_sql_server	Variables globales para la connecting string
conn_str_sql_server_windows_authentication	Variables globales para la connecting string
conn_str_sql_server_login_password_authentication	Variables globales para la connecting string
conn_str_sql_server	Variables globales para la connecting string
indicador_sql_server_comentario_linea	Variables globales para localizar comentarios en código SQL
indicador_sql_server_comentario_bloque_linea_ini	Variables globales para localizar comentarios en código SQL
indicador_sql_server_comentario_bloque_linea_fin	Variables globales para localizar comentarios en código SQL
query_sql_server_bbdd_permisos	Query para localizar si el usuario tiene permisos para acceder a los códigos SQL de los objetos
query_lista_bbdd_sql_server	Query que permite listar todas las bases de datos de un servidor
query_objetos_sql_server	Query que permite listar los objetos según sus tipologías de una base de datos
query_objetos_sql_server_parametros	Query que permite para cada objeto dentro de una base de datos listar los parámetros que requiere en caso de que los hubiese
query_campos_tablas_sql_server	Query que permite acceder a los campos con sus respectivos tipos de datos para un objeto de base de datos
query_helptext_sql_server	Query que permite almacenar en una tabla temporal SQL (las que vienen precedidas de # en T-SQL) el código SQL de un objeto que no es de tipo tabla
query_drop_if_exists_objeto	Query que permite eliminar un objeto si existe o no en una base de datos SQL
func_sql_server_tipo_conexion_servidor	Función que permite probar el tipo de conexión que tiene el usuario en un servidor SQL (windows authentication o con login / password)
func_sql_server_df_codigo_objeto	Función que permite reconstruir el código SQL de un objeto sea tabla o sea objeto con un script asociado

Con estos cambios adaptados, el resto de los cálculos hasta el output final en fichero Excel sería el mismo para todos los proveedores SQL que el que está actualmente desarrollado en el app.

Futuros desarrollos

4

Ampliar el control de versiones:

- otros proveedores SQL sean Oracle, MySQL, postgre-SQL etc etc.
- otras suites de MS Office (especialmente Excel).

Ampliar también el control de versiones a **otros lenguajes de programación** sean Python u otros. La idea es aquí hacerlo tuneable por el usuario. Funcionaria de la forma siguiente usando la base de datos SQLite mencionada en los 4 anteriores futuros desarrollos.

Desde un fichero Excel, cada hoja seria un lenguaje de programación y se configuraría ahí algo similar al diccionario **dicc_objetos_vba** (módulo APP_3_BACK_END_MS_ACCESS) a la subkey_3 asociada a la key_1 RUTINAS.

```
87      , "RUTINAS":
88      {
89          "RUTINA_PRIVADA":
90          {
91              "TIPO_RUTINA": "RUTINA"
92              , "TIPO_DECLARACION": "PRIVADA"
93              , "LISTA_DECLARACION_INICIO": ["Private Sub"]
94              , "LISTA_DECLARACION_FIN": ["End Sub"]
95          }
96          , "RUTINA_PUBLICA":
97          {
98              "TIPO_RUTINA": "RUTINA"
99              , "TIPO_DECLARACION": "PUBLICA"
100              , "LISTA_DECLARACION_INICIO": ["Sub", "Public Sub", "Global Sub"]
101              , "LISTA_DECLARACION_FIN": ["End Sub"]
102          }
103      }
104  }
```

Esto permitiría localizar en un módulo las rutinas / funciones y su código asociado. Los módulos para cada proyecto a comparar deberían estar en subcarpetas separadas que el usuario indicaría mediante un filedialog en la GUI.

Para seleccionar el lenguaje de programación, en la GUI habría un combobox cuya lista de opciones serian los nombres de las hojas del Excel mencionado.

Futuros desarrollos

5 Limpiador de parámetros y variables locales OBSOLETOS de rutinas / funciones (MS ACCESS, SQL SERVER)

Para cada rutina y función seria localizar que parámetros no se usan, lo mismo con las variables locales y darle la posibilidad al usuario de realizar la limpieza automáticamente en el script.

Seria interesante sobre todo para código VBA. El resto de los lenguajes de programación que se pueden enlazar a Visual Studio Code (la interfaz de programación que uso) esto se ve enseguida.

6 Controlar el flujo de ejecución del app y de los errores usando la librería logging

Tanto los flujos de ejecución como los errores se almacenarían en la base de datos **SQLite** comentada para otros futuros desarrollos con la posibilidad también de acceder a un histórico.

Para los errores, la idea aquí es cuando surge alguno que el usuario pueda informar desde el app (y que se actualizaría en la base de datos SQLite) un mensaje propio que explique a que se debe el error y poder mostrarlo en los logs de errores que se generan en ficheros .txt en caso de que se produjera en un futuro uso del app.

Daria más indicaciones que el error que sale por defecto que muchas veces no da muchas pistas para los usuarios que no conocen el código del app.

Futuros desarrollos

11

Controlar el flujo de ejecución del app y de los errores usando la librería logging

Log Errores ACTUAL

```
MS_ACCESS: ERRORES EN BBDD_01: C:/Users/user/Desktop/APP_CONTROL_VERSIONES/PRUEBA_APP/PRUEBA_CONTROL_VERSIONES_1.accdb
-----
MODULO PYTHON: APP_3_BACK_END_MS_ACCESS.py
RUTINA PYTHON: def_proceso_access_1_import
LINEA ERROR: 1767
ERROR: (-2147352567, 'Ocurrió una excepción.', (0, None, 'La base de datos ya está abierta.', None, -1, -2146820421), None)
```

Log Errores PREVISTO

```
MS_ACCESS: ERRORES EN BBDD_01: C:/Users/user/Desktop/APP_CONTROL_VERSIONES/PRUEBA_APP/PRUEBA_CONTROL_VERSIONES_1.accdb
-----
MODULO PYTHON: APP_3_BACK_END_MS_ACCESS.py
RUTINA PYTHON: def_proceso_access_1_import
LINEA ERROR: 1767
ERROR: (-2147352567, 'Ocurrió una excepción.', (0, None, 'La base de datos ya está abierta.', None, -1, -2146820421), None)
ERROR_DESC: No puedes tener la base de datos MS ACCESS abierta al ejecutar los procesos de control de versiones o de diagnostico de dependencias
```

ANEXOS

ANEXO_01

RUTINA

def_proceso_access_1_import

ANEXO_01

RUTINA: def_proceso_access_1_import

La rutina permite importar el código VBA de los distintos módulos VBA de una base de datos MS ACCESS. Asimismo, recupera las estructuras de las tablas locales como las connecting string de los vínculos ODBC y vínculos Otros (los que son hacia otras bases de datos MS ACCESS).

Tras la importación realiza cálculos intermedios para preparar los datos para poder realizar los cálculos de los procesos de control de versiones y el de diagnostico de dependencias de objetos.

La rutina tiene 3 parámetros:

PARAMETRO	DESCRIPCIÓN
proceso_id	PROCESO_01 (control de versiones) o PROCESO_02 (diagnostico). Son las keys_1 del diccionario <code>dicc_procesos</code> (módulo APP_2_GENERAL).
opcion_bbdd	BBDD_01 o BBDD_02. Son las keys_1 del diccionario <code>dicc_codigos_bbdd</code> (módulo APP_2_GENERAL).
path_bbdd	es la ruta donde esta ubicada la base de datos MS ACCESS

La rutina nada más empezar se encapsula en un bloque de flujos de errores (**try except**).

El bloque **except** permite almacenar en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL), en las subkeys_3 (LISTA_DICC_ERRORES_IMPORTACION_BBDD_01 y LISTA_DICC_ERRORES_IMPORTACION_BBDD_01) asociadas a las key_1 (PROCESO_01 o PROCESO_02, según el parámetro de la rutina), los sucesivos errores de ejecución, en caso de que los hubiese, lo que permite generar en otra rutina logs en formatos .txt.

La rutina tiene varios bloques de cálculos:

BLOQUE	DESCRIPCIÓN
1	Importación código VBA en un dataframe y cálculo de las estructuras de tablas y vínculos que se almacenan en una lista de diccionarios
2	Cálculo de columnas intermedias en el dataframe mencionado
3	Generación de las listas de diccionarios para las rutinas y variables publicas VBA que se suman a la lista de diccionarios mencionada y se almacena al final en el diccionario <code>dicc_codigos_bbdd</code> (módulo <code>APP_2_GENERAL</code>)
4	Calculo en el dataframe mencionado de columnas especificas al proceso de DIAGNOSTICO
5	Almacenamiento del dataframe en el diccionario <code>dicc_codigos_bbdd</code> (módulo <code>APP_2_GENERAL</code>)

1

Importación código VBA en un dataframe

Con el parámetro **path_bbdd** (que es la ruta donde se ubica la base de datos MS ACCESS), se abre la base de datos, mediante la librería Python **win32com.client**, y se procede a recuperar los códigos VBA de todos los módulos.

Se crea la lista de diccionarios **lista_dicc_objetos** para almacenar a lo largo de la rutina Python los datos relacionados con tablas, vínculos, rutinas VBA y variables públicas VBA.

```

2150 lista_dicc_objetos = []
2151
2152
2153 #se abre el access y se accede a su codigo VBA
2154 access_app = win32com.client.Dispatch("Access.Application")
2155 access_app.OpenCurrentDatabase(path_bbdd)
2156
2157 current_bbdd = access_app.CurrentDb()
2158 vba_project = access_app.VBE.ActiveVBProject

```


ANEXO_01

RUTINA: def_proceso_access_1_import

1

Importación código VBA en un dataframe

Nada más abrirse la base de datos MS ACCESS, se procede a almacenar en el diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) la lista de librerías DLL usadas en el código VBA. Esto es para el control de versiones cuando se accede a la GUI de merge en base de datos física, permite informar, comparando las 2 bases de datos seleccionadas, que librerías DLL el usuario ha de activar manualmente en la BBDD_02 (que es por defecto la base de datos donde se realiza el merge).

```
2161 #se listan las librerías DLL usadas en el código VBA y se almacenan en dicc_codigos_bbdd
2162 lista_librerias_dll = [dll.Description for dll in vba_project.References]
2163 lista_librerias_dll = sorted(lista_librerias_dll)
2164
2165 mod_gen.dicc_codigos_bbdd[opcion_bbdd]["MS_ACCESS"]["LISTA_LIBRERIAS_DLL"] = lista_librerias_dll if len(lista_librerias_dll) != 0 else None
```

El siguiente paso consiste en crear el dataframe **df_bbdd_codigo** que almacena todos los códigos VBA de los distintos módulos VBA de la base de datos MS ACCESS.

Se crea mediante bucle sobre los componentes VBA de la base de datos donde a cada iteración se concatena al dataframe creado. En este paso tan solo se recuperan los datos siguientes:

- Tipo módulo
- Nombre módulo
- Número de línea del código dentro del módulo
- Código VBA

ANEXO_01

RUTINA: def_proceso_access_1_import

1

Importación código VBA en un dataframe

```
2178 df_bbdd_codigo = pd.DataFrame()
2179 cont = 0
2180 for component in vba_project.VBComponents:
2181
2182     cont += 1
2183
2184     lambda tipo modulo = lambda x: "Estandar" if x == 1 else "Clase" if x == 2 else "UserForm" if x == 3 else "Formulario/Reporte" if x == 100 else None
2185     tipo_modulo = lambda_tipo_modulo(component.type,
2186
2187     nombre_modulo = component.Name
2188     code_module = component.CodeModule
2189     num_lines = code_module.CountOfLines
2190     code_lines = code_module.Lines(1, num_lines)
2191
2192     lista_temp_1 = code_lines.split("\n")
2193     lista_temp_2 = [[tipo_modulo, nombre_modulo, ind + 1, linea] for ind, linea in enumerate(lista_temp_1)]
2194
2195     df_temp = pd.DataFrame(lista_temp_2, columns = lista_headers_df_bbdd_codigo)
2196     df_bbdd_codigo = df_temp if cont == 1 else pd.concat([df_bbdd_codigo, df_temp])
2197
2198     del df_temp
2199
2200 df_bbdd_codigo.reset_index(drop = True, inplace = True)
```

El siguiente paso es recuperar las estructuras de las tablas locales y de los vínculos (sean ODBC u otros) para almacenarlos en la lista de diccionarios **lista_dicc_objetos** mencionada en páginas anteriores. Este paso requiere que la base de datos MS ACCESS este abierta.

Como paso previo, se crea un dataframe temporal **df_access_vinculos** para almacenar las connecting string de los vínculos ODBC u otros.

ANEXO_01

RUTINA: def_proceso_access_1_import

1

Importación código VBA en un dataframe

Se realiza abriendo un Recordset sobre la query **query_access_vínculos** (módulo APP_3_BACK_END_MS_ACCESS) y mediante bucle sobre el mismo se crea el dataframe que contiene las siguientes columnas:

- Nombre del vinculo
- Connecting String
- Objeto de origen

```
24 query_access_vinculos = """SELECT Name AS NOMBRE_VINCULO_ACCESS, Connect AS LINK_ODBC, ForeignName AS OBJETO_ORIGEN
25 | | | | | | | FROM MSysObjects
26 | | | | | | | """
```

```
2210 MiRecordset = current_bbdd.OpenRecordset(query_access_vinculos)
2211
2212 lista_temp = []
2213 while not MiRecordset.EOF:
2214     nombre_vinculo = MiRecordset.Fields("NOMBRE_VINCULO_ACCESS").Value
2215     link_connect = MiRecordset.Fields("LINK_ODBC").Value
2216     objeto_origen = MiRecordset.Fields("OBJETO_ORIGEN").Value
2217
2218     lista_temp.append([nombre_vinculo, link_connect, objeto_origen])
2219
2220     MiRecordset.MoveNext()
2221
2222 MiRecordset.Close()
2223
2224 df_access_vinculos = pd.DataFrame(lista_temp, columns = lista_headers_df_access_vinculos) if len(lista_temp) != 0 else None
```

ANEXO_01

RUTINA: def_proceso_access_1_import

1

Importación código VBA en un dataframe

Pasado este paso previo, se pasa a calcular la lista de diccionarios **lista_dicc_objetos** para las tablas locales y vínculos.

Cada diccionario que integran esta lista es por objeto. Su estructura es la misma según el tipo de objeto (sean tablas locales, vínculos ODBC u otros, rutinas VBA o variables públicas VBA) por lo que algunas keys se usan o no según el tipo de objeto.

KEY	TIPO OBJETO					DESCRIPCIÓN
	TABLAS_LOCALES	VINCULOS_ODBC	VINCULOS_OTROS	RUTINAS_VBA	VARIABLES_VBA	
TIPO_OBJETO	X	X	X	X	X	TABLA_LOCAL, VINCULO_ODBC,VINCULO_OTRO, RUTINAS_VBA, VARIABLES_VBA (son las subkeys_3 del diccionario dicc_control_versiones_tipo_objeto asociado MS_ACCESS)
TIPO_MODULO				X	X	Formulario, UserForm VBA, módulo estándar VBA
NOMBRE_MODULO				X	X	Nombre del módulo
NOMBRE_OBJETO	X	X	X	X		Nombre del objeto
TIPO Rutina				X		Rutina o función
TIPO_DECLARACION_Rutina				X		Pública o privada
PARAMETROS_Rutina				X		Concatenación de los parámetros de la rutina VBA con sus tipos de datos
TIPO_VARIABLE_PUBLICA					X	Variable pública normal o variable pública definida por el usuario
TIPO_DATO_VARIABLE_PUBLICA					X	Tipo de dato para las variables públicas normales que no son constantes
VARIABLE_PUBLICA_ES_CONSTANTE					X	Indica si la variable pública es constante o no
VARIABLE_PUBLICA_CONSTANTE_VALOR					X	Valor de la constante para las variables públicas normales que son constantes
VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES					X	Concatenación de las sub-variables asociadas a una variable pública definida por el usuario
DF_CODIGO	X	X	X	X	X	Dataframe con el código del objeto (sentencia CREATE TABLE para las tablas, connecting string para los vínculos, código VBA para el resto)
CONNECTING_STRING_VINCULOS	X	X	X	X	X	El la connecting string para los vínculos ODBC u otros (es la key DF_OBJETO convertida en string para este tipo de objetos). Se usa solo en el diagnostico porque se exporta a Excel

Se realiza mediante bucle sobre los distintos objetos de la base de datos MS ACCESS usando el método **TableDefs** donde se excluye en la búsqueda las tablas de sistema (las que empiezan por MSys). Se determina si un objeto es tabla local o vinculo con el atributo **connect** asociado a **TableDefs** (si es False es tabla local, si es True es vinculo).

ANEXO_01

RUTINA: def_proceso_access_1_import

1

Importación código VBA en un dataframe

Dentro del bucle, se calcula el código del objeto, mediante la función **func_diagnostico_access_tablas_df_create_table** (módulo APP_3_BACK_END_MS_ACCESS). Para las tablas locales es la sentencia SQL CREATE TABLE y para los vínculos es la connecting string a la cual se le agrega el objeto de origen.

La función **func_diagnostico_access_tablas_df_create_table** no se detalla en el presente documento, con referirse al código Python de la función considero que es suficiente.

```
2229         tipo_objeto = None
2230         nombre_objeto = None
2231         df_codigo = pd.DataFrame(columns = ["CODIGO"])
2232         for tabledef in current_bbdd.TableDefs:
2233             nombre_objeto = tabledef.Name
2234
2235             if not tabledef.Name.startswith("MSys"):
```

Hay que matizar que la función **func_diagnostico_access_tablas_df_create_table** tan solo crea la sentencia SQL de CREATE TABLE pero no crea las primary keys (PK). En caso de querer modificar este punto habría que modificar la función.

ANEXO_01

RUTINA: def_proceso_access_1_import

1

Importación código VBA en un dataframe

TABLA_LOCAL

```
2235 if not tabledef.Name.startswith("MSys"):
2236
2237     #TABLAS LOCALES
2238     if not tabledef.Connect:
2239         tipo_objeto = "TABLA_LOCAL"
2240         link_connect = None
2241
2242         prop_tabla = current_bbdd.TableDefs(nombre_objeto)
2243
2244         df_codigo = func_diagnostico_access_tablas_df_create_table(tipo_objeto, prop_tabla = prop_tabla, nombre_objeto = nombre_objeto) if proceso_id == "PROCESO_01" else None
2245
2246         dicc_temp = {"TIPO_OBJETO": tipo_objeto
2247                     , "TIPO_MODULO": None #key que no aplica
2248                     , "NOMBRE_MODULO": None #key que no aplica
2249                     , "NOMBRE_OBJETO": nombre_objeto
2250                     , "TIPO Rutina": None #key que no aplica
2251                     , "TIPO_DECLARACION_Rutina": None #key que no aplica
2252                     , "PARAMETROS_Rutina": None #key que no aplica
2253                     , "TIPO_VARIABLE_PUBLICA": None #key que no aplica
2254                     , "TIPO_DATO_VARIABLE_PUBLICA": None #key que no aplica
2255                     , "VARIABLE_PUBLICA_ES_CONSTANTE": None #key que no aplica
2256                     , "VARIABLE_PUBLICA_CONSTANTE_VALOR": None #key que no aplica
2257                     , "VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES": None #key que no aplica
2258                     , "DF_CODIGO": df_codigo
2259                     , "CONNECTING_STRING_VINCULOS": None #key que no aplica
2260                     }
2261
2262         lista_dicc_objetos.append(dicc_temp)
```

ANEXO_01

RUTINA: def_proceso_access_1_import

1

Importación código VBA en un dataframe

VINCULO_ODBC y VINCULO_OTRO

```
2266 # VINCULOS
2267 elif tabledef.Connect:
2268
2269     tipo_objeto = "VINCULO_ODBC" if tabledef.Connect.upper().startswith("ODBC") else "VINCULO_OTRO"
2270     link_connect = tabledef.Connect
2271
2272     df_codigo = func_diagnostico_access_tablas_df_create_table(tipo_objeto, link_connect = link_connect, nombre_vinculo = nombre_objeto, df_access_vinculos = df_access_vinculos)
2273     connecting_string_vinculos = "\n".join([df_codigo.iloc[ind, 0] for ind in df_codigo.index])
2274
2275     dicc_temp = {"TIPO_OBJETO": tipo_objeto
2276                 , "TIPO_MODULO": None #key que no aplica
2277                 , "NOMBRE_MODULO": None #key que no aplica
2278                 , "NOMBRE_OBJETO": nombre_objeto
2279                 , "TIPO Rutina": None #key que no aplica
2280                 , "TIPO_DECLARACION_Rutina": None #key que no aplica
2281                 , "PARAMETROS_Rutina": None #key que no aplica
2282                 , "TIPO_VARIABLE_PUBLICA": None #key que no aplica
2283                 , "TIPO_DATO_VARIABLE_PUBLICA": None #key que no aplica
2284                 , "VARIABLE_PUBLICA_ES_CONSTANTE": None #key que no aplica
2285                 , "VARIABLE_PUBLICA_CONSTANTE_VALOR": None #key que no aplica
2286                 , "VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLEES": None #key que no aplica
2287                 , "DF_CODIGO": df_codigo
2288                 , "CONNECTING_STRING_VINCULOS": connecting_string_vinculos
2289                 }
2290
2291     lista_dicc_objetos.append(dicc_temp)
```

Se pasa como parámetro kwargs
el dataframe df_access_vinculos
comentado anteriormente

Llegado as este paso, ya se dispone de todo lo necesario para poder realizar los cálculos de control de versiones y diagnóstico de dependencias por loque se cierra la base de datos MS ACCESS.

```
2301 access_app.CloseCurrentDatabase()
2302 access_app.Quit()
2303 access_app = None
```

ANEXO_01

RUTINA: def_proceso_access_1_import

2

Cálculo de columnas intermedias en el dataframe df_bbdd_codigo

Se calculan las siguientes columnas por bloques.

BLOQUE	COLUMNA	DESCRIPCIÓN
A	INDICE	es el indice del df (se usa para localizar el nombre de la rutina / funcion y poder localizar su codigo asociado en CODIGO)
	CODIGO_MAYUSC_SIN_TAB_TRIMEADO	es la columna CODIGO puesta en mayusculas donde se quitan las tabulaciones y se trimea
	CODIGO_SIN_COMENTARIOS	es la columna CODIGO a la que se le borran todos los comentarios para poder realizar el diagnostico (los que son lineas completas de comentarios y las que estan a la derecha del codigo)
	CODIGO_SIN_COMENTARIOS_MAYUSC	es la columna CODIGO_SIN_COMENTARIOS puesta en mayusculas (se usa para calcular las columnas CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI y CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO)
	CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS	es la columna CODIGO donde se quitan las tabulaciones y se trimea y al cual se le quitan los comentarios (los que son lineas completas de comentarios y las que estan a la derecha del codigo)
	ES_ENCABEZADO_MODULO	es la columna CODIGO_MAYUSC_SIN_TAB_TRIMEADO donde se filtra todo lo que empieza por OPTION que son las sentencias de modulo (SI / NO)
B	TIPO_RUTINA	rutina o funcion
	TIPO_DECLARACION_RUTINA	publica o privada
	NOMBRE_RUTINA	nombre rutina / funcion
	NUMERO_LINEA_CODIGO_RUTINA	numero de linea del codigo dentro de la rutina / funcion
	PARAMETROS_RUTINA	parametros de la rutina / funcion con los tipos de datos asociados
	TIPO_DATO_PARAMETROS_Y_VARIABLES_LOCALES_RUTINA	recopila los tipos de dato de los parametros y variables locales de la rutina, se usa en el diagnostico de dependencias para localizar si variables publicas definidas por el usuario se usan en la rutina
	ES_VARIABLE_PUBLICA	4 valores distintos: NO, NO_ES_CONSTANTE, SI_ES_CONSTANTE y DEFINIDA_POR_USUARIO_iii --> NO_ES_CONSTANTE (para variables que se declaran en 1 sola linea, las que empiezan por "Public" o "Global") --> SI_ES_CONSTANTE (para variables que se declaran en 1 sola linea, las que empiezan por ""Const", "Public Const" o "Global Const") --> DEFINIDA_POR_USUARIO_iii (para variables definidas por el usuario, se declaran en varias lineas, se crea un literal unico para todas las lineas que componen la declaracion de una misma variable, iii es un numero de variable unico para cada una de estas variables)
	ES_VARIABLE_PUBLICA_CONTROL_VERSIONES	es la columna ES_VARIABLE_PUBLICA != NO donde no se eliminan los comentarios /se agregan las lineas de codigo que son todo comentarios y que no estan asociados a ninguna rutina / funcion
	ES_VARIABLE_PUBLICA_DIAGNOSTICO	es la columna ES_VARIABLE_PUBLICA != NO donde si se eliminan todos los comentarios

ANEXO_01

RUTINA: def_proceso_access_1_import

2

Cálculo de columnas intermedias en el dataframe df_bbdd_codigo

Las columnas del bloque A se calculan mediante los métodos **apply** (lambda) de la librería Python pandas.

```
2363 df_bbdd_codigo["CODIGO"] = df_bbdd_codigo["CODIGO"].apply(lambda x: x.replace("\r", "").replace("\'", indicador_comentario_vba_access))
2364 df_bbdd_codigo["INDICE"] = df_bbdd_codigo.index
2365 df_bbdd_codigo["CODIGO_MAYUSC_SIN_TAB_TRIMEADO"] = df_bbdd_codigo["CODIGO"].apply(lambda x: x.upper().replace("\t", "").strip())
2366
2367 df_bbdd_codigo["CODIGO_SIN_COMENTARIOS"] = df_bbdd_codigo["CODIGO"].apply(lambda x: func_codigo_vba_quitar_comentarios(str(x)))
2368 df_bbdd_codigo["CODIGO_SIN_COMENTARIOS_MAYUSC"] = df_bbdd_codigo["CODIGO_SIN_COMENTARIOS"].apply(lambda x: x.upper())
2369
2370 df_bbdd_codigo["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"] = df_bbdd_codigo["CODIGO"].apply(lambda x: func_codigo_vba_quitar_comentarios(str(x).replace("\t", "").strip()))
2371
2372 df_bbdd_codigo["ES_ENCABEZADO_MODULO"] = df_bbdd_codigo["CODIGO_MAYUSC_SIN_TAB_TRIMEADO"].apply(lambda x: "SI" if len(x) >= 6 and x.upper()[0:6] == "OPTION" else "NO")
```

Las columnas CODIGO_SIN_COMENTARIOS y CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS se calculan con la función **func_codigo_vba_quitar_comentarios** (módulo APP_3_BACK_END_MS_ACCESS), no se detalla en el presente documento.

Esta función permite quitar todos los comentarios:

- Los que aplican a la línea de código entera (las que empiezan por la comilla simple)
- Los que están a la derecha de una línea de código activa

Se hace mediante una función propia porque la librería Python **re**, preparada normalmente para este tipo de manipulaciones de string, no dio los resultados esperados.

<pre>'es una linea de comentario completa Public Public_Prueba_1 As Long 'mi comentario Public Const Public_Prueba_2 = "'aaa'" 'mi comentario Public Const Public_Prueba_3 = "'aaa'" 'mi comentario Public Const Public_Prueba_4 = "'bbb'" 'mi com'entario</pre>		<pre>Public Public_Prueba_1 As Long Public Const Public_Prueba_2 = "'aaa'" Public Const Public_Prueba_3 = "'aaa'" Public Const Public_Prueba_4 = "'bbb'"</pre>
--	---	--

2

Cálculo de columnas intermedias en el dataframe df_bbdd_codigo

Las columnas del bloque B se calculan, mediante la función **func_access_df_bbdd_codigo_cálculos** (módulo APP_3_BACK_END_MS_ACCESS).

```

2389         #se crean las columnas adicionales en el df df_bbdd_codigo para las RUTINAS usando la lista
2390         #que devuelve la funcion func_access_bbdd_codigo_calculos que se convierte en df y se realiza merge
2391         #con el df df_bbdd_codigo por la columna INDICE
2392         lista_calculos_rutinas = func_access_bbdd_codigo_calculos("LISTA_RUTINAS", df_bbdd_codigo_param = df_bbdd_codigo)
2393
2394         df_temp_rutinas = pd.DataFrame(lista_calculos_rutinas, columns = lista_headers_nuevas_columnas_rutinas)
2395
2396         df_bbdd_codigo = (pd.merge(df_bbdd_codigo[[i for i in df_bbdd_codigo.columns]], df_temp_rutinas[[i for i in df_temp_rutinas.columns]],
2397                                   how = "left", left_on = "INDICE", right_on = "INDICE"))

```

```

2401         #se crean las columnas adicionales en el df df_bbdd_codigo para las VARIABLES PUBLICAS usando la lista
2402         #que devuelve la funcion func_access_bbdd_codigo_calculos que se convierte en df y se realiza merge
2403         #con el df df bbdd codigo por la columna INDICE
2404         lista_calculos_variables_publicas = func_access_bbdd_codigo_calculos("LISTA_VARIABLES_PUBLICAS", df_bbdd_codigo_param = df_bbdd_codigo)
2405
2406         df_temp_variables_publicas = pd.DataFrame(lista_calculos_variables_publicas, columns = lista_headers_nuevas_columnas_variables_publicas)
2407
2408         df_bbdd_codigo = (pd.merge(df_bbdd_codigo[[i for i in df_bbdd_codigo.columns]],
2409                                   df_temp_variables_publicas[[i for i in df_temp_variables_publicas.columns if i not in ["TIPO_MODULO", "NOMBRE_MODULO", "CODIGO"]]],
2410                                   how = "left", left_on = "INDICE", right_on = "INDICE"))

```

Las nuevas columnas relacionadas con rutinas / funciones y variables públicas VBA, en ambos casos, con la función **func_access_df_bbdd_codigo_calculos**, se crea una lista de listas con los datos necesarios que se convierte en dataframe para poder realizar el merge con el dataframe **df_bbdd_codigo** y disponer así de las nuevas columnas.

La función **func_access_df_bbdd_codigo_cálculos** se explica más en profundidad en el anexo **ANEXO_08**.

ANEXO_01

RUTINA: def_proceso_access_1_import

3

Generación de las listas de diccionarios para objetos tipo rutinas VBA y variables públicas VBA

Se pasa a calcular estas listas donde cada diccionario ha de tener la misma estructura que los que componen la lista de diccionarios **lista_dicc_objetos** comentada en páginas anteriores para las tablas locales y vínculos.

Para el tipo de objeto RUTINAS_VBA, se extrae del dataframe **df_bbdd_codigo** un nuevo dataframe filtrado por la columna **NOMBRE_RUTINA** no nula.

```
2429 lista_campos_rutina = ["TIPO_MODULO", "NOMBRE_MODULO", "NOMBRE_RUTINA", "TIPO_RUTINA", "TIPO_DECLARACION_RUTINA", "PARAMETROS_RUTINA"]
2430
2431 df_rutinas = df_bbdd_codigo.loc[df_bbdd_codigo["NOMBRE_RUTINA"].isnull() == False, lista_campos_rutina]
2432 df_rutinas.drop_duplicates(subset = lista_campos_rutina, keep = "last", inplace = True)
2433 df_rutinas.reset_index(drop = True, inplace = True)
```

Mediante bucle sobre los índices de este nuevo dataframe, se construye la lista de diccionarios mencionada.

```
2436 for ind in df_rutinas.index:
2437
2438     tipo_modulo = df_rutinas.iloc[ind, df_rutinas.columns.get_loc("TIPO_MODULO")]
2439     nombre_modulo = df_rutinas.iloc[ind, df_rutinas.columns.get_loc("NOMBRE_MODULO")]
2440     nombre_rutina = df_rutinas.iloc[ind, df_rutinas.columns.get_loc("NOMBRE_RUTINA")]
```

Según que se opte por el proceso de control de versiones (PROCESO_01) o el proceso de diagnostico de dependencias (PROCESO_02), la key **DF_CODIGO** de los diccionarios varia:

- CONTROL VERSIONES: se agrega el código original sin alteraciones
- DIAGNOSTICO: no se agrega el código (se agrega un dataframe vacío)

ANEXO_01

RUTINA: def_proceso_access_1_import

3

Generación de las listas de diccionarios para objetos tipo rutinas VBA y variables públicas VBA

```
2445 if proceso_id == "PROCESO_01":
2446     df_codigo = (df_bbdd_codigo.loc[(df_bbdd_codigo["TIPO_MODULO"] == tipo_modulo) & (df_bbdd_codigo["NOMBRE_MODULO"] == nombre_modulo) &
2447                                     (df_bbdd_codigo["NOMBRE Rutina"] == nombre_rutina), ["CODIGO"]])
2448
2449     df_codigo.reset_index(drop = True, inplace = True)
2450
2451 elif proceso_id == "PROCESO_02":
2452     df_codigo = pd.DataFrame(columns = ["CODIGO"])
2453
2454
2455 #se agregan los datos a la lista lista_dicc_objetos
2456 dicc_temp = {"TIPO_OBJETO": "RUTINAS_VBA"
2457             , "TIPO_MODULO": tipo_modulo
2458             , "NOMBRE_MODULO": nombre_modulo
2459             , "NOMBRE_OBJETO": nombre_rutina
2460             , "TIPO_RUTINA": df_rutinas.iloc[ind, df_rutinas.columns.get_loc("TIPO_RUTINA")]
2461             , "TIPO_DECLARACION_RUTINA": df_rutinas.iloc[ind, df_rutinas.columns.get_loc("TIPO_DECLARACION_RUTINA")]
2462             , "PARAMETROS_RUTINA": df_rutinas.iloc[ind, df_rutinas.columns.get_loc("PARAMETROS_RUTINA")]
2463             , "TIPO_VARIABLE_PUBLICA": None #key que no aplica
2464             , "TIPO_DATO_VARIABLE_PUBLICA": None #key que no aplica
2465             , "VARIABLE_PUBLICA_ES_CONSTANTE": None #key que no aplica
2466             , "VARIABLE_PUBLICA_CONSTANTE_VALOR": None #key que no aplica
2467             , "VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES": None #key que no aplica
2468             , "DF_CODIGO": df_codigo
2469             , "CONNECTING_STRING_VINCULOS": None #key que no aplica
2470             }
2471
2472 lista_dicc_objetos.append(dicc_temp)
```

Para las variables públicas VBA, se genera la lista de diccionarios **lista_dicc_objetos_variables_publicas** mediante la función **func_access_df_bbdd_codigo_calculos_varios** (opción LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS) porque requiere muchos cálculos intermedios (es para aligerar el código de la presente rutina).

```
2487 lista_dicc_objetos_variables_publicas = func_access_bbdd_codigo_calculos("LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS", lista_calculos_variables_publicas = lista_calculos_variables_publicas)
```

ANEXO_01

RUTINA: def_proceso_access_1_import

3

Generación de las listas de diccionarios para objetos tipo rutinas VBA y variables públicas VBA

Se fusiona **lista_dicc_objetos** con **lista_dicc_objetos_variables_publicas** y se almacena la lista resultante en el diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_OBJETOS) asociada a la key_1 (MS_ACCESS) y subkey_2 (LISTA_DICC_OBJETOS).

```
2495 #se fusiona lista_dicc_objetos con lista_dicc_objetos_variables_publicas
2496 lista_dicc_objetos = lista_dicc_objetos + lista_dicc_objetos_variables_publicas
2497
2498
2499 #se almacena lista_dicc_objetos en dicc_codigos_bbdd[opcion_bbdd]["MS_ACCESS"]["LISTA_OBJETOS_CON_O_SIN_CODIGO"]
2500 mod_gen.dicc_codigos_bbdd[opcion_bbdd]["MS_ACCESS"]["LISTA_DICC_OBJETOS"] = lista_dicc_objetos
```

4

Cálculo de columnas específicas al diagnostico de dependencias en el dataframe df_bbdd_codigo

COLUMNA	DESCRIPCIÓN
CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI	es la columna CODIGO_SIN_COMENTARIOS_MAYUSC donde se excluye todo segun que no se adapte al diccionario dicc_diagnostico_tablas_access_donde_buscar y sirve para el calculo de dependencias de tablas / vinculos con la funcion func_diagnostico_access_tablas_codigo_preservar
CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO	es la columna CODIGO_SIN_COMENTARIOS_MAYUSC con todo lo incluido entre comillas dobles y que no este incluido en CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI se usa en el diagnostico para informar de las tablas que se usan (o no) fuera del ambito de las logicas configuradas en el diccionario dicc_diagnostico_tablas_access_donde_buscar es el usuario quien tiene que chequear manualmente desde el excel final si se ha de considerar dependencia o no (es una lista con el numero de linea de la rutina / funcion y el codigo de la linea)
CODIGO_DIAGNOSTICO Rutinas_Y_Variables	es la columna CODIGO_SIN_COMENTARIOS_MAYUSC donde se quita todo lo incluido entre "" (usar un objeto llamado aaa no es lo mismo que usar un "aaa"), sirve para el calculo de dependencias de rutinas VBA y variables VBA

ANEXO_01

RUTINA: def_proceso_access_1_import

4

Cálculo de columnas específicas al diagnostico de dependencias en el dataframe **df_bbdd_código**

```
2525 if proceso_id == "PROCESO_02":
2526
2527     #se calcula CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_SI y CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO usando
2528     #la funcion func_diagnostico_access_tablas_codigo_preservar
2529     df_bbdd_codigo["CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI"] = (df_bbdd_codigo.apply(lambda x:
2530     func_diagnostico_access_tablas_codigo_preservar("PRESERVAR_SI", x["CODIGO_SIN_COMENTARIOS_MAYUSC"])
2531     , axis = 1))
2532
2533
2534     df_bbdd_codigo["CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO"] = (df_bbdd_codigo.apply(lambda x:
2535     func_diagnostico_access_tablas_codigo_preservar("PRESERVAR_NO", x["CODIGO_SIN_COMENTARIOS_MAYUSC"])
2536     , axis = 1))
2537
2538
2539     #se calcula CODIGO_DIAGNOSTICO Rutinas y Variables
2540     #usando CODIGO_SIN_COMENTARIOS_MAYUSC se quita todo lo incluido entre "" para evitar que localice en el diagnostico objetos tipo rutinas o variables
2541     #incluidos en string (ejemplo: usar un objeto llamado aaa no es lo mismo que usar un "aaa")
2542     df_bbdd_codigo["CODIGO_DIAGNOSTICO_RUTINAS_Y_VARIABLES"] = (df_bbdd_codigo["CODIGO_SIN_COMENTARIOS_MAYUSC"].apply(lambda x:
2543     func_access_bbdd_codigo_calculos_varios("QUITAR_ENCAPSULADO_ENTRE_COMILLAS_DOBLAS", string_param = x.upper())))
```

Las 2 primeras columnas se calculan con la función **func_diagnostico_access_tablas_codigo_preservar** (módulo APP_3_BACK_END_MS_ACCESS). Esta función se detalla más en profundidad en el anexo **ANEXO_10**.

La última se calcula con la función **func_vba_string_quitar_encapsulado_entre_caracteres** (módulo APP_3_BACK_END_MS_ACCESS) para excluir todo lo incluido en el código entre comillas dobles. No se documenta en el presente documento.

5

Cálculo de columnas específicas al diagnostico de dependencias en el dataframe **df_bbdd_codigo**

Se guarda el dataframe **df_bbdd_codigo** en el diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) en la subkey_3 () asociada a la key_1 (BBDD_01 o BBDD02, parámetro de la presente rutina) y a la subkey_2 (DF_CODIGO_CALCULADO_TRAS_IMPORT).

```
2550 mod_gen.dicc_codigos_bbdd[opcion_bbdd]["MS_ACCESS"]["DF_CODIGO_CALCULADO_TRAS_IMPORT"] = df_bbdd_codigo
```

ANEXO_01

RUTINA: def_proceso_access_1_import

Por último, se crea el bloque **except** para almacenar los posibles errores de ejecución de la rutina en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_IMPORTACION_BBDD_01 o LISTA_DICC_ERRORES_IMPORTACION_BBDD_02, según la opción de base de datos, parámetro de la presente rutina) asociada a la key_1 (**proceso_id**, parámetro de la presente rutina) y subkey_2 (MS_ACCESS).

```
2555     except Exception as Err:
2556
2557         traceback_error = traceback.extract_tb(Err.__traceback__)
2558         modulo_python = os.path.basename(traceback_error[0].filename)
2559         rutina_python = traceback_error[0].name
2560         linea_error = traceback_error[0].lineno
2561
2562         lista_dicc_errores_migracion = mod_gen.dicc_errores_procesos[proceso_id]["MS_ACCESS"]["LISTA_DICC_ERRORES_IMPORTACION_" + opcion_bbdd]
2563
2564         dicc_errores_temp = {"TIPO_BBDD": "MS_ACCESS"
2565                             , "MODULO_PYTHON": modulo_python
2566                             , "RUTINA_PYTHON": rutina_python
2567                             , "LINEA_ERROR": linea_error
2568                             , "ERRORES": str(Err)
2569                             }
2570
2571         if isinstance(lista_dicc_errores_migracion, list):
2572             lista_dicc_errores_migracion.append(dicc_errores_temp)
2573         else:
2574             lista_dicc_errores_migracion = [dicc_errores_temp]
2575
2576         mod_gen.dicc_errores_procesos[proceso_id]["MS_ACCESS"]["LISTA_DICC_ERRORES_IMPORTACION_" + opcion_bbdd] = lista_dicc_errores_migracion
2577         del dicc_errores_temp
2578         del lista_dicc_errores_migracion
2579         pass #es pass para que se localicen todos los posibles errores en el proceso de import
```

ANEXO_02

RUTINA

def_proceso_sql_server_1_import

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

La rutina permite importar el código T-SQL de los distintos esquemas de una base de datos SQL SERVER. Asimismo, recupera las estructuras de las tablas.

Tras la importación realiza cálculos intermedios para preparar los datos para poder realizar los cálculos de los procesos de control de versiones y el de diagnostico de dependencias de objetos.

La rutina tiene 4 parámetros:

PARAMETRO	DESCRIPCIÓN
proceso_id	PROCESO_01 (control de versiones) o PROCESO_03 (diagnostico). Son las keys_1 del diccionario <code>dicc_procesos</code> (módulo <code>APP_2_GENERAL</code>).
opcion_bbdd	BBDD_01 o BBDD_02. Son las keys_1 del diccionario <code>dicc_codigos_bbdd</code> (módulo <code>APP_2_GENERAL</code>).
sql_server_servidor	es el servidor SQL SERVER
lista_sql_server_bbdd	es la lista de bases de datos donde ejecutar el proceso

La rutina nada más empezar se encapsula en un bloque de flujos de errores (**try except**).

El bloque **except** permite almacenar en el diccionario **dicc_errores_procesos** (módulo `APP_2_GENERAL`), en las subkeys_3 (`LISTA_DICC_ERRORES_IMPORTACION_BBDD_01` y `LISTA_DICC_ERRORES_IMPORTACION_BBDD_01`) asociadas a las key_1 (`PROCESO_01` o `PROCESO_03`, según el parámetro de la rutina), los sucesivos errores de ejecución, en caso de que los hubiese, lo que permite generar en otra rutina logs en formatos .txt.

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

El 1er paso es conectarse al servidor seleccionado usando el parámetro de la función.

A modo recordatorio, en la GUI de inicio al seleccionar el servidor SQL SERVER hay un proceso integrado que permite localizar si la conexión es por Windows Authentication o es mediante login y password. Por lo que, en esta etapa, ya se sabe si es por un modo u otro y por lo tanto la connecting string ya se adapta a ello. Esta ultima, se recupera del valor de la subkey_3 (CONNECTING_STRING) del diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL).

La conexión con el servidor del parámetro de la rutina se realiza usando la base datos de sistema “master” para completar la connecting string.

```
486 conn_str = mod_gen.dicc_codigos_bbdd[opcion_bbdd][\"SQL_SERVER\"][\"CONNECTING_STRING\"].replace(\"REEMPLAZA_SERVIDOR\", sql_server_servidor).replace(\"REEMPLAZA_BBDD\", \"master\")
487 MiConexion = pyodbc.connect(conn_str)
488 cursor = MiConexion.cursor()
```

El siguiente paso es almacenar en la lista temporal **lista_sql_server_bbdd_servidor** todas las bases de datos que conforman el servidor seleccionado (es necesario para el cálculo del proceso de diagnostico de dependencias).

```
491 #se crea la lista de bbdd que conforman el servidor (se usa solo en el diagnostico de dependencias)
492 df_temp = pd.read_sql_query(query_lista_bbdd_sql_server, MiConexion)
493 lista_sql_server_bbdd_servidor = [df_temp.iloc[ind, 0] for ind in df_temp.index] if proceso_id == \"PROCESO_03\" else None
```

Se obtiene mediante la query **query_lista_bbdd_sql_server** (módulo APP_3_BACK_END_SQL_SERVER):

```
68 query_lista_bbdd_sql_server = \"\"\"SELECT UPPER(name) FROM master.dbo.sysdatabases
69 WHERE name not in ('master', 'tempdb', 'model', 'msdb')
70 ORDER BY name\"\"\"
```

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

Se crean los dataframes **df_objetos_sql_server** y **df_objetos_sql_server_parámetros** que almacenan respectivamente los objetos de las bases de datos seleccionadas y los parámetros de los stored procedures y funciones (este último dataframe se calcula tan solo si se opta por el proceso de diagnostico de dependencias).

```
500 df_objetos_sql_server = None
501 df_objetos_sql_server_parametros = None
502 for ind, bbdd in enumerate(lista_sql_server_bbdd):
503
504     df_temp_1 = pd.read_sql_query(query_objetos_sql_server.replace("REEMPLAZA_BBDD", bbdd), MiConexion)
505     df_objetos_sql_server = df_temp_1 if ind == 0 else pd.concat([df_objetos_sql_server, df_temp_1])
506     del df_temp_1
507
508
509     if proceso_id == "PROCESO_03":
510
511         df_temp_2 = pd.read_sql_query(query_objetos_sql_server_parametros.replace("REEMPLAZA_BBDD", bbdd), MiConexion)
512         df_objetos_sql_server_parametros = df_temp_2 if ind == 0 else pd.concat([df_objetos_sql_server_parametros, df_temp_2])
513         del df_temp_2
514
515 df_objetos_sql_server.reset_index(drop = True, inplace = True)
516
517 if proceso_id == "PROCESO_03":
518     df_objetos_sql_server_parametros.reset_index(drop = True, inplace = True)
```

El dataframe **df_objetos_sql_server** se crea mediante la query **query_objetos_sql_server** (módulo APP_3_BACK_END_SQL_SERVER):

```
81 query_objetos_sql_server = """SELECT
82     'REEMPLAZA_BBDD' AS BBDD
83     , T2.name AS REPOSITORIO
84     , CASE WHEN T1.type = 'U' THEN 'TABLAS'
85           WHEN T1.type = 'V' THEN 'VIEWS'
86           WHEN T1.type = 'P' THEN 'STORED_PROCEDURES'
87           WHEN T1.type IN ('FN', 'IF', 'TF', 'AF') THEN 'FUNCIONES'
88     END AS TIPO_OBJETO
89     , T1.name AS NOMBRE_OBJETO
90     , 'REEMPLAZA_BBDD' + '.' + T2.name + '.' + T1.name AS OBJETO_LONG
91     , T2.name + '.' + T1.name AS OBJETO_SHORT
92
93     FROM REEMPLAZA_BBDD.sys.objects T1
94     LEFT JOIN REEMPLAZA_BBDD.sys.schemas T2 on T1.schema_id = T2.schema_id
95
96     WHERE UPPER(T1.type) IN ('U', 'V', 'P', 'FN', 'IF', 'TF', 'AF')
97
98     ORDER BY T1.type, T2.name, T1.name
99 """
```

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

El dataframe **df_objetos_sql_server_parametros** se crea mediante la query **query_objetos_sql_server_parametros** (módulo APP_3_BACK_END_SQL_SERVER):

```
101 query_objetos_sql_server_parametros = """SELECT
102     'REEMPLAZA_BBDD' AS BBDD
103     , T3.name AS REPOSITORIO
104     , CASE WHEN T2.type = 'U' THEN 'TABLAS'
105           WHEN T2.type = 'V' THEN 'VIEWS'
106           WHEN T2.type = 'P' THEN 'STORED_PROCEDURES'
107           WHEN T2.type IN ('FN', 'IF', 'TF', 'AF') THEN 'FUNCIONES'
108     END AS TIPO_OBJETO
109     , T2.name AS NOMBRE_OBJETO
110     , CASE WHEN UPPER(T4.name) LIKE '%CHAR%' THEN CONCAT(T1.name, ' ', UPPER(T4.name), '(', CAST(T1.max_length AS VARCHAR(100)), ')')
111     ELSE CONCAT(T1.name, ' ', UPPER(T4.name))
112     END AS PARAMETROS
113
114     FROM REEMPLAZA_BBDD.sys.parameters T1
115     LEFT JOIN REEMPLAZA_BBDD.sys.objects T2 ON T1.object_id = T2.object_id
116         | LEFT JOIN REEMPLAZA_BBDD.sys.schemas T3 on T2.schema_id = T3.schema_id
117
118     LEFT JOIN REEMPLAZA_BBDD.sys.types T4 ON T1.user_type_id = T4.user_type_id
119
120     WHERE T2.type IN ('P', 'FN', 'IF', 'TF') AND LEN(COALESCE(T1.name, '')) <> 0
121     ORDER BY T2.name, T1.parameter_id
122     """
```

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

El siguiente paso consiste en crear la lista de diccionarios **lista_dicc_objetos_sql_server** que sirve de base para el cálculo de los procesos de control de versiones y diagnostico de dependencias.

Mediante bucle (en adelante bucle_1) sobre los índices del dataframe **df_objetos_sql_server**, agregan diccionarios por objetos a esta lista. Los diccionarios tienen la estructura siguiente:

KEY	DESCRIPCIÓN
SERVIDOR	Es el servidor seleccionado (parámetro de la función)
BBDD	Es la base de datos donde se ubica el objeto
REPOSITORIO	Es el esquema donde se ubica el objeto
TIPO_OBJETO	Es el tipo de objeto (TABLAS, VIEWS, STORED_PROCEDURES o FUNCIONES)
NOMBRE_OBJETO	Nombre del objeto
OBJETO_LONG	Nombre del objeto (en formato mi_bbdd.mi_esquema.mi_objeto)
OBJETO_SHORT	Nombre del objeto (en formato mi_esquema.mi_objeto)
PARAMETROS	Parámetros del objeto (solo para STORED_PROCEDURES o FUNCIONES)
DF_CODIGO_ORIGINAL	Dataframe con el código original T-SQL del objeto (incluyendo comentarios). Es el que sirve para el cálculo del proceso de control de versiones pero también se usa en el diagnostico de dependencias cuando se opta por exportar los scripts T-.SQL en ficheros .sql.
LISTA_CODIGO_LIMPIADO	Lista donde cada item es una línea de código "limpiada" del script T-SQL del objeto (ver más abajo en los comentarios en que consiste esta limpieza). Se usa solo en el cálculo del diagnostico de dependencias. Es lista y no dataframe porque el cálculo del diagnostico es más rápido si se itera por listas en vez de dataframes.

```
523 lista_dicc_objetos_sql_server = []
524 for ind in df_objetos_sql_server.index:
525
526     bbdd = df_objetos_sql_server.iloc[ind, df_objetos_sql_server.columns.get_loc("BBDD")]
527     repositorio = df_objetos_sql_server.iloc[ind, df_objetos_sql_server.columns.get_loc("REPOSITORIO")]
528     tipo_objeto = df_objetos_sql_server.iloc[ind, df_objetos_sql_server.columns.get_loc("TIPO_OBJETO")]
529     nombre_objeto = df_objetos_sql_server.iloc[ind, df_objetos_sql_server.columns.get_loc("NOMBRE_OBJETO")]
530     objeto_long = df_objetos_sql_server.iloc[ind, df_objetos_sql_server.columns.get_loc("OBJETO_LONG")]
531     objeto_short = df_objetos_sql_server.iloc[ind, df_objetos_sql_server.columns.get_loc("OBJETO_SHORT")]
```

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

Dentro de la iteración sobre bucle_1, se calculan los valores de las keys DF_CODIGO_ORIGINAL y LISTA_CODIGO_LIMPIADO del diccionario mencionado en la página anterior mediante la función **func_sql_server_df_codigo_objeto** (módulo APP_3_BACK_END_SQL_SERVER).

```
538 #se crea el df con el codigo del objeto usando la funcion func_sql_server_df_codigo_objeto
539 df_codigo_original = func_sql_server_df_codigo_objeto("DF_CODIGO_ORIGINAL", tipo_objeto = tipo_objeto, MiConexion = MiConexion, bbdd = bbdd, objeto_long = objeto_long,
540 cursor = cursor, objeto_short = objeto_short)
541
542
543 #se limpia el df de codigo original usando la funcion func_sql_server_df_codigo_objeto (aplica solo para el diagnostico de dependencias)
544 lista_codigo_limpiado = func_sql_server_df_codigo_objeto("LISTA_CODIGO_LIMPIO", df_codigo_original = df_codigo_original) if proceso_id == "PROCESO_03" else None
```

Esta función no se explica con más detalle en este diseño funcional porque considero que con referirse directamente al código de la misma y a los comentarios ahí insertados es más que suficiente para entender su funcionamiento. No obstante, aquí se explica brevemente que hace.

La key **DF CODIGO ORIGINAL** se calcula con la opción “DF_CODIGO_ORIGINAL” que permite generar el código T-SQL de un objeto:

Si el objeto es una tabla, es la sentencia CREATE TABLE que se genera mediante el uso de la query **query_campos_tablas_sql_server**.

```
130 query_campos_tablas_sql_server = """SELECT
131 CASE WHEN CHARACTER_MAXIMUM_LENGTH IS NOT NULL THEN CONCAT('[', COLUMN_NAME, ']', UPPER(DATA_TYPE), '(', CHARACTER_MAXIMUM_LENGTH, ')')
132 ELSE CONCAT('[', COLUMN_NAME, ']', UPPER(DATA_TYPE))
133 END AS CAMPO_CON_TIPO_DATO
134 FROM REEMPLAZA_BBDD.INFORMATION_SCHEMA.COLUMNS
135 WHERE TABLE_CATALOG + '.' + TABLE_SCHEMA + '.' + TABLE_NAME = 'REEMPLAZA_OBJETO_LONG'
136 ORDER BY TABLE_CATALOG + '.' + TABLE_SCHEMA + '.' + TABLE_NAME, ORDINAL_POSITION
137 """
```

	CAMPO_CON_TIPO_DATO
1	[ErrorLogID] INT
2	[ErrorTime] DATETIME
3	[UserName] NVARCHAR(128)
4	[ErrorNumber] INT
5	[ErrorSeverity] INT
6	[ErrorState] INT
7	[ErrorProcedure] NVARCHAR(126)
8	[ErrorLine] INT
9	[ErrorMessage] NVARCHAR(4000)

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

Se extrae un dataframe por objeto de esta query y mediante bucle sobre los índices del mismo se crea la sentencia CREATE TABLE mediante otro dataframe (al cual se le agrega el número de línea).

Matizar que la sentencia CREATE TABLE en esta función **se limita a la declaración de campos usados** no a las primary keys (PK) ni a los índices que pudiesen tener las tablas. En caso de querer agregar esta información al app habría que modificar la función comentada.

Si el objeto no es una tabla (VIEWS, STORED_PROCEDURES o FUNCIONES), el calculo se hace mediante la instrucción SQL SERVER **sp_helptext** mediante la query **query_helptext_sql_server**.

```
148 query_helptext_sql_server = """USE REEMPLAZA_BBDD
149
150 IF OBJECT_ID('tempdb.dbo.#TEMP', 'U') IS NOT NULL
151     DROP TABLE #TEMP;
152
153 CREATE TABLE #TEMP (
154     NUMERO_LINEA_CODIGO INT IDENTITY(1,1)
155     , CODIGO NVARCHAR(MAX)
156 );
157
158 INSERT INTO #TEMP (CODIGO)
159 EXEC sp_helptext 'REEMPLAZA_OBJETO_LONG';
160 """
```

	NUMERO_LINEA_CODIGO	CODIGO
1	1	
2	2	CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
3	3	
4	4	@Tabla varchar(255), @RutaCSV varchar(255), @Separ...
5	5	
6	6	AS
7	7	BEGIN
8	8	
9	9	
10	10	declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 ...

La query **query_helptext_sql_server** permite mediante tabla temporal SQL SERVER (las que vienen precedidas de un #) encapsular el código de un script T-SQL. En la rutina Python aquí comentada se almacenan estos datos en un dataframe.

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

La key **LISTA CODIGO LIMPIADO** se calcula con la opción "LISTA_CODIGO_LIMPIO" y realiza los pasos siguientes:

PASO	DESCRIPCIÓN
1	Usando el dataframe calculado con esta misma función con la opción DF_CODIGO_ORIGINAL, se hace una 1era limpieza donde se quitan las tabulaciones, saltos de linea, se trimea, se quitan los corchetes "[" y "]", se quitan las lineas vacias
2	Se convierte el dataframe en string para poder localizar las lineas de comentarios (las que van por bloques /* y */)
3	Se localizan los comentarios por bloques y los que van por linea (los que vienen precedidos de "--") y se eliminan del string del paso 2
4	En SQL Server el uso de un objeto (sea objeto long o objeto short) puede realizarse envarias lineas. Aquí lo que se hace es unificarlos en una solo linea haciendo replace de "\n.\n", "\n." y ".\n" por nada (\n es en Python un salto de linea)
5	Se splitea el string por "\n" para obtener la lista de código limpiado

Volviendo a la iteración sobre bucle_1, el siguiente paso es calcular la key PARAMETROS usando el dataframe **df_objetos_sql_server_parametros** comentado en páginas anteriores que se convierte en string de cara a su correcta exportación en el fichero Excel de diagnostico de dependencias. La conversión a string se hace solo si se ha optado por el proceso de diagnostico de dependencias.

```
548     if proceso_id == "PROCESO_03":
549
550
551         df_parametros_temp = (df_objetos_sql_server_parametros.loc[(df_objetos_sql_server_parametros["BBDD"] == bbdd) &
552                             (df_objetos_sql_server_parametros["REPOSITORIO"] == repositorio) &
553                             (df_objetos_sql_server_parametros["NOMBRE_OBJETO"] == nombre_objeto) &
554                             (df_objetos_sql_server_parametros["TIPO_OBJETO"] == tipo_objeto), ["PARAMETROS"]])
555
556         df_parametros_temp.reset_index(drop = True, inplace = True)
557         string_parametros = ", ".join([df_parametros_temp.iloc[ind, 0] for ind in df_parametros_temp.index]) if len(df_parametros_temp) != 0 else None
```


ANEXO_02

RUTINA: def_proceso_sql_server_1_import

Con todos los datos listos en la iteración sobre bucle_1, se pasa a crear el diccionario y a agregarlo a la lista **lista_dicc_objetos_sql_server**.

```
562     dicc_temp = {"SERVIDOR": sql_server_servidor
563                 , "BBDD": bbdd
564                 , "REPOSITORIO": repositorio
565                 , "TIPO_OBJETO": tipo_objeto
566                 , "NOMBRE_OBJETO": nombre_objeto
567                 , "OBJETO_LONG": objeto_long
568                 , "OBJETO_SHORT": objeto_short
569                 , "PARAMETROS": string_parametros
570                 , "DF_CODIGO_ORIGINAL": df_codigo_original
571                 , "LISTA_CODIGO_LIMPIADO": lista_codigo_limpiado
572                 }
573
574     lista_dicc_objetos_sql_server.append(dicc_temp)
```

Una vez finalizado el bucle sobre bucle_1, se pasa a almacenar las listas **lista_dicc_objetos_sql_server** y **lista_sql_server_bbdd_servidor** en el diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) en las subkeys_3 (LISTA_DICC_OBJETOS y LISTA_BBDD_SERVIDOR) respectivamente asociadas a la key_1 (BBDD_01 o BBDD_02, parámetro de la función que se explica aquí) y subkey_2 (SQL_SERVER).

```
591     mod_gen.dicc_codigos_bbdd[opcion_bbdd]["SQL_SERVER"]["LISTA_DICC_OBJETOS"] = lista_dicc_objetos_sql_server if len(lista_dicc_objetos_sql_server) != 0 else None
592     mod_gen.dicc_codigos_bbdd[opcion_bbdd]["SQL_SERVER"]["LISTA_BBDD_SERVIDOR"] = lista_sql_server_bbdd_servidor
```

ANEXO_02

RUTINA: def_proceso_sql_server_1_import

Por último, se crea el bloque **except** para almacenar los posibles errores de ejecución de la rutina en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_IMPORTACION_BBDD_01 o LISTA_DICC_ERRORES_IMPORTACION_BBDD_02, según la opción de base de datos, parámetro de la presente rutina) asociada a la key_1 (BBDD_01 o BBDD_02, parámetro de la presente rutina) y subkey_2 (SQL_SERVER).

```
596 except Exception as Err:
597
598     traceback_error = traceback.extract_tb(Err.__traceback__)
599     modulo_python = os.path.basename(traceback_error[0].filename)
600     rutina_python = traceback_error[0].name
601     linea_error = traceback_error[0].lineno
602
603     lista_dicc_errores_migracion = mod_gen.dicc_errores_procesos[proceso_id]["SQL_SERVER"]["LISTA_DICC_ERRORES_IMPORTACION_" + opcion_bbdd]
604
605     dicc_errores_temp = {"TIPO_BBDD": "SQL_SERVER"
606                          , "MODULO_PYTHON": modulo_python
607                          , "RUTINA_PYTHON": rutina_python
608                          , "LINEA_ERROR": linea_error
609                          , "ERRORES": str(Err)
610                          }
611
612     if isinstance(lista_dicc_errores_migracion, list):
613         lista_dicc_errores_migracion.append(dicc_errores_temp)
614     else:
615         lista_dicc_errores_migracion = [dicc_errores_temp]
616
617     mod_gen.dicc_errores_procesos[proceso_id]["SQL_SERVER"]["LISTA_DICC_ERRORES_IMPORTACION_" + opcion_bbdd] = lista_dicc_errores_migracion
618     del dicc_errores_temp
619     del lista_dicc_errores_migracion
620     pass #es pass para que se localicen todos los posibles errores en el proceso de import
```

ANEXO_03

FUNCIÓN

func_dicc_control_versiones

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

La función **func_dicc_control_versiones** permite realizar la comparación de 2 scripts (sean VBA o T-SQL) sobre un mismo objeto presente en las 2 bases de datos (BBDD_01 y BBDD_02) y localizar que en que líneas de código de ambos scripts hay cambios.

La función tiene parámetros **kwargs** porque según el tipo de base de datos (MS ACCESS o SQL SERVER) y según los tipos de objetos se pasan más o menos parámetros según los casos.

TIPO BBDD	TIPO OBJETO	PARAMETROS **KWARGS							
		tipo_bbdd	check_objeto	tipo_objeto	tipo_repositorio	repositorio	nombre_objeto	df_codigo_bbdd_1	df_codigo_bbdd_2
MS ACCESS	TABLA_LOCAL	X	X	X			X	X	X
	VINCULO_ODBC	X	X	X			X	X	X
	VINCULO_OTRO	X	X	X			X	X	X
	RUTINAS_VBA	X	X	X	X	X	X	X	X
	VARIABLES_VBA	X	X	X	X	X		X	X
SQL SERVER	TABLAS	X	X	X		X	X	X	X
	VIEWS	X	X	X		X	X	X	X
	STORED_PROCEDURES	X	X	X		X	X	X	X
	FUNCIONES	X	X	X		X	X	X	X

PARAMETRO	DESCRIPCIÓN
tipo_bbdd	MS_ACCESS o SQL_SERVER
check_objeto	Valores de las keys del diccionario dicc_control_versiones_tipo_concepto (YA_EXISTE, SOLO_EN_BBDD_01 o SOLO_EN_BBDD_02)
tipo_objeto	Los tipos de objeto del cuadro de arriba
tipo_repositorio	Solo para MS ACCESS (formulario, userform o módulo VBA estandar)
repositorio	Nombre del módulo VBA para MS ACCESS y nombre del esquema para SQL SERVER
nombre_objeto	Nombre del objeto
df_codigo_bbdd_1	Dataframe con el código del objeto en BBDD_01. Para VARIABLES_VBA es el código del módulo con solo las variables, sin las rutinas / funciones).
df_codigo_bbdd_2	Dataframe con el código del objeto en BBDD_02. Para VARIABLES_VBA es el código del módulo con solo las variables, sin las rutinas / funciones).

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

La función devuelve un **diccionario** con las siguientes keys:

KEYS	DESCRIPCIÓN
TIPO_BBDD	MS_ACCESS o SQL_SERVER
CHECK_OBJETO	Valores de las keys del diccionario <code>dicc_control_versiones_tipo_concepto</code> (YA_EXISTE, SOLO_EN_BBDD_01 o SOLO_EN_BBDD_02)
TIPO_OBJETO	Los tipos de objeto mencionados en el 1er cuadro de la página anterior
TIPO_OBJETO_SUBFORM	Tipo de objeto con el literal que sale en la GUI de control de versiones y merge en base de datos física es el valor de las subkeys_4, TIPO_OBJETO_SUBFORM, del diccionario <code>dicc_control_versiones_tipo_objeto</code>)
TIPO_REPOSITORIO	Solo para MS ACCESS (formulario, userform o módulo VBA estandar)
REPOSITORIO	Nombre del módulo VBA para MS ACCESS y nombre del esquema para SQL SERVER
NOMBRE_OBJETO	Nombre del objeto
NUM_CAMBIOS_SCRIPT_1	Número de líneas de código con cambios en el script de BBDD_01 (se usa en el sub-formulario del a GUI de control de versiones)
NUM_CAMBIOS_SCRIPT_2	Número de líneas de código con cambios en el script de BBDD_02 (se usa en el sub-formulario del a GUI de control de versiones)
DF_CODIGO_ORIGINAL_1	Dataframe con el código del script de BBDD_01 donde las líneas con cambios con respecto al script de BBDD_02 estan localizadas
DF_CODIGO_ORIGINAL_2	Dataframe con el código del script de BBDD_02 donde las líneas con cambios con respecto al script de BBDD_01 estan localizadas
LISTA_DICC_MERGE_HECHOS	None al ejecutarse la función. Se informa a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)
DF_CODIGO_ACTUAL_1	Al ejecutarse la función es de inicio el mismo dataframe que DF_CODIGO_ORIGINAL_1. Se actualiza a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)
DF_CODIGO_ACTUAL_2	Al ejecutarse la función es de inicio el mismo dataframe que DF_CODIGO_ORIGINAL_2. Se actualiza a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)

La función, cuando se ejecuta en las rutinas **def_proceso_access_2_control_versiones** y **def_proceso_sql_server_2_control_versiones**, y se localizan objetos con cambios en los scripts VBA y/o T-SQL entre BBDD_01 y BBDD_02, es lo que permite crear la lista de diccionarios, comentada anteriormente en este documento, que se almacena en la subkey_4 (**LISTA DICC OBJETOS CONTROL VERSIONES**) del **diccionario `dicc_control_versiones_tipo_objeto`**.

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

A continuación, se pasa a detallar el contenido del código de la función.

El 1er paso es crear una copia de los dataframes de los parámetros **df_codigo_bbdd_1** y **df_codigo_bbdd_2** (ambos vienen con el código original tal cual se importo de las bases de datos MS ACCESS o SQL SERVER, con los comentarios si los hubiese). En estas copias se agregan 2 nuevas columnas.

Estos ajustes se realizan tan solo si los parámetros **df_codigo_bbdd_1** y **df_codigo_bbdd_2**. No lo son (son None) cuando el parámetro **check_objeto** es:

- YA_EXISTE: **df_codigo_bbdd_1** y **df_codigo_bbdd_2** son ambos dataframes
- SOLO_EN_BBDD_01: **df_codigo_bbdd_1** es un dataframe y **df_codigo_bbdd_2** no lo es (es None)
- SOLO_EN_BBDD_02: **df_codigo_bbdd_1** no es un dataframe (es None) y **df_codigo_bbdd_2** si es un dataframe

```
764 #se modifican los df_codigo_bbdd_1 + df_codigo_bbdd_2 (que ya vienen con la columna C
765 #y se reordenan dentro de los nuevos df:
766 # --> NUM_LINEA
767 # --> CODIGO_CON_NUM_LINEA
768 # --> CONTROL_CAMBIOS_ORIGINAL
769 # --> CONTROL CAMBIOS ACTUAL
770 df_temp1 = pd.DataFrame()
771 if isinstance(df_codigo_bbdd_1, pd.DataFrame):
772     df_temp1 = df_codigo_bbdd_1.copy()
773
774     df_temp1[lista_headers_df_codigo_control_versiones_1] = [None, None, None, None]
775     df_temp1 = df_temp1[lista_headers_df_codigo_control_versiones_2]
776
777
778 df_temp2 = pd.DataFrame()
779 if isinstance(df_codigo_bbdd_2, pd.DataFrame):
780     df_temp2 = df_codigo_bbdd_2.copy()
```

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

El cálculo del diccionario resultante de la función, comentado en las paginas anteriores varia según el valor del parámetro **check_objeto**.

BLOQUE	check_objeto
1	YA EXISTE
2	SOLO_EN_BBDD_01
3	SOLO_EN_BBDD_02

1

YA_EXISTE: el objeto esta en las 2 bases de datos

Mediante la librería **difflib** se realiza la localización de las líneas con cambios de un script a otro. Previamente se convierten los dataframes de código (se usan los que vienen como parámetros, no los que se copiaron y se agregaron nuevas columnas) en listas usando la columna CODIGO.

```
788 lista_script_1 = df_codigo_bbdd_1["CODIGO"].tolist()
789 lista_script_2 = df_codigo_bbdd_2["CODIGO"].tolist()
790
791 diff = difflib.ndiff(lista_script_1, lista_script_2)
```

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

1

YA_EXISTE: el objeto esta en las 2 bases de datos

Para explicar lo que hace la librería difflib se usa el siguiente ejemplo:

Script BBDD_01

```
01 Sub Config_Usuario()  
02 'se hace mediante conexionEXT y no import pq la tabla  
03 'contiene los password de dichas DDBB  
04  
05 Dim MiConexion As New ADODB.Connection  
06 Dim MiConexionEXT As New ADODB.Connection  
07 Dim MiRecordset1 As New ADODB.Recordset  
08 Dim MiRecordset2 As New ADODB.Recordset  
09 Dim MiRecordset3 As New ADODB.Recordset  
10 Dim MiSQL1 As String, MiSQL2 As String  
11  
12 On Error Resume Next  
13  
14 Set MiConexion = CurrentProject.Connection  
15  
16  
17 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre " & _  
18         & "FROM [T_00_ConfigUsuario]"  
19  
20 MiRecordset1.Open MiSQL1, MiConexion, 1, 2  
21 MiRecordset1.MoveFirst  
22  
23 Public_IDUsuario = MiRecordset1(0)  
24 Public_Usuario_Nom = MiRecordset1(1)  
25 Public_RutaDDBB = CurrentProject.Path & "\BBDD\DDBB.accdb"  
26  
27 MiRecordset1.Close  
28 Set MiRecordset1 = Nothing  
29  
30  
31  
32 MiConexion.Close  
33 Set MiConexion = othing  
34  
35 End Sub
```

Script BBDD_02

```
01 Sub Config_Usuario()  
02 'se hace mediante conexionEXT y no import pq la tabla  
03 'contiene los password de dichas DDBB  
04  
05 Dim MiConexion As New ADODB.Connection  
06 Dim MiConexionEXT As New ADODB.Connection  
07 Dim MiRecordset1 As New ADODB.Recordset  
08 Dim MiRecordset2 As New ADODB.Recordset  
09 Dim MiRecordset3 As New ADODB.Recordset  
10 Dim MiSQL1 As String, MiSQL2 As String  
11  
12  
13 Set MiConexion = CurrentProject.Connection  
14  
15  
16 MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre, Usuario_Telefono " & _  
17         & "[T_00_ConfigUsuario]"  
18  
19 MiRecordset1.Open MiSQL1, MiConexion, 1, 2  
20 MiRecordset1.MoveFirst  
21  
22 Public_ID_Usuario = MiRecordset1(0)  
23 Public_Usuario_Nom = MiRecordset1(1)  
24 Public_Usuario_Telefono = MiRecordset1(2)  
25 Public_RutaDDBB = CurrentProject.Path & "\BBDD\DDBB.accdb"  
26  
27 MiRecordset1.Close  
28 Set MiRecordset1 = Nothing  
29  
30  
31  
32 MiConexion.Close  
33 Set MiConexion = Nothing  
34  
35 End Sub
```

Se marcan en **rojo** las diferencias de un script a otro.

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

1

YA_EXISTE: el objeto esta en las 2 bases de datos

El método **ndiff** de la librería **difflib**, mencionado en las páginas anteriores realiza los siguiente al principio de cada línea de código con cambios de un script a otro:

- Marca un signo menos “-” cuando se localiza un cambio del script 1 con respecto al script 2
- Marca un signo más “+” cuando se localiza un cambio del script 2 con respecto al script 1

Se crean 2 listas temporales **lista_cambios_script_1** y **lista_cambios_script_2** para almacenar respectivamente aquellas líneas de código que empiecen por el signo “-” y por otra parte las que empiecen por el signo “+”. Se crean también los contadores de cambios en cada script (). Mediante bucle sobre estas diferencias generadas con el método **ndiff** se actualizan estas listas y contadores.

```
801 lista_cambios_script_1 = []
802 lista_cambios_script_2 = []
803
804 num_cambios_script_1 = 0
805 num_cambios_script_2 = 0
806
807 for line in diff:
808
809     if len(line.replace(" ", "").replace("\t", "")) != 0:
810
811         if line.startswith('- ') and len(line[2:].replace(" ", "").replace("\t", "")) != 0:
812             lista_cambios_script_1.append(line[2:])
813             num_cambios_script_1 += 1
814
815         elif line.startswith('+ ') and len(line[2:].replace(" ", "").replace("\t", "")) != 0:
816             lista_cambios_script_2.append(line[2:])
817             num_cambios_script_2 += 1
```

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

1

YA_EXISTE: el objeto esta en las 2 bases de datos

El bucle sobre el método **ndiff** mencionado, con el ejemplo anterior, devuelve:

```
Sub Config_Usuario()  
  
    'se hace mediante conexionEXT y no import pq la tabla contiene los password de dichas DDBB  
  
    Dim MiConexion As New ADODB.Connection  
    Dim MiConexionEXT As New ADODB.Connection  
    Dim MiRecordset1 As New ADODB.Recordset  
    Dim MiRecordset2 As New ADODB.Recordset  
    Dim MiRecordset3 As New ADODB.Recordset  
    Dim MiSQL1 As String, MiSQL2 As String  
  
    - On Error Resume Next  
  
    Set MiConexion = CurrentProject.Connection  
  
    - MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre FROM [T_00_ConfigUsuario]"  
    + MiSQL1 = "SELECT Usuario_ID, Usuario_Nombre, Usuario_Telefono FROM [T_00_ConfigUsuario]"  
  
    MiRecordset1.Open MiSQL1, MiConexion, 1, 2  
    MiRecordset1.MoveFirst  
  
    - Public_IDUsuario = MiRecordset1(0)  
    + Public_ID_Usuario_NEW = MiRecordset1(0)  
  
    Public_Usuario_Nom = MiRecordset1(1)  
    + Public_Usuario_Telefono = MiRecordset1(2)  
  
    Public_RutaDDBB = CurrentProject.Path & "\BBDD\DDBB.accdb"  
  
    MiRecordset1.Close  
    Set MiRecordset1 = Nothing  
  
    MiConexion.Close  
    Set MiConexion = Nothing  
  
End Sub
```

Líneas con cambios en el script 1

Líneas con cambios en el script 2

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

1

YA_EXISTE: el objeto esta en las 2 bases de datos

El siguiente paso consiste en calcular los **números de línea de códigos** donde hay cambios en los 2 scripts para poder actualizar en los dataframes que se copiaron de los parámetros de la función y se les agregaron columnas adicionales. Las columnas que se actualizan son CONTROL_CAMBIOS_ORIGINAL y CONTROL_CAMBIOS_ACTUAL ambas con el valor "CAMBIOS LOCALIZADOS".

Los números de línea con cambios en los 2 scripts se almacenan en las listas temporales **lista_cambios_lineas_script_1** y **lista_cambios_lineas_script_2**.

```
821 lista_cambios_lineas_script_1 = []
822 if len(lista_cambios_script_1) != 0:
823
824     lista_codigo_con_num_linea = [[ind + 1, item] for ind, item in enumerate(lista_script_1)]
825
826     for linea_cambio in lista_cambios_script_1:
827         for num_linea, linea_codigo in lista_codigo_con_num_linea:
828
829             if linea_cambio.strip() == linea_codigo.strip():
830                 lista_cambios_lineas_script_1.append(num_linea)
831
832                 break
```

```
835 for ind in df_temp1.index:
836     num_linea = ind + 1
837
838     for item in lista_cambios_lineas_script_1:
839         if num_linea == item:
840             df_temp1.iloc[ind, df_temp1.columns.get_loc("CONTROL_CAMBIOS_ORIGINAL")] = "CAMBIOS_LOCALIZADOS"
841             df_temp1.iloc[ind, df_temp1.columns.get_loc("CONTROL_CAMBIOS_ACTUAL")] = "CAMBIOS_LOCALIZADOS"
842             break
```

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

1

YA_EXISTE: el objeto esta en las 2 bases de datos

```
846 lista_cambios_lineas_script_2 = []
847 if len(lista_cambios_script_2) != 0:
848
849     lista_codigo_con_num_linea = [[ind + 1, item] for ind, item in enumerate(lista_script_2)]
850
851     for linea_cambio in lista_cambios_script_2:
852         for num_linea, linea_codigo in lista_codigo_con_num_linea:
853
854             if linea_cambio.strip() == linea_codigo.strip():
855                 lista_cambios_lineas_script_2.append(num_linea)
856
857                 break
```

```
860         for ind in df_temp2.index:
861             num_linea = ind + 1
862
863             for item in lista_cambios_lineas_script_2:
864                 if num_linea == item:
865                     df_temp2.iloc[ind, df_temp2.columns.get_loc("CONTROL_CAMBIOS_ORIGINAL")] = "CAMBIOS_LOCALIZADOS"
866                     df_temp2.iloc[ind, df_temp2.columns.get_loc("CONTROL_CAMBIOS_ACTUAL")] = "CAMBIOS_LOCALIZADOS"
867                     break
```

En la GUI de control de versiones, la columna **CONTROL_CAMBIOS_ORIGINAL** permite actualizar los scripts en pantalla con los colores **verdes** de cambios cuando en el apartado MERGE se opta por revertir todos los cambios o revertir el último cambio cuando el usuario ha realizado un solo cambio. Asimismo, la columna **CONTROL_CAMBIOS_ACTUAL**, en la misma GUI, permite guardar los cambios realizados por el usuario (migrar de BBDD_01 a BBDD_02 o quitar en BBDD_02).

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

1

YA_EXISTE: el objeto esta en las 2 bases de datos

Como ultimo paso, es crear el diccionario que sirve de resultado de la función. Se crea la variable temporal **hay_diferencias** para poder actualizar a None las distintas keys del diccionario cuando no se localizan diferencias entre los 2 scripts.

```
871 hay_diferencias = "SI" if len(lista_cambios_script_1) + len(lista_cambios_script_2) != 0 else "NO"
872
873 dicc_temp = {"TIPO_BBDD": tipo_bbdd
874             , "CHECK_OBJETO": check_objeto if hay_diferencias == "SI" else None
875             , "TIPO_OBJETO": tipo_objeto if hay_diferencias == "SI" else None
876             , "TIPO_OBJETO_SUBFORM": tipo_objeto_subform if hay_diferencias == "SI" else None
877             , "TIPO_REPOSITORIO": tipo_repositorio if hay_diferencias == "SI" else None
878             , "REPOSITORIO": repositorio if hay_diferencias == "SI" else None
879             , "NOMBRE_OBJETO": nombre_objeto if hay_diferencias == "SI" else None
880             , "NUM_CAMBIOS_SCRIPT_1": num_cambios_script_1 if hay_diferencias == "SI" else None
881             , "NUM_CAMBIOS_SCRIPT_2": num_cambios_script_2 if hay_diferencias == "SI" else None
882             , "DF_CODIGO_ORIGINAL_1": df_temp1 if hay_diferencias == "SI" else None
883             , "DF_CODIGO_ORIGINAL_2": df_temp2 if hay_diferencias == "SI" else None
884             , "LISTA_DICC_MERGE_HECHOS": None #se usa para saber que merge se han hecho (aqui es None al iniciar el app)
885             , "DF_CODIGO_ACTUAL_1": df_temp1 if hay_diferencias == "SI" else None
886             , "DF_CODIGO_ACTUAL_2": df_temp2 if hay_diferencias == "SI" else None
887             }
888
889
890 #resultado de la funcion (devuelve el diccionario dicc_temp si al menos uno de los valores asociados a las keys es distinto de None sino devuelve None)
891 return dicc_temp if sum(1 if isinstance(dicc_temp[key], pd.DataFrame) else 1 if dicc_temp[key] != None else 0 for key in dicc_temp.keys()) != 0 else None
```

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

2

SOLO_EN_BBDD_01: el objeto solo esta en BBDD_01

Aquí no es necesario usar la librería **difflib** porque al no haber script en la BBDD_02 todo se marca como “CAMBIOS LOCALIZADOS” en las columnas CONTROL_CAMBIOS_ORIGINAL y CONTROL_CAMBIOS_ACTUAL en el dataframe del script 1 que se copio de los parámetros.

```
896 elif check_objeto == dicc_control_versiones_tipo_concepto["SOLO_EN_BBDD_01"]:
897
898     num_cambios_script_1 = len(df_temp1)
899     df_temp1[["CONTROL_CAMBIOS_ORIGINAL", "CONTROL_CAMBIOS_ACTUAL"]] = ["CAMBIOS_LOCALIZADOS", "CAMBIOS_LOCALIZADOS"]
900
901
902     dicc_temp = {
903         "TIPO_BBDD": tipo_bbdd
904         , "CHECK_OBJETO": check_objeto
905         , "TIPO_OBJETO": tipo_objeto
906         , "TIPO_OBJETO_SUBFORM": tipo_objeto_subform
907         , "TIPO_REPOSITORIO": tipo_repositorio
908         , "REPOSITORIO": repositorio
909         , "NOMBRE_OBJETO": nombre_objeto
910         , "NUM_CAMBIOS_SCRIPT_1": num_cambios_script_1
911         , "NUM_CAMBIOS_SCRIPT_2": 0
912         , "DF_CODIGO_ORIGINAL_1": df_temp1
913         , "DF_CODIGO_ORIGINAL_2": None
914         , "LISTA_DICC_MERGE_HECHOS": None #se usa para saber que merge se han hecho (aquí es None al iniciar el app)
915         , "DF_CODIGO_ACTUAL_1": df_temp1
916         , "DF_CODIGO_ACTUAL_2": None
917     }
918
919     #resultado de la funcion (devuelve el diccionario dicc_temp si al menos uno de los valores asociados a las keys es distinto de None sino devuelve None)
920     return dicc_temp if sum(1 if isinstance(dicc_temp[key], pd.DataFrame) else 1 if dicc_temp[key] != None else 0 for key in dicc_temp.keys()) != 0 else None
```

ANEXO_03

FUNCIÓN: func_dicc_control_versiones

3

SOLO_EN_BBDD_02: el objeto solo esta en BBDD_02

Al igual que en la página anterior, aquí tampoco es necesario usar la librería **difflib** porque al no haber script en la BBDD_01 todo se marca como “CAMBIOS LOCALIZADOS” en las columnas CONTROL_CAMBIOS_ORIGINAL y CONTROL_CAMBIOS_ACTUAL en el dataframe del script 2 que se copio de los parámetros.

```
924 elif check_objeto == dicc_control_versiones_tipo_concepto["SOLO_EN_BBDD_02"]:
925
926     num_cambios_script_2 = len(df_temp2)
927     df_temp2[["CONTROL_CAMBIOS_ORIGINAL", "CONTROL_CAMBIOS_ACTUAL"]] = ["CAMBIOS_LOCALIZADOS", "CAMBIOS_LOCALIZADOS"]
928
929
930     dicc_temp = {"TIPO_BBDD": tipo_bbdd
931                 , "CHECK_OBJETO": check_objeto
932                 , "TIPO_OBJETO": tipo_objeto
933                 , "TIPO_OBJETO_SUBFORM": tipo_objeto_subform
934                 , "TIPO_REPOSITORIO": tipo_repositorio
935                 , "REPOSITORIO": repositorio
936                 , "NOMBRE_OBJETO": nombre_objeto
937                 , "NUM_CAMBIOS_SCRIPT_1": 0
938                 , "NUM_CAMBIOS_SCRIPT_2": num_cambios_script_2
939                 , "DF_CODIGO_ORIGINAL_1": None
940                 , "DF_CODIGO_ORIGINAL_2": df_temp2
941                 , "LISTA_DICC_MERGE_HECHOS": None          #se usa para saber que merge se han hecho (aquí es None al iniciar el app)
942                 , "DF_CODIGO_ACTUAL_1": None
943                 , "DF_CODIGO_ACTUAL_2": df_temp2
944                 }
945
946     #resultado de la funcion (devuelve el diccionario dicc_temp si al menos uno de los valores asociados a las keys es distinto de None sino devuelve None)
947     return dicc_temp if sum(1 if isinstance(dicc_temp[key], pd.DataFrame) else 1 if dicc_temp[key] != None else 0 for key in dicc_temp.keys()) != 0 else None
```

ANEXO_04

RUTINA

def_proceso_access_2_control_versiones

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

La rutina permite realizar el cálculo del control de versiones en MS ACCESS

La rutina nada más empezar se encapsula en un bloque de flujos de errores (**try except**).

El bloque **except** permite almacenar en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL), en la subkey_3 (LISTA_DICC_ERRORES_CALCULO) asociada a las key_1 (PROCESO_01) y subkey_2 (MS_ACCESS), los sucesivos errores de ejecución, en caso de que los hubiese, lo que permite generar en otra rutina logs en formatos .txt.

El 1er paso es recuperar los datos necesarios para BBDD_01 y BBDD_02 del diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) para el proceso de control de versiones.

```
2594 #se recuperan los datos necesarios del diccionario dicc_codigos_bbdd
2595 lista_dicc_objetos_bbdd_1 = mod_gen.dicc_codigos_bbdd["BBDD_01"]["MS_ACCESS"]["LISTA_DICC_OBJETOS"]
2596 lista_dicc_objetos_bbdd_2 = mod_gen.dicc_codigos_bbdd["BBDD_02"]["MS_ACCESS"]["LISTA_DICC_OBJETOS"]
2597
2598 df_bbdd_codigo_1 = mod_gen.dicc_codigos_bbdd["BBDD_01"]["MS_ACCESS"]["DF_CODIGO_CALCULADO_TRAS_IMPORT"]
2599 df_bbdd_codigo_2 = mod_gen.dicc_codigos_bbdd["BBDD_02"]["MS_ACCESS"]["DF_CODIGO_CALCULADO_TRAS_IMPORT"]
```

Antes de proceder a calcular el control de versiones hay que realizar unos ajustes previos para las variables públicas VBA. Las listas de diccionarios **lista_dicc_objetos_bbdd_1** y **lista_dicc_objetos_bbdd_2**, listan estas variables 1 a 1 (cada diccionario que las componen es un objeto).

El control de versiones para las variables públicas VBA se realiza a nivel de módulo no de variable pública por lo que es necesario reconstruir las listas **lista_dicc_objetos_bbdd_1** y **lista_dicc_objetos_bbdd_2**.

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

TIPO BBDD	TIPO OBJETO	REPOSITORIO
MS_ACCESS	Variables VBA	Mod_00_Config

BBDD_01	PRUEBA_CONTROL_VERSIONES_1.accdB	BBDD_02	PRUEBA_CONTROL_VERSIONES_2.accdB
0001	Public Public_Version As String	0001	Public Public_RutaDB As String, aa As String, bb As Long
0002	Public Public_UsuarioID As String, Public_UsuarioNombre As String, Public_Array(1 To 10, 2	0002	Public Public_IDUsuario As String, Public_Usuario_FechaCreacion As Date
0003	Public Const Public_Nombre_Access = "BBDD"	0003	Public Public_Usuario_Nom As String
0004	Type UserType Prueba_1	0004	Public Public_Usuario_Password As String
0005	SubVar_1 As String	0005	Public Public_Usuario_Telef As String
0006	SubVar_2 As Long	0006	Public Public_Usuario_Email As String
0007	End Type		
0008	Enum Enum_Prueba_1		
0009	SubVar_1		
0010	SubVar_2 = 1		
0011	SubVar_3		
0012	End Enum		
0013	'datos varios para el usuario		
0014	Public Public_IDUsuario_OLD As String		
0015	Public Public_Usuario_Nom As String		
0016	Public Public_Usuario_NomComp As String		
0017	Public Public_Usuario_Password As String		
0018	Public Public_Usuario_Telef As String		
0019	Public Public_Usuario_Email As String		

El control de versiones para las variables públicas VBA se realiza a nivel de módulo no de variable pública por lo que es necesario reconstruir las listas **lista_dicc_objetos_bbdd_1** y **lista_dicc_objetos_bbdd_2**.

Los pasos a seguir para reconstruir estas listas temporales son:

PASO	DESCRIPCIÓN
1	mediante el código almacenado en el diccionario dicc_codigos_bbdd (módulo APP_2_GENERAL) se extraen los tipos y nombre de módulos donde se han localizado variables publicas para el control de versiones (columna ES_VARIABLE_PUBLICA_CONTROL_VERSIONES = SI) y se almacenan estos tipos y nombres de módulos en las listas temporales lista_temp_1 y lista_temp_2 a las cuales se les quita los duplicados
2	mediante bucle sobre lista_temp_1 y lista_temp_2 se reconstruyen las listas lista_dicc_objetos_bbdd_1 y lista_dicc_objetos_bbdd_2 solo para VARIABLES_VBA , en cada iteracion se extrae de los dataframes de código (donde columna ES_VARIABLE_PUBLICA_CONTROL_VERSIONES = SI y donde el tipo y nombre del módulo del df de código corresponde con los de la iteración), en cada diccionario que se agrega a estas listas reconstruidas tan solo se informa las keys TIPO_OBJETO , TIPO_MODULO , NOMBRE_MODULO y DF_CODIGO
3	las listas definitivas lista_dicc_objetos_bbdd_1 y lista_dicc_objetos_bbdd_2 son la fusión entre las listas originales (sin VARIABLES_VBA) y de estas listas reconstruidas

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

1

```
2626 #se extraen los codigos por tipo y nombre de modulo de los df bdd_codigo_1 y df bdd_codigo_2
2627 df_temp_1 = df_bdd_codigo_1.loc[df_bdd_codigo_1["ES_VARIABLE_PUBLICA_CONTROL_VERSIONES"] == "SI", ["TIPO_MODULO", "NOMBRE_MODULO", "CODIGO"]]
2628 df_temp_2 = df_bdd_codigo_2.loc[df_bdd_codigo_2["ES_VARIABLE_PUBLICA_CONTROL_VERSIONES"] == "SI", ["TIPO_MODULO", "NOMBRE_MODULO", "CODIGO"]]
2629
2630 df_temp_1.reset_index(drop = True, inplace = True)
2631 df_temp_2.reset_index(drop = True, inplace = True)
2632
2633
2634 #se listan los tipos y nombres de modulos en cada base de datos y se quitan los duplicados
2635 lista_temp_1 = []
2636 lista_temp_2 = []
2637
2638 if len(df_temp_1) != 0:
2639     lista_temp_1 = [[df_temp_1.iloc[ind, df_temp_1.columns.get_loc("TIPO_MODULO")],
2640                     df_temp_1.iloc[ind, df_temp_1.columns.get_loc("NOMBRE_MODULO")]] for ind in df_temp_1.index]
2641
2642     lista_temp_1 = [sublista for i, sublista in enumerate(lista_temp_1) if sublista not in lista_temp_1[:i]]
2643
2644 if len(df_temp_2) != 0:
2645     lista_temp_2 = [[df_temp_2.iloc[ind, df_temp_2.columns.get_loc("TIPO_MODULO")],
2646                     df_temp_2.iloc[ind, df_temp_2.columns.get_loc("NOMBRE_MODULO")]] for ind in df_temp_2.index]
2647
2648     lista_temp_2 = [sublista for i, sublista in enumerate(lista_temp_2) if sublista not in lista_temp_2[:i]]
2649
```

2

```
2653 lista_dicc_objetos_variables_publicas_bbdd_1 = []
2654 if len(lista_temp_1) != 0:
2655
2656     for tipo_modulo, nombre_modulo in lista_temp_1:
2657
2658         df_codigo = df_temp_1.loc[(df_temp_1["TIPO_MODULO"] == tipo_modulo) & (df_temp_1["NOMBRE_MODULO"] == nombre_modulo), ["CODIGO"]]
2659         df_codigo.reset_index(drop = True, inplace = True)
2660
2661         df_codigo = df_codigo if len(df_codigo) != 0 else None
2662
2663
2664         dicc_temp = {"TIPO_OBJETO": "VARIABLES_VBA"
2665                     , "TIPO_MODULO": tipo_modulo
2666                     , "NOMBRE_MODULO": nombre_modulo
2667                     , "NOMBRE_OBJETO": None #key que no aplica aqui
2668                     , "TIPO Rutina": None #key que no aplica aqui
2669                     , "TIPO_DECLARACION_Rutina": None #key que no aplica aqui
2670                     , "PARAMETROS_Rutina": None #key que no aplica aqui
2671                     , "TIPO_VARIABLE_PUBLICA": None #key que no aplica aqui
2672                     , "TIPO_DATO_VARIABLE_PUBLICA": None #key que no aplica aqui
2673                     , "VARIABLE_PUBLICA_ES_CONSTANTE": None #key que no aplica aqui
2674                     , "VARIABLE_PUBLICA_CONSTANTE_VALOR": None #key que no aplica aqui
2675                     , "VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES": None #key que no aplica aqui
2676                     , "DF_CODIGO": df_codigo
2677                     }
2678
2679         lista_dicc_objetos_variables_publicas_bbdd_1.append(dicc_temp)
```

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

2

```
2685 lista_dicc_objetos_variables_publicas_bbdd_2 = []
2686 if len(lista_temp_2) != 0:
2687
2688     for tipo_modulo, nombre_modulo in lista_temp_2:
2689
2690         df_codigo = df_temp_2.loc[(df_temp_2["TIPO_MODULO"] == tipo_modulo) & (df_temp_2["NOMBRE_MODULO"] == nombre_modulo), ["CODIGO"]]
2691         df_codigo.reset_index(drop = True, inplace = True)
2692
2693         df_codigo = df_codigo if len(df_codigo) != 0 else None
2694
2695
2696         dicc_temp = {"TIPO_OBJETO": "VARIABLES_VBA"
2697                     , "TIPO_MODULO": tipo_modulo
2698                     , "NOMBRE_MODULO": nombre_modulo
2699                     , "NOMBRE_OBJETO": None                    #key que no aplica aqui
2700                     , "TIPO Rutina": None                    #key que no aplica aqui
2701                     , "TIPO_DECLARACION_Rutina": None        #key que no aplica aqui
2702                     , "PARAMETROS_Rutina": None              #key que no aplica aqui
2703                     , "TIPO_VARIABLE_PUBLICA": None          #key que no aplica aqui
2704                     , "TIPO_DATO_VARIABLE_PUBLICA": None     #key que no aplica aqui
2705                     , "VARIABLE_PUBLICA_ES_CONSTANTE": None  #key que no aplica aqui
2706                     , "VARIABLE_PUBLICA_CONSTANTE_VALOR": None #key que no aplica aqui
2707                     , "VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLEES": None #key que no aplica aqui
2708                     , "DF_CODIGO": df_codigo
2709                     }
2710
2711         lista_dicc_objetos_variables_publicas_bbdd_2.append(dicc_temp)
```

3

```
2725 lista_dicc_objetos_bbdd_1 = [dicc for dicc in lista_dicc_objetos_bbdd_1 if dicc["TIPO_OBJETO"] != "VARIABLES_VBA"] + lista_dicc_objetos_variables_publicas_bbdd_1
2726 lista_dicc_objetos_bbdd_2 = [dicc for dicc in lista_dicc_objetos_bbdd_2 if dicc["TIPO_OBJETO"] != "VARIABLES_VBA"] + lista_dicc_objetos_variables_publicas_bbdd_2
```

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

Tras estos ajustes previos, ahora se pasa a calcular el control de versiones entre BBDD_01 y BBDD_02.

El siguiente paso es crear las listas de temporales **lista_objetos_conso_1** y **lista_objetos_conso_2** usando las listas **lista_dicc_objetos_bbdd_1** y **lista_dicc_objetos_bbdd_2** reconstruidas. Son listas de listas donde cada sublista tiene la estructura siguiente:

INDICE SUBLISTA	DESCRIPCIÓN
0	Tipo objeto
1	Tipo módulo
2	Nombre módulo
3	Nombre objeto
4	indicador de si el objeto esta en BBDD_01 (se establece a 0 de inicio)
5	indicador de si el objeto esta en BBDD_02 (se establece a 0 de inicio)
6	dataframe con el código del objeto en BBDD_01 (se establece a None de inicio)
7	dataframe con el código del objeto en BBDD_02 (se establece a None de inicio)

```
2756 lista_objetos_conso_1 = [[dicc["TIPO_OBJETO"]
2757 , dicc["TIPO_MODULO"]
2758 , dicc["NOMBRE_MODULO"]
2759 , dicc["NOMBRE_OBJETO"]
2760 , 0
2761 , 0
2762 , None
2763 , None] for dicc in lista_dicc_objetos_bbdd_1]
2764
2765 lista_objetos_conso_2 = [[dicc["TIPO_OBJETO"]
2766 , dicc["TIPO_MODULO"]
2767 , dicc["NOMBRE_MODULO"]
2768 , dicc["NOMBRE_OBJETO"]
2769 , 0
2770 , 0
2771 , None
2772 , None] for dicc in lista_dicc_objetos_bbdd_2]
```

Se fusionan **lista_objetos_conso_1** y **lista_objetos_conso_2** en la lista temporal **lista_objetos_conso** a la cual se le quitan los duplicados. Esta lista permite disponer de todos los objetos tanto de BBDD_01 como de BBDD_02.

```
2774 lista_objetos_conso = lista_objetos_conso_1 + lista_objetos_conso_2
2775 lista_objetos_conso = [sublista for i, sublista in enumerate(lista_objetos_conso) if sublista not in lista_objetos_conso[:i]]
```

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

El siguiente paso es calcular en **lista_objetos_conso** los ítems de cada una de sus sublistas asociados a los índices 4 a 7. Se realiza mediante bucle sobre **lista_objetos_conso**.

En cada iteración sobre este bucle se busca el objeto tanto en **lista_objetos_conso_1** como en **lista_objetos_conso_2**:

- Si hay cacheo para BBDD_01, se informan los ítems con índices de sublista 4 y 6.
- Si hay cacheo para BBDD_02, se informan los ítems con índices de sublista 5 y 7.

El cacheo varia según el tipo de objeto

TIPO OBJETO	CACHEO			
	TIPO OBJETO	TIPO MODULO	NOMBRE MODULO	NOMBRE OBJETO
TABLA_LOCAL	X			X
VINCULO_ODBC	X			X
VINCULO_OTRO	X			X
RUTINAS_VBA	X	X	X	X
VARIABLES_VBA	X	X	X	

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

```
2786 for indice_lista, item in enumerate(lista_objetos_conso):
2787
2788     tipo_objeto_conso = item[indice_tipo_objeto]
2789     tipo_modulo_conso = item[indice_tipo_modulo]
2790     nombre_modulo_conso = item[indice_nombre_modulo]
2791     nombre_objeto_conso = item[indice_nombre_objeto]
```

BBDD_01



```
2794 #BBDD_01 (se informan los indices 4 y 6)
2795 cont_1 = 0
2796 for dicc in lista_dicc_objetos_bbdd_1:
2797
2798     tipo_objeto_seek = dicc["TIPO_OBJETO"]
2799     tipo_modulo_seek = dicc["TIPO_MODULO"]
2800     nombre_modulo_seek = dicc["NOMBRE_MODULO"]
2801     nombre_objeto_seek = dicc["NOMBRE_OBJETO"]
2802     df_codigo_seek = dicc["DF_CODIGO"]
2803
2804     if tipo_objeto_conso in ["TABLA_LOCAL", "VINCULO_ODBC", "VINCULO_OTRO"]:
2805         if tipo_objeto_conso == tipo_objeto_seek and nombre_objeto_conso == nombre_objeto_seek:
2806             cont_1 = 1
2807             break
2808
2809     elif tipo_objeto_conso == "RUTINAS_VBA":
2810         if tipo_objeto_conso == tipo_objeto_seek and tipo_modulo_conso == tipo_modulo_seek and nombre_modulo_conso == nombre_modulo_seek and nombre_objeto_conso == nombre_objeto_seek:
2811             cont_1 = 1
2812             break
2813
2814     elif tipo_objeto_conso == "VARIABLES_VBA":
2815         if tipo_objeto_conso == tipo_objeto_seek and tipo_modulo_conso == tipo_modulo_seek and nombre_modulo_conso == nombre_modulo_seek:
2816             cont_1 = 1
2817             break
2818
2819     if cont_1 == 1:
2820         lista_objetos_conso[indice_lista][indice_esta_en_bbdd_1] = 1
2821         lista_objetos_conso[indice_lista][indice_df_codigo_1] = df_codigo_seek
```

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

```
2375     for indice_lista, item in enumerate(lista_objetos_conso):
2376
2377         tipo_objeto_conso = item[indice_tipo_objeto]
2378         tipo_modulo_conso = item[indice_tipo_modulo]
2379         nombre_modulo_conso = item[indice_nombre_modulo]
2380         nombre_objeto_conso = item[indice_nombre_objeto]
```

BBDD_02



```
2825     #BBDD_02 (se informan los indices 5 y 7)
2826     cont_2 = 0
2827     for dicc in lista_dicc_objetos_bbdd_2:
2828
2829         tipo_objeto_seek = dicc["TIPO_OBJETO"]
2830         tipo_modulo_seek = dicc["TIPO_MODULO"]
2831         nombre_modulo_seek = dicc["NOMBRE_MODULO"]
2832         nombre_objeto_seek = dicc["NOMBRE_OBJETO"]
2833         df_codigo_seek = dicc["DF_CODIGO"]
2834
2835         if tipo_objeto_conso in ["TABLA_LOCAL", "VINCULO_ODBC", "VINCULO_OTRO"]:
2836             if tipo_objeto_conso == tipo_objeto_seek and nombre_objeto_conso == nombre_objeto_seek:
2837                 cont_2 = 1
2838                 break
2839
2840         elif tipo_objeto_conso == "RUTINAS_VBA":
2841             if tipo_objeto_conso == tipo_objeto_seek and tipo_modulo_conso == tipo_modulo_seek and nombre_modulo_conso == nombre_modulo_seek and nombre_objeto_conso == nombre_objeto_seek:
2842                 cont_2 = 1
2843                 break
2844
2845         elif tipo_objeto_conso == "VARIABLES_VBA":
2846             if tipo_objeto_conso == tipo_objeto_seek and tipo_modulo_conso == tipo_modulo_seek and nombre_modulo_conso == nombre_modulo_seek:
2847                 cont_2 = 1
2848                 break
2849
2850     if cont_2 == 1:
2851         lista_objetos_conso[indice_lista][indice_esta_en_bbdd_2] = 1
2852         lista_objetos_conso[indice_lista][indice_df_codigo_2] = df_codigo_seek
```


ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

Tras finalizar el bucle sobre **lista_objetos_conso** se pasa a calcular, se realiza otro bucle de nuevo sobre **lista_objetos_conso** para generar las listas de diccionarios siguientes:

```
2857 lista_control_versiones_tablas_access = []
2858 lista_control_versiones_vinculos_odbc = []
2859 lista_control_versiones_vinculos_otros = []
2860 lista_control_versiones_rutinas_vba = []
2861 lista_control_versiones_variables_vba = []
```

En cada iteración de este bucle:

- Se localiza si el objeto esta en las 2 bases de datos o solo en 1 y cual (se realiza mediante comparativa de los índices de sublistas 4 y 5)
- Se calcula el control de versiones usando la función **func_dicc_control_versiones** (módulo APP_2_GENERAL) pasándole como parámetros, entre otros, los dataframes almacenados en los índices de sublistas 6 y 7).

```
2863 for tipo_objeto_conso, tipo_modulo_conso, nombre_modulo_conso, nombre_objeto_conso, en_bbdd_1_conso, en_bbdd_2_conso, df_codigo_1_conso, df_codigo_2_conso in lista_objetos_conso:
2864
2865     #segun se localice si el objeto esta en las 2 bbdd o solo en 1 se le asigna el valor correspondiente de la key
2866     #del diccionario dicc_control_versiones_tipo_concepto
2867     if en_bbdd_1_conso == 1 and en_bbdd_2_conso == 1:
2868         check_objeto = mod_gen.dicc_control_versiones_tipo_concepto["YA_EXISTE"]
2869
2870     elif en_bbdd_1_conso == 1 and en_bbdd_2_conso == 0:
2871         check_objeto = mod_gen.dicc_control_versiones_tipo_concepto["SOLO_EN_BBDD_01"]
2872
2873     elif en_bbdd_1_conso == 0 and en_bbdd_2_conso == 1:
2874         check_objeto = mod_gen.dicc_control_versiones_tipo_concepto["SOLO_EN_BBDD_02"]
```

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

```
2863 for tipo_objeto_conso, tipo_modulo_conso, nombre_modulo_conso, nombre_objeto_conso, en_bbdd_1_conso, en_bbdd_2_conso, df_codigo_1_conso, df_codigo_2_conso in lista_objetos_conso:
```

```
2879     if tipo_objeto_conso in ["TABLA_LOCAL", "VINCULO_ODBC", "VINCULO_OTRO"]:  
2880  
2881         dicc_temp = mod_gen.func_dicc_control_versiones(tipo_bbdd = "MS_ACCESS"  
2882                                                         , check_objeto = check_objeto  
2883                                                         , tipo_objeto = tipo_objeto_conso  
2884                                                         , nombre_objeto = nombre_objeto_conso  
2885                                                         , df_codigo_bbdd_1 = df_codigo_1_conso  
2886                                                         , df_codigo_bbdd_2 = df_codigo_2_conso  
2887                                                         )  
2888  
2889         if isinstance(dicc_temp, dict):  
2890             if tipo_objeto_conso == "TABLA_LOCAL":  
2891                 lista_control_versiones_tablas_access.append(dicc_temp)  
2892  
2893             elif tipo_objeto_conso == "VINCULO_ODBC":  
2894                 lista_control_versiones_vinculos_odbc.append(dicc_temp)  
2895  
2896             elif tipo_objeto_conso == "VINCULO_OTRO":  
2897                 lista_control_versiones_vinculos_otros.append(dicc_temp)
```

```
2901     elif tipo_objeto_conso == "RUTINAS_VBA":  
2902  
2903         dicc_temp = mod_gen.func_dicc_control_versiones(tipo_bbdd = "MS_ACCESS"  
2904                                                         , check_objeto = check_objeto  
2905                                                         , tipo_objeto = tipo_objeto_conso  
2906                                                         , tipo_repositorio = tipo_modulo_conso  
2907                                                         , repositorio = nombre_modulo_conso  
2908                                                         , nombre_objeto = nombre_objeto_conso  
2909                                                         , df_codigo_bbdd_1 = df_codigo_1_conso  
2910                                                         , df_codigo_bbdd_2 = df_codigo_2_conso  
2911                                                         )  
2912  
2913         if isinstance(dicc_temp, dict):  
2914             lista_control_versiones_rutinas_vba.append(dicc_temp)
```

```
2918     elif tipo_objeto_conso == "VARIABLES_VBA":  
2919  
2920         dicc_temp = mod_gen.func_dicc_control_versiones(tipo_bbdd = "MS_ACCESS"  
2921                                                         , check_objeto = check_objeto  
2922                                                         , tipo_objeto = tipo_objeto_conso  
2923                                                         , tipo_repositorio = tipo_modulo_conso  
2924                                                         , repositorio = nombre_modulo_conso  
2925                                                         , df_codigo_bbdd_1 = df_codigo_1_conso  
2926                                                         , df_codigo_bbdd_2 = df_codigo_2_conso  
2927                                                         )  
2928  
2929         if isinstance(dicc_temp, dict):  
2930             lista_control_versiones_variables_vba.append(dicc_temp)
```

Según el tipo de objeto, los parámetros kwargs de la función `func_dicc_control_versiones` varían

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

Tras finalizar este último bucle se pasa al almacenar las listas de diccionarios en el diccionario **dicc_control_versiones_tipo_objeto** en las subkeys_3 (van por tipo de objeto) asociadas a la key_1 (MS_ACCESS) y subkey_2 (TIPO_OBJETO).

```
2939 mod_gen.dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"]["TABLA_LOCAL"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_tablas_access if len(lista_c
2940 mod_gen.dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"]["VINCULO_ODBC"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_vinculos_odbc if len(lista_c
2941 mod_gen.dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"]["VINCULO_OTRO"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_vinculos_otros if len(lista
2942 mod_gen.dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"]["RUTINAS_VBA"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_rutinas_vba if len(lista_con
2943 mod_gen.dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"]["VARIABLES_VBA"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_variables_vba if len(lista
```

Por último, se informa la subkey_3 (TODOS) del diccionario **dicc_control_versiones_tipo_objeto** mediante bucle sobre las 5 subkeys_3 anteriores.

```
2954 lista_tipo_objetos = list(mod_gen.dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"].keys())
2955
2956 lista_control_versiones_todo = []
2957 for tipo_objeto in lista_tipo_objetos:
2958     if tipo_objeto != "TODOS":
2959
2960         try:
2961             for dicc in mod_gen.dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"][tipo_objeto]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"]:
2962                 lista_control_versiones_todo.append(dicc)
2963
2964         except:#puede haber tipo de objetos que no tengan cambios
2965             pass
2966
2967 mod_gen.dicc_control_versiones_tipo_objeto["MS_ACCESS"]["TIPO_OBJETO"]["TODOS"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_todo if len(lista_control_versiones_todo) != 0 else None
```

ANEXO_04

RUTINA: def_proceso_access_2_control_versiones

Por último, se crea el bloque **except** para almacenar los posibles errores de ejecución de la rutina en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_CALCULO) asociada a la key_1 (PROCESO_01) y subkey_2 (MS_ACCESS).

```
2984     except Exception as Err:
2985
2986         traceback_error = traceback.extract_tb(Err.__traceback__)
2987         modulo_python = os.path.basename(traceback_error[0].filename)
2988         rutina_python = traceback_error[0].name
2989         linea_error = traceback_error[0].lineno
2990
2991         lista_dicc_errores_migracion = mod_gen.dicc_errores_procesos["PROCESO_01"]["MS_ACCESS"]["LISTA_DICC_ERRORES_CALCULO"]
2992
2993         dicc_errores_temp = {"TIPO_BBDD": "MS_ACCESS"
2994                             , "MODULO_PYTHON": modulo_python
2995                             , "RUTINA_PYTHON": rutina_python
2996                             , "LINEA_ERROR": linea_error
2997                             , "ERRORES": str(Err)
2998                             }
2999
3000         if isinstance(lista_dicc_errores_migracion, list):
3001             lista_dicc_errores_migracion.append(dicc_errores_temp)
3002         else:
3003             lista_dicc_errores_migracion = [dicc_errores_temp]
3004
3005         mod_gen.dicc_errores_procesos["PROCESO_01"]["MS_ACCESS"]["LISTA_DICC_ERRORES_CALCULO"] = lista_dicc_errores_migracion
3006         del dicc_errores_temp
3007         del lista_dicc_errores_migracion
3008         pass #es pass para que se localicen todos los posibles errores en el proceso
```

ANEXO 05

RUTINA

def_proceso_sql_server_2_control_versiones

ANEXO_05

RUTINA: def_proceso_sql_server_2_control_versiones

La rutina permite realizar el cálculo del control de versiones en SQL SERVER

La rutina nada más empezar se encapsula en un bloque de flujos de errores (**try except**).

El bloque **except** permite almacenar en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL), en la subkey_3 (LISTA_DICC_ERRORES_CALCULO) asociada a las key_1 (PROCESO_01) y subkey_2 (SQL SERVER), los sucesivos errores de ejecución, en caso de que los hubiese, lo que permite generar en otra rutina logs en formatos .txt.

El 1er paso es recuperar los datos necesarios para BBDD_01 y BBDD_02 del diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) para el proceso de control de versiones.

```
637 lista_dicc_objetos_bbdd_1 = mod_gen.dicc_codigos_bbdd["BBDD_01"]["SQL_SERVER"]["LISTA_DICC_OBJETOS"]
638 lista_dicc_objetos_bbdd_2 = mod_gen.dicc_codigos_bbdd["BBDD_02"]["SQL_SERVER"]["LISTA_DICC_OBJETOS"]
```

ANEXO_05

RUTINA: def_proceso_sql_server_2_control_versiones

Tras estos ajustes previos, ahora se pasa a calcular el control de versiones entre BBDD_01 y BBDD_02.

El siguiente paso es crear las listas de temporales **lista_objetos_conso_1** y **lista_objetos_conso_2** usando las listas **lista_dicc_objetos_bbdd_1** y **lista_dicc_objetos_bbdd_2** reconstruidas. Son listas de listas donde cada sublista tiene la estructura siguiente:

INDICE SUBLISTA	DESCRIPCIÓN
0	Tipo objeto
1	Nombre repositorio
2	Nombre objeto
3	indicador de si el objeto esta en BBDD_01 (se establece a 0 de inicio)
4	indicador de si el objeto esta en BBDD_02 (se establece a 0 de inicio)
5	dataframe con el código del objeto en BBDD_01 (se establece a None de inicio)
6	dataframe con el código del objeto en BBDD_02 (se establece a None de inicio)

```
660 lista_objetos_conso_1 = [[dicc["TIPO_OBJETO"]
661                          , dicc["REPOSITORIO"]
662                          , dicc["NOMBRE_OBJETO"]
663                          , 0
664                          , 0
665                          , None
666                          , None] for dicc in lista_dicc_objetos_bbdd_1]
667
668 lista_objetos_conso_2 = [[dicc["TIPO_OBJETO"]
669                          , dicc["REPOSITORIO"]
670                          , dicc["NOMBRE_OBJETO"]
671                          , 0
672                          , 0
673                          , None
674                          , None] for dicc in lista_dicc_objetos_bbdd_2]
```

Se fusionan **lista_objetos_conso_1** y **lista_objetos_conso_2** en la lista temporal **lista_objetos_conso** a la cual se le quitan los duplicados. Esta lista permite disponer de todos los objetos tanto de BBDD_01 como de BBDD_02.

```
676 lista_objetos_conso = lista_objetos_conso_1 + lista_objetos_conso_2
677 lista_objetos_conso = [sublista for i, sublista in enumerate(lista_objetos_conso) if sublista not in lista_objetos_conso[:i]]#
```

ANEXO_05

RUTINA: def_proceso_sql_server_2_control_versiones

El siguiente paso es calcular en **lista_objetos_conso** los ítems de cada una de sus sublistas asociados a los índices 3 a 6. Se realiza mediante bucle sobre **lista_objetos_conso**.

En cada iteración sobre este bucle se busca el objeto tanto en **lista_objetos_conso_1** como en **lista_objetos_conso_2**:

- Si hay cacheo para BBDD_01, se informan los ítems con índices de sublista 3 y 5.
- Si hay cacheo para BBDD_02, se informan los ítems con índices de sublista 4 y 6.

El cacheo se hace por el tipo de objeto, el nombre del repositorio y el nombre del objeto.

```
692     for ind, item in enumerate(lista_objetos_conso):
693
694         tipo_objeto_conso = item[indice_tipo_objeto]
695         repositorio_conso = item[indice_repositorio]
696         nombre_objeto_conso = item[indice_nombre_objeto]
```

BBDD_01

```
699         if isinstance(lista_dicc_objetos_bbdd_1, list):
700             for dicc in lista_dicc_objetos_bbdd_1:
701
702                 tipo_objeto_seek = dicc["TIPO_OBJETO"]
703                 repositorio_seek = dicc["REPOSITORIO"]
704                 nombre_objeto_seek = dicc["NOMBRE_OBJETO"]
705
706                 if tipo_objeto_conso == tipo_objeto_seek and repositorio_conso == repositorio_seek and nombre_objeto_conso == nombre_objeto_seek:
707                     lista_objetos_conso[ind][indice_esta_en_bbdd_1] = 1
708                     lista_objetos_conso[ind][indice_df_codigo_1] = dicc["DF_CODIGO_ORIGINAL"]
709
710                     break
```

BBDD_02

```
714         if isinstance(lista_dicc_objetos_bbdd_2, list):
715             for dicc in lista_dicc_objetos_bbdd_2:
716
717                 tipo_objeto_seek = dicc["TIPO_OBJETO"]
718                 repositorio_seek = dicc["REPOSITORIO"]
719                 nombre_objeto_seek = dicc["NOMBRE_OBJETO"]
720
721                 if tipo_objeto_conso == tipo_objeto_seek and repositorio_conso == repositorio_seek and nombre_objeto_conso == nombre_objeto_seek:
722                     lista_objetos_conso[ind][indice_esta_en_bbdd_2] = 1
723                     lista_objetos_conso[ind][indice_df_codigo_2] = dicc["DF_CODIGO_ORIGINAL"]
724
725                     break
```


Tras finalizar el bucle sobre **lista_objetos_conso** se pasa a calcular, se realiza otro bucle de nuevo sobre **lista_objetos_conso** para generar las listas de diccionarios siguientes:

```
734 lista_control_versiones_sql_server_tablas = []
735 lista_control_versiones_sql_server_views = []
736 lista_control_versiones_sql_server_stored_procedures = []
737 lista_control_versiones_sql_server_funciones = []
```

En cada iteración de este bucle:

- Se localiza si el objeto esta en las 2 bases de datos o solo en 1 y cual (se realiza mediante comparativa de los índices de sublistas 3 y 4)
- Se calcula el control de versiones usando la función **func_dicc_control_versiones** (módulo APP_2_GENERAL) pasándole como parámetros, entre otros, los dataframes almacenados en los índices de sublistas 5 y 6).

```
739 for tipo_objeto_conso, repositorio_conso, nombre_objeto_conso, en_bbdd_1_conso, en_bbdd_2_conso, df_codigo_1_conso, df_codigo_2_conso in lista_objetos_conso:
740
741     if en_bbdd_1_conso == 1 and en_bbdd_2_conso == 1:
742         check_objeto = mod_gen.dicc_control_versiones_tipo_concepto["YA_EXISTE"]
743
744     elif en_bbdd_1_conso == 1 and en_bbdd_2_conso == 0:
745         check_objeto = mod_gen.dicc_control_versiones_tipo_concepto["SOLO_EN_BBDD_01"]
746
747     elif en_bbdd_1_conso == 0 and en_bbdd_2_conso == 1:
748         check_objeto = mod_gen.dicc_control_versiones_tipo_concepto["SOLO_EN_BBDD_02"]
```

ANEXO_05

RUTINA: def_proceso_sql_server_2_control_versiones

```
739 for tipo_objeto_conso, repositorio_conso, nombre_objeto_conso, en_bbdd_1_conso, en_bbdd_2_conso, df_codigo_1_conso, df_codigo_2_conso in lista_objetos_conso:
752     dicc_temp = mod_gen.func_dicc_control_versiones(tipo_bbdd = "SQL_SERVER"
753     , check_objeto = check_objeto
754     , tipo_objeto = tipo_objeto_conso
755     , repositorio = repositorio_conso
756     , nombre_objeto = nombre_objeto_conso
757     , df_codigo_bbdd_1 = df_codigo_1_conso
758     , df_codigo_bbdd_2 = df_codigo_2_conso
759     )
760
761     if isinstance(dicc_temp, dict):
762
763         if tipo_objeto_conso == "TABLAS":
764             lista_control_versiones_sql_server_tablas.append(dicc_temp)
765
766         if tipo_objeto_conso == "VIEWS":
767             lista_control_versiones_sql_server_views.append(dicc_temp)
768
769         if tipo_objeto_conso == "STORED_PROCEDURES":
770             lista_control_versiones_sql_server_stored_procedures.append(dicc_temp)
771
772         if tipo_objeto_conso == "FUNCIONES":
773             lista_control_versiones_sql_server_funciones.append(dicc_temp)
774
```



Tras finalizar este último bucle se pasa al almacenar las listas de diccionarios en el diccionario **dicc_control_versiones_tipo_objeto** en las subkeys_3 (van por tipo de objeto) asociadas a la key_1 (SQL SERVER) y subkey_2 (TIPO OBJETO).

```
780 mod_gen.dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"]["TABLAS"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_sql_server_tablas if len(lista_c
781 mod_gen.dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"]["VIEWS"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_sql_server_views if len(lista_con
782 mod_gen.dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"]["STORED_PROCEDURES"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_sql_server_stored_pro
783 mod_gen.dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"]["FUNCIONES"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_sql_server_funciones if len(lis
```

Por último, se informa la subkey_3 (TODOS) del diccionario **dicc_control_versiones_tipo_objeto** mediante bucle sobre las 4 subkeys_3 anteriores.

```
793 lista_tipo_objetos = list(mod_gen.dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"].keys())
794
795 lista_control_versiones_todo = []
796 for tipo_objeto in lista_tipo_objetos:
797     if tipo_objeto != "TODOS":
798
799         try:
800             for dicc in mod_gen.dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"][tipo_objeto]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"]:
801                 lista_control_versiones_todo.append(dicc)
802
803         except:#puede haber tipo de objetos que no tengan cambios
804             pass
805
806 mod_gen.dicc_control_versiones_tipo_objeto["SQL_SERVER"]["TIPO_OBJETO"]["TODOS"]["LISTA_DICC_OBJETOS_CONTROL_VERSIONES"] = lista_control_versiones_todo if len(lis
```

ANEXO_05

RUTINA: def_proceso_sql_server_2_control_versiones

Por último, se crea el bloque **except** para almacenar los posibles errores de ejecución de la rutina en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_CALCULO) asociada a la key_1 (PROCESO_01) y subkey_2 (SQL_SERVER).

```
816     except Exception as Err:
817
818         traceback_error = traceback.extract_tb(Err.__traceback__)
819         modulo_python = os.path.basename(traceback_error[0].filename)
820         rutina_python = traceback_error[0].name
821         linea_error = traceback_error[0].lineno
822
823         lista_dicc_errores_migracion = mod_gen.dicc_errores_procesos["PROCESO_01"]["SQL_SERVER"]["LISTA_DICC_ERRORES_CALCULO"]
824
825         dicc_errores_temp = {"TIPO_BBDD": "SQL_SERVER"
826                             , "MODULO_PYTHON": modulo_python
827                             , "RUTINA_PYTHON": rutina_python
828                             , "LINEA_ERROR": linea_error
829                             , "ERRORES": str(Err)
830                             }
831
832         if isinstance(lista_dicc_errores_migracion, list):
833             lista_dicc_errores_migracion.append(dicc_errores_temp)
834         else:
835             lista_dicc_errores_migracion = [dicc_errores_temp]
836
837         mod_gen.dicc_errores_procesos["PROCESO_01"]["SQL_SERVER"]["LISTA_DICC_ERRORES_CALCULO"] = lista_dicc_errores_migracion
838         del dicc_errores_temp
839         del lista_dicc_errores_migracion
840         pass #es pass para que se localicen todos los posibles errores en el proceso
```

ANEXO_06

FUNCIÓN

def_proceso_merge_realizar_cambios

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

La rutina permite registrar los movimientos que el usuario realiza de un script a otro entre la BBDD_01 y la BBDD_02 en la GUI de control de versiones. 6 tipos de acciones están configuradas en la rutina:

ACCIONES	DESCRIPCIÓN
Migrar todo	Migre el script de BBDD_01 en BBDD_02, reemplazando el que habia (el script reemplazado queda visible en pantalla, al final del script migrado para tener trazabilidad de la acción realizada).
Migrar por líneas	Migra una línea o bloque de líneas del script de BBDD_01 al de BBDD_02 (los parámetros líneas de origen y destino son obligatorios).
Quitar todo	Elimina por completo el script de BBDD_02
Quitar por líneas	Quita una línea o bloques de líneas en el script de BBDD_02 (el parámetro líneas de origen es obligatorio).
Revertir todo	Revierten todos los cambios realizadas (sean migraciones o eliminaciones) y restituye el código original.
Revertir Último	Revierte el último cambio realizado por el usuario.

Estos movimientos se almacenan dentro del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL) con una doble finalidad:

FINALIDAD
Sirve de base para calcular los objetos donde el usuario ha realizado cambios y poder de esta forma acceder a la GUI de merge en base de datos física donde se listan estos objetos.
Si el usuario realiza cambios en un objeto y acto seguido decide consultar otro objeto (haciendo también, o no, una migración de líneas de código entre una base de datos y otra), al consultar de nuevo el 1er objeto los cambios que realizo anteriormente no se pierden y visualizara en pantalla los scripts de este objeto tal como los dejo antes de consultar el otro objeto.

A la diferencia de los otros Anexos, las explicaciones que se detallan aquí no se enfocan en nada en el código (con tal de referirse a los comentarios ahí incorporados considero que es mas que suficiente). El enfoque es más para que se vea como funcionan los procesos mencionados en el cuadro anterior **“desde dentro del app”** viendo como interactúan con el diccionario de configuración **dicc_control_versiones_tipo_objeto** y se ilustra la explicación con ejemplos.

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

Al ejecutar el proceso de cálculo del control de versiones, desde la GUI de inicio, sea en MS ACCESS y/o SQL SERVER, los objetos donde se localizan diferencias a nivel de línea de código entre sus scripts en las bases de datos BBDD_01 y BBDD_02 se almacenan en las subkeys_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES) del diccionario **dicc_control_versiones_tipo_objeto** según el tipo de objeto y el tipo de entorno de base datos.

Estas diferencias se calculan mediante la función **func_dicc_control_versiones** (módulo APP_2_GENERAL), ver **ANEXO_03**, y devuelve un diccionario por objeto que informa entre otros de los cambios localizados entre su script entre BBDD_01 y BBDD_02.

Para todos los objetos con cambios entre BBDD_01 y BBDD_02, se agregan sus diccionarios a una lista que pasa a almacenarse en las subkeys_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES) del diccionario **dicc_control_versiones_tipo_objeto** mencionado.

dicc_control_versiones_tipo_objeto			
KEY 1	KEY 2	KEY 3	KEY 4
MS_ACCESS	TIPO_OBJETO	TODOS	COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		TABLA_LOCAL	COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		VINCULO_ODBC	COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
		VINCULO_OTRO	COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
	MERGE_BBDD_FISICA	RUTINAS_VBA	COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		VARIABLES_VBA	COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		AJUSTES_MANUALES	COMBOBOX_GUI
			LISTA_DICC_OBJETOS
			COMBOBOX_GUI
			LISTA_DICC_OBJETOS
SQL_SERVER	TIPO_OBJETO	TODOS	COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		TABLAS	COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		VIEWS	LISTA_OBJECT_ID
			COMBOBOX_CONTROL_VERSIONES
			TIPO_OBJETO_SUBFORM
		STORED_PROCEDURES	LISTA_DICC_OBJETOS_CONTROL_VERSIONES
			LISTA_OBJECT_ID
			COMBOBOX_CONTROL_VERSIONES
	MERGE_BBDD_FISICA	FUNCIONES	TIPO_OBJETO_SUBFORM
			LISTA_DICC_OBJETOS_CONTROL_VERSIONES
			LISTA_OBJECT_ID
		OBJETOS_A_MIGRAR	COMBOBOX_GUI
			LISTA_DICC_OK_MIGRACION
			LISTA_DICC_ERRORES_MIGRACION

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

Cada diccionario que compone las listas almacenadas en las subkeys_4 mencionadas del diccionario **dicc_control_versiones_tipo_objeto** tiene la estructura siguiente:

KEYS	DESCRIPCIÓN
TIPO_BBDD	MS_ACCESS o SQL_SERVER
CHECK_OBJETO	Valores de las keys del diccionario dicc_control_versiones_tipo_concepto (YA_EXISTE, SOLO_EN_BBDD_01 o SOLO_EN_BBDD_02)
TIPO_OBJETO	Los tipos de objeto mencionados en el 1er cuadro de la página anterior
TIPO_OBJETO_SUBFORM	Tipo de objeto con el literal que sale en la GUI de control de versiones y merge en base de datos física es el valor de las subkeys_4, TIPO_OBJETO_SUBFORM, del diccionario dicc_control_versiones_tipo_objeto)
TIPO_REPOSITORIO	Solo para MS ACCESS (formulario, userform o módulo VBA estandar)
REPOSITORIO	Nombre del módulo VBA para MS ACCESS y nombre del esquema para SQL SERVER
NOMBRE_OBJETO	Nombre del objeto
NUM_CAMBIOS_SCRIPT_1	Número de líneas de código con cambios en el script de BBDD_01 (se usa en el sub-formulario del a GUI de control de versiones)
NUM_CAMBIOS_SCRIPT_2	Número de líneas de código con cambios en el script de BBDD_02 (se usa en el sub-formulario del a GUI de control de versiones)
DF_CODIGO_ORIGINAL_1	Dataframe con el código del script de BBDD_01 donde las líneas con cambios con respecto al script de BBDD_02 estan localizadas
DF_CODIGO_ORIGINAL_2	Dataframe con el código del script de BBDD_02 donde las líneas con cambios con respecto al script de BBDD_01 estan localizadas
LISTA_DICC_MERGE_HECHOS	None al ejecutarse la función. Se informa a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)
DF_CODIGO_ACTUAL_1	Al ejecutarse la función es de inicio el mismo dataframe que DF_CODIGO_ORIGINAL_1. Se actualiza a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)
DF_CODIGO_ACTUAL_2	Al ejecutarse la función es de inicio el mismo dataframe que DF_CODIGO_ORIGINAL_2. Se actualiza a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)

Las keys marcadas en **ROJO** son sobre las cuales el app actúa cuando el usuario realiza cambios de scripts de una base de datos a otra y también es a partir de las cuales se permite realizar el **merge** en base de datos física (ver **ANEXO_07**).

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

Las subkeys_4 del diccionario **dicc_control_versiones_tipo_objeto** mencionadas en la página anterior sirven para:

DF_CODIGO_ORIGINAL_1
DF_CODIGO_ORIGINAL_2



Son los dataframes que contienen el **código original** de los scripts de un objeto con los cambios (es decir antes de que el usuario interactué con ellos). Estos dataframes son inamovibles de todos los cambios que el usuario pueda realizar (esto es para recuperar el script original si **el usuario opta por la opción en la GUI de “Revertir todo”**)

DF_CODIGO_ACTUAL_1
DF_CODIGO_ACTUAL_2



Son los dataframes con **los últimos cambios realizados por el usuario**. Cuando se accede a la GUI de control de versiones, es decir antes de que el usuario haga cambios, estos dataframes son idénticos a los de DF_CODIGO_ORIGINAL (ver arriba). Difieren cuando el usuario realiza cambios.

LISTA_DICC_MERGE_HECHOS



Es una **lista de diccionarios** donde cada diccionario contiene los dataframes actuales de las 2 bases de datos tras el último cambio. Se usa para registrar todos los cambios realizados por el usuario, dentro de una misma sesión del app, porque este puede realizar el proceso en varias etapas (por ejemplo, migra 3 líneas de BBDD_01 a BBDD_02, luego decide migrar otras 2 líneas, se da cuenta que se equivoco y necesita revertir el cambio, etc etc). Es necesario para poder **revertir el último cambio** cuando el usuario ha realizado más de 2 cambios y así poder acceder al cambio intermedio.

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

Se ilustra lo comentado con el ejemplo siguiente:

Al ejecutar el proceso de control de versiones, se localizan 2 objetos SQL SERVER con cambios

TIPO OBJETO	ESQUEMA	NOMBRE OBJETO	UBICACIÓN EN BBDD
STORED PROCEDURE	dbo	PROC_BULK_INSERT	Objetos ya existentes en las 2 BBDD's
STORED PROCEDURE	dbo	PROC_PrintError	Objetos ya existentes en las 2 BBDD's

Tipo Objeto

SQL SERVER: stored procedures

Tipo Concepto

Objetos ya existentes en las 2 BBDD's

VER

EXCEL

TIPO BBDD	TIPO OBJETO	REPOSITORIO	OBJETO	BBDD_01	BBDD_02
SQL_SERVER	Stored Procedures	dbo	PROC_BULK_INSERT	6	5
SQL_SERVER	Stored Procedures	dbo	PROC_PrintError	1	2

BBDD_01	BBDD_ORIGEN	BBDD_02	BBDD_DESTINO
0002	CREATE PROCEDURE dbo.[PROC_BULK_INSERT]	0001	CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0003		0002	
0004	@Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)	0003	@Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0005		0004	
0006	AS	0005	AS
0007	BEGIN	0006	BEGIN
0008		0007	
0009		0008	
0010	declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)	0009	
0011		0010	declare @CadenaSQL3 nvarchar(4000), @CadenaSQL4 nvarchar(4000)
0012		0011	
0013	set @CadenaSQL1='truncate table ' + @Tabla	0012	
0014		0013	set @CadenaSQL3='truncate table ' + @Tabla
0015	set @CadenaSQL2='BULK INSERT ' + @Tabla + '	0014	
0016	FROM ''' + @RutaCSV + '''	0015	set @CadenaSQL4='BULK INSERT ' + @Tabla + '
0017	WITH	0016	FROM ''' + @RutaCSV + '''
0018	(	0017	WITH
0019	FIRSTROW = 2,	0018	(
0020	CODEPAGE='ACP',	0019	FIRSTROW = 2,
0021	FIELDTERMINATOR = ''' + @Separador + ''',	0020	CODEPAGE='ACP',
0022	ROWTERMINATOR = ''\n'',	0021	FIELDTERMINATOR = ''' + @Separador + ''',
0023	TABLOCK	0022	ROWTERMINATOR = ''\n'',
0024	)'	0023	TABLOCK
0025	--CODEPAGE=ACP permite agregar los acentos	0024	)'
0026		0025	
0027	exec sp_executesql @CadenaSQL1	0026	exec sp_executesql @CadenaSQL3
0028	exec sp_executesql @CadenaSQL2	0027	exec sp_executesql @CadenaSQL4
0029		0028	
0030		0029	
0031	END	0030	END

BBDD_01	BBDD_ORIGEN	BBDD_02	BBDD_DESTINO
0001		0001	
0002	CREATE PROCEDURE [dbo].[PROC_PrintError]	0002	CREATE PROCEDURE [dbo].[PROC_PrintError]
0003	AS	0003	AS
0004	BEGIN	0004	BEGIN
0005	SET NOCOUNT ON;	0005	--SET NOCOUNT ON;
0006		0006	
0007	PRINT 'Error ' + CONVERT(varchar(50), ERROR_NUMBER()) +	0007	PRINT 'Error ' + CONVERT(varchar(50), ERROR_NUMBER()) +
0008	', Severity ' + CONVERT(varchar(5), ERROR_SEVERITY()) +	0008	', Severity ' + CONVERT(varchar(5), ERROR_SEVERITY()) +
0009	', Procedure ' + ISNULL(ERROR_PROCEDURE(), '-');	0009	', State ' + CONVERT(varchar(5), ERROR_STATE()) +
0010	PRINT ERROR_MESSAGE();	0010	', Procedure ' + ISNULL(ERROR_PROCEDURE(), '-');
0011	END;	0011	PRINT ERROR_MESSAGE();
		0012	END;

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

Diccionario *dicc_control_versiones_tipo_objeto*

```
{SQL_SERVER:
  {'TIPO_OBJETO':
    {'STORED_PROCEDURES':
      {'LISTA_DICC_OBJETOS_CONTROL_VERSIONES':
        [
          {'TIPO_BBDD': 'SQL_SERVER'
            , 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's'
            , 'TIPO_OBJETO': 'STORED_PROCEDURES'
            , 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
            , 'TIPO_REPOSITORIO': None
            , 'REPOSITORIO': 'dbo'
            , 'NOMBRE_OBJETO': 'PROC_BULK_INSERT'
            , 'NUM_CAMBIOS_SCRIPT_1': 6
            , 'NUM_CAMBIOS_SCRIPT_2': 5
            , 'DF_CODIGO_ORIGINAL_1': 1
            , 'DF_CODIGO_ORIGINAL_2': 2
            , 'LISTA_DICC_MERGE_HECHOS': None
            , 'DF_CODIGO_ACTUAL_1': 3
            , 'DF_CODIGO_ACTUAL_2': 4
          ]
        }
      }
    }
  }
}
```

None
al
inicio



B
B
D
D
0
1

B
B
D
D
0
2

1

3

2

4

Dataframes de código

ORIGINAL

NUM LINEA	CODIGO	CODIGO_CON_NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001	0001	None	None
1	0002	0002	None	None
2	0003	0003	None	None
3	0004	0004	None	None
4	0005	0005	None	None
5	0006	0006	None	None
6	0007	0007	None	None
7	0008	0008	None	None
8	0009	0009	None	None
9	0010	0010	None	None
10	0011	0011	None	None
11	0012	0012	None	None
12	0013	0013	None	None
13	0014	0014	None	None
14	0015	0015	None	None
15	0016	0016	None	None

ACTUAL

NUM LINEA	CODIGO	CODIGO_CON_NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001	0001	None	None
1	0002	0002	None	None
2	0003	0003	None	None
3	0004	0004	None	None
4	0005	0005	None	None
5	0006	0006	None	None
6	0007	0007	None	None
7	0008	0008	None	None
8	0009	0009	None	None
9	0010	0010	None	None
10	0011	0011	None	None
11	0012	0012	None	None
12	0013	0013	None	None
13	0014	0014	None	None
14	0015	0015	None	None
15	0016	0016	None	None

ORIGINAL

NUM LINEA	CODIGO	CODIGO_CON_NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001	0001	None	None
1	0002	0002	None	None
2	0003	0003	None	None
3	0004	0004	None	None
4	0005	0005	None	None
5	0006	0006	None	None
6	0007	0007	None	None
7	0008	0008	None	None
8	0009	0009	None	None
9	0010	0010	None	None
10	0011	0011	None	None
11	0012	0012	None	None
12	0013	0013	None	None
13	0014	0014	None	None
14	0015	0015	None	None
15	0016	0016	None	None

ACTUAL

NUM LINEA	CODIGO	CODIGO_CON_NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001	0001	None	None
1	0002	0002	None	None
2	0003	0003	None	None
3	0004	0004	None	None
4	0005	0005	None	None
5	0006	0006	None	None
6	0007	0007	None	None
7	0008	0008	None	None
8	0009	0009	None	None
9	0010	0010	None	None
10	0011	0011	None	None
11	0012	0012	None	None
12	0013	0013	None	None
13	0014	0014	None	None
14	0015	0015	None	None
15	0016	0016	None	None

IGUALES
al inicio

IGUALES
al inicio

CAMBIOS_LOCALIZADOS

Es el flag que permite en la GUI marcar
las líneas de color **VERDE**

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

Diccionario *dicc_control_versiones_tipo_objeto*

```
{SQL_SERVER:
  {'TIPO_OBJETO':
    {'STORED PROCEDURES':
      {'LISTA_DICC_OBJETOS_CONTROL_VERSIONES':
        [
          {'TIPO_BBDD': 'SQL_SERVER'
            , 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's'
            , 'TIPO_OBJETO': 'STORED PROCEDURES'
            , 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
            , 'TIPO_REPOSITORIO': None
            , 'REPOSITORIO': 'dbo'
            , 'NOMBRE_OBJETO': 'PROC_BULK_INSERT'
            , 'NUM_CAMBIOS_SCRIPT_1': 6
            , 'NUM_CAMBIOS_SCRIPT_2': 5

            , 'DF_CODIGO_ORIGINAL_1':
            , 'DF_CODIGO_ORIGINAL_2':

            , 'LISTA_DICC_MERGE_HECHOS': None

            , 'DF_CODIGO_ACTUAL_1':
            , 'DF_CODIGO_ACTUAL_2':
          }

          , {'TIPO_BBDD': 'SQL_SERVER'
            , 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's'
            , 'TIPO_OBJETO': 'STORED PROCEDURES'
            , 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
            , 'TIPO_REPOSITORIO': None
            , 'REPOSITORIO': 'dbo'
            , 'NOMBRE_OBJETO': 'PROC_PrintError'
            , 'NUM_CAMBIOS_SCRIPT_1': 1
            , 'NUM_CAMBIOS_SCRIPT_2': 2

            , 'DF_CODIGO_ORIGINAL_1': 5
            , 'DF_CODIGO_ORIGINAL_2': 6
            , 'LISTA_DICC_MERGE_HECHOS': None
            , 'DF_CODIGO_ACTUAL_1': 7
            , 'DF_CODIGO_ACTUAL_2': 8
          }
        ]
      }
    }
  }
}
```

None
al
inicio



B
B
D
D

5

0
1

7

B
B
D
D

6

0
2

8

Dataframes de código

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	None	None	None	None
1	None	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None	None
2	None	AS\r	None	None
3	None	BEGIN\r	None	None
4	None	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	None	None
5	None	, Severity ' + CONVERT(varchar(5), ...	None	None
6	None	, Procedure ' + ISNULL(ERROR_PROCE...	None	None
7	None	PRINT ERROR_MESSAGE();\r	None	None
8	None	END;	None	None
9	None	None	None	None
10	None	None	None	None

ORIGINAL

CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	None	None	None	None
1	None	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None	None
2	None	AS\r	None	None
3	None	BEGIN\r	None	None
4	None	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	None	None
5	None	, Severity ' + CONVERT(varchar(5), ...	None	None
6	None	, Procedure ' + ISNULL(ERROR_PROCE...	None	None
7	None	PRINT ERROR_MESSAGE();\r	None	None
8	None	END;	None	None
9	None	None	None	None
10	None	None	None	None

ACTUAL

CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS

IGUALES
al inicio

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	None	None	None	None
1	None	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None	None
2	None	AS\r	None	None
3	None	BEGIN\r	None	None
4	None	--SET NOCOUNT ON;\r	None	None
5	None	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	None	None
6	None	, Severity ' + CONVERT(varchar(5), ...	None	None
7	None	, State ' + CONVERT(varchar(5), ...	None	None
8	None	, Procedure ' + ISNULL(ERROR_PROCE...	None	None
9	None	PRINT ERROR_MESSAGE();\r	None	None
10	None	END;	None	None
11	None	None	None	None

ORIGINAL

CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS

CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS

IGUALES
al inicio

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	None	None	None	None
1	None	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None	None
2	None	AS\r	None	None
3	None	BEGIN\r	None	None
4	None	--SET NOCOUNT ON;\r	None	None
5	None	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	None	None
6	None	, Severity ' + CONVERT(varchar(5), ...	None	None
7	None	, State ' + CONVERT(varchar(5), ...	None	None
8	None	, Procedure ' + ISNULL(ERROR_PROCE...	None	None
9	None	PRINT ERROR_MESSAGE();\r	None	None
10	None	END;	None	None
11	None	None	None	None

ACTUAL

CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS

CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS

CAMBIOS_LOCALIZADOS

Es el flag que permite en la GUI marcar
las líneas de color VERDE

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

Siempre en el mismo ejemplo, supongamos que el usuario **realiza los cambios** siguientes:

PROC BULK INSERT

CAMBIO 1 – Migración de una línea de BBDD_01 a BBDD_02

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino: **ACCIÓN** **MERGE**

BBDD_01 BBDD_ORIGEN

```
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009
0010 declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0011
0012
0013 set @CadenaSQL1='truncate table ' + @Tabla
0014
0015 set @CadenaSQL2='BULK INSERT ' + @Tabla + '
0016 FROM ''' + @RutaCSV + '''
0017 WITH
0018 (
0019     FIRSTROW = 2,
0020     CODEPAGE='ACP'',
0021     FIELDTERMINATOR = ''' + @Separador + ''',
0022     ROWTERMINATOR = ''\\n'',
0023     TABLOCK
0024 )'
0025
0026 --CODEPAGE=ACP permite agregar los acentos
0027
0028 exec sp_executesql @CadenaSQL1
0029 exec sp_executesql @CadenaSQL2
0030
```

BBDD_02 BBDD_DESTINO

```
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009
0010 declare @CadenaSQL3 nvarchar(4000), @CadenaSQL4 nvarchar(4000)
0011
0012 declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0013
0014 set @CadenaSQL3='truncate table ' + @Tabla
0015
0016 set @CadenaSQL4='BULK INSERT ' + @Tabla + '
0017 FROM ''' + @RutaCSV + '''
0018 WITH
0019 (
0020     FIRSTROW = 2,
0021     CODEPAGE='ACP'',
0022     FIELDTERMINATOR = ''' + @Separador + ''',
0023     ROWTERMINATOR = ''\\n'',
0024     TABLOCK
0025 )'
0026
0027 exec sp_executesql @CadenaSQL3
0028 exec sp_executesql @CadenaSQL4
0029
0030
```

CAMBIO 2 – Migración de una línea de BBDD_01 a BBDD_02

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino: **ACCIÓN** **MERGE**

BBDD_01 BBDD_ORIGEN

```
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009
0010 declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0011
0012
0013 set @CadenaSQL1='truncate table ' + @Tabla
0014
0015 set @CadenaSQL2='BULK INSERT ' + @Tabla + '
0016 FROM ''' + @RutaCSV + '''
0017 WITH
0018 (
0019     FIRSTROW = 2,
0020     CODEPAGE='ACP'',
0021     FIELDTERMINATOR = ''' + @Separador + ''',
0022     ROWTERMINATOR = ''\\n'',
0023     TABLOCK
0024 )'
0025
0026 --CODEPAGE=ACP permite agregar los acentos
0027
0028 exec sp_executesql @CadenaSQL1
0029 exec sp_executesql @CadenaSQL2
0030
```

BBDD_02 BBDD_DESTINO

```
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009
0010 declare @CadenaSQL3 nvarchar(4000), @CadenaSQL4 nvarchar(4000)
0011
0012 declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0013
0014 set @CadenaSQL3='truncate table ' + @Tabla
0015
0016 set @CadenaSQL4='BULK INSERT ' + @Tabla + '
0017 FROM ''' + @RutaCSV + '''
0018 WITH
0019 (
0020     FIRSTROW = 2,
0021     CODEPAGE='ACP'',
0022     FIELDTERMINATOR = ''' + @Separador + ''',
0023     ROWTERMINATOR = ''\\n'',
0024     TABLOCK
0025 )'
0026
0027 exec sp_executesql @CadenaSQL3
0028 exec sp_executesql @CadenaSQL4
0029
0030
```

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

PROC BULK INSERT

CAMBIO 3 – Eliminar todo el script de BBDD_02

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino: **ACCIÓN** **MERGE**

```
BBDD_01 BBDD_ORIGEN
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009
0010 declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0011
0012
0013 set @CadenaSQL1='truncate table ' + @Tabla
0014
0015 set @CadenaSQL2='BULK INSERT ' + @Tabla + '
0016 FROM ''' + @RutaCSV + '''
0017 WITH
0018 (
0019     FIRSTROW = 2,
0020     CODEPAGE='ACP'',
0021     FIELDTERMINATOR = ''' + @Separador + ''',
0022     ROWTERMINATOR = ''\\n'',
0023     TABLOCK
0024 )'
0025
0026 --CODEPAGE=ACP permite agregar los acentos
0027
0028 exec sp_executesql @CadenaSQL1
0029 exec sp_executesql @CadenaSQL2
0030
```

```
BBDD_02 BBDD_DESTINO
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009
0010 declare @CadenaSQL3 nvarchar(4000), @CadenaSQL4 nvarchar(4000)
0011
0012 declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0013
0014 set @CadenaSQL3='truncate table ' + @Tabla
0015
0016 set @CadenaSQL4='BULK INSERT ' + @Tabla + '
0017 FROM ''' + @RutaCSV + '''
0018 WITH
0019 (
0020     FIRSTROW = 2,
0021     CODEPAGE='ACP'',
0022     FIELDTERMINATOR = ''' + @Separador + ''',
0023     ROWTERMINATOR = ''\\n'',
0024     TABLOCK
0025 )'
0026
0027 --CODEPAGE=ACP permite agregar los acentos
0028
0029 exec sp_executesql @CadenaSQL3
0030 exec sp_executesql @CadenaSQL4
0031
```

CAMBIO 4 – Revertir el último cambio

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino: **ACCIÓN** **MERGE**

```
BBDD_01 BBDD_ORIGEN
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009
0010 declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0011
0012
0013 set @CadenaSQL1='truncate table ' + @Tabla
0014
0015 set @CadenaSQL2='BULK INSERT ' + @Tabla + '
0016 FROM ''' + @RutaCSV + '''
0017 WITH
0018 (
0019     FIRSTROW = 2,
0020     CODEPAGE='ACP'',
0021     FIELDTERMINATOR = ''' + @Separador + ''',
0022     ROWTERMINATOR = ''\\n'',
0023     TABLOCK
0024 )'
0025
0026 --CODEPAGE=ACP permite agregar los acentos
0027
0028 exec sp_executesql @CadenaSQL1
0029 exec sp_executesql @CadenaSQL2
0030
```

```
BBDD_02 BBDD_DESTINO
0001 CREATE PROCEDURE dbo.[PROC_BULK_INSERT]
0002
0003     @Tabla varchar(255), @RutaCSV varchar(255), @Separador varchar(255)
0004
0005 AS
0006 BEGIN
0007
0008
0009
0010 declare @CadenaSQL3 nvarchar(4000), @CadenaSQL4 nvarchar(4000)
0011
0012 declare @CadenaSQL1 nvarchar(4000), @CadenaSQL2 nvarchar(4000)
0013
0014 set @CadenaSQL3='truncate table ' + @Tabla
0015
0016 set @CadenaSQL4='BULK INSERT ' + @Tabla + '
0017 FROM ''' + @RutaCSV + '''
0018 WITH
0019 (
0020     FIRSTROW = 2,
0021     CODEPAGE='ACP'',
0022     FIELDTERMINATOR = ''' + @Separador + ''',
0023     ROWTERMINATOR = ''\\n'',
0024     TABLOCK
0025 )'
0026
0027 --CODEPAGE=ACP permite agregar los acentos
0028
0029 exec sp_executesql @CadenaSQL3
0030 exec sp_executesql @CadenaSQL4
0031
```

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

PROC PrintError

CAMBIO 5 – Quitar líneas de BBDD_02

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino:

ACCIÓN **MERGE**

BBDD_01 **BBDD_ORIGEN**

```
0001 CREATE PROCEDURE [dbo].[PROC_PrintError]
0002 AS
0003 BEGIN
0004 SET NOCOUNT ON;
0005
0006 PRINT 'Error ' + CONVERT(varchar(50), ERROR_NUMBER()) +
0007      ', Severity ' + CONVERT(varchar(5), ERROR_SEVERITY()) +
0008      ', Procedure ' + ISNULL(ERROR_PROCEDURE(), '-');
0009 PRINT ERROR_MESSAGE();
0010 END;
```

BBDD_02 **BBDD_DESTINO**

```
0001
0002 CREATE PROCEDURE [dbo].[PROC_PrintError]
0003 AS
0004 BEGIN
0005 --SET NOCOUNT ON;
0006
0007 PRINT 'Error ' + CONVERT(varchar(50), ERROR_NUMBER()) +
0008      ', Severity ' + CONVERT(varchar(5), ERROR_SEVERITY()) +
0009      ', State ' + CONVERT(varchar(5), ERROR_STATE()) +
0010      ', Procedure ' + ISNULL(ERROR_PROCEDURE(), '-');
0011 PRINT ERROR_MESSAGE();
0012 END;
```

CAMBIO 6 – Migrar líneas de BBDD_01 a BBDD_02

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino:

ACCIÓN **MERGE**

BBDD_01 **BBDD_ORIGEN**

```
0001 CREATE PROCEDURE [dbo].[PROC_PrintError]
0002 AS
0003 BEGIN
0004 SET NOCOUNT ON;
0005
0006 PRINT 'Error ' + CONVERT(varchar(50), ERROR_NUMBER()) +
0007      ', Severity ' + CONVERT(varchar(5), ERROR_SEVERITY()) +
0008      ', Procedure ' + ISNULL(ERROR_PROCEDURE(), '-');
0009 PRINT ERROR_MESSAGE();
0010 END;
```

BBDD_02 **BBDD_DESTINO**

```
0001
0002 CREATE PROCEDURE [dbo].[PROC_PrintError]
0003 AS
0004 BEGIN
0005 --SET NOCOUNT ON;
0006
0007 PRINT 'Error ' + CONVERT(varchar(50), ERROR_NUMBER()) +
0008      ', Severity ' + CONVERT(varchar(5), ERROR_SEVERITY()) +
0009      ', State ' + CONVERT(varchar(5), ERROR_STATE()) +
0010      ', Procedure ' + ISNULL(ERROR_PROCEDURE(), '-');
0011 PRINT ERROR_MESSAGE();
0012 END;
```

CAMBIO 7 – Revertir todos los cambios

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino:

ACCIÓN **MERGE**

BBDD_01 **BBDD_ORIGEN**

```
0001 CREATE PROCEDURE [dbo].[PROC_PrintError]
0002 AS
0003 BEGIN
0004 SET NOCOUNT ON;
0005
0006 PRINT 'Error ' + CONVERT(varchar(50), ERROR_NUMBER()) +
0007      ', Severity ' + CONVERT(varchar(5), ERROR_SEVERITY()) +
0008      ', Procedure ' + ISNULL(ERROR_PROCEDURE(), '-');
0009 PRINT ERROR_MESSAGE();
0010 END;
```

BBDD_02 **BBDD_DESTINO**

```
0001 CREATE PROCEDURE [dbo].[PROC_PrintError]
0002 AS
0003 BEGIN
0004 --SET NOCOUNT ON;
0005
0006 PRINT 'Error ' + CONVERT(varchar(50), ERROR_NUMBER()) +
0007      ', Severity ' + CONVERT(varchar(5), ERROR_SEVERITY()) +
0008      ', State ' + CONVERT(varchar(5), ERROR_STATE()) +
0009      ', Procedure ' + ISNULL(ERROR_PROCEDURE(), '-');
0010 PRINT ERROR_MESSAGE();
0011 END;
```

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

Tras presentar los ejemplos de cambios realizados por el usuario, ahora toca ver como se materializan en el diccionario **dicc_control_versiones_tipo_objeto**.

Los cambios se realizan en las keys siguientes de los diccionarios que componen la lista almacenada en la subkey_4 (**LISTA_DICC_OBJETOS_CONTROL_VERSIONES**) del diccionario **dicc_control_versiones_tipo_objeto**.

- LISTA_DICC_MERGE_HECHOS
- DF_CODIGO_ACTUAL_1
- DF_CODIGO_ACTUAL_2

```
{SQL_SERVER:
  {'TIPO_OBJETO':
    {'STORED PROCEDURES':
      {'LISTA_DICC_OBJETOS_CONTROL_VERSIONES':
        [
          {
            'TIPO_BDD': 'SQL_SERVER'
            , 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BDD's'
            , 'TIPO_OBJETO': 'STORED PROCEDURES'
            , 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
            , 'TIPO_REPOSITORIO': None
            , 'REPOSITORIO': None
            , 'NOMBRE_OBJETO': 'PROC_BULK_INSERT'
            , 'NUM_CAMBIOS_SCRIPT_1': 1
            , 'NUM_CAMBIOS_SCRIPT_2': 5
            , 'DF_CODIGO_ORIGINAL_1':
            , 'DF_CODIGO_ORIGINAL_2':
            , 'LISTA_DICC_MERGE_HECHOS': None
            , 'DF_CODIGO_ACTUAL_1':
            , 'DF_CODIGO_ACTUAL_2':
          }
          , {
            'TIPO_BDD': 'SQL_SERVER'
            , 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BDD's'
            , 'TIPO_OBJETO': 'STORED PROCEDURES'
            , 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
            , 'TIPO_REPOSITORIO': None
            , 'REPOSITORIO': None
            , 'NOMBRE_OBJETO': 'PROC_PrintError'
            , 'NUM_CAMBIOS_SCRIPT_1': 1
            , 'NUM_CAMBIOS_SCRIPT_2': 2
            , 'DF_CODIGO_ORIGINAL_1':
            , 'DF_CODIGO_ORIGINAL_2':
            , 'LISTA_DICC_MERGE_HECHOS': None
            , 'DF_CODIGO_ACTUAL_1':
            , 'DF_CODIGO_ACTUAL_2':
          }
        ]
      }
    }
  }
}
```


ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

CAMBIO 1

MERGE LEYENDA COLORES: Cambios localizados Agregado en bdd MERGE Eliminado en bdd MERGE

BDD: Acción: Líneas Origen: Líneas Destino: **ACCIÓN** **MERGE**

```
{'TIPO_BDD': 'SQL_SERVER'
, 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BDD's'
, 'TIPO_OBJETO': 'STORED PROCEDURES'
, 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
, 'TIPO_REPOSITORIO': None
, 'REPOSITORIO': 'dbo'
, 'NOMBRE_OBJETO': 'PROC_BULK_INSERT'
, 'NUM_CAMBIOS_SCRIPT_1': 6
, 'NUM_CAMBIOS_SCRIPT_2': 5

, 'DF_CODIGO_ORIGINAL_1':
, 'DF_CODIGO_ORIGINAL_2':

, 'LISTA_DICC_MERGE_HECHOS'
, 'DF_CODIGO_ACTUAL_1': 1
, 'DF_CODIGO_ACTUAL_2': 2
}
```

LISTA_DICC_MERGE_HECHOS

```
[
  {'DF_CODIGO_ACTUAL_1': 1
  , 'DF_CODIGO_ACTUAL_2': 2
  }
```

AGREGADO

Es el flag que permite en la GUI marcar las líneas de color **NARANJA**

1

```
DF_CODIGO_ACTUAL_1
NUM LINEA CODIGO CODIGO_CON NUM LINEA CONTROL_CAMBIOS_ORIGINAL CONTROL_CAMBIOS_ACTUAL
0 0001\t \r 0001\t\r 0001\t\r None None
1 0002\t CREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r 0002\tCREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r None None
2 0003\t \r 0003\t\r 0003\t\r None None
3 0004\t \t@Tabla varchar(255), @RutaCSV varchar(255), ... 0004\t\t@Tabla varchar(255), @RutaCSV varchar(... None None
4 0005\t \r 0005\t\r 0005\t\r None None
5 0006\t AS\r 0006\tAS\r None None
6 0007\t BEGIN\r 0007\tBEGIN\r None None
7 0008\t \r 0008\t\r 0008\t\r None None
8 0009\t \r 0009\t\r 0009\t\r None None
9 0010\t declare @CadenaSQL1 nvarchar(4000), @CadenaSQL... 0010\tdeclare @CadenaSQL1 nvarchar(4000), @Cad... CAMBIOS_LOCALIZADOS AGREGADO
10 0011\t \r 0011\t\r 0011\t\r None None
11 0012\t \r 0012\t\r 0012\t\r None None
12 0013\t set @CadenaSQL1='truncate table ' + @Tabla\r 0013\tset @CadenaSQL1='truncate table ' + @Tab... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
13 0014\t \r 0014\t\r 0014\t\r None None
14 0015\t set @CadenaSQL2='BULK INSERT ' + @Tabla + '\r 0015\tset @CadenaSQL2='BULK INSERT ' + @Tabla ... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
15 0016\t \tFROM ' + @RutaCSV + ' '\r 0016\t\tFROM ' + @RutaCSV + ' '\r None None
16 0017\t \tWITH\r 0017\t\tWITH\r None None
17 0018\t \t \r 0018\t\t \r None None
18 0019\t \tFIRSTROW = 2,\r 0019\t\tFIRSTROW = 2,\r None None
19 0020\t \tCODEPAGE='ACP',\r 0020\t\tCODEPAGE='ACP',\r None None
20 0021\t \tFIELDTERMINATOR = ' ' + @Separador + ' '\r 0021\t\tFIELDTERMINATOR = ' ' + @Separador + ' ... None None
21 0022\t \tROWTERMINATOR = ' '\r 0022\t\tROWTERMINATOR = ' '\r None None
22 0023\t \tTABLELOCK\r 0023\t\tTABLELOCK\r None None
23 0024\t \t)\r 0024\t\t)\r None None
24 0025\t \t--CODEPAGE=ACP permite agregar los acentos\r 0025\t\t--CODEPAGE=ACP permite agregar los ace... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
25 0026\t \r 0026\t\r 0026\t\r None None
26 0027\t exec sp_executesql @CadenaSQL1\r 0027\texec sp_executesql @CadenaSQL1\r CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
27 0028\t exec sp_executesql @CadenaSQL2\r 0028\texec sp_executesql @CadenaSQL2\r CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
28 0029\t \r 0029\t\r 0029\t\r None None
29 0030\t \r 0030\t\r 0030\t\r None None
30 0031\t END 0031\tEND None None
```

2

```
DF_CODIGO_ACTUAL_2
NUM LINEA CODIGO CODIGO_CON NUM LINEA CONTROL_CAMBIOS_ORIGINAL CONTROL_CAMBIOS_ACTUAL
0 0001\t \r 0001\t\r 0001\t\r None None
1 0002\t CREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r 0002\tCREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r None None
2 0003\t \r 0003\t\r 0003\t\r None None
3 0004\t \t@Tabla varchar(255), @RutaCSV varchar(255), ... 0004\t\t@Tabla varchar(255), @RutaCSV varchar(... None None
4 0005\t \r 0005\t\r 0005\t\r None None
5 0006\t AS\r 0006\tAS\r None None
6 0007\t BEGIN\r 0007\tBEGIN\r None None
7 0008\t \r 0008\t\r 0008\t\r None None
8 0009\t \r 0009\t\r 0009\t\r None None
9 0010\t declare @CadenaSQL3 nvarchar(4000), @CadenaSQL... 0010\tdeclare @CadenaSQL3 nvarchar(4000), @Cad... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
10 0011\t \r 0011\t\r 0011\t\r None None
11 0012\t declare @CadenaSQL1 nvarchar(4000), @CadenaSQL... 0012\tdeclare @CadenaSQL1 nvarchar(4000), @Cad... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
12 0013\t \r 0013\t\r 0013\t\r None None
13 0014\t set @CadenaSQL3='truncate table ' + @Tabla\r 0014\tset @CadenaSQL3='truncate table ' + @Tab... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
14 0015\t \r 0015\t\r 0015\t\r None None
15 0016\t set @CadenaSQL4='BULK INSERT ' + @Tabla + '\r 0016\tset @CadenaSQL4='BULK INSERT ' + @Tabla ... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
16 0017\t \tFROM ' + @RutaCSV + ' '\r 0017\t\tFROM ' + @RutaCSV + ' '\r None None
17 0018\t \tWITH\r 0018\t\tWITH\r None None
18 0019\t \t \r 0019\t\t \r None None
19 0020\t \tFIRSTROW = 2,\r 0020\t\tFIRSTROW = 2,\r None None
20 0021\t \tCODEPAGE='ACP',\r 0021\t\tCODEPAGE='ACP',\r None None
21 0022\t \tFIELDTERMINATOR = ' ' + @Separador + ' '\r 0022\t\tFIELDTERMINATOR = ' ' + @Separador + ' ... None None
22 0023\t \tROWTERMINATOR = ' '\r 0023\t\tROWTERMINATOR = ' '\r None None
23 0024\t \tTABLELOCK\r 0024\t\tTABLELOCK\r None None
24 0025\t \t)\r 0025\t\t)\r None None
25 0026\t \r 0026\t\r 0026\t\r None None
26 0027\t exec sp_executesql @CadenaSQL3\r 0027\texec sp_executesql @CadenaSQL3\r CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
27 0028\t exec sp_executesql @CadenaSQL4\r 0028\texec sp_executesql @CadenaSQL4\r CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
28 0029\t \r 0029\t\r 0029\t\r None None
29 0030\t \r 0030\t\r 0030\t\r None None
30 0031\t END 0031\tEND None None
```


ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

CAMBIO 2

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino: ACCIÓN:

```
{'TIPO_BBDD': 'SQL_SERVER',
 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's',
 'TIPO_OBJETO': 'STORED PROCEDURES',
 'TIPO_OBJETO_SUBFORM': 'Stored Procedures',
 'TIPO_REPOSITORIO': None,
 'REPOSITORIO': 'dbe',
 'NOMBRE_OBJETO': 'PROC_BULK_INSERT',
 'NUM_CAMBIOS_SCRIPT_1': 6,
 'NUM_CAMBIOS_SCRIPT_2': 5,
 'DF_CODIGO_ORIGINAL_1':,
 'DF_CODIGO_ORIGINAL_2':,
 'LISTA_DICC_MERGE_HECHOS':,
 'DF_CODIGO_ACTUAL_1': 3,
 'DF_CODIGO_ACTUAL_2': 4,
}
```

```
LISTA_DICC_MERGE_HECHOS
-----
[
  {'DF_CODIGO_ACTUAL_1': 1,
   'DF_CODIGO_ACTUAL_2': 2},
  {'DF_CODIGO_ACTUAL_1': 3,
   'DF_CODIGO_ACTUAL_2': 4},
]
```

Dataframes
CAMBIO 1

AGREGADO

Es el flag que permite en la GUI marcar las líneas de color **NARANJA**

3

```
DF_CODIGO_ACTUAL_1
NUM_LINEA CODIGO CODIGO_CON NUM_LINEA CONTROL_CAMBIOS_ORIGINAL CONTROL_CAMBIOS_ACTUAL
0 0001\t CREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r 0002\tCREATE PROCEDURE dbo.[PROC_BULK_INSERT]\r None None
1 0002\t \t@Tabla varchar(255), @RutaCSV varchar(255), ... 0003\t\t@Tabla varchar(255), @RutaCSV varchar(...) None None
2 0003\t AS\r 0004\tAS\r None None
3 0004\t BEGIN\r 0005\tBEGIN\r None None
4 0005\t \r 0006\t\r None None
5 0006\t declare @CadenaSQL1 nvarchar(4000), @CadenaSQL... 0007\tdeclare @CadenaSQL1 nvarchar(4000), @Cad... None None
6 0007\t \r 0008\t\r None None
7 0008\t set @CadenaSQL1='truncate table ' + @Tabla\r 0009\tset @CadenaSQL1='truncate table ' + @Tab... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
8 0009\t \r 0010\t\r None None
9 0010\t set @CadenaSQL2='BULK INSERT ' + @Tabla + '\r 0011\tset @CadenaSQL2='BULK INSERT ' + @Tabla ... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
10 0011\t \tFROM ' + @RutaCSV + ' '\r 0012\t\tFROM ' + @RutaCSV + ' '\r None None
11 0012\t \tWITH\r 0013\t\tWITH\r None None
12 0013\t \tFIRSTROW = 2,\r 0014\t\tFIRSTROW = 2,\r None None
13 0014\t \tCODEPAGE='ACP',\r 0015\t\tCODEPAGE='ACP',\r None None
14 0015\t \tFIELDTERMINATOR = ' ' + @Separador + ' '\r 0016\t\tFIELDTERMINATOR = ' ' + @Separador + ' '\r None None
15 0016\t \tROWTERMINATOR = ' '\r 0017\t\tROWTERMINATOR = ' '\r None None
16 0017\t \tTABLELOCK\r 0018\t\tTABLELOCK\r None None
17 0018\t \t)\r 0019\t\t)\r None None
18 0019\t \t--CODEPAGE=ACP permite agregar los acentos\r 0020\t\t--CODEPAGE=ACP permite agregar los ace... CAMBIOS_LOCALIZADOS CAMBIOS_LOCALIZADOS
19 0020\t exec sp_executesql @CadenaSQL1\r 0021\texec sp_executesql @CadenaSQL1\r None None
20 0021\t exec sp_executesql @CadenaSQL2\r 0022\texec sp_executesql @CadenaSQL2\r None None
21 0022\t \r 0023\t\r None None
22 0023\t \r 0024\t\r None None
23 0024\t \r 0025\t\r None None
24 0025\t \r 0026\t\r None None
25 0026\t \r 0027\t\r None None
26 0027\t \r 0028\t\r None None
27 0028\t \r 0029\t\r None None
28 0029\t \r 0030\t\r None None
29 0030\t \r 0031\t\r None None
30 0031\t \r 0032\t\r None None
31 0032\t \r 0033\t\r None None
32 0033\t \r 0034\t\r None None
33 0034\t \r 0035\t\r None None
34 0035\t \r 0036\t\r None None
35 0036\t \r 0037\t\r None None
36 0037\t \r 0038\t\r None None
37 0038\t \r 0039\t\r None None
38 0039\t \r 0040\t\r None None
39 0040\t \r 0041\t\r None None
40 0041\t \r 0042\t\r None None
41 0042\t \r 0043\t\r None None
42 0043\t \r 0044\t\r None None
43 0044\t \r 0045\t\r None None
44 0045\t \r 0046\t\r None None
45 0046\t \r 0047\t\r None None
46 0047\t \r 0048\t\r None None
47 0048\t \r 0049\t\r None None
48 0049\t \r 0050\t\r None None
49 0050\t \r 0051\t\r None None
50 0051\t \r 0052\t\r None None
51 0052\t \r 0053\t\r None None
52 0053\t \r 0054\t\r None None
53 0054\t \r 0055\t\r None None
54 0055\t \r 0056\t\r None None
55 0056\t \r 0057\t\r None None
56 0057\t \r 0058\t\r None None
57 0058\t \r 0059\t\r None None
58 0059\t \r 0060\t\r None None
59 0060\t \r 0061\t\r None None
60 0061\t \r 0062\t\r None None
61 0062\t \r 0063\t\r None None
62 0063\t \r 0064\t\r None None
63 0064\t \r 0065\t\r None None
64 0065\t \r 0066\t\r None None
65 0066\t \r 0067\t\r None None
66 0067\t \r 0068\t\r None None
67 0068\t \r 0069\t\r None None
68 0069\t \r 0070\t\r None None
69 0070\t \r 0071\t\r None None
70 0071\t \r 0072\t\r None None
71 0072\t \r 0073\t\r None None
72 0073\t \r 0074\t\r None None
73 0074\t \r 0075\t\r None None
74 0075\t \r 0076\t\r None None
75 0076\t \r 0077\t\r None None
76 0077\t \r 0078\t\r None None
77 0078\t \r 0079\t\r None None
78 0079\t \r 0080\t\r None None
79 0080\t \r 0081\t\r None None
80 0081\t \r 0082\t\r None None
81 0082\t \r 0083\t\r None None
82 0083\t \r 0084\t\r None None
83 0084\t \r 0085\t\r None None
84 0085\t \r 0086\t\r None None
85 0086\t \r 0087\t\r None None
86 0087\t \r 0088\t\r None None
87 0088\t \r 0089\t\r None None
88 0089\t \r 0090\t\r None None
89 0090\t \r 0091\t\r None None
90 0091\t \r 0092\t\r None None
91 0092\t \r 0093\t\r None None
92 0093\t \r 0094\t\r None None
93 0094\t \r 0095\t\r None None
94 0095\t \r 0096\t\r None None
95 0096\t \r 0097\t\r None None
96 0097\t \r 0098\t\r None None
97 0098\t \r 0099\t\r None None
98 0099\t \r 0100\t\r None None
99 0100\t \r 0101\t\r None None
100 0101\t \r 0102\t\r None None
101 0102\t \r 0103\t\r None None
102 0103\t \r 0104\t\r None None
103 0104\t \r 0105\t\r None None
104 0105\t \r 0106\t\r None None
105 0106\t \r 0107\t\r None None
106 0107\t \r 0108\t\r None None
107 0108\t \r 0109\t\r None None
108 0109\t \r 0110\t\r None None
109 0110\t \r 0111\t\r None None
110 0111\t \r 0112\t\r None None
111 0112\t \r 0113\t\r None None
112 0113\t \r 0114\t\r None None
113 0114\t \r 0115\t\r None None
114 0115\t \r 0116\t\r None None
115 0116\t \r 0117\t\r None None
116 0117\t \r 0118\t\r None None
117 0118\t \r 0119\t\r None None
118 0119\t \r 0120\t\r None None
119 0120\t \r 0121\t\r None None
120 0121\t \r 0122\t\r None None
121 0122\t \r 0123\t\r None None
122 0123\t \r 0124\t\r None None
123 0124\t \r 0125\t\r None None
124 0125\t \r 0126\t\r None None
125 0126\t \r 0127\t\r None None
126 0127\t \r 0128\t\r None None
127 0128\t \r 0129\t\r None None
128 0129\t \r 0130\t\r None None
129 0130\t \r 0131\t\r None None
130 0131\t \r 0132\t\r None None
131 0132\t \r 0133\t\r None None
132 0133\t \r 0134\t\r None None
133 0134\t \r 0135\t\r None None
134 0135\t \r 0136\t\r None None
135 0136\t \r 0137\t\r None None
136 0137\t \r 0138\t\r None None
137 0138\t \r 0139\t\r None None
138 0139\t \r 0140\t\r None None
139 0140\t \r 0141\t\r None None
140 0141\t \r 0142\t\r None None
141 0142\t \r 0143\t\r None None
142 0143\t \r 0144\t\r None None
143 0144\t \r 0145\t\r None None
144 0145\t \r 0146\t\r None None
145 0146\t \r 0147\t\r None None
146 0147\t \r 0148\t\r None None
147 0148\t \r 0149\t\r None None
148 0149\t \r 0150\t\r None None
149 0150\t \r 0151\t\r None None
150 0151\t \r 0152\t\r None None
151 0152\t \r 0153\t\r None None
152 0153\t \r 0154\t\r None None
153 0154\t \r 0155\t\r None None
154 0155\t \r 0156\t\r None None
155 0156\t \r 0157\t\r None None
156 0157\t \r 0158\t\r None None
157 0158\t \r 0159\t\r None None
158 0159\t \r 0160\t\r None None
159 0160\t \r 0161\t\r None None
160 0161\t \r 0162\t\r None None
161 0162\t \r 0163\t\r None None
162 0163\t \r 0164\t\r None None
163 0164\t \r 0165\t\r None None
164 0165\t \r 0166\t\r None None
165 0166\t \r 0167\t\r None None
166 0167\t \r 0168\t\r None None
167 0168\t \r 0169\t\r None None
168 0169\t \r 0170\t\r None None
169 0170\t \r 0171\t\r None None
170 0171\t \r 0172\t\r None None
171 0172\t \r 0173\t\r None None
172 0173\t \r 0174\t\r None None
173 0174\t \r 0175\t\r None None
174 0175\t \r 0176\t\r None None
175 0176\t \r 0177\t\r None None
176 0177\t \r 0178\t\r None None
177 0178\t \r 0179\t\r None None
178 0179\t \r 0180\t\r None None
179 0180\t \r 0181\t\r None None
180 0181\t \r 0182\t\r None None
181 0182\t \r 0183\t\r None None
182 0183\t \r 0184\t\r None None
183 0184\t \r 0185\t\r None None
184 0185\t \r 0186\t\r None None
185 0186\t \r 0187\t\r None None
186 0187\t \r 0188\t\r None None
187 0188\t \r 0189\t\r None None
188 0189\t \r 0190\t\r None None
189 0190\t \r 0191\t\r None None
190 0191\t \r 0192\t\r None None
191 0192\t \r 0193\t\r None None
192 0193\t \r 0194\t\r None None
193 0194\t \r 0195\t\r None None
194 0195\t \r 0196\t\r None None
195 0196\t \r 0197\t\r None None
196 0197\t \r 0198\t\r None None
197 0198\t \r 0199\t\r None None
198 0199\t \r 0200\t\r None None
199 0200\t \r 0201\t\r None None
200 0201\t \r 0202\t\r None None
201 0202\t \r 0203\t\r None None
202 0203\t \r 0204\t\r None None
203 0204\t \r 0205\t\r None None
204 0205\t \r 0206\t\r None None
205 0206\t \r 0207\t\r None None
206 0207\t \r 0208\t\r None None
207 0208\t \r 0209\t\r None None
208 0209\t \r 0210\t\r None None
209 0210\t \r 0211\t\r None None
210 0211\t \r 0212\t\r None None
211 0212\t \r 0213\t\r None None
212 0213\t \r 0214\t\r None None
213 0214\t \r 0215\t\r None None
214 0215\t \r 0216\t\r None None
215 0216\t \r 0217\t\r None None
216 0217\t \r 0218\t\r None None
217 0218\t \r 0219\t\r None None
218 0219\t \r 0220\t\r None None
219 0220\t \r 0221\t\r None None
220 0221\t \r 0222\t\r None None
221 0222\t \r 0223\t\r None None
222 0223\t \r 0224\t\r None None
223 0224\t \r 0225\t\r None None
224 0225\t \r 0226\t\r None None
225 0226\t \r 0227\t\r None None
226 0227\t \r 0228\t\r None None
227 0228\t \r 0229\t\r None None
228 0229\t \r 0230\t\r None None
229 0230\t \r 0231\t\r None None
230 0231\t \r 0232\t\r None None
231 0232\t \r 0233\t\r None None
232 0233\t \r 0234\t\r None None
233 0234\t \r 0235\t\r None None
234 0235\t \r 0236\t\r None None
235 0236\t \r 0237\t\r None None
236 0237\t \r 0238\t\r None None
237 0238\t \r 0239\t\r None None
238 0239\t \r 0240\t\r None None
239 0240\t \r 0241\t\r None None
240 0241\t \r 0242\t\r None None
241 0242\t \r 0243\t\r None None
242 0243\t \r 0244\t\r None None
243 0244\t \r 0245\t\r None None
244 0245\t \r 0246\t\r None None
245 0246\t \r 0247\t\r None None
246 0247\t \r 0248\t\r None None
247 0248\t \r 0249\t\r None None
248 0249\t \r 0250\t\r None None
249 0250\t \r 0251\t\r None None
250 0251\t \r 0252\t\r None None
251 0252\t \r 0253\t\r None None
252 0253\t \r 0254\t\r None None
253 0254\t \r 0255\t\r None None
254 0255\t \r 0256\t\r None None
255 0256\t \r 0257\t\r None None
256 0257\t \r 0258\t\r None None
257 0258\t \r 0259\t\r None None
258 0259\t \r 0260\t\r None None
259 0260\t \r 0261\t\r None None
260 0261\t \r 0262\t\r None None
261 0262\t \r 0263\t\r None None
262 0263\t \r 0264\t\r None None
263 0264\t \r 0265\t\r None None
264 0265\t \r 0266\t\r None None
265 0266\t \r 0267\t\r None None
266 0267\t \r 0268\t\r None None
267 0268\t \r 0269\t\r None None
268 0269\t \r 0270\t\r None None
269 0270\t \r 0271\t\r None None
270 0271\t \r 0272\t\r None None
271 0272\t \r 0273\t\r None None
272 0273\t \r 0274\t\r None None
273 0274\t \r 0275\t\r None None
274 0275\t \r 0276\t\r None None
275 0276\t \r 0277\t\r None None
276 0277\t \r 0278\t\r None None
277 0278\t \r 0279\t\r None None
278 0279\t \r 0280\t\r None None
279 0280\t \r 0281\t\r None None
280 0281\t \r 0282\t\r None None
281 0282\t \r 0283\t\r None None
282 0283\t \r 0284\t\r None None
283 0284\t \r 0285\t\r None None
284 0285\t \r 0286\t\r None None
285 0286\t \r 0287\t\r None None
286 0287\t \r 0288\t\r None None
287 0288\t \r 0289\t\r None None
288 0289\t \r 0290\t\r None None
289 0290\t \r 0291\t\r None None
290 0291\t \r 0292\t\r None None
291 0292\t \r 0293\t\r None None
292 0293\t \r 0294\t\r None None
293 0294\t \r 0295\t\r None None
294 0295\t \r 0296\t\r None None
295 0296\t \r 0297\t\r None None
296 0297\t \r 0298\t\r None None
297 0298\t \r 0299\t\r None None
298 0299\t \r 0300\t\r None None
299 0300\t \r 0301\t\r None None
300 0301\t \r 0302\t\r None None
301 0302\t \r 0303\t\r None None
302 0303\t \r 0304\t\r None None
303 0304\t \r 0305\t\r None None
304 0305\t \r 0306\t\r None None
305 0306\t \r 0307\t\r None None
306 0307\t \r 0308\t\r None None
307 0308\t \r 0309\t\r None None
308 0309\t \r 0310\t\r None None
309 0310\t \r 0311\t\r None None
310 0311\t \r 0312\t\r None None
311 0312\t \r 0313\t\r None None
312 0313\t \r 0314\t\r None None
313 0314\t \r 0315\t\r None None
314 0315\t \r 0316\t\r None None
315 0316\t \r 0317\t\r None None
316 0317\t \r 0318\t\r None None
317 0318\t \r 0319\t\r None None
318 0319\t \r 0320\t\r None None
319 0320\t \r 0321\t\r None None
320 0321\t \r 0322\t\r None None
321 0322\t \r 0323\t\r None None
322 0323\t \r 0324\t\r None None
323 0324\t \r 0325\t\r None None
324 0325\t \r 0326\t\r None None
325 0326\t \r 0327\t\r None None
326 0327\t \r 0328\t\r None None
327 0328\t \r 0329\t\r None None
328 0329\t \r 0330\t\r None None
329 0330\t \r 0331\t\r None None
330 0331\t \r 0332\t\r None None
331 0332\t \r 0333\t\r None None
332 0333\t \r 0334\t\r None None
333 0334\t \r 0335\t\r None None
334 0335\t \r 0336\t\r None None
335 0336\t \r 0337\t\r None None
336 0337\t \r 0338\t\r None None
337 0338\t \r 0339\t\r None None
338 0339\t \r 0340\t\r None None
339 0340\t \r 0341\t\r None None
340 0341\t \r 0342\t\r None None
341 0342\t \r 0343\t\r None None
342 0343\t \r 0344\t\r None None
343 0344\t \r 0345\t\r None None
344 0345\t \r 0346\t\r None None
345 0346\t \r 0347\t\r None None
346 0347\t \r 0348\t\r None None
347 0348\t \r 0349\t\r None None
348 0349\t \r 0350\t\r None None
349 0350\t \r 0351\t\r None None
350 0351\t \r 0352\t\r None None
351 0352\t \r 0353\t\r None None
352 0353\t \r 0354\t\r None None
353 0354\t \r 0355\t\r None None
354 0355\t \r 0356\t\r None None
355 0356\t \r 0357\t\r None None
356 0357\t \r 0358\t\r None None
357 0358\t \r 0359\t\r None None
358 0359\t \r 0360\t\r None None
359 0360\t \r 0361\t\r None None
360 0361\t \r 0362\t\r None None
361 0362\t \r 0363\t\r None None
362 0363\t \r 0364\t\r None None
363 0364\t \r 0365\t\r None None
364 0365\t \r 0366\t\r None None
365 0366\t \r 0367\t\r None None
366 0367\t \r 0368\t\r None None
367 0368\t \r 0369\t\r None None
368 0369\t \r 0370\t\r None None
369 0370\t \r 0371\t\r None None
370 0371\t \r 0372\t\r None None
371 0372\t \r 0373\t\r None None
372 0373\t \r 0374\t\r None None
373 0374\t \r 0375\t\r None None
374 0375\t \r 0376\t\r None None
375 0376\t \r 0377\t\r None None
376 0377\t \r 0378\t\r None None
377 0378\t \r 0379\t\r None None
378 0379\t \r 0380\t\r None None
379 0380\t \r 0381\t\r None None
380 0381\t \r 0382\t\r None None
381 0382\t \r 0383\t\r None None
382 0383\t \r 0384\t\r None None
383 0384\t \r 0385\t\r None None
384 0385\t \r 0386\t\r None None
385 0386\t \r 0387\t\r None None
386 0387\t \r 0388\t\r None None
387 0388\t \r 0389\t\r None None
388 0389\t \r 0390\t\r None None
389 0390\t \r 0391\t\r None None
390 0391\t \r 0392\t\r None None
391 0392\t \r 0393\t\r None None
392 0393\t \r 0394\t\r None None
393 0394\t \r 0395\t\r None None
394 0395\t \r 0396\t\r None None
395 0396\t \r 0397\t\r None None
396 0397\t \r 0398\t\r None None
397 0398\t \r 0399\t\r None None
398 0399\t \r 0400\t\r None None
399 0400\t \r 0401\t\r None None
400 0401\t \r 0402\t\r None None
401 0402\t \r 0403\t\r None None
402 0403\t \r 0404\t\r None None
403 0404\t \r 0405\t\r None None
404 0405\t \r 0406\t\r None None
405 0406\t \r 0407\t\r None None
406 0407\t \r 0408\t\r None None
407 0408\t \r 0409\t\r None None
408 0409\t \r 0410\t\r None None
409 0410\t \r 0411\t\r None None
410 0411\t \r 0412\t\r None None
411 0412\t \r 0413\t\r None None
412 0413\t \r 0414\t\r None None
413 0414\t \r 0415\t\r None None
414 0415\t \r 0416\t\r None None
415 0416\t \r 0417\t\r None None
416 0417\t \r 0418\t\r None None
417 0418\t \r 0419\t\r None None
418 0419\t \r 0420\t\r None None
419 0420\t \r 0421\t\r None None
420 0421\t \r 0422\t\r None None
421 0422\t \r 0423\t\r None None
422 0423\t \r 0424\t\r None None
423 0424\t \r 0425\t\r None None
424 0425\t \r 0426\t\r None None
425 0426\t \r 0427\t\r None None
426 0427\t \r 0428\t\r None None
427 0428\t \r 0429\t\r None None
428 0429\t \r 0430\t\r None None
429 0430\t \r 0431\t\r None None
430 0431\t \r 0432\t\r None None
431 0432\t \r 0433\t\r None None
432 0433\t \r 0434\t\r None None
433 0434\t \r 0435\t\r None None
434 0435\t \r 0436\t\r None None
435 0436\t \r 0437\t\r None None
436 0437\t \r 0438\t\r None None
437 0438\t \r 0439\t\r None None
438 0439\t \r 0440\t\r None None
439 0440\t \r 0441\t\r None None
440 0441\t \r 0442\t\r None None
441 0442\t \r 0443\t\r None None
442 0443\t \r 0444\t\r None None
443 0444\t \r 0445\t\r None None
444 0445\t \r 0446\t\r None None
445 0446\t \r 0447\t\r None None
446 0447\t \r 0448\t\r None None
447 0448\t \r 0449\t\r None None
448 0449\t \r 0450\t\r None None
449 0450\t \r 0451\t\r None None
450 0451\t \r 0452\t\r None None
451 0452\t \r 0453\t\r None None
452 0453\t \r 0454\t\r None None
453 0454\t \r 0455\t\r None None
454 0455\t \r 0456\t\r None None
455 0456\t \r 0457\t\r None None
456 0457\t \r 0458\t\r None None
457 0458\t \r 0459\t\r None None
458 0459\t \r 0460\t\r None None
459 0460\t \r 0461\t\r None None
460 0461\t \r 0462\t\r None None
461 0462\t \r 0463\t\r None None
462 0463\t \r 0464\t\r None None
463 0464\t \r 0465\t\r None None
464 0465\t \r 0466\t\r None None
465 0466\t \r 0467\t\r None None
466 0467\t \r 0468\t\r None None
467 0468\t \r 0469\t\r None None
468 0469\t \r 0470\t\r None None
469 0470\t \r 0471\t\r None None
470 0471\t \r 0472\t\r None None
471 0472\t \r 0473\t\r None None
472 0473\t \r 0474\t\r None None
473 0474\t \r 0475\t\r None None
474 0475\t \r 0476\t\r None None
475 0476\t \r 0477\t\r None None
476 0477\t \r 0478\t\r None None
477 0478\t \r 0479\t\r None None
478 0479\t \r 0480\t\r None None
479 0480\t \r 0481\t\r None None
480 0481\t \r 0482\t\r None None
481 0482\t \r 0483\t\r None None
482 0483\t \r 0484\t\r None None
483 0484\t \r 0485\t\r None None
484 0485\t \r 0486\t\r None None
485 0486\t \r 0487\t\r None None
486 0487\t \r 0488\t\r None None
487 0488\t \r 0489\t\r None None
488 0489\t \r 0490\t\r None None
489 0490\t \r 0491\t\r None None
490 0491\t \r 0492\t\r None None
491 0492\t \r 0493\t\r None None
492 0493\t \r 0494\t\r None None
493 0494\t \r 0495\t\r None None
494 0495\t \r 0496\t\r None None
495 0496\t \r 0497\t\r None None
496 0497\t \r 0498\t\r None None
497 0498\t \r 0499\t\r None None
498 0499\t \r 0500\t\r None None
499 0500\t \r 0501\t\r None None
500 0501\t \r 0502\t\r None None
501 0502\t \r 0503\t\r None None
502 0503\t \r 0504\t\r None None
503 0504\t \r 0505\t\r None None
504 0505\t \r 0506\t\r None None
505 0506\t \r 0507\t\r None None
506 0507\t \r 0508\t\r None None
507 0508\t \r 0509\t\r None None
508 0509\t \r 0510\t\r None None
509 0510\t \r 0511\t\r None None
510 0511\t \r 0512\t\r None None
511 0512\t \r 0513\t\r None None
512 0513\t \r 0514\t\r None None
513 0514\t \r 0515\t\r None None
514 0515\t \r 0516\t\r None None
515 0516\t \r 0517\t\r None None
516 0517\t \r 0518\t\r None None
517 0518\t \r 0519\t\r None None
518 0519\t \r 0520\t\r None None
519 0520\t \r 0521\t\r None None
520 0521\t \r 0522\t\r None None
521 0522\t \r 0523\t\r None None
522 0523\t \r 0524\t\r None None
523 0524\t \r 0525\t\r None None
524 0525\t \r 0526\t\r None None
525 0526\t \r 0527\t\r None None
526 0527\t \r 0528\t\r None None
527 0528\t \r 0529\t\r None None
528 0529\t \r 0530\t\r None None
529 0530\t \r 0531\t\r None None
530 0531\t \r 0532\t\r None None
531 0532\t \r 0533\t\r None None
532 0533\t \r 0534\t\r None None
533 0534\t \r 0535\t\r None None
534 0535\t \r 0536\t\r None None
535 0536\t \r 0537\t\r None None
536 0537\t \r 0538\t\r None None
537 0538\t \r 0539\t\r None None
538 0539\t \r 0540\t\r None None
539 0540\t \r 0541\t\r None None
540 0541\t \r 0542\t\r None None
541 0542\t \
```

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

CAMBIO 3

MERGE LEYENDA COLORES: Cambios localizados Agregado en bddd MERGE Eliminado en bddd MERGE

BDDD: Acción: Líneas Origen: Líneas Destino: ACCIÓN **MERGE**

```
{'TIPO_BBDD': 'SQL_SERVER'
, 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's'
, 'TIPO_OBJETO': 'STORED PROCEDURES'
, 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
, 'TIPO_REPOSITORIO': None
, 'REPOSITORIO': 'dbo'
, 'NOMBRE_OBJETO': 'PROC_BULK_INSERT'
, 'NUM_CAMBIOS_SCRIPT_1': 0
, 'NUM_CAMBIOS_SCRIPT_2': 5
, 'DF_CODIGO_ORIGINAL_1':
, 'DF_CODIGO_ORIGINAL_2':
, 'LISTA_DICC_MERGE_HECHOS':
, 'DF_CODIGO_ACTUAL_1': 5
, 'DF_CODIGO_ACTUAL_2': 6
}
```

LISTA_DICC_MERGE_HECHOS

```
[
  {'DF_CODIGO_ACTUAL_1': 1
  , 'DF_CODIGO_ACTUAL_2': 2
  },
  {'DF_CODIGO_ACTUAL_1': 3
  , 'DF_CODIGO_ACTUAL_2': 4
  },
  {'DF_CODIGO_ACTUAL_1': 5
  , 'DF_CODIGO_ACTUAL_2': 6
  }
]
```

Dataframes
CAMBIO 1
Dataframes
CAMBIO 2

AGREGADO

Es el flag que permite en la GUI marcar las líneas de color **NARANJA**

ELIMINADO

Es el flag que permite en la GUI marcar las líneas de color **AZUL**

5

DF_CODIGO_ACTUAL_1	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t	0001\t	None	None
1	0002\t	0002\t	None	None
2	0003\t	0003\t	None	None
3	0004\t	0004\t	None	None
4	0005\t	0005\t	None	None
5	0006\t	0006\t	None	None
6	0007\t	0007\t	None	None
7	0008\t	0008\t	None	None
8	0009\t	0009\t	None	None
9	0010\t	0010\t	None	None
10	0011\t	0011\t	None	None
11	0012\t	0012\t	None	None
12	0013\t	0013\t	None	None
13	0014\t	0014\t	None	None
14	0015\t	0015\t	None	None
15	0016\t	0016\t	None	None
16	0017\t	0017\t	None	None
17	0018\t	0018\t	None	None
18	0019\t	0019\t	None	None
19	0020\t	0020\t	None	None
20	0021\t	0021\t	None	None
21	0022\t	0022\t	None	None
22	0023\t	0023\t	None	None
23	0024\t	0024\t	None	None
24	0025\t	0025\t	None	None
25	0026\t	0026\t	None	None
26	0027\t	0027\t	None	None
27	0028\t	0028\t	None	None
28	0029\t	0029\t	None	None
29	0030\t	0030\t	None	None
30	0031\t	0031\t	None	None
31	0032\t	0032\t	None	None

CAMBIO 1

AGREGADO

CAMBIO 2

AGREGADO

6

DF_CODIGO_ACTUAL_2	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t	0001\t	None	ELIMINADO
1	0002\t	0002\t	None	ELIMINADO
2	0003\t	0003\t	None	ELIMINADO
3	0004\t	0004\t	None	ELIMINADO
4	0005\t	0005\t	None	ELIMINADO
5	0006\t	0006\t	None	ELIMINADO
6	0007\t	0007\t	None	ELIMINADO
7	0008\t	0008\t	None	ELIMINADO
8	0009\t	0009\t	None	ELIMINADO
9	0010\t	0010\t	None	ELIMINADO
10	0011\t	0011\t	None	ELIMINADO
11	0012\t	0012\t	None	ELIMINADO
12	0013\t	0013\t	None	ELIMINADO
13	0014\t	0014\t	None	ELIMINADO
14	0015\t	0015\t	None	ELIMINADO
15	0016\t	0016\t	None	ELIMINADO
16	0017\t	0017\t	None	ELIMINADO
17	0018\t	0018\t	None	ELIMINADO
18	0019\t	0019\t	None	ELIMINADO
19	0020\t	0020\t	None	ELIMINADO
20	0021\t	0021\t	None	ELIMINADO
21	0022\t	0022\t	None	ELIMINADO
22	0023\t	0023\t	None	ELIMINADO
23	0024\t	0024\t	None	ELIMINADO
24	0025\t	0025\t	None	ELIMINADO
25	0026\t	0026\t	None	ELIMINADO
26	0027\t	0027\t	None	ELIMINADO
27	0028\t	0028\t	None	ELIMINADO
28	0029\t	0029\t	None	ELIMINADO
29	0030\t	0030\t	None	ELIMINADO
30	0031\t	0031\t	None	ELIMINADO
31	0032\t	0032\t	None	ELIMINADO

CAMBIO 3

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

CAMBIO 4

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: BBDD_02 Acción: Revertir Último Líneas Origen: Líneas Destino: ACCIÓN MERGE

```
{'TIPO_BBDD': 'SQL_SERVER'
, 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's'
, 'TIPO_OBJETO': 'STORED PROCEDURES'
, 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
, 'TIPO_REPOSITORIO': None
, 'REPOSITORIO': 'dbo'
, 'NOMBRE_OBJETO': 'PROC_BULK_INSERT'
, 'NUM_CAMBIOS_SCRIPT_1': 0
, 'NUM_CAMBIOS_SCRIPT_2': 5
, 'DF_CODIGO_ORIGINAL_1':
, 'DF_CODIGO_ORIGINAL_2':
, 'LISTA_DICC_MERGE_HECHOS':
, 'DF_CODIGO_ACTUAL_1': 3
, 'DF_CODIGO_ACTUAL_2': 4
}
```

LISTA_DICC_MERGE_HECHOS

```
[
  {'DF_CODIGO_ACTUAL_1': 1
  , 'DF_CODIGO_ACTUAL_2': 2
  }
, {'DF_CODIGO_ACTUAL_1': 3
  , 'DF_CODIGO_ACTUAL_2': 4
  }
, {'DF_CODIGO_ACTUAL_1': 5
  , 'DF_CODIGO_ACTUAL_2': 6
  }
]
```

Se elimina

LISTA_DICC_MERGE_HECHOS

```
[
  {'DF_CODIGO_ACTUAL_1': 1
  , 'DF_CODIGO_ACTUAL_2': 2
  }
, {'DF_CODIGO_ACTUAL_1': 3
  , 'DF_CODIGO_ACTUAL_2': 4
  }
]
```

Dataframes
CAMBIO 1
Dataframes
CAMBIO 2

Dataframes
CAMBIO 1
Dataframes
CAMBIO 2

AGREGADO

Es el flag que permite en la GUI marcar las líneas de color **NARANJA**

3

DF_CODIGO_ACTUAL_1

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\tr	0001\tr	None	None
1	0002\tr	0002\tr	None	None
2	0003\tr	0003\tr	None	None
3	0004\tr	0004\tr	None	None
4	0005\tr	0005\tr	None	None
5	0006\tr	0006\tr	None	None
6	0007\tr	0007\tr	None	None
7	0008\tr	0008\tr	None	None
8	0009\tr	0009\tr	None	None
9	0010\tr	0010\tr	None	None
10	0011\tr	0011\tr	None	None
11	0012\tr	0012\tr	None	None
12	0013\tr	0013\tr	None	None
13	0014\tr	0014\tr	None	None
14	0015\tr	0015\tr	None	None
15	0016\tr	0016\tr	None	None
16	0017\tr	0017\tr	None	None
17	0018\tr	0018\tr	None	None
18	0019\tr	0019\tr	None	None
19	0020\tr	0020\tr	None	None
20	0021\tr	0021\tr	None	None
21	0022\tr	0022\tr	None	None
22	0023\tr	0023\tr	None	None
23	0024\tr	0024\tr	None	None
24	0025\tr	0025\tr	None	None
25	0026\tr	0026\tr	None	None
26	0027\tr	0027\tr	None	None
27	0028\tr	0028\tr	None	None
28	0029\tr	0029\tr	None	None
29	0030\tr	0030\tr	None	None
30	0031\tr	0031\tr	None	None

CAMBIO 1 AGREGADO

CAMBIO 2 AGREGADO

4

DF_CODIGO_ACTUAL_2

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\tr	0001\tr	None	None
1	0002\tr	0002\tr	None	None
2	0003\tr	0003\tr	None	None
3	0004\tr	0004\tr	None	None
4	0005\tr	0005\tr	None	None
5	0006\tr	0006\tr	None	None
6	0007\tr	0007\tr	None	None
7	0008\tr	0008\tr	None	None
8	0009\tr	0009\tr	None	None
9	0010\tr	0010\tr	None	None
10	0011\tr	0011\tr	None	None
11	0012\tr	0012\tr	None	None
12	0013\tr	0013\tr	None	None
13	0014\tr	0014\tr	None	None
14	0015\tr	0015\tr	None	None
15	0016\tr	0016\tr	None	None
16	0017\tr	0017\tr	None	None
17	0018\tr	0018\tr	None	None
18	0019\tr	0019\tr	None	None
19	0020\tr	0020\tr	None	None
20	0021\tr	0021\tr	None	None
21	0022\tr	0022\tr	None	None
22	0023\tr	0023\tr	None	None
23	0024\tr	0024\tr	None	None
24	0025\tr	0025\tr	None	None
25	0026\tr	0026\tr	None	None
26	0027\tr	0027\tr	None	None
27	0028\tr	0028\tr	None	None
28	0029\tr	0029\tr	None	None
29	0030\tr	0030\tr	None	None
30	0031\tr	0031\tr	None	None

CAMBIO 1 AGREGADO

CAMBIO 2 AGREGADO

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

CAMBIO 5

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: BBDD_02 Acción: Quitar por líneas Líneas Origen: 2 5 Líneas Destino: ACCIÓN MERGE

```
{'TIPO_BBDD': 'SQL_SERVER'  
, 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's'  
, 'TIPO_OBJETO': 'STORED_PROCEDURES'  
, 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'  
, 'TIPO_REPOSITORIO': 'None'  
, 'REPOSITORIO': 'dbo'  
, 'NOMBRE_OBJETO': 'PROC_PrintError'  
, 'NUM_CAMBIOS_SCRIPT_1': 1  
, 'NUM_CAMBIOS_SCRIPT_2': 2  
  
, 'DF_CODIGO_ORIGINAL_1':  
, 'DF_CODIGO_ORIGINAL_2':  
, 'LISTA_DICC_MERGE_HECHOS':  
  
, 'DF_CODIGO_ACTUAL_1': 5  
, 'DF_CODIGO_ACTUAL_2': 6  
}
```

```
LISTA_DICC_MERGE_HECHOS  
-----  
[  
  {'DF_CODIGO_ACTUAL_1': 5  
  , 'DF_CODIGO_ACTUAL_2': 6  
  }  
]
```

ELIMINADO

Es el flag que permite en la GUI marcar las líneas de color **AZUL**

5

NUM LINEA	CODIGO	CODIGO_CON	NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t		0001\t	None	None
1	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None
2	0003\t	AS\r	0003\t	AS\r	None
3	0004\t	BEGIN\r	0004\t	BEGIN\r	None
4	0005\t	SET NOCOUNT ON;\r	0005\t	SET NOCOUNT ON;\r	CAMBIOS_LOCALIZADOS
5	0006\t		0006\t		None
6	0007\t	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	0007\t	PRINT 'Error ' + CONVERT(varchar(50)...	None
7	0008\t	, Severity ' + CONVERT(varchar(5), ...	0008\t	, Severity ' + CONVERT(varchar(5)...	None
8	0009\t	, Procedure ' + ISNULL(ERROR PROCED...	0009\t	, Procedure ' + ISNULL(ERROR PROCED...	None
9	0010\t	PRINT ERROR_MESSAGE();\r	0010\t	PRINT ERROR_MESSAGE();\r	None
10	0011\t	END;	0011\t	END;	None

6

NUM LINEA	CODIGO	CODIGO_CON	NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t		0001\t	None	None
1	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None
2	0003\t	AS\r	0003\t	AS\r	None
3	0004\t	BEGIN\r	0004\t	BEGIN\r	None
4	0005\t	--SET NOCOUNT ON;\r	0005\t	--SET NOCOUNT ON;\r	CAMBIOS_LOCALIZADOS
5	0006\t		0006\t		None
6	0007\t	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	0007\t	PRINT 'Error ' + CONVERT(varchar(50)...	None
7	0008\t	, Severity ' + CONVERT(varchar(5), ...	0008\t	, Severity ' + CONVERT(varchar(5)...	None
8	0009\t	, State ' + CONVERT(varchar(5), ERR...	0009\t	, State ' + CONVERT(varchar(5)...	CAMBIOS_LOCALIZADOS
9	0010\t	, Procedure ' + ISNULL(ERROR PROCED...	0010\t	, Procedure ' + ISNULL(ERROR PROCED...	None
10	0011\t	PRINT ERROR_MESSAGE();\r	0011\t	PRINT ERROR_MESSAGE();\r	None
11	0012\t	END;	0012\t	END;	None

CAMBIO 5

ELIMINADO

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

CAMBIO 6

MERGE

LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: BBDD_01 Acción: Migrar por líneas

Líneas Origen: 2 5 Líneas Destino: 4

ACCIÓN MERGE

```
{'TIPO_BBDD': 'SQL_SERVER'
, 'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's'
, 'TIPO_OBJETO': 'STORED PROCEDURES'
, 'TIPO_OBJETO_SUBFORM': 'Stored Procedures'
, 'TIPO_REPOSITORIO': None
, 'REPOSITORIO': 'dbo'
, 'NOMBRE_OBJETO': 'PROC_PrintError'
, 'NUM_CAMBIOS_SCRIPT_1': 1
, 'NUM_CAMBIOS_SCRIPT_2': 2

, 'DF_CODIGO_ORIGINAL_1':
, 'DF_CODIGO_ORIGINAL_2':
, 'LISTA_DICC_MERGE_HECHOS':
, 'DF_CODIGO_ACTUAL_1': 7
, 'DF_CODIGO_ACTUAL_2': 8
}
```

LISTA_DICC_MERGE_HECHOS

```
[
  {'DF_CODIGO_ACTUAL_1': 5
   , 'DF_CODIGO_ACTUAL_2': 6
  },
  {'DF_CODIGO_ACTUAL_1': 7
   , 'DF_CODIGO_ACTUAL_2': 8
  }
]
```

Dataframes
CAMBIO 5

AGREGADO

Es el flag que permite en la GUI marcar las líneas de color **NARANJA**

ELIMINADO

Es el flag que permite en la GUI marcar las líneas de color **AZUL**

7

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t	\r	None	None
1	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None	AGREGADO
2	0003\t	AS\r	None	AGREGADO
3	0004\t	BEGIN\r	None	AGREGADO
4	0005\t	SET NOCOUNT ON;\r	None	AGREGADO
5	0006\t	\r	None	None
6	0007\t	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	None	None
7	0008\t	, Severity ' + CONVERT(varchar(5), ...	None	None
8	0009\t	, Procedure ' + ISNULL(ERROR_PROCE...	None	None
9	0010\t	PRINT ERROR_MESSAGE();\r	None	None
10	0011\t	END;	None	None

CAMBIO 6

8

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t	\r	None	None
1	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None	ELIMINADO
2	0003\t	AS\r	None	ELIMINADO
3	0004\t	BEGIN\r	None	ELIMINADO
4	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None	AGREGADO
5	0003\t	AS\r	None	AGREGADO
6	0004\t	BEGIN\r	None	AGREGADO
7	0005\t	SET NOCOUNT ON;\r	None	AGREGADO
8	0005\t	--SET NOCOUNT ON;\r	None	AGREGADO
9	0002\t	\r	None	None
10	0003\t	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	None	None
11	0004\t	, Severity ' + CONVERT(varchar(5), ...	None	None
12	0005\t	, State ' + CONVERT(varchar(5), ERR...	None	None
13	0006\t	, Procedure ' + ISNULL(ERROR_PROCE...	None	None
14	0007\t	PRINT ERROR_MESSAGE();\r	None	None
15	0008\t	END;	None	None

CAMBIO 5

CAMBIO 6

CAMBIO 5

ANEXO_06

RUTINA: def_proceso_merge_realizar_cambios

CAMBIO 7

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD: Acción: Líneas Origen: Líneas Destino: **ACCIÓN** **MERGE**

```
{'TIPO_BBDD': 'SQL_SERVER',
'CHECK_OBJETO': 'Objetos ya existentes en las 2 BBDD's',
'TIPO_OBJETO': 'STORED PROCEDURES',
'TIPO_OBJETO_SUBFORM': 'Stored Procedures',
'TIPO_REPOSITORIO': 'None',
'REPOSITORIO': 'dbo',
'NOMBRE_OBJETO': 'PROC_PrintError',
'NUM_CAMBIOS_SCRIPT_1': 1,
'NUM_CAMBIOS_SCRIPT_2': 2,

'DF_CODIGO_ORIGINAL_1':
'DF_CODIGO_ORIGINAL_2':
'LISTA_DICC_MERGE_HECHOS',

'DF_CODIGO_ACTUAL_1':
'DF_CODIGO_ACTUAL_2':
}
```

LISTA_DICC_MERGE_HECHOS

```
{'DF_CODIGO_ACTUAL_1': 5,
'DF_CODIGO_ACTUAL_2': 6,
'DF_CODIGO_ORIGINAL_1': 7,
'DF_CODIGO_ORIGINAL_2': 8}
```

Se elimina la lista

Dataframes
CAMBIO 5
Dataframes
CAMBIO 6

LISTA_DICC_MERGE_HECHOS

None

Los dataframes DF_CODIGO_ACTUAL vuelven a ser los mismos que los dataframes DF_CODIGO_ORIGINAL

CAMBIOS_LOCALIZADOS

Es el flag que permite en la GUI marcar las líneas de color

VERDE

DF_CODIGO_ACTUAL_1 = DF_CODIGO_ORIGINAL_1

NUM LINEA	CODIGO	CODIGO_CON	NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t		0001\t	None	None
1	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None
2	0003\t	AS \r	0003\t	AS \r	None
3	0004\t	BEGIN \r	0004\t	BEGIN \r	None
4	0005\t	SET NOCOUNT ON; \r	0005\t	SET NOCOUNT ON; \r	CAMBIOS_LOCALIZADOS
5	0006\t		0006\t		CAMBIOS_LOCALIZADOS
6	0007\t	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	0007\t	PRINT 'Error ' + CONVERT(varchar(50)...	None
7	0008\t	, Severity ' + CONVERT(varchar(5), ...	0008\t	, Severity ' + CONVERT(varchar(5)...	None
8	0009\t	, Procedure ' + ISNULL(ERROR_PROCED...	0009\t	, Procedure ' + ISNULL(ERROR...	None
9	0010\t	PRINT ERROR_MESSAGE(); \r	0010\t	PRINT ERROR_MESSAGE(); \r	None
10	0011\t	END;	0011\t	END;	None

DF_CODIGO_ACTUAL_2 = DF_CODIGO_ORIGINAL_2

NUM LINEA	CODIGO	CODIGO_CON	NUM LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t		0001\t	None	None
1	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	0002\t	CREATE PROCEDURE [dbo].[PROC_PrintError] \r	None
2	0003\t	AS \r	0003\t	AS \r	None
3	0004\t	BEGIN \r	0004\t	BEGIN \r	None
4	0005\t	--SET NOCOUNT ON; \r	0005\t	--SET NOCOUNT ON; \r	CAMBIOS_LOCALIZADOS
5	0006\t		0006\t		CAMBIOS_LOCALIZADOS
6	0007\t	PRINT 'Error ' + CONVERT(varchar(50), ERRO...	0007\t	PRINT 'Error ' + CONVERT(varchar(50)...	None
7	0008\t	, Severity ' + CONVERT(varchar(5), ...	0008\t	, Severity ' + CONVERT(varchar(5)...	None
8	0009\t	, State ' + CONVERT(varchar(5), ERR...	0009\t	, State ' + CONVERT(varchar(5)...	CAMBIOS_LOCALIZADOS
9	0010\t	, Procedure ' + ISNULL(ERROR_PROCED...	0010\t	, Procedure ' + ISNULL(ERROR...	CAMBIOS_LOCALIZADOS
10	0011\t	PRINT ERROR_MESSAGE(); \r	0011\t	PRINT ERROR_MESSAGE(); \r	None
11	0012\t	END;	0012\t	END;	None

ANEXO_07

FUNCIÓN

def_merge_bbdd_fisicas

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

La rutina **def_merge_bbdd_fisicas** (módulo APP_2_GENERAL) permite:

- realizar los **merge** en bases de datos físicas sean MS ACCESS o SQL SERVER.
- generar la **documentación** del proceso.

La rutina tiene 3 parámetros:

PARÁMETRO	DESCRIPCIÓN
tipo_bbdd_selecc	MS_ACCESS o SQL_SERVER (es la key_1 del diccionario dicc_control_versiones_tipo_objeto , módulo APP_2_GENERAL)
lista_dicc_objetos_migrar_bbdd_fisica	Es una lista de diccionarios (uno por objeto) donde el usuario ha realizado cambios en el script de BBDD_02 en la GUI de control de versiones.
ruta_export	Es la ruta donde se documenta el proceso con los objetos afectados mediante ficheros en formato .txt. Sirve tambien para guardar los posibles logs de errores, tambien en formato .txt.

Antes de empezar con la explicación de la rutina, es necesario matizar el origen del parámetro **lista_dicc_objetos_migrar_bbdd_fisica**. Es la lista almacenada en la subkey_4 (LISTA_DICC_OBJETOS) del diccionario **dicc_control_versiones_tipo_objeto** (módulo APP_2_GENERAL), asociada a las subkeys_1 (MS_ACCESS o SQL_SERVER), subkey_2 (MERGE_BBDD_FISICA) y subkey_3 (OBJETOS_A_MIGRAR).

La lista almacenada en la subkeys_4 (LISTA_DICC_OBJETOS) se genera nada más abrir la GUI de merge en base de datos física mediante la rutina **def_merge_bbdd_fisica_lista_objetos** (módulo APP_2_GENERAL).

```
1665 class gui_ventana_merge_bbdd_fisicas:
1666     #clase que permite generar la GUI para
1667
1668     def __init__(self, master, **kwargs):
```



```
1683     #se calculan las listas de objetos donde realiz
1684     mod_gen.def_merge_bbdd_fisica_lista_objetos()
```


ANEXO_07

RUTINA: def_merge_bbdd_fisicas

Esta rutina, que no se documenta en el presente documento, permite recopilar de las listas de diccionarios almacenadas en las subkeys_4 (LISTA_DICC_OBJETOS_CONTRO_VERSIONES) del mismo diccionario **dicc_control_versiones_tipo_objeto** los objetos donde el usuario ha realizado cambios.

dicc_control_versiones_tipo_objeto			
KEY 1	KEY 2	KEY 3	KEY 4
MS_ACCESS	TIPO_OBJETO	TODOS	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		TABLA_LOCAL	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		VINCULO_ODBC	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		VINCULO_OTRO	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		RUTINAS_VBA	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		VARIABLES_VBA	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES
	MERGE_BBDD_FISICA	AJUSTES_MANUALES	COMBOBOX_GUI LISTA_DICC_OBJETOS
		OBJETOS_A_MIGRAR	COMBOBOX_GUI LISTA_DICC_OBJETOS
		LISTA_DICC_OK_MIGRACION	
		LISTA_DICC_ERRORES_MIGRACION	
SQL_SERVER	TIPO_OBJETO	TODOS	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES
		TABLAS	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES LISTA_OBJECT_ID
		VIEWS	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES LISTA_OBJECT_ID
		STORED_PROCEDURES	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES LISTA_OBJECT_ID
		FUNCIONES	COMBOBOX_CONTROL_VERSIONES TIPO_OBJETO_SUBFORM LISTA_DICC_OBJETOS_CONTROL_VERSIONES LISTA_OBJECT_ID
	MERGE_BBDD_FISICA	OBJETOS_A_MIGRAR	COMBOBOX_GUI LISTA_DICC_OBJETOS
		LISTA_DICC_OK_MIGRACION	
		LISTA_DICC_ERRORES_MIGRACION	

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A modo recordatorio del **ANEXO_06**, los diccionarios que integran las listas almacenadas en las subkeys_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES) tienen la estructura siguiente:

KEYS	DESCRIPCIÓN
TIPO_BBDD	MS_ACCESS o SQL_SERVER
CHECK_OBJETO	Valores de las keys del diccionario <code>dicc_control_versiones_tipo_concepto</code> (YA_EXISTE, SOLO_EN_BBDD_01 o SOLO_EN_BBDD_02)
TIPO_OBJETO	Los tipos de objeto mencionados en el 1er cuadro de la página anterior
TIPO_OBJETO_SUBFORM	Tipo de objeto con el literal que sale en la GUI de control de versiones y merge en base de datos física es el valor de las subkeys_4, TIPO_OBJETO_SUBFORM, del diccionario <code>dicc_control_versiones_tipo_objeto</code>
TIPO_REPOSITORIO	Solo para MS ACCESS (formulartio, userform o módulo VBA estandar)
REPOSITORIO	Nombre del módulo VBA para MS ACCESS y nombre del esquema para SQL SERVER
NOMBRE_OBJETO	Nombre del objeto
NUM_CAMBIOS_SCRIPT_1	Número de líneas de código con cambios en el script de BBDD_01 (se usa en el sub-formulario del a GUI de control de versiones)
NUM_CAMBIOS_SCRIPT_2	Número de líneas de código con cambios en el script de BBDD_02 (se usa en el sub-formulario del a GUI de control de versiones)
DF_CODIGO_ORIGINAL_1	Dataframe con el código del script de BBDD_01 donde las líneas con cambios con respecto al script de BBDD_02 estan localizadas
DF_CODIGO_ORIGINAL_2	Dataframe con el código del script de BBDD_02 donde las líneas con cambios con respecto al script de BBDD_01 estan localizadas
LISTA_DICC_MERGE_HECHOS	None al ejecutarse la función. Se informa a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)
DF_CODIGO_ACTUAL_1	Al ejecutarse la función es de inicio el mismo dataframe que DF_CODIGO_ORIGINAL_1. Se actualiza a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)
DF_CODIGO_ACTUAL_2	Al ejecutarse la función es de inicio el mismo dataframe que DF_CODIGO_ORIGINAL_2. Se actualiza a posteriori cuando el usuario realiza cambios de scripts de una base de datos a otra en la GUI de control de versiones (apartado MERGE)

Cada vez que el usuario realiza un cambio en los scripts en la GUI de control de versiones, se agrega un diccionario adicional en la lista almacenada en la key LISTA_DICC_MERGE_HECHOS (sus keys son DF_CODIGO_ACTUAL_1 y DF_CODIGO_ACTUAL_2 y contienen respectivamente los dataframes actualizados con los cambios realizados de los scripts de BBDD_01 y BBDD_02).

La rutina **def_merge_bbdd_fisica_lista_objetos** mencionada en páginas anteriores chequea en cada diccionario (es un objeto por diccionario) de las listas almacenadas en las subkeys_4 (LISTA_DICC_OBJETOS_CONTROL_VERSIONES) si el valor de la key LISTA_DICC_MERGE_HECHOS es **lista** y en caso de serlo recupera dentro del último diccionario de la misma el dataframe almacenado en la key DF_CODIGO_ACTUAL_2 (BBDD_02 es por defecto la base de datos en la cual se realiza el merge en base de datos física).

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

El objeto con su dataframe de código con cambios más actual se agrega a una lista que genera la rutina **def_merge_bbdd_fisica_lista_objetos** la cual al final del proceso es la que se almacena en la subkey_4 (LISTA_DICC_OBJETOS) mencionada en páginas anteriores.

Estas listas contienen diccionarios (uno por objeto donde se ha realizado un cambio en el script de BBDD_02) con la estructura siguiente:

KEY	DESCRIPCIÓN
TIPO_BBDD	MS_ACCESS o SQL_SERVER (es la key_1 del diccionario dicc_control_versiones_tipo_objeto, módulo APP_2_GENERAL)
CHECK_OBJETO	Valores de las keys (YA_EXISTE, SOLO_EN_BBDD_01 y SOLO_EN_BBDD_02) del diccionario dicc_control_versiones_tipo_concepto
TIPO_OBJETO_SUBFORM	Valores de las subkeys_4 (TIPO_OBJETO_SUBFORM) del diccionario dicc_control_versiones_tipo_objeto
TIPO_REPOSITORIO	Tipo de módulo (estander, UserForm VBA, formulario)
REPOSITORIO	Nombre del módulo VBA
NOMBRE_OBJETO	Nombre del objeto
ESTADO_MIGRACION	Valor estatico "Pendiente migrar"
DF_CODIGO	Dataframe con el código con cambios más actual del script de BBDD_02

El parámetro **lista_dicc_objetos_migrar_bbdd_fisica** de la presente rutina que se detalla en este anexo es la lista de diccionarios comentada en el párrafo anterior pero filtrada por el tipo de base de datos (MS ACCESS o SQL SERVER).

Las explicaciones a continuación se realizan según el entorno de base de datos (MS ACCESS o SQL SERVER). Solo en el caso de MS ACCESS, se ilustran las explicaciones con ejemplos.

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

MS ACCESS

Antes de empezar a detallar el código de la rutina del presente anexo supongamos 2 bases de datos (BBDD_01 y BBDD_02) cada una con un módulo VBA llamado **Modulo_PRUEBA**.

BBDD_01

```
Option Compare Database
Option Explicit
Option Base 1

Public Public_Version As String
Public Public_UsuarioID As String, Public_UsuarioNombre As String, Public_Array(1 To 10, 2) As Variant
Public Const Public_Nombre_Access = "BBDD"

Type UserType_Prueba_1
    SubVar_1 As String
    SubVar_2 As Long
End Type

Sub ActivarPanelNavegacion()

On Error Resume Next

DoCmd.LockNavigationPane (False)
DoCmd.SelectObject acTable, "MSysObjects", True
DoCmd.RunCommand acCmdWindowUnhide

End Sub

Function Func_CorreccionCIF(xCIF As String) As String

Dim i As Integer
Dim MayuscCIF As String
Dim CIFCorr As String

MayuscCIF = UCase(Replace(xCIF, " ", ""))

CIFCorr = ""
For i = 1 To Len(MayuscCIF)
    If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And (Asc(Mid(MayuscCIF, i, 1)) <= 90)) Or _
        ((Asc(Mid(MayuscCIF, i, 1)) >= 48) And (Asc(Mid(MayuscCIF, i, 1)) <= 57)) Then
        CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
    End If
Next i

Func_CorreccionCIF = CIFCorr

End Function
```

BBDD_02

```
Option Compare Database
Option Explicit
Option Base 1

Public Public_Version As String
Public Public_UsuarioID As String, Public_UsuarioNombre As String, Public_Array(1 To 10, 2) As Variant
Public Const Public_Nombre_Access = "BBDD_2"

Type UserType_Prueba_1
    SubVar_1 As String
    SubVar_2 As Long
    SubVar_3 As Date
End Type

Enum Enum_Prueba_1
    SubVar_1
    SubVar_2 = 1
    SubVar_3
End Enum

Function Func_CorreccionCIF(xCIF As String) As String

Dim i As Integer
Dim MayuscCIF As String
Dim CIFCorr As String

'funcion que permite limpiar el CIF
MayuscCIF = UCase(Replace(xCIF, " ", ""))

CIFCorr = ""
For i = 1 To Len(MayuscCIF)
    If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And (Asc(Mid(MayuscCIF, i, 1)) <= 90)) Or _
        ((Asc(Mid(MayuscCIF, i, 1)) >= 48) And (Asc(Mid(MayuscCIF, i, 1)) <= 57)) Then
        CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
    End If
Next i

Func_CorreccionCIF = CIFCorr

End Function
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

MS ACCESS

Supongamos que las migraciones que el usuario quiere realizar en BBDD_02 (base de datos donde se realiza el merge) son las siguientes:

→ Variables públicas VBA

BBDD_01 BBDD_01.accdb

0001 Public Public_Version As String
0002 Public Public_UsuarioID As String, Public UsuarioNombre As String, Public Array(1 To 10, 2
0003 Public Const Public_Nombre_Access = "BBDD"
0004 Type UserType_Prueba_1
0005 SubVar_1 As String
0006 SubVar_2 As Long
0007 End Type

BBDD_02 BBDD_02.accdb

0001 Public Public_Version As String
0002 Public Public_UsuarioID As String, Public UsuarioNombre As String, Public Array(1 To 10, 2
0003 Public Const Public_Nombre_Access = "BBDD_2"
0004 Type UserType_Prueba_1
0005 SubVar_1 As String
0006 SubVar_2 As Long
0007 SubVar_3 As Date
0008 End Type
0009 Enum Enum_Prueba_1
0010 SubVar_1
0011 SubVar_2 = 1
0012 SubVar_3
0013 End Enum

BBDD BBDD_02 **Acción** Quitar por líneas **Lineas Origen** 3 **Lineas Destino**

BBDD BBDD_01 **Acción** Migrar por líneas **Lineas Origen** 3 **Lineas Destino** 3

BBDD BBDD_02 **Acción** Quitar por líneas **Lineas Origen** 11 12 **Lineas Destino**

BBDD_02 BBDD_02.accdb

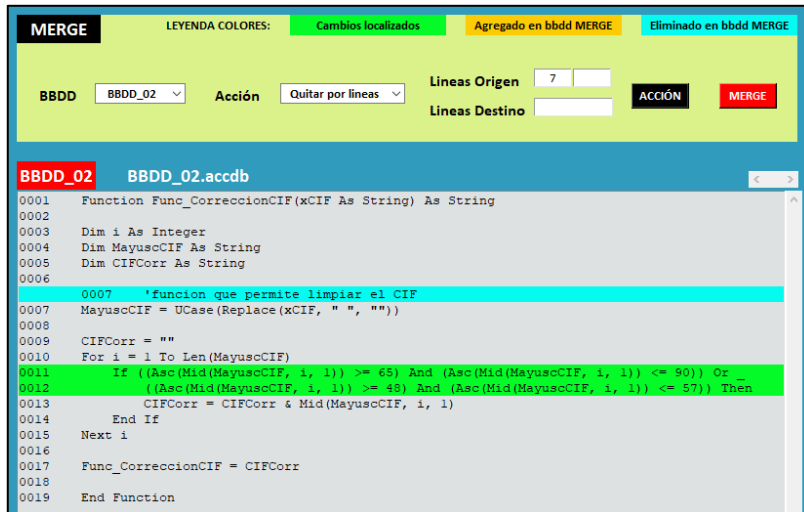
0001 Public Public_Version As String
0002 Public Public_UsuarioID As String, Public UsuarioNombre As String, Public Array(1 To 10, 2
0003 Public Const Public_Nombre_Access = "BBDD_2"
0004 Public Const Public_Nombre_Access = "BBDD"
0004 Type UserType_Prueba_1
0005 SubVar_1 As String
0006 SubVar_2 As Long
0007 SubVar_3 As Date
0008 End Type
0009 Enum Enum_Prueba_1
0010 SubVar_1
0011 SubVar_2 = 1
0012 SubVar_3
0011 End Enum

ANEXO_07

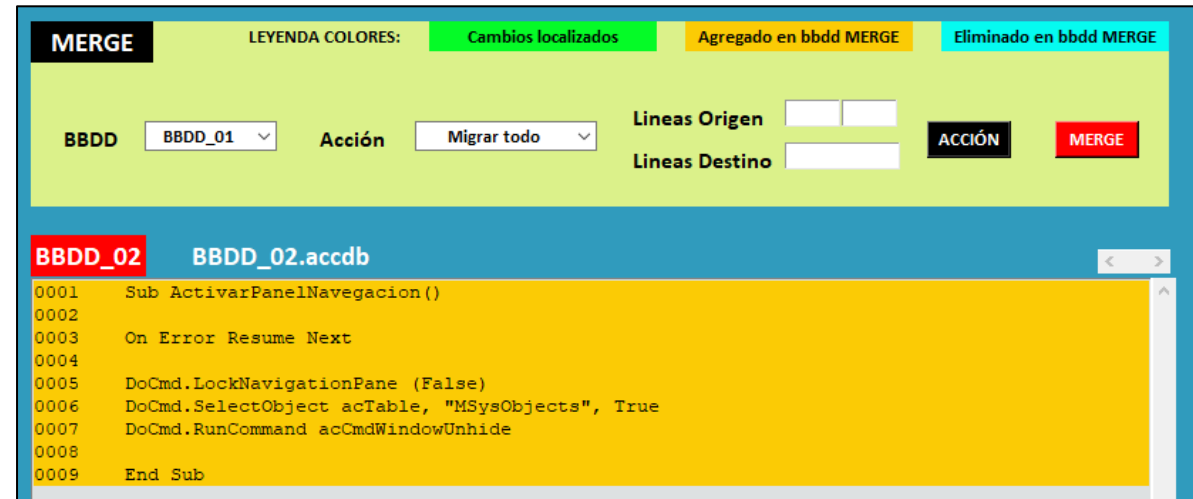
RUTINA: def_merge_bbdd_fisicas

MS ACCESS

→ Rutinas / funciones VBA



Se elimina una línea en la función **Func_CorreccionCIF** de BBDD_02



Se agrega la rutina **ActivarPanelNavegacion** a BBDD_02

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

MS ACCESS

El módulo **Modulo_PRUEBA** ha de quedar así:

```
Option Compare Database
Option Explicit
Option Base 1

Public Public_Version As String
Public Public_UsuarioID As String, Public_UsuarioNombre As String, Public_Array(1 To 10, 2) As Variant
Public Const Public_Nombre_Access = "BBDD"

Type UserType_Prueba_1
    SubVar_1 As String
    SubVar_2 As Long
    SubVar_3 As Date
End Type

Enum Enum_Prueba_1
    SubVar_1
End Enum

Function Func_CorreccionCIF(xCIF As String) As String

    Dim i As Integer
    Dim MayuscCIF As String
    Dim CIFCorr As String

    MayuscCIF = UCase(Replace(xCIF, " ", ""))

    CIFCorr = ""
    For i = 1 To Len(MayuscCIF)
        If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And (Asc(Mid(MayuscCIF, i, 1)) <= 90)) _
            Or ((Asc(Mid(MayuscCIF, i, 1)) >= 48) And (Asc(Mid(MayuscCIF, i, 1)) <= 57)) Then
            CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
        End If
    Next i

    Func_CorreccionCIF = CIFCorr
End Function

Sub ActivarPanelNavegacion()

    On Error Resume Next

    DoCmd.LockNavigationPane (False)
    DoCmd.SelectObject acTable, "MSysObjects", True
    DoCmd.RunCommand acCmdWindowUnhide

End Sub
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

MS ACCESS

Por último, supongamos una tabla y un vinculo ODBC presentes en BBDD_01 que el usuario decide también migrar a BBDD_02.

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD Acción Líneas Origen Líneas Destino ACCIÓN MERGE

BBDD_02 BBDD_02.accdb

```
0001 CREATE TABLE [T_00_PROVINCIAS]
0002 (
0003     [ID_PROVINCIA] TEXT(255)
0004     , [DESC_PROVINCIA] TEXT(255)
0005     , [FECHA_ADD] TEXT(255)
0006 )
```

MERGE LEYENDA COLORES: Cambios localizados Agregado en bbdd MERGE Eliminado en bbdd MERGE

BBDD Acción Líneas Origen Líneas Destino ACCIÓN MERGE

BBDD_02 BBDD_02.accdb

```
0001 ODBC
0002 DRIVER=SQL Server Native Client 11.0
0003 SERVER=
0004 Trusted_Connection=Yes
0005 APP=Microsoft Office
0006 DATABASE=BBDD_DESTINO
0007 LANGUAGE=Español
0008
0009 SOURCE: dbo.ErrorLog
```


ANEXO_07

RUTINA: def_merge_bbdd_fisicas

MS ACCESS

Una vez presentados los ejemplos, las explicaciones de la rutina del presente anexo para el entorno MS ACCESS se segmentan en 4 bloques:

TIPO BBDD	BLOQUE	DESCRIPCIÓN	
MS ACCESS	A1	Cálculos previos (tablas / vinculos)	Se crea la lista de diccionarios lista_dicc_tablas_y_vinculos_con_cambios donde cada diccionario corresponde a un objeto del tipo tabla o vinculo en el cual el usuario ha realizado cambios en su script en BBDD_02. Se agregan a estos diccionarios los datos necesarios para crear el objeto en la base de datos fisica y tambien para poder documentar el objeto.
	A2	Cálculos previos (módulos VBA)	Se crea la lista de diccionarios lista_dicc_modulos_con_cambios donde cada diccionario corresponde a un módulo VBA en el cual el usuario ha realizado cambios en BBDD_02 (sea en variables públicas y/o en rutinas / funciones. Se agregan a estos diccionarios los datos necesarios para poder reconstruir el módulo en la base de datos fisica y tambien para poder documentar el el módulo con sus variables públicas y/o rutinas / funciones.
	A3	Documentación del proceso	Se crea en la ruta seleccionada (parámetro ruta_export), la documentación en ficheros .txt mediante bucle sobre las listas lista_dicc_tablas_y_vinculos_con_cambios y lista_dicc_modulos_con_cambios
	A4	Merge en base de datos fisica	Se generan los merge en la base de datos fisica mediante bucle sobre las listas lista_dicc_tablas_y_vinculos_con_cambios y lista_dicc_modulos_con_cambios donde se integran bloques try except anidados para poder crear los logs de errores de migración (en caso de que los hubiese).

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A1

MS ACCESS - cálculos previos TABLAS y VINCULOS

Se calcula la lista de diccionarios **lista_dicc_tablas_y_vinculos_con_cambios** mediante bucle sobre la lista parámetro de la rutina **lista_dicc_objetos_migrar_bbdd_fisica**.

```
1976 lista_dicc_tablas_y_vinculos_con_cambios = []
1977 for dicc in lista_dicc_objetos_migrar_bbdd_fisica:
1978     if dicc["TIPO_OBJETO_SUBFORM"] in [tipo_objeto_subform_tablas, tipo_objeto_subform_vinculos_odbc, tipo_objeto_subform_vinculos_otros]:
1979
1980         tipo_objeto = func_dicc_control_versiones_tipo_objeto_buscar_en_dicc("TIPO_OBJETO_DESDE_SUBFORM", valor = dicc["TIPO_OBJETO_SUBFORM"])
1981         nombre_objeto = dicc["NOMBRE_OBJETO"]
1982         df_codigo = dicc["DF_CODIGO"]
```

Se crea el dataframe **df_codigo_documentacion** en el cual al principio de cada línea de código se agrega el flag ADD o DEL según que el usuario haya agregado o eliminado líneas

```
1987 df_codigo_documentacion = df_codigo[["CODIGO_CON_NUM_LINEA", "CONTROL_CAMBIOS_ACTUAL"]].copy()
1988
1989 df_codigo_documentacion["ADD_DEL"] = (df_codigo_documentacion.apply(lambda x: "ADD\t" if x["CONTROL_CAMBIOS_ACTUAL"] == "AGREGADO" else "DEL\t"
1990     if x["CONTROL_CAMBIOS_ACTUAL"] == "ELIMINADO" else "---\t",
1991     axis = 1))
1992
1993
1994 df_codigo_documentacion["CODIGO"] = df_codigo_documentacion[["ADD_DEL", "CODIGO_CON_NUM_LINEA"]].astype(str).apply("", join, axis = 1)
1995 df_codigo_documentacion = df_codigo_documentacion[["CODIGO"]]
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A1

MS ACCESS - cálculos previos TABLAS y VINCULOS

tipo_objeto TABLA_LOCAL
nombre_objeto T_00_PROVINCIAS

NUM_LINEA	CODIGO	CODIGO_CON	NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t CREATE TABLE [T_00_PROVINCIAS]	0001\t CREATE TABLE [T_00_PROVINCIAS]		CAMBIOS_LOCALIZADOS	AGREGADO
1	0002\t (	0002\t (		CAMBIOS_LOCALIZADOS	AGREGADO
2	0003\t \t[ID_PROVINCIA] TEXT(255)	0003\t\t[ID_PROVINCIA] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
3	0004\t \t, [DESC_PROVINCIA] TEXT(255)	0004\t\t, [DESC_PROVINCIA] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
4	0005\t \t, [FECHA_ADD] TEXT(255)	0005\t\t, [FECHA_ADD] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
5	0006\t)	0006\t)		CAMBIOS_LOCALIZADOS	AGREGADO

NUM_LINEA	CODIGO	CODIGO_CON	NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t CREATE TABLE [T_00_PROVINCIAS]	0001\t CREATE TABLE [T_00_PROVINCIAS]		CAMBIOS_LOCALIZADOS	AGREGADO
1	0002\t (	0002\t (		CAMBIOS_LOCALIZADOS	AGREGADO
2	0003\t \t[ID_PROVINCIA] TEXT(255)	0003\t\t[ID_PROVINCIA] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
3	0004\t \t, [DESC_PROVINCIA] TEXT(255)	0004\t\t, [DESC_PROVINCIA] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
4	0005\t \t, [FECHA_ADD] TEXT(255)	0005\t\t, [FECHA_ADD] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
5	0006\t)	0006\t)		CAMBIOS_LOCALIZADOS	AGREGADO

NUM_LINEA	CODIGO	CODIGO_CON	NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	ADD\t0001\t CREATE TABLE [T_00_PROVINCIAS]	ADD\t0001\t CREATE TABLE [T_00_PROVINCIAS]		CAMBIOS_LOCALIZADOS	AGREGADO
1	ADD\t0002\t (	ADD\t0002\t (		CAMBIOS_LOCALIZADOS	AGREGADO
2	ADD\t0003\t \t[ID_PROVINCIA] TEXT(255)	ADD\t0003\t\t[ID_PROVINCIA] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
3	ADD\t0004\t \t, [DESC_PROVINCIA] TEXT(255)	ADD\t0004\t\t, [DESC_PROVINCIA] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
4	ADD\t0005\t \t, [FECHA_ADD] TEXT(255)	ADD\t0005\t\t, [FECHA_ADD] TEXT(255)		CAMBIOS_LOCALIZADOS	AGREGADO
5	ADD\t0006\t)	ADD\t0006\t)		CAMBIOS_LOCALIZADOS	AGREGADO

tipo_objeto VINCULO_ODBC
nombre_objeto dbo_ErrorLog

NUM_LINEA	CODIGO	CODIGO_CON	NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t ODBC	0001\t ODBC		CAMBIOS_LOCALIZADOS	AGREGADO
1	0002\t DRIVER=SQL Server Native Client 11.0	0002\t DRIVER=SQL Server Native Client 11.0		CAMBIOS_LOCALIZADOS	AGREGADO
2	0003\t SERVER=	0003\t SERVER=		CAMBIOS_LOCALIZADOS	AGREGADO
3	0004\t Trusted Connection=Yes	0004\t Trusted Connection=Yes		CAMBIOS_LOCALIZADOS	AGREGADO
4	0005\t APP=Microsoft Office	0005\t APP=Microsoft Office		CAMBIOS_LOCALIZADOS	AGREGADO
5	0006\t DATABASE=BBDD_DESTINO	0006\t DATABASE=BBDD_DESTINO		CAMBIOS_LOCALIZADOS	AGREGADO
6	0007\t LANGUAGE=Español	0007\t LANGUAGE=Español		CAMBIOS_LOCALIZADOS	AGREGADO
7	0008\t	0008\t		CAMBIOS_LOCALIZADOS	AGREGADO
8	0009\t SOURCE: dbo.ErrorLog	0009\t SOURCE: dbo.ErrorLog		CAMBIOS_LOCALIZADOS	AGREGADO

NUM_LINEA	CODIGO	CODIGO_CON	NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t ODBC	0001\t ODBC		CAMBIOS_LOCALIZADOS	AGREGADO
1	0002\t DRIVER=SQL Server Native Client 11.0	0002\t DRIVER=SQL Server Native Client 11.0		CAMBIOS_LOCALIZADOS	AGREGADO
2	0003\t SERVER=	0003\t SERVER=		CAMBIOS_LOCALIZADOS	AGREGADO
3	0004\t Trusted Connection=Yes	0004\t Trusted Connection=Yes		CAMBIOS_LOCALIZADOS	AGREGADO
4	0005\t APP=Microsoft Office	0005\t APP=Microsoft Office		CAMBIOS_LOCALIZADOS	AGREGADO
5	0006\t DATABASE=BBDD_DESTINO	0006\t DATABASE=BBDD_DESTINO		CAMBIOS_LOCALIZADOS	AGREGADO
6	0007\t LANGUAGE=Español	0007\t LANGUAGE=Español		CAMBIOS_LOCALIZADOS	AGREGADO
7	0008\t	0008\t		CAMBIOS_LOCALIZADOS	AGREGADO
8	0009\t SOURCE: dbo.ErrorLog	0009\t SOURCE: dbo.ErrorLog		CAMBIOS_LOCALIZADOS	AGREGADO

NUM_LINEA	CODIGO	CODIGO_CON	NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	ADD\t0001\t ODBC	ADD\t0001\t ODBC		CAMBIOS_LOCALIZADOS	AGREGADO
1	ADD\t0002\t DRIVER=SQL Server Native Client 11.0	ADD\t0002\t DRIVER=SQL Server Native Client 11.0		CAMBIOS_LOCALIZADOS	AGREGADO
2	ADD\t0003\t SERVER=	ADD\t0003\t SERVER=		CAMBIOS_LOCALIZADOS	AGREGADO
3	ADD\t0004\t Trusted Connection=Yes	ADD\t0004\t Trusted Connection=Yes		CAMBIOS_LOCALIZADOS	AGREGADO
4	ADD\t0005\t APP=Microsoft Office	ADD\t0005\t APP=Microsoft Office		CAMBIOS_LOCALIZADOS	AGREGADO
5	ADD\t0006\t DATABASE=BBDD_DESTINO	ADD\t0006\t DATABASE=BBDD_DESTINO		CAMBIOS_LOCALIZADOS	AGREGADO
6	ADD\t0007\t LANGUAGE=Español	ADD\t0007\t LANGUAGE=Español		CAMBIOS_LOCALIZADOS	AGREGADO
7	ADD\t0008\t	ADD\t0008\t		CAMBIOS_LOCALIZADOS	AGREGADO
8	ADD\t0009\t SOURCE: dbo.ErrorLog	ADD\t0009\t SOURCE: dbo.ErrorLog		CAMBIOS_LOCALIZADOS	AGREGADO

Se prepara la sentencia create table y el objeto de origen (para los vínculos sean ODBC u otros), eliminando del dataframe el flag ELIMINADO en la columna CONTROL_CAMBIOS_ACTUAL.

```
1998 #se prepara la sentencia create (quitando el flag ELIMINADO)
1999 df_codigo_sin_flag_eliminado = df_codigo.loc[df_codigo["CONTROL_CAMBIOS_ACTUAL"] != "ELIMINADO", ["CODIGO"]]
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A1

MS ACCESS - cálculos previos TABLAS y VINCULOS

```

2006 if tipo_objeto == "TABLA_LOCAL":
2007
2008     sentencia_create = "\n".join([df_codigo_sin_flag_eliminado.iloc[ind, 0] for ind in df_codigo_sin_flag_eliminado.index]) if len(df_codigo_sin_flag_eliminado) != 0 else None
2009     objeto_origen = None
2010
2011
2012 elif tipo_objeto == "VINCULO_ODBC":
2013
2014     sentencia_create = "\n".join([df_codigo_sin_flag_eliminado.iloc[ind, 0] for ind in df_codigo_sin_flag_eliminado.index
2015     if len(df_codigo_sin_flag_eliminado.iloc[ind, 0]) != 0 and
2016     df_codigo_sin_flag_eliminado.iloc[ind, 0][0:6].upper() != "SOURCE"])).replace("\n", ";") if len(df_codigo_sin_flag_eliminado) != 0 else None
2017
2018     objeto_origen = df_codigo_sin_flag_eliminado.iloc[len(df_codigo_sin_flag_eliminado) - 1, 0].replace("SOURCE: ", "").strip() if len(df_codigo_sin_flag_eliminado) != 0 else None
2019
2020
2021 elif tipo_objeto == "VINCULO_OTRO":
2022     sentencia_create = "\n".join([df_codigo_sin_flag_eliminado.iloc[ind, 0]
2023     for ind in df_codigo_sin_flag_eliminado.index if len(df_codigo_sin_flag_eliminado.iloc[ind, 0]) != 0 and
2024     df_codigo_sin_flag_eliminado.iloc[ind, 0][0:6].upper() != "SOURCE"])).replace("\n", ";") if len(df_codigo_sin_flag_eliminado) != 0 else None
2025
2026     objeto_origen = df_codigo_sin_flag_eliminado.iloc[len(df_codigo_sin_flag_eliminado) - 1, 0].replace("SOURCE: ", "").strip() if len(df_codigo_sin_flag_eliminado) != 0 else None

```

tipo_objeto TABLA_LOCAL
nombre_objeto T_00_PROVINCIAS

```

df_codigo_sin_flag_eliminado
CODIGO
0 CREATE TABLE [T_00_PROVINCIAS]
1 (
2     \t[ID_PROVINCIA] TEXT(255)
3     \t, [DESC_PROVINCIA] TEXT(255)
4     \t, [FECHA_ADD] TEXT(255)
5 )

```

```

sentencia_create CREATE TABLE [T_00_PROVINCIAS]
(
    [ID_PROVINCIA] TEXT(255)
    , [DESC_PROVINCIA] TEXT(255)
    , [FECHA_ADD] TEXT(255)
)

```

objeto_origen None

tipo_objeto VINCULO_ODBC
nombre_objeto dbo_ErrorLog

```

df_codigo_sin_flag_eliminado
CODIGO ODBC
1 DRIVER=SQL Server Native Client 11.0
2 SERVER=
3 Trusted_Connection=Yes
4 APP=Microsoft Office
5 DATABASE=BBDD_DESTINO
6 LANGUAGE=Español
7
8 SOURCE: dbo.ErrorLog

```

```

sentencia_create ODBC;DRIVER=SQL Server Native Client 11.0;SERVER=;Trusted_Connection=Yes;APP=Microsoft Office;DATABASE=BBDD_DESTINO;LANGUAGE=Español

```

objeto_origen dbo.ErrorLog

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A1

MS ACCESS - cálculos previos TABLAS y VINCULOS

Se encapsulan todos los datos calculados en un diccionario temporal que se agrega a la lista `lista_dicc_tablas_y_vinculos_con_cambios`.

```

2030     dicc_temp = {"TIPO_OBJETO": func_dicc_control_versiones_tipo_objeto_buscar_en_dicc("TIPO_OBJETO_DESDE_SUBFORM", valor = dicc["TIPO_OBJETO_SUBFORM"])}
2031                 , "NOMBRE_OBJETO": nombre_objeto
2032                 , "DF_CODIGO_DOCUMENTACION": df_codigo_documentacion
2033
2034                 , "PARAMETROS_MERGE": {"SENTENCIA_CREATE": sentencia_create
2035                                     , "OBJETO_ORIGEN": objeto_origen
2036                                     }
2037             }
2038
2039     lista_dicc_tablas_y_vinculos_con_cambios.append(dicc_temp)

```

A2

MS ACCESS - cálculos previos módulos VBA

Se extrae el dataframe **df_codigos_bbdd**, resultante de la importación de código VBA de la rutina **def_proceso_access_1_import** para la base de datos BBDD_02.

Mediante la lista parámetro de la rutina **lista_dicc_objetos_migrar_bbdd_fisica**, se localizan los módulos VBA con cambios realizados por el usuario creando la lista temporal **lista_modulos_con_cambios**.

```

2051 df_codigos_bbdd = dicc_codigos_bbdd["BBDD_02"]["MS_ACCESS"]["DF_CODIGO_CALCULADO_TRAS_IMPORT"]
2052
2053
2054 #se crea la lista de los modulos donde se han realizado cambios
2055 lista_modulos_con_cambios = [[dicc["TIPO_REPOSITORIO"], dicc["REPOSITORIO"]]
2056 | | | | | | | | | | for dicc in lista_dicc_objetos_migrar_bbdd_fisica if dicc["TIPO_OBJETO_SUBFORM"] in [tipo_objeto_subform_variables_publicas, tipo_objeto_subform_rutinas]]
2057
2058 lista_modulos_con_cambios = [sublista for i, sublista in enumerate(lista_modulos_con_cambios) if sublista not in lista_modulos_con_cambios[:i]]

```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Se crea la lista de diccionarios **lista_dicc_modulos_con_cambios** donde cada diccionario que la compone es un módulo VBA con los datos necesarios para poder realizar el merge en base de datos física y para documentar el proceso.

La generación de esta lista se hace mediante un bucle (en adelante bucle_1), sobre la lista temporal **lista_modulos_con_cambios**, en el cual se reconstruye el módulo VBA (en forma de dataframe) de la iteración con las variables públicas con y sin cambios realizados por el usuario (si las hubiese), con las rutinas / funciones con y sin cambios realizados por el usuario (si las hubiese), se agregan las rutinas / funciones de la base de datos BBDD_01 que no estaban en BBDD_02 (si el usuario ha realizado merge de este tipo).

Se crea también la documentación en forma de listas de dataframes para las rutinas / funciones, un dataframe por rutina) y en forma de un único dataframe para las variables públicas del módulo de la iteración.

Al final de cada iteración, se crea un diccionario temporal donde se almacenan los datos comentados que se agrega a la lista **lista_dicc_modulos_con_cambios**.

```
2066 lista_dicc_modulos_con_cambios = []
2067 if len(lista_modulos_con_cambios) != 0:
2068
2069     for tipo_modulo_con_cambios, nombre_modulo_con_cambios in lista_modulos_con_cambios:
```

El primer paso dentro de la iteración sobre bucle_1 es extraer el código VBA original del módulo de BBDD_02 antes de todos los posibles cambios realizados por el usuario, mediante el dataframe **df_codigo_original_modulo**.

```
2072 df_codigo_original_modulo = (df_codigos_bbdd.loc[(df_codigos_bbdd["TIPO_MODULO"] == tipo_modulo_con_cambios) & (df_codigos_bbdd["NOMBRE_MODULO"] == nombre_modulo_con_cambios),
2073 [i for i in df_codigos_bbdd.columns]])
2074
2075 df_codigo_original_modulo.reset_index(drop = True, inplace = True)
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Los pasos a seguir dentro de la iteración sobre bucle_1 son:

PASO	DESCRIPCIÓN
Encabezado de módulo	Se crea el dataframe df_código_módulo_encabezado_reconstruido filtrando el dataframe df_código_original_módulo por el valor distinto a NO en la columna ES_VARIABLE_PUBLICA.
Variables públicas	Se crea el dataframe df_código_módulo_variables_publicas_antes_cambios con las variables públicas del módulo antes de BBDD_020 antes de cualquier cambio (si los hubiese) realizado por el usuario. Se crea el dataframe df_código_módulo_variables_publicas_para_reconstruccion, filtrando el dataframe de las variables públicas con cambios (va a nivel de módulo no de variable pública) donde se filtra se quitan las líneas de código con el flag ELIMINADO en la columna CONTROL_CAMBIOS_ACTUAL. Se crea el dataframe df_código_módulo_variables_publicas_para_documentación que sirve para documentar el proceso donde en cada línea de código se agrega al principio un flag ADD o DEL según que se hayan agregado o eliminado líneas. Se crea el dataframe df_código_módulo_variables_publicas_reconstruido que es el dataframe df_código_módulo_variables_publicas_para_reconstrucción si se han localizado cambios de variables públicas en el módulo de la iteración, en caso contrario pasa a ser el dataframe df_código_módulo_variables_publicas_antes_cambios.
Rutinas / funciones	Se crea la lista lista_rutinas_para_reconstruccion_modulo de las rutinas / funciones dentro del módulo de la iteración. Es una lista de lista donde cada sublista contiene: --> el número de orden de la rutina / función dentro del módulo --> el nombre de la rutina / función del módulo --> el código de la rutina con los cambios (si los hay y sino el código original) --> el tipo de acción a realizar (MANTENER o ELIMINAR) Se recuperan todas las rutinas / funciones del módulo de BBDD_02 independientemente de que el usuario haya realizado cambios en algunas o no para obtener en que orden aparecen en el módulo y mediante bucle sobre esta lista de rutinas / funciones en cada iteración se localizan las que tienen cambios realizados por el usuario (si los tienen se conserva el código de la rutina con cambios, quitando las líneas donde el flag pone ELIMINADO en caso contrario se conserva el código original), se informa también de la acción a realizar (si el dataframe de código de la rutina / función tras quitar las líneas con el flag ELIMINADO devuelve un dataframe vacío no se agrega la rutina / función en la reconstrucción del módulo para el merge en bbdd física). Se agrega también el dataframe con cambios preparado para su exportación en fichero .txt donde en cada línea al principio se agrega ADD o DEL según que la línea se haya agregado o eliminado respectivamente. Al final del proceso se agregan a esta lista las rutinas que no estaban en BBDD_02 pero que el usuario ha migrado desde la BBDD_01. Estas rutinas dentro del módulo reconstruido se agregan al final del módulo para conservar más o menos el mismo orden del módulo original antes de los cambios. La lista lista_rutinas_para_reconstruccion_modulo se re-ordena por el número de orden de la rutina dentro del módulo.
Módulo reconstruido	Se crea el dataframe df_codigo_modulo_reconstruido de código reconstruido para el módulo de la iteración sobre bucle_1 concatenando los dataframes df_codigo_modulo_encabezado_reconstruido, df_codigo_modulo_variables_publicas_reconstruido y df_codigo_modulo_rutinas_reconstruido.
Agregación de los datos a la lista	Se crea un diccionario temporal para almacenar los datos comentados en pasos anteriores y se agrega este diccionario a la lista lista_dicc_modulos_con_cambios.

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Encabezado de módulo

```
2083 df_codigo_modulo_encabezado_reconstruido = df_codigo_original_modulo.loc[df_codigo_original_modulo["ES_ENCABEZADO_MODULO"] == "SI", ["CODIGO"]]
2084
2085 df_codigo_modulo_encabezado_reconstruido = (pd.concat([df_codigo_modulo_encabezado_reconstruido, pd.DataFrame({"CODIGO": [""]})])
2086 | if len(df_codigo_modulo_encabezado_reconstruido) != 0 else pd.DataFrame({"CODIGO": [""]}))
2087
2088 df_codigo_modulo_encabezado_reconstruido.reset_index(drop = True, inplace = True)
```



```
df_codigo_modulo_encabezado_reconstruido
CODIGO
0 Option Compare Database
1 Option Explicit
2 Option Base 1
3
```

Variables públicas

```
2097 df_codigo_modulo_variables_publicas_antes_cambios = df_codigo_original_modulo.loc[df_codigo_original_modulo["ES_VARIABLE_PUBLICA"] != "NO", ["CODIGO"]]
2098 df_codigo_modulo_variables_publicas_antes_cambios = pd.concat([df_codigo_modulo_variables_publicas_antes_cambios, pd.DataFrame({"CODIGO": [""]})])
2099 df_codigo_modulo_variables_publicas_antes_cambios.reset_index(drop = True, inplace = True)
```



```
df_codigo_modulo_variables_publicas_antes_cambios
CODIGO
0 Public Public_Version As String
1 Public Public_UsuarioID As String, Public Usua...
2 Public Const Public_Nombre_Access = "BBDD_2"
3 Type UserType_Prueba_1
4 SubVar_1 As String
5 SubVar_2 As Long
6 SubVar_3 As Date
7 End Type
8 Enum Enum_Prueba_1
9 SubVar_1
10 SubVar_2 = 1
11 SubVar_3
12 End Enum
13
```


ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Variables públicas

```
2145 df_codigo_modulo_variables_publicas_reconstruido = (df_codigo_modulo_variables_publicas_para_reconstruccion
2146 if isinstance(df_codigo_modulo_variables_publicas_para_reconstruccion, pd.DataFrame)
2147 else df_codigo_modulo_variables_publicas_antes_cambios)
```



```
df_codigo_modulo_variables_publicas_reconstruido

CODIGO
0      Public Public_Version As String
1  Public Public_UsuarioID As String, Public Usua...
2      Public Const Public_Nombre_Access = "BBDD"
3          Type UserType_Prueba_1
4              SubVar_1 As String
5              SubVar_2 As Long
6              SubVar_3 As Date
7          End Type
8          Enum Enum_Prueba_1
9              SubVar_1
10         End Enum
11
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

Se crea el dataframe **df_codigo_original_modulo_rutinas**, filtrando el dataframe **df_codigo_original_modulo**, comentado en páginas anteriores, por las líneas de código del módulo de la iteración sobre bucle_1 asociadas a rutinas / funciones VBA. Se extraen las columnas NOMBRE_RUTINA, ORDEN_RUTINA_EN_MODULO y CODIGO.

```
2186 df_codigo_original_modulo_rutinas = df_codigo_original_modulo.loc[df_codigo_original_modulo["NOMBRE_RUTINA"].isnull() == False, ["NOMBRE_RUTINA", "ORDEN_RUTINA_EN_MODULO", "CODIGO"]
2187 df_codigo_original_modulo_rutinas.reset_index(drop = True, inplace = True)
```



	NOMBRE_RUTINA	ORDEN_RUTINA_EN_MODULO	CODIGO
0	Func_CorreccionCIF	1.0	Function Func_CorreccionCIF(xCIF As String) As...
1	Func_CorreccionCIF	1.0	
2	Func_CorreccionCIF	1.0	Dim i As Integer
3	Func_CorreccionCIF	1.0	Dim MayuscCIF As String
4	Func_CorreccionCIF	1.0	Dim CIFCorr As String
5	Func_CorreccionCIF	1.0	
6	Func_CorreccionCIF	1.0	'funcion que permite limpiar el CIF
7	Func_CorreccionCIF	1.0	MayuscCIF = UCase(Replace(xCIF, " ", ""))
8	Func_CorreccionCIF	1.0	
9	Func_CorreccionCIF	1.0	CIFCorr = ""
10	Func_CorreccionCIF	1.0	For i = 1 To Len(MayuscCIF)
11	Func_CorreccionCIF	1.0	If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And ...
12	Func_CorreccionCIF	1.0	((Asc(Mid(MayuscCIF, i, 1)) >= 48) And...
13	Func_CorreccionCIF	1.0	CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
14	Func_CorreccionCIF	1.0	End If
15	Func_CorreccionCIF	1.0	Next i
16	Func_CorreccionCIF	1.0	
17	Func_CorreccionCIF	1.0	Func_CorreccionCIF = CIFCorr
18	Func_CorreccionCIF	1.0	
19	Func_CorreccionCIF	1.0	End Function

Mediante bucle sobre los índices del dataframe, mencionado en párrafo anterior, se crea la lista temporal **lista_rutinas_para_reconstruccion_modulo** que contiene la información de código VBA a nivel de rutinas / funciones del módulo antes de los cambios realizados por el usuario.

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

Es lista de listas donde cada sublista contiene:

ITEMS SUBLISTA
Número de orden de la rutina dentro del módulo
Nombre de la rutina
Datframe de código VBA con cambios realizados por el usuario
Datframe de código VBA para la documentación
Tipo de acción a realizar cuando se pase a reconstruir el módulo a nivel de rutinas / funciones

Los ítems 3, 4 y 5 se establecen a None de inicio y se calculan más adelante.

```
2190 lista_rutinas_para_reconstruccion_modulo = []
2191 if len(df_codigo_original_modulo_rutinas) != 0:
2192
2193     lista_rutinas_para_reconstruccion_modulo = [[df_codigo_original_modulo_rutinas.iloc[ind, df_codigo_original_modulo_rutinas.columns.get_loc("ORDEN_RUTINA_EN_MODULO")]
2194     , df_codigo_original_modulo_rutinas.iloc[ind, df_codigo_original_modulo_rutinas.columns.get_loc("NOMBRE_RUTINA")]
2195     , None #es para el df de codigo con cambios (se calcula mas adelante)
2196     , None #es para el df de codigo para documentar (se calcula mas adelante)
2197     , None #es para el tipo de accion a realizar (se calcula mas adelante)
2198     ] for ind in df_codigo_original_modulo_rutinas.index]
2199
2200
2201 #se quitan los duplicados y se ordena por el numero de orden de la rutina dentro del modulo
2202 lista_rutinas_para_reconstruccion_modulo = [sublista for i, sublista in enumerate(lista_rutinas_para_reconstruccion_modulo) if sublista not in lista_rutinas_para_reconstruccion_modulo[:i]]
2203 lista_rutinas_para_reconstruccion_modulo = sorted(lista_rutinas_para_reconstruccion_modulo, key = lambda x: x[0])
```



```
lista_rutinas_para_reconstruccion_modulo
[
  [
    1.0
    , Func_CorreccionCIF
    , None
    , None
    , None
  ]
]
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

Se inicia bucle (en adelante bucle_2), sobre la lista **lista_rutinas_para_reconstruccion_modulo** para poder completar los ítems 3, 4 y 5 de cada una de sus sublistas.

```
2207 for indice_lista_rutinas_modulo, item_lista_rutinas_modulo in enumerate(lista_rutinas_para_reconstruccion_modulo):
2208
2209     orden_rutina_en_modulo = item_lista_rutinas_modulo[indice_lista_numero_orden_rutina]
2210     nombre_rutina = item_lista_rutinas_modulo[indice_lista_nombre_rutina]
```

Los pasos que se realizan en cada una de las iteraciones sobre bucle_2 son:

PASO	DESCRIPCIÓN
1	Se crea el dataframe df_codigo_original_rutina con el código original de la rutina / función (al cual se le agrega una línea de código en blanco al final para tener separación con otras rutinas). Se usa más adelante para crear el dataframe df_codigo_rutina_reconstruido
2	Se crea el dataframe df_codigo_rutina_con_cambios con el código de la rutina / función con los cambios realizados por el usuario. Se usa para crear los dataframes df_codigo_rutina_para_reconstruccion y df_codigo_rutina_para_documentacion .
3	Se crea el dataframe df_codigo_rutina_para_reconstruccion con el código de la rutina / función donde se quitan las líneas con el flag ELIMINADO en la columna CONTROL_CAMBIOS_ACTUAL
4	Se crea el dataframe df_codigo_rutina_para_documentacion con el código de la rutina / función donde en cada línea, al principio, se agrega el flag ADD o DEL según que se hayan agregado o eliminado líneas
5	Se crea la variable temporal accion_rutina_a_realizar que localiza el tipo de acción a realizar cuando se reconstruya el módulo (CON_CAMBIOS o ELIMINAR, este último es cuando en la GUI el usuario en una rutina / función de BBDD_02 ha optado por la opción "Quitar todo")
6	Se crea el dataframe df_codigo_rutina_reconstruido mediante combinación de los dataframes df_codigo_rutina_para_reconstruccion y df_codigo_original_rutina (es df_codigo_rutina_para_reconstruccion si este es dataframe sino es df_codigo_original_rutina)
7	Se informan los ítems 3, 4 y 5 de las sublistas de la lista lista_rutinas_para_reconstruccion_modulo

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

1

```
2214 df_codigo_original_rutina = df_codigo_original_modulo.loc[df_codigo_original_modulo["ORDEN_RUTINA_EN_MODULO"] == orden_rutina_en_modulo, ["CODIGO"]]  
2215 df_codigo_original_rutina = pd.concat([df_codigo_original_rutina, pd.DataFrame({"CODIGO": [""]})])  
2216 df_codigo_original_rutina.reset_index(drop = True, inplace = True)
```



	NOMBRE_RUTINA	ORDEN_RUTINA_EN_MODULO	CODIGO
0	Func_CorreccionCIF	1.0	Function Func_CorreccionCIF(xCIF As String) As...
1	Func_CorreccionCIF	1.0	
2	Func_CorreccionCIF	1.0	Dim i As Integer
3	Func_CorreccionCIF	1.0	Dim MayuscCIF As String
4	Func_CorreccionCIF	1.0	Dim CIFCorr As String
5	Func_CorreccionCIF	1.0	
6	Func_CorreccionCIF	1.0	'funcion que permite limpiar el CIF
7	Func_CorreccionCIF	1.0	MayuscCIF = UCase(Replace(xCIF, " ", ""))
8	Func_CorreccionCIF	1.0	
9	Func_CorreccionCIF	1.0	CIFCorr = ""
10	Func_CorreccionCIF	1.0	For i = 1 To Len(MayuscCIF)
11	Func_CorreccionCIF	1.0	If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And ...
12	Func_CorreccionCIF	1.0	((Asc(Mid(MayuscCIF, i, 1)) >= 48) And...
13	Func_CorreccionCIF	1.0	CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
14	Func_CorreccionCIF	1.0	End If
15	Func_CorreccionCIF	1.0	Next i
16	Func_CorreccionCIF	1.0	
17	Func_CorreccionCIF	1.0	Func_CorreccionCIF = CIFCorr
18	Func_CorreccionCIF	1.0	
19	Func_CorreccionCIF	1.0	End Function

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

2

```
2226 lista_df_rutinas_con_cambios = [dicc["DF_CODIGO"] for dicc in lista_dicc_objetos_migrar_bbdd_fisica
2227                                     if dicc["TIPO_REPOSITORIO"] == tipo_modulo_con_cambios and dicc["REPOSITORIO"] == nombre_modulo_con_cambios and
2228                                     dicc["TIPO_OBJETO_SUBFORM"] == tipo_objeto_subform_rutinas and dicc["NOMBRE_OBJETO"] == nombre_rutina
2229                                     ]
2230
2231
2232 accion_rutina_a_realizar = None
2233 df_codigo_rutina_para_reconstruccion = None
2234 df_codigo_rutina_para_documentacion = None
2235
2236 if len(lista_df_rutinas_con_cambios) != 0:
2237
2238     df_codigo_rutina_con_cambios = lista_df_rutinas_con_cambios[0]
```



df_codigo_rutina_con_cambios					
NUM_LINEA	CODIGO		CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0	0001\t	Function Func_CorreccionCIF(xCIF As String) As...	0001\tFunction Func_CorreccionCIF(xCIF As Stri...	None	None
1	0002\t		0002\t	None	None
2	0003\t	Dim i As Integer	0003\tDim i As Integer	None	None
3	0004\t	Dim MayuscCIF As String	0004\tDim MayuscCIF As String	None	None
4	0005\t	Dim CIFCorr As String	0005\tDim CIFCorr As String	None	None
5	0006\t		0006\t	None	None
6	0007\t	'funcion que permite limpiar el CIF	\t0007\t'funcion que permite limpiar el CIF	CAMBIOS_LOCALIZADOS	ELIMINADO
7	0007\t	MayuscCIF = UCase(Replace(xCIF, " ", ""))	0007\tMayuscCIF = UCase(Replace(xCIF, " ", ""))	None	None
8	0008\t		0008\t	None	None
9	0009\t	CIFCorr = ""	0009\tCIFCorr = ""	None	None
10	0010\t	For i = 1 To Len(MayuscCIF)	0010\tFor i = 1 To Len(MayuscCIF)	None	None
11	0011\t	If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And ...	0011\t If ((Asc(Mid(MayuscCIF, i, 1)) >= 65...	CAMBIOS_LOCALIZADOS	CAMBIOS_LOCALIZADOS
12	0012\t	((Asc(Mid(MayuscCIF, i, 1)) >= 48) And...	0012\t ((Asc(Mid(MayuscCIF, i, 1)) >= 4...	CAMBIOS_LOCALIZADOS	CAMBIOS_LOCALIZADOS
13	0013\t	CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)	0013\t CIFCorr = CIFCorr & Mid(MayuscCI...	None	None
14	0014\t	End If	0014\t End If	None	None
15	0015\t	Next i	0015\tNext i	None	None
16	0016\t		0016\t	None	None
17	0017\t	Func_CorreccionCIF = CIFCorr	0017\tFunc_CorreccionCIF = CIFCorr	None	None
18	0018\t		0018\t	None	None
19	0019\t	End Function	0019\tEnd Function	None	None

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

3

```
2240 if isinstance(df_codigo_rutina_con_cambios, pd.DataFrame):
2241
2242     #se crea el df df_codigo_rutina_para_reconstruccion
2243     df_codigo_rutina_para_reconstruccion = df_codigo_rutina_con_cambios.loc[df_codigo_rutina_con_cambios["CONTROL_CAMBIOS_ACTUAL"] != "ELIMINADO", ["CODIGO"]]
2244     df_codigo_rutina_para_reconstruccion = pd.concat([df_codigo_rutina_para_reconstruccion, pd.DataFrame({"CODIGO": [""]})])
2245     df_codigo_rutina_para_reconstruccion.reset_index(drop = True, inplace = True)
```



```
df_codigo_rutina_para_reconstruccion

                                CODIGO
0  Function Func_CorreccionCIF(xCIF As String) As...
1
2                                Dim i As Integer
3                                Dim MayuscCIF As String
4                                Dim CIFCorr As String
5
6                                MayuscCIF = UCase(Replace(xCIF, " ", ""))
7
8                                CIFCorr = ""
9                                For i = 1 To Len(MayuscCIF)
10                               If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And ...
11                               ((Asc(Mid(MayuscCIF, i, 1)) >= 48) And...
12                               CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
13                               End If
14                               Next i
15
16                               Func_CorreccionCIF = CIFCorr
17
18                               End Function
19
```


ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

4

```
2249 df_codigo_rutina_para_documentacion = df_codigo_rutina_con_cambios[["CODIGO_CON_NUM_LINEA", "CONTROL_CAMBIOS_ACTUAL"]].copy()
2250
2251 df_codigo_rutina_para_documentacion["ADD_DEL"] = (df_codigo_rutina_para_documentacion.apply(lambda x: "ADD\t" if x["CONTROL_CAMBIOS_ACTUAL"] == "AGREGADO" else "DEL\t"
2252                                                    if x["CONTROL_CAMBIOS_ACTUAL"] == "ELIMINADO" else "---\t",
2253                                                    axis = 1))
2254
2255 df_codigo_rutina_para_documentacion["CODIGO"] = df_codigo_rutina_para_documentacion[["ADD_DEL", "CODIGO_CON_NUM_LINEA"]].astype(str).apply("", join, axis = 1)
2256 df_codigo_rutina_para_documentacion = df_codigo_rutina_para_documentacion[["CODIGO"]]
```



```
df_codigo_rutina_para_documentacion
CODIGO
0 ---\t0001\tFunction Func_CorreccionCIF(xCIF As...
1 ---\t0002\t
2 ---\t0003\tDim i As Integer
3 ---\t0004\tDim MayuscCIF As String
4 ---\t0005\tDim CIFCorr As String
5 ---\t0006\t
6 DEL\t \t0007\t'funcion que permite limpiar ...
7 ---\t0007\tMayuscCIF = UCase(Replace(xCIF, " "...
8 ---\t0008\t
9 ---\t0009\tCIFCorr = ""
10 ---\t0010\tFor i = 1 To Len(MayuscCIF)
11 ---\t0011\t If ((Asc(Mid(MayuscCIF, i, 1)) ...
12 ---\t0012\t ((Asc(Mid(MayuscCIF, i, 1))...
13 ---\t0013\t CIFCorr = CIFCorr & Mid(May...
14 ---\t0014\t End If
15 ---\t0015\tNext i
16 ---\t0016\t
17 ---\t0017\tFunc_CorreccionCIF = CIFCorr
18 ---\t0018\t
19 ---\t0019\tEnd Function
```

5

```
2260 if isinstance(df_codigo_rutina_con_cambios, pd.DataFrame):
2261     accion_rutina_a_realizar = "CON_CAMBIOS" if len(df_codigo_rutina_con_cambios) != 0 else "ELIMINAR"
```



```
accion_rutina_a_realizar
CON_CAMBIOS
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

6

```
2266 #se crea el df de codigo de rutina reconstruido
2267 df_codigo_rutina_reconstruido = df_codigo_rutina_para_reconstruccion if isinstance(df_codigo_rutina_para_reconstruccion, pd.DataFrame) else df_codigo_original_rutina
```



```
df_codigo_rutina_reconstruido
CODIGO
0 Function Func_CorreccionCIF(xCIF As String) As...
1
2 Dim i As Integer
3 Dim MayuscCIF As String
4 Dim CIFCorr As String
5
6 MayuscCIF = UCase(Replace(xCIF, " ", ""))
7
8 CIFCorr = ""
9 For i = 1 To Len(MayuscCIF)
10 If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And ...
11 ((Asc(Mid(MayuscCIF, i, 1)) >= 48) And...
12 CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
13 End If
14 Next i
15
16 Func_CorreccionCIF = CIFCorr
17
18 End Function
19
```

7

```
2270 #se informan los df y la accion a realizar en la sublista de lista_rutinas_para_reconstruccion_modulo
2271 lista_rutinas_para_reconstruccion_modulo[indice_lista_rutinas_modulo][indice_lista_df_codigo_rutina_reconstruida] = df_codigo_rutina_reconstruido
2272 lista_rutinas_para_reconstruccion_modulo[indice_lista_rutinas_modulo][indice_lista_df_codigo_rutina_documentacion] = df_codigo_rutina_para_documentacion
2273 lista_rutinas_para_reconstruccion_modulo[indice_lista_rutinas_modulo][indice_lista_accion_rutina_a_realizar] = accion_rutina_a_realizar
```

```
2180 indice_lista_df_codigo_rutina_reconstruida = 2
2181 indice_lista_df_codigo_rutina_documentacion = 3
2182 indice_lista_accion_rutina_a_realizar = 4
```



```
[
  1.0
  , Func_CorreccionCIF
  , 6
  , 4
  , CON_CAMBIOS
]
```

4 Dataframe del paso 4

6 Dataframe del paso 6

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

Una vez finalizado el bucle_2 con la generación de la lista **lista_rutinas_para_reconstruccion_modulo** se pasa a agregar a la misma aquellas rutinas / funciones que no estaban de inicio en BBDD_02 pero si en BBDD_01 y que el usuario ha decidido migrar.

PASO	DESCRIPCIÓN
1	Se crea la lista lista_nuevas_rutinas que almacena las rutinas / funciones nuevas en BBDD_02 migradas desde BBDD_01. Se realiza la búsqueda en la lista parámetro de la presente rutina que se detalla en este anexo (lista_dicc_objetos_migrar_bbdd_fisica) donde se buscan los diccionarios (por objeto) cuya keys de tipo y nombre de módulo corresponden con los valores de la iteración sobre bucle_1 y tambien donde la key CHECK_OBJETO sea SOLO_EN_BBDD_01 (key del diccionario de configuración dicc_control_versiones_tipo_concepto). Es lista de listas donde cada sublista que la componen contiene: --> nombre del objeto --> dataframe con el código del objeto
2	Mediante bucle sobre la lista lista_nuevas_rutinas, se crean los dataframes df_codigo_nueva_rutina_reconstruida (al cual se quitan las lineas con el flag ELIMINADO en la columna CONTROL_CAMBIOS_ACTUAL) y df_codigo_nueva_rutina_documentacion (al cual se agrega en cada linea al principio el flag ADD o DEL según que el usuario haya agregado o eliminado lineas). Se calcula tambien el número de orden de la nueva rutina / función dentro del módulo (es la longitud de la lista lista_rutinas_para_reconstruccion_modulo a la cual se le agrega 1)
3	Se agregan los dataframes df_codigo_nueva_rutina_reconstruida y df_codigo_nueva_rutina_documentacion a la lista lista_rutinas_para_reconstruccion_modulo. En el item 5 de cada una de sus sublistas se informa el flag NEW

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

1

```
2290 lista_nuevas_rutinas = [[dicc["NOMBRE_OBJETO"], dicc["DF_CODIGO"]] for dicc in lista_dicc_objetos_migrar_bbdd_fisica
2291 if dicc["TIPO_REPOSITORIO"] == tipo_modulo_con_cambios and dicc["REPOSITORIO"] == nombre_modulo_con_cambios and
2292 func_dicc_control_versiones_tipo_objeto_buscar_en_dicc("TIPO_OBJETO_DESDE_SUBFORM", valor = dicc["TIPO_OBJETO_SUBFORM"]) == "RUTINAS_VBA" and
2293 dicc["CHECK_OBJETO"] == texto_check_objeto_solo_bbdd_01]
```

```
[
  [
    , ActivarPanelNavegacion
  ]
]
```

NUM_LINEA	CODIGO	CODIGO_CON_NUM_LINEA	CONTROL_CAMBIOS_ORIGINAL	CONTROL_CAMBIOS_ACTUAL
0 0001\t	Sub ActivarPanelNavegacion()	0001\tSub ActivarPanelNavegacion()	CAMBIOS_LOCALIZADOS	AGREGADO
1 0002\t		0002\t	CAMBIOS_LOCALIZADOS	AGREGADO
2 0003\t	On Error Resume Next	0003\tOn Error Resume Next	CAMBIOS_LOCALIZADOS	AGREGADO
3 0004\t		0004\t	CAMBIOS_LOCALIZADOS	AGREGADO
4 0005\t	DoCmd.LockNavigationPane (False)	0005\tDoCmd.LockNavigationPane (False)	CAMBIOS_LOCALIZADOS	AGREGADO
5 0006\t	DoCmd.SelectObject acTable, "MSysObjects", True	0006\tDoCmd.SelectObject acTable, "MSysObjects..."	CAMBIOS_LOCALIZADOS	AGREGADO
6 0007\t	DoCmd.RunCommand acCmdWindowUnhide	0007\tDoCmd.RunCommand acCmdWindowUnhide	CAMBIOS_LOCALIZADOS	AGREGADO
7 0008\t		0008\t	CAMBIOS_LOCALIZADOS	AGREGADO
8 0009\t	End Sub	0009\tEnd Sub	CAMBIOS_LOCALIZADOS	AGREGADO

2

```
2295 if len(lista_nuevas_rutinas) != 0:
2296     for item_nuevas_rutinas in lista_nuevas_rutinas:
2297
2298         numero_orden_en_modulo_nueva_rutina = len(lista_rutinas_para_reconstruccion_modulo) + 1
2299         nombre_nueva_rutina = item_nuevas_rutinas[0]
2300         df_codigo_nueva_rutina = item_nuevas_rutinas[1]
```

```
numero_orden_en_modulo_nueva_rutina    2
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

2

```
2304 df_codigo_nueva_rutina_reconstruida = df_codigo_nueva_rutina.loc[df_codigo_nueva_rutina["CONTROL_CAMBIOS_ACTUAL"] != "ELIMINADO", ["CODIGO"]]
2305 df_codigo_nueva_rutina_reconstruida.reset_index(drop = True, inplace = True)
2306
2307 df_codigo_nueva_rutina_reconstruida = pd.concat([df_codigo_nueva_rutina_reconstruida, pd.DataFrame({"CODIGO": [""]})])
```



```
df_codigo_nueva_rutina_reconstruida
CODIGO
0      Sub ActivarPanelNavegacion()
1
2      On Error Resume Next
3
4      DoCmd.LockNavigationPane (False)
5 DoCmd.SelectObject acTable, "MSysObjects", True
6      DoCmd.RunCommand acCmdWindowUnhide
7
8      End Sub
0
```

```
2311 df_codigo_nueva_rutina_documentacion = df_codigo_nueva_rutina[["CODIGO_CON_NUM_LINEA", "CONTROL_CAMBIOS_ACTUAL"]].copy()
2312
2313 df_codigo_nueva_rutina_documentacion["ADD_DEL"] = (df_codigo_nueva_rutina_documentacion.apply(lambda x: "ADD\t" if x["CONTROL_CAMBIOS_ACTUAL"] == "AGREGADO" else "DEL\t"
2314                                                    if x["CONTROL_CAMBIOS_ACTUAL"] == "ELIMINADO" else "---\t",
2315                                                    axis = 1))
2316
2317 df_codigo_nueva_rutina_documentacion["CODIGO"] = df_codigo_nueva_rutina_documentacion[["ADD_DEL", "CODIGO_CON_NUM_LINEA"]].astype(str).apply("".join, axis = 1)
2318 df_codigo_nueva_rutina_documentacion = df_codigo_nueva_rutina_documentacion[["CODIGO"]]
```



```
df_codigo_nueva_rutina_documentacion
CODIGO
0      ADD\t0001\tSub ActivarPanelNavegacion()
1      ADD\t0002\t
2      ADD\t0003\tOn Error Resume Next
3      ADD\t0004\t
4      ADD\t0005\tDoCmd.LockNavigationPane (False)
5 ADD\t0006\tDoCmd.SelectObject acTable, "MSysOb...
6      ADD\t0007\tDoCmd.RunCommand acCmdWindowUnhide
7      ADD\t0008\t
8      ADD\t0009\tEnd Sub
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

3

```
2322 lista_rutinas_para_reconstruccion_modulo.append([numero_orden_en_modulo_nueva_rutina
2323 , nombre_nueva_rutina
2324 , df_codigo_nueva_rutina_reconstruida
2325 , df_codigo_nueva_rutina_documentacion
2326 , "NEW"])
```



```
[
  [
    2
    , ActivarPanelNavegacion
    , 'NEW'
  ]
]
```

```
df_codigo_nueva_rutina_reconstruida
CODIGO
0 Sub ActivarPanelNavegacion()
1
2 On Error Resume Next
3
4 DoCmd.LockNavigationPane (False)
5 DoCmd.SelectObject acTable, "MSysObjects", True
6 DoCmd.RunCommand acCmdWindowUnhide
7
8 End Sub
0
```

```
df_codigo_nueva_rutina_documentacion
CODIGO
0 ADD\t0001\tSub ActivarPanelNavegacion()
1 ADD\t0002\t
2 ADD\t0003\tOn Error Resume Next
3 ADD\t0004\t
4 ADD\t0005\tDoCmd.LockNavigationPane (False)
5 ADD\t0006\tDoCmd.SelectObject acTable, "MSysOb...
6 ADD\t0007\tDoCmd.RunCommand acCmdWindowUnhide
7 ADD\t0008\t
8 ADD\t0009\tEnd Sub
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

Una vez finalizado el bucle sobre la lista **lista_nuevas_rutinas**, siempre dentro de bucle_1, se pasa a reconstruir el código VBA de las rutinas / funciones del módulo mediante la creación del dataframe **df_codigo_modulo_rutinas_reconstruido**. Este último se genera mediante concatenación de los dataframes contenidos en las sublistas (item 3) de la lista **lista_rutinas_para_reconstruccion_modulo**, excluyendo aquellos donde el item 5 de las sublistas pone ELIMINAR.

```
2330 df_codigo_modulo_rutinas_reconstruido = pd.DataFrame(columns = ["CODIGO"])
2331 if len(lista_rutinas_para_reconstruccion_modulo):
2332     lista_rutinas_para_reconstruccion_modulo = sorted(lista_rutinas_para_reconstruccion_modulo, key = lambda x: x[0])
2333
2334     for numero_orden_rutina, nombre_rutina, df_codigo_para_reconstruccion_modulo, df_codigo_para_documentacion, accion_a_realizar in lista_rutinas_para_reconstruccion_modulo:
2335
2336         if accion_a_realizar != "ELIMINAR":
2337             df_codigo_modulo_rutinas_reconstruido = pd.concat([df_codigo_modulo_rutinas_reconstruido, df_codigo_para_reconstruccion_modulo])
```



```
0 CODIGO
1 Function Func_CorreccionCIF(xCIF As String) As...
2
3 Dim i As Integer
4 Dim MayuscCIF As String
5 Dim CIFCorr As String
6
7 MayuscCIF = UCase(Replace(xCIF, " ", ""))
8
9 CIFCorr = ""
10 For i = 1 To Len(MayuscCIF)
11 If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And ...
12 ((Asc(Mid(MayuscCIF, i, 1)) >= 48) And...
13 CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
14 End If
15 Next i
16
17 Func_CorreccionCIF = CIFCorr
18
19 End Function
```

```
0 Sub ActivarPanelNavegacion()
1
2 On Error Resume Next
3
4 DoCmd.LockNavigationPane (False)
5 DoCmd.SelectObject acTable, "MSysObjects", True
6 DoCmd.RunCommand acCmdWindowUnhide
7
8 End Sub
9
```

*Los 2 pantallazos son el mismo dataframe
(no cabe en pantalla el pantallazo)*

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

El siguiente paso, siempre dentro de bucle_1, es reconstruir el código del módulo creando el dataframe **df_codigo_modulo_reconstruido** como la concatenación (en este orden) de los dataframes mencionados en páginas anteriores:

- **df_codigo_modulo_encabezado_reconstruido**
- **df_codigo_modulo_variables_publicas_reconstruido**
- **df_codigo_modulo_rutinas_reconstruido**

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

df_codigo_modulo_encabezado_reconstruido

```
CODIGO
0 Option Compare Database
1 Option Explicit
2 Option Base 1
3
```

df_codigo_modulo_variables_publicas_reconstruido

```
CODIGO
0 Public Public_Version As String
1 Public Public_UsuarioID As String, Public Usua...
2 Public Const Public_Nombre_Access = "BBDD_2"
3 Type UserType_Prueba_1
4 SubVar_1 As String
5 SubVar_2 As Long
6 SubVar_3 As Date
7 End Type
8 Enum Enum_Prueba_1
9 SubVar_1
10 SubVar_2 = 1
11 SubVar_3
12 End Enum
13
```

```
CODIGO
0 Function Func_CorreccionCIF(xCIF As String) As...
1
2 Dim i As Integer
3 Dim MayuscCIF As String
4 Dim CIFCorr As String
5
6 MayuscCIF = UCase(Replace(xCIF, " ", ""))
7
8 CIFCorr = ""
9 For i = 1 To Len(MayuscCIF)
10 If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And ...
11 ((Asc(Mid(MayuscCIF, i, 1)) >= 48) And...
12 CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
13 End If
14 Next i
15
16 Func_CorreccionCIF = CIFCorr
17
18 End Function
```

```
Sub ActivarPanelNavegacion()
1
2 On Error Resume Next
3
4 DoCmd.LockNavigationPane (False)
5 DoCmd.SelectObject acTable, "MSysObjects", True
6 DoCmd.RunCommand acCmdWindowUnhide
7
8 End Sub
```

df_codigo_modulo_reconstruido

```
CODIGO
0 Option Compare Database
1 Option Explicit
2 Option Base 1
3
4 Public Public_Version As String
5 Public Public_UsuarioID As String, Public Usua...
6 Public Const Public_Nombre_Access = "BBDD_2"
7 Type UserType_Prueba_1
8 SubVar_1 As String
9 SubVar_2 As Long
10 SubVar_3 As Date
11 End Type
12 Enum Enum_Prueba_1
13 SubVar_1
14 SubVar_2 = 1
15 SubVar_3
16 End Enum
17
18 Function Func_CorreccionCIF(xCIF As String) As...
19
20 Dim i As Integer
21 Dim MayuscCIF As String
22 Dim CIFCorr As String
23
24 MayuscCIF = UCase(Replace(xCIF, " ", ""))
25
26 CIFCorr = ""
27 For i = 1 To Len(MayuscCIF)
28 If ((Asc(Mid(MayuscCIF, i, 1)) >= 65) And ...
29 ((Asc(Mid(MayuscCIF, i, 1)) >= 48) And...
30 CIFCorr = CIFCorr & Mid(MayuscCIF, i, 1)
31 End If
32 Next i
33
34 Func_CorreccionCIF = CIFCorr
35
36 End Function
37
38 Sub ActivarPanelNavegacion()
39
40 On Error Resume Next
41
42 DoCmd.LockNavigationPane (False)
43 DoCmd.SelectObject acTable, "MSysObjects", True
44 DoCmd.RunCommand acCmdWindowUnhide
45
46 End Sub
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A2

MS ACCESS - cálculos previos módulos VBA

Rutinas / funciones

Como último paso en bucle_1, se agregan los datos mencionados en páginas anteriores a un diccionario temporal el cual se agrega a la lista **lista_dicc_modulos_con_cambios**.

Como paso previo, se crea la lista **lista_rutinas_modulo_para_documentacion** como extracción de la lista **lista_rutinas_para_reconstruccion_modulo** donde cada una de sus sublistas contiene el nombre de la rutina / función con cambios realizados por el usuario y el dataframe listo para generar la documentación (es el que tiene el flag ADD o DEL al principio de cada línea de código).

```
2357 lista_rutinas_modulo_para_documentacion = [[item[1], item[3]] for item in lista_rutinas_para_reconstruccion_modulo if item[4] != None]
2358
2359
2360 #se crea diccionario temporal que se agrega a lista_dicc_modulos_con_cambios
2361 dicc_temp = {"TIPO_REPOSITORIO": tipo_modulo_con_cambios
2362             , "NOMBRE_REPOSITORIO": nombre_modulo_con_cambios
2363             , "DF_VARIABLES_PUBLICAS_DOCUMENTACION": df_codigo_modulo_variables_publicas_para_documentacion
2364             , "LISTA_DF_RUTINAS_DOCUMENTACION": lista_rutinas_modulo_para_documentacion
2365             , "DF_CODIGO_REPOSITORIO": df_codigo_modulo_reconstruido
2366             }
2367
2368 lista_dicc_modulos_con_cambios.append(dicc_temp)
```

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A3

MS ACCESS – documentación del proceso

Usando las listas **lista_dicc_tablas_y_vinculos_con_cambios** y **lista_dicc_modulos_con_cambios**, mencionadas en páginas anteriores, y usando el parámetro **ruta_export** de la presente rutina que se detalla en este anexo se pasa crear la documentación del proceso mediante ficheros .txt.

No hay más explicaciones del código Python en este documento sobre este proceso porque con referirse directamente al código de la rutina considero que es suficiente para entender su funcionamiento.

A4

MS ACCESS – merge en base de datos fisica

El proceso de merge en base de datos física se hace usando las 2 listas diccionarios **lista_dicc_tablas_y_vinculos_con_cambios** y **lista_dicc_modulos_con_cambios**, mencionadas en las páginas anteriores.

Se realiza mediante el uso de bloques **try except else** anidados unos dentro de los otros y que se resumen en el cuadro siguiente:

Con el cuadro que se muestra a continuación es más que suficiente para entender el proceso de merge por lo que no se muestra ningún pantallazo de código.

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

A4

MS ACCESS – merge en base de datos fisica

BLOQUE	DESCRIPCIÓN
try	Se prueba abrir la base de datos MS ACCESS configurada para BBDD_02 en modo escritura
except	En caso de error, se almacena el error (en formato de diccionario junto con otros datos como el módulo y rutina Python, el número de la línea de código en Python) en el diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_MIGRACION) asociada a la key_1 (MS_ACCESS) y subkey_2 (MERGE_BBDD_FISICA).
else	En caso de que no haya error, se pasa a realizar la migración de los objetos con cambios usando las listas lista_dicc_tablas_y_vinculos_con_cambios (para las tablas y vinculos) y lista_dicc_modulos_con_cambios (para los módulos VBA)
	TABLAS y VINCULOS
	BLOQUE
	try
	Mediante bucle sobre la lista lista_dicc_tablas_y_vinculos_con_cambios , se recuperan de cada diccionario los valores de las keys TIPO_OBJETO, NOMBRE_OBJETO y de la key PARAMETROS_MERGE los valores de las subkeys SENTENCIA_CREATE y ORIGEN_OBJETO y con los datos recuperados en cada iteración se elimina el objeto (por si ya existe en la base datos) y se pasa a crearlo
	except
	En caso de error, se almacena el error (en formato de diccionario junto con otros datos como el módulo y rutina Python, el número de la línea de código en Python y el tipo y nombre de objeto que se ha intentado crear) en el diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_MIGRACION) asociada a la key_1 (MS_ACCESS) y subkey_2 (MERGE_BBDD_FISICA).
	else
	En caso de que no haya error, se almacena el OK del tipo y nombre de objeto (en formato de diccionario) en el diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_OK_MIGRACION) asociada a la key_1 (MS_ACCESS) y subkey_2 (MERGE_BBDD_FISICA).
	MODULOS VBA
	BLOQUE
	try
	Mediante bucle sobre la lista lista_dicc_modulos_con_cambios , se recuperan de cada diccionario los valores de las keys TIPO_REPOSITORIO, NOMBRE_REPOSITORIO y DF_CODIGO_REPOSITORIO y con los datos recuperados en cada iteración reemplaza en módulo VBA
	except
	En caso de error, se almacena el error (en formato de diccionario junto con otros datos como el módulo y rutina Python, el número de la línea de código en Python y el tipo y nombre del módulo VBA que se ha intentado modificar) en el diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_MIGRACION) asociada a la key_1 (MS_ACCESS) y subkey_2 (MERGE_BBDD_FISICA).
	else
	En caso de que no haya error, se almacena el OK del tipo y nombre de módulo VBA (en formato de diccionario) en el diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_OK_MIGRACION) asociada a la key_1 (MS_ACCESS) y subkey_2 (MERGE_BBDD_FISICA).
finally	Se cierra la base de datos MS ACCESS

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

SQL SERVER

El proceso para el entorno SQL SERVER es **mucho más simple** que en el caso de MS ACCESS por lo que no es necesario ilustrar las explicaciones con un ejemplo.

Los pasos a seguir son estos:

PASO	DESCRIPCIÓN
1	Se crea la lista de diccionarios <code>lista_dicc_sql_server_objetos_con_cambios</code> que contiene todos los objetos de la base de datos SQL SERVER con cambios realizados por el usuario. Cada diccionario que la compone (uno por objeto) tienen la estructura siguiente: --> REPOSITORIO --> TIPO_OBJETO --> NOMBRE_OBJETO --> DF_CODIGO_DOCUMENTACION --> SENTENCIA_SQL La lista se crea mediante bucle sobre la lista <code>lista_dicc_sql_server_objetos_con_cambios</code>, parámetro de la rutina del presente anexo. El orden de las iteraciones se hace por tipo de objeto en este orden: TABLAS, VIEWS, FUNCIONES y por último STORED_PROCEDURES. Es para dejar preparada la lista de cara al merge en base de datos física. Si no se ejecuta en este orden podrían haber funciones, views y stored procedures dependientes de objetos que estén en la lista de cambios realizados por el usuario. Los commit generarían logs de errores. En cada iteración se calculan para cada objeto un dataframe para documentar en fichero .txt (donde a cada línea de código se le agrega al principio el flag ADD o DEL según que el usuario haya agregado o eliminado líneas) y una sentencia SQL para poder realizar los commit cuando se haga el merge (es la concatenación en string del dataframe del objeto al cual se quita previamente las líneas con el flag ELIMINADO en la columna CONTROL_CAMBIOS_ACTUAL)
2	Ajuste previo para realizar la documentación y el merge en base de datos física
3	Generación de la documentación del proceso en ficheros .txt
4	Realización del merge en base de datos física

Al igual que en MS ACCESS, esto se realiza mediante un bloque **try except else** anidado dentro de otro bloque **try except else**. El proceso aquí es mucho más simple.

ANEXO_07

RUTINA: def_merge_bbdd_fisicas

SQL SERVER

BLOQUE	DESCRIPCIÓN	
try	Se prueba conectar al servidor y base de datos donde realizar el merge recuperando la connecting string del diccionario dicc_codigos_bbdd (módulo APP_2_GENERAL) en la subkey_3 (CONNECTING_STRING) asociada a la key_1 (BBDD_02) y subkey_2 (SQL_SERVER). La connecting string almacenada en el diccionario dicc_codigos_bbdd ya esta adaptada al tipo de conexión (Windows Authentication o SQL Server authentication)	
except	En caso de error, se almacena el error (en formato de diccionario junto con otros datos como el módulo y rutina Python, el número de la linea de código en Python) en el diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_MIGRACION) asociada a la key_1 (SQL_SERVER) y subkey_2 (MERGE_BBDD_FISICA).	
else	En caso de que no haya error, se pasa e realizar la migración de los objetos con cambios usando la lista lista_dicc_sql_server_objetos_con_cambios . Como paso previo, en dicha lista se recuperan los esquemas que no estan creados en BBDD_02 y se crean para evitar que todos los objetos asociados a estos esquemas generen logs de errores. La lista lista_dicc_sql_server_objetos_con_cambios ya esta ordenada por tipo de objeto (tablas, views, funciones y stored procedures) y los commit se realizan en este orden	
	BLOQUE	DESCRIPCIÓN
	try	Mediante bucle sobre la lista lista_dicc_sql_server_objetos_con_cambios , se recuperan de cada diccionario los valores de las keys REPOSITORIO, , TIPO_OBJETO, NOMBRE_OBJETO y SENTENCIA_SQL NOMBRE_OBJETO y con los datos recuperados en cada iteración se elimina el objeto (por si ya existe en la base datos) y se pasa a crearlo
	except	En caso de error, se almacena el error (en formato de diccionario junto con otros datos como el módulo y rutina Python, el número de la linea de código en Python, el nombre del esquema, el tipo y nombre de objeto que se ha intentado crear) en el diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_MIGRACION) asociada a la key_1 (SQL_SERVER) y subkey_2 (MERGE_BBDD_FISICA).
	else	En caso de que no haya error, se almacena el OK del tipo y nombre de objeto (en formato de diccionario) en el diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_OK_MIGRACION) asociada a la key_1 (SQL_SERVER) y subkey_2 (MERGE_BBDD_FISICA).

ANEXO_08

FUNCIÓN

func_access_bbdd_codigo_calculos

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

La función **func_access_bbdd_codigo_calculos** permite realizar cálculos relacionados con rutinas y variables públicas VBA: Devuelve en todos los casos una lista, que según la opción seleccionada pueden ser listas de listas o listas de diccionarios.

BLOQUE	OPCIÓN	DESCRIPCIÓN	CONTENIDO SUBLISTAS / DICCIONARIOS	
			ITEMS / KEYS	APLICA
1	LISTA Rutinas	La opción permite crear una lista de listas (cuya longitud es la misma que el dataframe df_bbdd_param). Sirve para calcular columnas adicionales asociadas con rutinas / funciones VBA en el dataframe df_bbdd_param	Indice del dataframe df_bbdd_param	X
			Tipo rutina (rutina o función)	X
			Tipo declaración de rutina (pública o privada)	X
			Nombre de la rutina	X
			Número de línea de código dentro de la rutina	X
			String que concatena los parámetros (si los hubiese) de la rutina / función con los tipos de datos asociados	X
2	LISTA Variables Públicas	La opción permite crear una lista de listas (cuya longitud es la misma que el dataframe df_bbdd_param). Sirve para calcular columnas adicionales asociadas con variables públicas VBA en el dataframe df_bbdd_param	String que concatena todos los tipos de datos de los parámetros y de las variables locales (es la base sobre la cual, en el diagnostico de dependencias, para saber si una variable pública definida por el usuario se usa o no en una rutina / función VBA)	X
			Tipo de módulo	X
			Nombre del módulo	X
			Indice de línea de código dentro del dataframe df_bbdd_param	X
			Flag que identifica si la variable pública es constante o no (solo para variables públicas normales) o flag unico dentro un módulo que aplica a todas las lineas de código asociadas a una variable pública definida por el usuario	X
			Línea de código	X
3	LISTA Dicc. Objetos Variables Públicas	La opción permite crear una lista de diccionarios que lista una a una todas las variables públicas normales y las que estan definidas por el usuario (estas al declararse en varias lineas se unifican en 1 sola concatenando sus sub-variables con su tipo de dato o valor). La estructura de los diccionarios que componen esta lista es común a todos los objetos MS ACCESS (tablas / vinculos, variables públicas VBA y rutinas / funciones VBA) por lo que algunos items no aplican y se establecen a None	TIPO_OBJETO	X
			TIPO_MODULO	X
			NOMBRE_MODULO	X
			NOMBRE_OBJETO	X
			TIPO Rutina	
			TIPO_DECLARACION_RUTINA	
			PARAMETROS_RUTINA	
			TIPO_VARIABLE_PUBLICA	X
			TIPO_DATO_VARIABLE_PUBLICA	X
			VARIABLE_PUBLICA_ES_CONSTANTE	X
			VARIABLE_PUBLICA_CONSTANTE_VALOR	X
			VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES	X
			DF_CODIGO	
			CONNECTING_STRING_VINCULOS	

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

La función tiene 2 parámetros kwargs:

- **df_bbdd_codigo_param**
- **lista_calculos_variables_publicas**

PARAMETROS KWARGS	OPCIONES FUNCIÓN			DESCRIPCIÓN
	LISTA Rutinas	LISTA VARIABLES_PUBLICAS	LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS	
df_bbdd_codigo_param	X	X		Dataframe que se calcula en la rutina def_proceso_access_1_import
lista_calculos_variables_publicas			X	En la rutina <code>def_proceso_access_1_import</code> se hace una 1era llamada a la función del presente Anexo <code>func_access_bbdd_codigo_calculos</code> (opción LISTA_RUTINAS). La lista resultante se usa como parámetro en otra llamada a la función <code>func_access_bbdd_codigo_calculos</code> (opción LISTA_DICC_OBJETOS_VARIABLES_PUBLICA) más adelante en la rutina <code>def_proceso_access_1_import</code>

1

opcion = LISTA_RUTINAS

Antes de realizar los cálculos de esta opción es necesario realizar una serie de **ajustes previos**.

AJUSTE	DESCRIPCIÓN
lista_declaracion_rutinas_vba_ini_ord_desc	Fusiona en una lista de listas los valores de las subkey_3 (TIPO Rutina, TIPO DECLARACION y LISTA DECLARACION_INICIO) asociadas a la key_1 (RUTINAS) del diccionario <code>dicc_objetos_vba</code> (módulo <code>APP_3_BACK_END_MS_ACCESS</code>). El 3er item de cada una de sus sublistas se trimea y se le agrega un espacio en blanco al final (es por si el usuario ha configurado mal el diccionario). La lista se re-ordena por la longitud del 3er item de cada una de sus sublistas de mayor a menor
tupla_declaracion_rutinas_vba_ini	Se recupera el 3 item de cada una de las sublistas de la lista <code>lista_declaracion_rutinas_vba_ini_ord_desc</code> y se convierte la lista resultante en tupla (es para que el metodo <code>startswith</code> de filtrado de dataframes funcione con selecciones múltiples)
lista_declaracion_rutinas_vba_fin	Fusiona en una lista las listas almacenadas en las subkey_3 (LISTA DECLARACION_FIN) asociadas a la key_1 (RUTINAS) del diccionario <code>dicc_objetos_vba</code> (módulo <code>APP_3_BACK_END_MS_ACCESS</code>). Cada item de la lista resultante se trimea
tupla_declaracion_rutinas_vba_fin	Se convierte la lista <code>lista_declaracion_rutinas_vba_fin</code> en tupla (es para que el metodo <code>startswith</code> de filtrado de dataframes funcione con selecciones múltiples)

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

```
lista_declaracion_rutinas_vba_ini_ord_desc  
[['FUNCION', 'PRIVADA', 'Private Function '],  
 ['FUNCION', 'PUBLICA', 'Public Function '],  
 ['FUNCION', 'PUBLICA', 'Global Function '],  
 ['RUTINA', 'PRIVADA', 'Private Sub '],  
 ['RUTINA', 'PUBLICA', 'Public Sub '],  
 ['RUTINA', 'PUBLICA', 'Global Sub '],  
 ['FUNCION', 'PUBLICA', 'Function '],  
 ['RUTINA', 'PUBLICA', 'Sub ']]
```

```
tupla_declaracion_rutinas_vba_ini  
  
('Private Function ',  
 'Public Function ',  
 'Global Function ',  
 'Private Sub ',  
 'Public Sub ',  
 'Global Sub ',  
 'Function ',  
 'Sub )
```

```
lista_declaracion_rutinas_vba_fin  
['End Sub', 'End Function']
```

```
tupla_declaracion_rutinas_vba_fin  
  
('End Sub', 'End Function')
```

Realizados estos ajustes, se pasa a realizar el cálculo de la lista de listas resultado de la función.

Para ilustrar las explicaciones detalladas del código se usa en adelante el ejemplo siguiente:

DESCRIPCIÓN
Un UserForm VBA con combobox de Provincia, actividad comercial y ranking, y un botón UPDATE
El botón UPDATE hace la llamada a una rutina (BotonUPDATE_Click) en el módulo privado UserForm_PRUEBA_1
La rutina BotonUPDATE_Click hace la llamada a la rutina Calc_Update_CIF (que tiene parámetros) del módulo público Modulo_PRUEBA_1
La rutina Calc_Update_CIF utiliza en su código la función Func_CorreccionCIF (también con parámetro) también ubicada en el módulo Modulo_PRUEBA_1

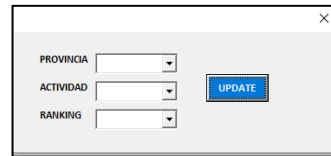
ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

UserForm_PRUEBA_1



```
0 Option Compare Database
1
2 Private Sub BotonUPDATE_Click()
3
4 Dim Msg As Variant
5
6 If IsNull(Me.ComboBox_Provincia) = True Or _
7 IsNull(Me.ComboBox_Provincia) = True Or _
8 IsNull(Me.ComboBox_Ranking) = True Then
9
10 MsgBox "La 3 OPCIONES son obligatorias.", vbOKCancel + vbCritical, ""
11 Exit Sub
12
13 Else
14 Msg = MsgBox("Estás segur@?", vbOKCancel + vbInformation, "")
15
16 If Msg = vbCancel Then
17 Exit Sub
18
19 Else
20 DoCmd.Hourglass True
21
22 Call Calc_Update_CIF(Me.ComboBox_Provincia, _
23 Me.ComboBox_ActivCom, _
24 Me.ComboBox_Ranking)
25
26 DoCmd.Hourglass False
27
28 MsgBox "OK", vbOKOnly + vbInformation, ""
29 End If
30 End If
31
32 End Sub
```

Modulo_PRUEBA_1

```
34 Option Compare Database
35 Option Base 1
36
37 Function Func_CorreccionCIF(xCIF As String) As String
38
39 Dim i As Integer, _
40 xTemp1 As String, _
41 xTemp2 As String
42
43 xTemp = UCase(Replace(xCIF, " ", ""))
44
45 xTemp2 = ""
46 For i = 1 To Len(xTemp)
47 If ((Asc(Mid(xTemp, i, 1)) >= 65) And (Asc(Mid(xTemp, i, 1)) <= 90)) _
48 Or ((Asc(Mid(xTemp, i, 1)) >= 48) _
49 And (Asc(Mid(xTemp, i, 1)) <= 57)) Then
50 xTemp2 = xTemp2 & Mid(xTemp1, i, 1)
51 End If
52 Next i
53
54 Func_CorreccionCIF = xTemp2
55
56 End Function
57
58 Sub Calc_Update_CIF(xProvincia_Sede As String, _
59 xActividadComercial As String, _
60 xRanking As Integer)
61
62 Dim MiConexion As New ADODB.Connection
63 Dim MiRecordset As New ADODB.Recordset
64
65 Dim MiSQL As String, _
66 i As Long
67
68 Set MiConexion = CurrentProject.Connection
69
70 MiSQL = "SELECT CIF, PROVINCIA_SEDE FROM [T_EMPRESAS]" & vbCrLf & _
71 & "WHERE ((PROVINCIA = '" & xProvincia_Sede & "') AND " & vbCrLf & _
72 & "(ACTIVIDAD_COMERCIAL = '" & xActividadComercial & "') AND " & _
73 & "(RANKING = '" & xRanking & "'))"
74
75 MiRecordset.Open MiSQL, MiConexion, 1, 2
76
77 With MiRecordset
78 .MoveFirst
79 While Not .EOF
80 MiRecordset(0) = Func_CorreccionCIF(MiRecordset(0))
81 .MoveNext
82 Wend
83 End With
84
85 End Sub
```

Indice del dataframe **df_bbdd_codigo** tras la importación del código VBA con la rutina **def_proceso_access_1_import**

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Tras los ajustes previos comentados en páginas anteriores, el 1er paso es calcular las listas temporales **lista_indices_rutinas_ini** y **lista_indices_rutinas_fin** que almacenan respectivamente sublistas con el tipo y nombre del módulo y el índice del dataframe **df_bbdd_codigo_param** correspondientes a las instrucciones VBA de inicio y fin de declaraciones de rutinas y funciones.

Se calculan mediante extracciones de dataframes sobre el dataframe **df_bbdd_codigo_param**. Los criterios de filtrado se hacen usando las tuplas **tupla_declaracion_rutinas_vba_ini** y **tupla_declaracion_rutinas_vba_fin**, calculadas en los ajustes previos. Mediante bucle sobre los índices de estos dataframes extraídos, se calculan las listas mencionadas y se ordenan por tipo y nombre de módulo y por el índice del dataframe.

```
413 df_temp_declaracion_rutinas_vba_ini = df_bbdd_codigo_param.loc[df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_rutinas_vba_ini), lista_campos_usados]
414 df_temp_declaracion_rutinas_vba_fin = df_bbdd_codigo_param.loc[df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_rutinas_vba_fin), lista_campos_usados]
415
416 df_temp_declaracion_rutinas_vba_ini.reset_index(drop = True, inplace = True)
417 df_temp_declaracion_rutinas_vba_fin.reset_index(drop = True, inplace = True)
418
419 lista_indices_rutinas_ini = []
420 lista_indices_rutinas_fin = []
421
422 if len(df_temp_declaracion_rutinas_vba_ini) != 0:
423     lista_indices_rutinas_ini = [(df_temp_declaracion_rutinas_vba_ini.iloc[ind, df_temp_declaracion_rutinas_vba_ini.columns.get_loc("TIPO_MODULO")],
424                                df_temp_declaracion_rutinas_vba_ini.iloc[ind, df_temp_declaracion_rutinas_vba_ini.columns.get_loc("NOMBRE_MODULO")],
425                                df_temp_declaracion_rutinas_vba_ini.iloc[ind, df_temp_declaracion_rutinas_vba_ini.columns.get_loc("INDICE")])
426                                for ind in df_temp_declaracion_rutinas_vba_ini.index]
427
428     lista_indices_rutinas_ini = sorted(lista_indices_rutinas_ini, key = lambda x: (x[0], x[1], x[2]))
429
430
431 if len(df_temp_declaracion_rutinas_vba_fin) != 0:
432     lista_indices_rutinas_fin = [(df_temp_declaracion_rutinas_vba_fin.iloc[ind, df_temp_declaracion_rutinas_vba_fin.columns.get_loc("TIPO_MODULO")],
433                                 df_temp_declaracion_rutinas_vba_fin.iloc[ind, df_temp_declaracion_rutinas_vba_fin.columns.get_loc("NOMBRE_MODULO")],
434                                 df_temp_declaracion_rutinas_vba_fin.iloc[ind, df_temp_declaracion_rutinas_vba_fin.columns.get_loc("INDICE")])
435                                 for ind in df_temp_declaracion_rutinas_vba_fin.index]
436
437     lista_indices_rutinas_fin = sorted(lista_indices_rutinas_fin, key = lambda x: (x[0], x[1], x[2]))
```

lista_indices_rutinas_ini

[['Estandar', 'Modulo_PRUEBA_1', 37], ['Estandar', 'Modulo_PRUEBA_1', 58], ['UserForm', 'UserForm_PRUEBA_1', 2]]

lista_indices_rutinas_fin

[['Estandar', 'Modulo_PRUEBA_1', 56], ['Estandar', 'Modulo_PRUEBA_1', 85], ['UserForm', 'UserForm_PRUEBA_1', 32]]

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Se inicia el proceso de cálculo de la lista de diccionarios **lista_dicc_datos_rutinas** que es la base sobre la cual se construye el dataframe resultado de la función, tan solo si la base de datos MS ACCESS tiene rutinas / funciones localizadas mediante comprobación que las listas **lista_indices_rutinas_ini** y **lista_indices_rutinas_fin** no están vacías y tienen la misma longitud.

```
446 lista_dicc_datos_rutinas = []
447 if len(lista_indices_rutinas_ini) != 0 and len(lista_indices_rutinas_ini) == len(lista_indices_rutinas_fin):
```

Se crea la lista temporal **lista_indices_y_codigo_inicio_rutina** mediante extracción del dataframe **df_bbdd_codigo_param** filtrado por las líneas de código correspondientes a inicios de declaración de rutinas / funciones donde se recupera el tipo y nombre de módulo, el índice del dataframe y la línea de código, sin tabulaciones, trimeada y sin comentarios.

La lista de listas resultante se ordena por tipo y nombre de módulo y índice.

```
452 lista_indices_ini_declar = [item[2] for item in lista_indices_rutinas_ini]
453 df_indices_y_codigo_inicio_rutina = (df_bbdd_codigo_param.loc[df_bbdd_codigo_param["INDICE"].isin(lista_indices_ini_declar),
454 ["TIPO_MODULO", "NOMBRE_MODULO", "INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
455
456 lista_indices_y_codigo_inicio_rutina = df_indices_y_codigo_inicio_rutina.values.tolist()
457 lista_indices_y_codigo_inicio_rutina = sorted(lista_indices_y_codigo_inicio_rutina, key = lambda x: (x[0], x[1], x[2]))
```



```
lista_indices_y_codigo_inicio_rutina

[['Estandar', 'Modulo_PRUEBA_1', 37, 'Function Func_CorreccionCIF(xCIF As String) As String'],
 ['Estandar', 'Modulo_PRUEBA_1', 58, 'Sub Calc_UpDate_CIF(xProvincia_Sede As String, _'],
 ['UserForm', 'UserForm_PRUEBA_1', 2, 'Private Sub BotonUPDATE_Click()']]
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Se inicia el cálculo de la lista de diccionarios **lista_dicc_datos_rutinas**, mediante bucle (en adelante bucle_rutinas) sobre la lista temporal **lista_indices_y_codigo_inicio_rutina**.

```
461 for tipo_modulo_ini, nombre_modulo_ini, indice_ini, linea_codigo_ini in lista_indices_y_codigo_inicio_rutina:
```

El primer bloque de cálculo dentro de **bucle_rutinas** es:

PASO	DESCRIPCIÓN
1	Se calcula el índice del dataframe df_bbdd_codigo_param correspondiente a la declaración de fin de rutina asociado al índice del dataframe correspondiente al inicio de declaración (el de la iteración sobre bucle_rutinas). Se realiza buscando el 1er item de la lista lista_indices_rutinas_fin inmediatamente posterior (para el mismo tipo y nombre de módulo) al índice de inicio de declaración de la iteración sobre bucle_rutinas .
2	Se calcula el tipo de rutina, el tipo de declaración de rutina y el nombre de la rutina usando la línea de código correspondiente al inicio de declaración, la cual se splitea por el parentesis de apertura "(" y se recupera el 1er item de la lista resultante. Usando este último y mediante bucle sobre la lista lista_declaracion_rutinas_vba_ini_ord_desc (calculada en los ajustes previos), se realizan los cálculos.
3	Se calcula el número de orden de la rutina / función dentro del módulo, buscando en que índice de lista (dentro de lista_indices_y_codigo_inicio_rutina) se encuentra el índice de inicio de declaración en el dataframe df_bbdd_codigo_param (para el mismo tipo y nombre de módulo de la iteración)

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

1

```
464 #se calcula el indice de fin de declaracion de la rutina buscando en lista_indices_rutinas_fin el 1er indice de fin de declaracion
465 #inmediatamente posterior a indice_ini siempre y cuando el tipo y nombre del modulo de lista_indices_y_codigo_inicio_rutina y lista_indices_rutinas_fin coincidan
466 indice_fin = [item[2] for item in lista_indices_rutinas_fin if item[0] == tipo_modulo_ini and item[1] == nombre_modulo_ini and item[2] > indice_ini][0]
```

2

```
469 #se calcula el tipo_rutina, tipo_declaracion_rutina y nombre_rutina
470 #
471 #se splitea la linea de codigo por el parentesis de apertura "(" para conservar solo el 1er item
472 #el 2ndo son los parametros (si los hubiese) que se calculan en este mismo bucle mas adelante
473 linea_codigo_spliteada_parentesis = linea_codigo_ini.split("(")[0]
474
475 tipo_rutina_localizada = None
476 tipo_declaracion_rutina_localizada = None
477 nombre_rutina_localizada = None
478
479 for tipo_rutina, tipo_declaracion_rutina, ini_declar in lista_declaracion_rutinas_vba_ini_ord_desc:
480
481     if ini_declar in linea_codigo_spliteada_parentesis:
482         tipo_rutina_localizada = tipo_rutina
483         tipo_declaracion_rutina_localizada = tipo_declaracion_rutina
484         nombre_rutina_localizada = linea_codigo_spliteada_parentesis.replace(ini_declar, "").strip()
485
486     break
```

3

```
488 #se calcula el numero de orden de la rutina dentro del modulo
489 numero_orden_rutina_en_modulo = [item[2] for item in lista_indices_y_codigo_inicio_rutina if item[0] == tipo_modulo_ini and item[1] == nombre_modulo_ini].index(indice_ini) + 1
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Usando el ejemplo comentado en páginas anteriores, el bloque de cálculo 1-2-3 daría:

UserForm_PRUEBA_1

```
tipo_modulo      UserForm
nombre_modulo    UserForm_PRUEBA_1
indice_ini       2
linea_codigo_ini Private Sub BotonUPDATE_Click()
-----
BLOQUES CALCULOS 1-2-3

lista_indices_rutinas_fin (recalculado según indice_ini)  RUTINA
indice_fin                                              32
tipo_rutina_localizada                                RUTINA
tipo_declaracion_rutina_localizada                     PRIVADA
nombre_rutina_localizada                             BotonUPDATE_Click
numero_orden_rutina_en_modulo                          1
*****
*****
```

Modulo_PRUEBA_1

```
tipo_modulo      Estandar
nombre_modulo    Modulo_PRUEBA_1
indice_ini       37
linea_codigo_ini Function Func_CorreccionCIF(xCIF As String) As String
-----
BLOQUES CALCULOS 1-2-3

lista_indices_rutinas_fin (recalculado según indice_ini)  FUNCION
indice_fin                                              56
tipo_rutina_localizada                                FUNCION
tipo_declaracion_rutina_localizada                     PUBLICA
nombre_rutina_localizada                             Func_CorreccionCIF
numero_orden_rutina_en_modulo                          1
*****
*****

tipo_modulo      Estandar
nombre_modulo    Modulo_PRUEBA_1
indice_ini       58
linea_codigo_ini Sub Calc_UpDate_CIF(xProvincia_Sede As String, _
-----
BLOQUES CALCULOS 1-2-3

lista_indices_rutinas_fin (recalculado según indice_ini)  RUTINA
indice_fin                                              85
tipo_rutina_localizada                                RUTINA
tipo_declaracion_rutina_localizada                     PUBLICA
nombre_rutina_localizada                             Calc_UpDate_CIF
numero_orden_rutina_en_modulo                          2
```


ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Siempre dentro de **bucle_rutinas**, el siguiente paso es calcular el **string de parámetros** de la rutina / función (si los hubiese).

PASO	DESCRIPCIÓN
4	Disponiendo de los índices de inicio y fin de declaración de la rutinas de la iteración sobre bucle_rutinas, se extraen las 60 primeras líneas de código de la rutina / función (si la rutina no llega a las 60 líneas, se extrae toda la rutina) desde el dataframe df_bbdd_codigo_param. Esto es porque VBA acepta como máximo 60 declaraciones de parámetros en una rutina / función que pueden declararse en varias líneas truncadas (las que acabab en "_")
5	Se concatena en string el dataframe extraido, donde se excluyen los "_" al final de la línea de código, y se comprueba si la rutina / función tiene parámetros (no los tiene si el primer paréntesis de cierre es inmediatamente consecutivo al primer paréntesis de apertura
6	Si la rutina tiene parametros se calcula string_parametros donde en string_60_primeras_lineas se reemplaza previamente los "(" por "@@" (@ es un caracter prohibido en VBA en declaraciones de parametros por lo que no hay riesgo de alterar el nombre o tipo de datos de un parametro) y se vuelve a calcular las posiciones dentro del string del 1er paréntesis de apertura y cierre (los "(" dentro de la declaracion de un parametro corresponden a arrays, si se hace este replace por "@@" el calculo del string de parametros seria incorrecto si hubiese arrays)

4

```
500 indice_fin_extracc = indice_ini + num_max_parametros_rutina_vba_access if indice_ini + num_max_parametros_rutina_vba_access < indice_fin else indice_fin
501
502 df_extracc_codigo = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["INDICE"] >= indice_ini) & (df_bbdd_codigo_param["INDICE"] <= indice_fin_extracc)] &
503                    (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.replace(" ", "") != indicador_linea_codigo_vba_truncada_a_linea_siguiente.strip())
504                    (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
505                    ["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
506
507 df_extracc_codigo.reset_index(drop = True, inplace = True)
```

Este filtro es para evitar estos casos (que es el mismo, VBA permite estas cosas):

```
Sub Calc_PRUEBA ( )
```

```
Sub Calc_PRUEBA( _
-
-
-
)
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

5

```
513 string_60_primeras_lineas = "".join([df_extracc_codigo.iloc[ind, 0].strip()
514                                     if df_extracc_codigo.iloc[ind, 0][-2:] != indicador_linea_codigo_vba_truncada_a_linea_siguiente
515                                     else df_extracc_codigo.iloc[ind, 0][-2:].strip()
516                                     for ind in df_extracc_codigo.index])
517
518 del df_extracc_codigo
519
520 #se comprueba si la rutina / funcion tiene parametros (el 1er parentesis de cierre si es inmediatamente posterior
521 #al 1er parentesis de apertura es que la rutina no tiene parametros)
522 lista_parentesis_apertura = list(re.finditer(r"\(", string_60_primeras_lineas))
523 primer_parentesis_apertura = [lista_parentesis_apertura[ind].start() for ind, item in enumerate(lista_parentesis_apertura)][0]
524
525 lista_parentesis_cierre = list(re.finditer(r"\)", string_60_primeras_lineas))
526 primer_parentesis_cierre = [lista_parentesis_cierre[ind].start() for ind, item in enumerate(lista_parentesis_cierre)][0]
527
528 rutina_tiene_parametros = "NO" if primer_parentesis_cierre == primer_parentesis_apertura + 1 else "SI"
```

6

```
536 string_parametros = ""
537 if rutina_tiene_parametros == "SI":
538
539     string_parametros = string_60_primeras_lineas.replace("()", "@@")
540
541     lista_parentesis_apertura = list(re.finditer(r"\(", string_parametros))
542     primer_parentesis_apertura = [lista_parentesis_apertura[ind].start() for ind, item in enumerate(lista_parentesis_apertura)][0]
543
544     lista_parentesis_cierre = list(re.finditer(r"\)", string_parametros))
545     primer_parentesis_cierre = [lista_parentesis_cierre[ind].start() for ind, item in enumerate(lista_parentesis_cierre)][0]
546
547     string_parametros = string_parametros[primer_parentesis_apertura + 1:primer_parentesis_cierre].replace("@@", "()")
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA RUTINAS

Usando el ejemplo comentado en páginas anteriores, el bloque de cálculo 4-5-6 daría:

UserForm_PRUEBA_1

```
tipo_modulo           UserForm
nombre_modulo         UserForm_PRUEBA_1
indice_ini            2
indice_fin            32
tipo_rutina_localizada RUTINA
tipo_declaracion_rutina_localizada PRIVADA
nombre_rutina_localizada BotonUPDATE_Click
numero_orden_rutina_en_modulo 1
-----
BLOQUES CALCULOS 4-5-6

df_extracc_codigo
    CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0      Private Sub BotonUPDATE_Click()
1          Dim Msg As Variant
2          If IsNull(Me.ComboBox_Provincia) = True Or _
3             IsNull(Me.ComboBox_Provincia) = True Or _
4             IsNull(Me.ComboBox_Ranking) = True Then
5              MsgBox "La 3 OPCIONES son obligatorias.", vbOK...
6              Exit Sub
7          Else
8              Msg = MsgBox("Estás seguro?", vbOKCancel + vbI...
9              If Msg = vbCancel Then
10                 Exit Sub
11             Else
12                 DoCmd.Hourglass True
13                 Call Calc_UpDate_CIF(Me.ComboBox_Provincia, _
14                     Me.ComboBox_ActivCom, _
15                     Me.ComboBox_Ranking)
16                 DoCmd.Hourglass False
17                 MsgBox "OK", vbOKOnly + vbInformation, ""
18                 End If
19             End If
20             End Sub

string_60_primeras_lineas Private Sub BotonUPDATE_Click()Dim Msg As VariantIf IsNull(Me.ComboBox_
As.", vbOKCancel + vbCritical, ""Exit SubElseMsg = MsgBox("Estás seguro?", vbOKCancel + vbInformation, "")If Ms
oCmd.Hourglass FalseMsgBox "OK", vbOKOnly + vbInformation, ""End IfEnd IfEnd Sub

rutina_tiene_parametros    NO
string_parametros
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Usando el ejemplo comentado en páginas anteriores, el bloque de cálculo 4-5-6 daría:

Modulo_PRUEBA_1

```
tipo_modulo      Estandar
nombre_modulo     Modulo_PRUEBA_1
indice_ini       37
indice_fin       56
tipo_rutina_localizada  FUNCION
tipo_declaracion_rutina_localizada  PUBLICA
nombre_rutina_localizada  Func_CorreccionCIF
numero_orden_rutina_en_modulo  1
-----
BLOQUES CALCULOS 4-5-6

df_extracc_codigo
CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0  Function Func_CorreccionCIF(xCIF As String) As...
1      Dim i As Integer, _
2      xTemp1 As String, _
3      xTemp2 As String
4      xTemp = UCase(Replace(xCIF, " ", ""))
5      xTemp2 = ""
6      For i = 1 To Len(xTemp)
7  If ((Asc(Mid(xTemp, i, 1)) >= 65) And (Asc(Mid...
8      Or ((Asc(Mid(xTemp, i, 1)) >= 48) _
9      And (Asc(Mid(xTemp, i, 1)) <= 57)) Then
10         xTemp2 = xTemp2 & Mid(xTemp1, i, 1)
11     End If
12 Next i
13     Func_CorreccionCIF = xTemp2
14 End Function

string_60_primeras_lineas      Function Func_CorreccionCIF(xCIF As String) As StringDim i As Int
5) And (Asc(Mid(xTemp, i, 1)) <= 90))Or ((Asc(Mid(xTemp, i, 1)) >= 48)And (Asc(Mid(xTemp, i, 1)) <= 57))

rutina_tiene_parametros        SI
string_parametros              xCIF As String
*****
*****
```

Modulo_PRUEBA_1

```
tipo_modulo      Estandar
nombre_modulo     Modulo_PRUEBA_1
indice_ini       58
indice_fin       85
tipo_rutina_localizada  RUTINA
tipo_declaracion_rutina_localizada  PUBLICA
nombre_rutina_localizada  Calc_Update_CIF
numero_orden_rutina_en_modulo  2
-----
BLOQUES CALCULOS 4-5-6

df_extracc_codigo
CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0  Sub Calc_Update_CIF(xProvincia_Sede As String, _
1      xActividadComercial As String, _
2      xRanking As Integer)
3      Dim MiConexion As New ADODB.Connection
4      Dim MiRecordset As New ADODB.Recordset
5      Dim MiSQL As String, _
6      i As Long
7      Set MiConexion = CurrentProject.Connection
8      MiSQL = "SELECT CIF, PROVINCIA_SEDE FROM [T_EM...
9      & "WHERE ((PROVINCIA = "" & xProvincia_Sede & ...
10     & "(ACTIVIDAD_COMERCIAL = "" & xActividadComer...
11     & "(RANKING = "" & xRanking & ""))"
12     MiRecordset.Open MiSQL, MiConexion, 1, 2
13     With MiRecordset
14         .MoveFirst
15         While Not .EOF
16             MiRecordset(0) = Func_CorreccionCIF(MiRecordse...
17         .MoveNext
18     Wend
19     End With
20 End Sub

string_60_primeras_lineas      Sub Calc_Update_CIF(xProvincia_Sede As String,xActividadComercial As String,xRanking As Integer)Dim i
MiConexion = CurrentProject.ConnectionMiSQL = "SELECT CIF, PROVINCIA_SEDE FROM [T_EMPRESAS]" & vbCrLf& "WHERE ((PROVINCIA = "" & xProvincia_
Ranking & ""))"MiRecordset.Open MiSQL, MiConexion, 1, 2With MiRecordset.MoveFirstWhile Not .EOFMiRecordset(0) = Func_CorreccionCIF(MiRecordse

rutina_tiene_parametros        SI
string_parametros              xProvincia_Sede As String,xActividadComercial As String,xRanking As Integer
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Siempre dentro de **bucle_rutinas**, el siguiente paso es calcular los tipos de datos de los parámetros y variables locales de la rutina / función de la iteración. Se usa en el diagnostico de dependencias para saber en que rutinas se usan las variables públicas definidas por el usuario.

```
Type UserType_1
  SubVar_1 As String
  SubVar_2 As Long
End Type
```

```
Sub Calc_PRUEBA_1(Parametro As UserType_1
  Dim xVariable As UserType_1
  MsgBox "hola"
End Sub
```

PASO	DESCRIPCIÓN
7	Se crea la lista lista_parametros_rutina mediante spliteo del string de parámetros string_parametros (ver páginas anteriores) por la coma "," (previamente se quita todo lo encapsulado entre comillas dobles y entre parentesis porque podria haber comas incrustadas ahí, lo que se quita no afecta al tipo de dato del parámetro)
8	Se crea la lista lista_variables_locales_rutina_linea_codigo para almacenar las lineas de código que corresponden con declaración de variables locales de rutina / función
9	Mediante bucle sobre los items de la lista lista_variables_locales_rutina_linea_codigo se netean las lineas de código quitando los "Dim ", "Cons " y los " _" de final de linea y además se quita todo lo encapsulado entre comillas dobles y todo y entre parentesis para poder a posteriori splitear por la coma "," (podria haber comas incrustadas dentro de los valores de las variables locales declaradas como constantes o dentro de las variables locales declaradas como arrays de más de 1 dimensión) y los items de la lista resultante se agregan a lista_variables_locales_rutina
10	Se fusionan las listas lista_parametros_rutina y lista_variables_locales_rutina en la lista lista_parametros_y_variables_locales y mediante bucle sobre esta última se splitea cada item por " As ", se recupera el 2ndo item (cuando la lista resultante es de longitud igual a 2) que se agrega a la lista lista_tipo_dato_parametros_y_variables_locales . Una vez finalizado el bucle mencionado, se quitan los duplicados de lista_tipo_dato_parametros_y_variables_locales y se concatenan sus items en el string string_tipo_dato_parametros_y_variables_locales

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

7

```
567 lista_parametros_rutina = []
568 if isinstance(string_parametros, str):
569
570     #se quita lo encapsulado entre comillas dobles y lo encapsulado entre parantesis
571     string_parametros_corr = func_access_bbdd_codigo_calculos_varios("QUITAR_ENCAPSULADO_ENTRE_COMILLAS_DOBLES", string_param = string_parametros)
572     string_parametros_corr = func_access_bbdd_codigo_calculos_varios("QUITAR_ENCAPSULADO_ENTRE_PARENTESIS", string_param = string_parametros_corr)
573
574     #se crea la lista lista_parametros_rutina spliteada por la coma ","
575     lista_parametros_rutina = string_parametros_corr.split(",")
```

La función **func_access_bbdd_codigo_calculos_varios** (módulo APP_3_BACK_END_MS_ACCESS) se documenta en el **ANEXO_09** del presente documento.

8

```
583 lista_variables_locales_rutina_linea_codigo = func_access_bbdd_codigo_calculos_varios("LISTA_DECLARACIONES_VARIABLES_LOCALES"
584                                             , indice_param_ini = indice_ini
585                                             , indice_param_fin = indice_fin
586                                             , df_bbdd_codigo_param = df_bbdd_codigo_param
587                                             )
```

Se usa la misma función que en el bloque anterior pero con otra opción, LISTA_DECLARACIONES_VARIABLES_LOCALES (ver **ANEXO_09**).

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

9

```
590 lista_variables_locales_rutina = []
591
592 if len(lista_variables_locales_rutina_linea_codigo) != 0:
593
594     for linea_codigo in lista_variables_locales_rutina_linea_codigo:
595
596         linea_codigo_corr = linea_codigo
597
598         #se quitan los " _" al final de la linea de codigo
599         if linea_codigo_corr[-2:] == indicador_linea_codigo_vba_truncada_a_linea_siguiente:
600             linea_codigo_corr = linea_codigo_corr[:-2]
601
602
603         #se quita lo encapsulado entre comillas dobles y lo encapsulado entre parantesis
604         linea_codigo_corr = func_access_bbdd_codigo_calculos_varios("QUITAR_ENCAPSULADO_ENTRE_COMILLAS_DOBLAS", string_param = linea_codigo_corr)
605         linea_codigo_corr = func_access_bbdd_codigo_calculos_varios("QUITAR_ENCAPSULADO_ENTRE_PARENTESIS", string_param = linea_codigo_corr)
606
607
608         #se crea la lista linea_codigo_corr_spliteada_por_coma spliteada por la coma ","
609         linea_codigo_corr_spliteada_por_coma = linea_codigo_corr.split(",")
610
611
612         #se agregan los items de linea_codigo_corr_spliteada_por_coma a lista_variables_locales_rutina
613         for item_split_coma in linea_codigo_corr_spliteada_por_coma:
614             if len(item_split_coma) != 0:
615                 lista_variables_locales_rutina.append(item_split_coma)
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

10

```
625 lista_parametros_y_variables_locales = lista_parametros_rutina + lista_variables_locales_rutina
626
627 lista_tipo_dato_parametros_y_variables_locales = []
628 for item in lista_parametros_y_variables_locales:
629
630     #se splitea el item por " As " y se conserva el 2do item de la lista resultante y se agrega a lista_tipo_dato_parametro
631     item_spliteado_por_as = item.strip().split(indicador_variable_vba_declaracion_tipo_dato)
632     tipo_dato_parametros_y_variables_locales = item_spliteado_por_as[1].strip() if len(item_spliteado_por_as) == 2 else ""
633
634     lista_tipo_dato_parametros_y_variables_locales.append(tipo_dato_parametros_y_variables_locales)
635
636 #se quitan los duplicados de lista_tipo_dato_parametros_y_variables_locales en string y se convierte en string
637 lista_tipo_dato_parametros_y_variables_locales = list(dict.fromkeys(lista_tipo_dato_parametros_y_variables_locales))
638 string_tipo_dato_parametros_y_variables_locales = ", ".join(lista_tipo_dato_parametros_y_variables_locales)
```


ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Usando el ejemplo comentado en páginas anteriores, el bloque de cálculo 7-8-9-10 daría:

UserForm_PRUEBA_1

```
tipo_modulo           UserForm
nombre_modulo          UserForm_PRUEBA_1
tipo_rutina_localizada RUTINA
tipo_declaracion_rutina_localizada PRIVADA
nombre_rutina_localizada BotonUPDATE_Click
*****
BLOQUES 7-8-9-10

lista_parametros_rutina      ['']

lista_variables_locales_rutina ['Dim Msg As Variant']

lista_tipo_dato_parametros_y_variables_locales ['', 'Variant']

string_tipo_dato_parametros_y_variables_locales , Variant
```

Modulo_PRUEBA_1

```
tipo_modulo           Estandar
nombre_modulo          Modulo_PRUEBA_1
tipo_rutina_localizada FUNCION
tipo_declaracion_rutina_localizada PUBLICA
nombre_rutina_localizada Func_CorreccionCIF
*****
BLOQUES 7-8-9-10

lista_parametros_rutina      ['xCIF As String']

lista_variables_locales_rutina ['Dim i As Integer', 'xTemp1 As String', 'xTemp2 As String']

lista_tipo_dato_parametros_y_variables_locales ['String', 'Integer']

string_tipo_dato_parametros_y_variables_locales String, Integer
```

Modulo_PRUEBA_1

```
tipo_modulo           Estandar
nombre_modulo          Modulo_PRUEBA_1
tipo_rutina_localizada RUTINA
tipo_declaracion_rutina_localizada PUBLICA
nombre_rutina_localizada Calc UpDate_CIF
*****
BLOQUES 7-8-9-10

lista_parametros_rutina      ['xProvincia_Sede As String', 'xActividadComercial As String', 'xRanking As Integer']

lista_variables_locales_rutina ['Dim MiConexion As New ADODB.Connection', 'Dim MiRecordset As New ADODB.Recordset', 'Dim MiSQL As String', 'i As Long']

lista_tipo_dato_parametros_y_variables_locales ['String', 'Integer', 'New ADODB.Connection', 'New ADODB.Recordset', 'Long']

string_tipo_dato_parametros_y_variables_locales String, Integer, New ADODB.Connection, New ADODB.Recordset, Long
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Tras finalizar todos los cálculos, siempre dentro de **bucle_rutinas**, se crea un diccionario temporal, donde agregar estos datos, que se agrega a la lista de diccionarios **lista_dicc_datos_rutinas**.

```
646     dicc_temp = {"TIPO_MODULO": tipo_modulo_ini
647                 , "NOMBRE_MODULO": nombre_modulo_ini
648                 , "INDICE_INICIO": indice_ini
649                 , "INDICE_FIN": indice_fin
650                 , "TIPO_RUTINA": tipo_rutina_localizada
651                 , "TIPO_DECLARACION_RUTINA": tipo_declaracion_rutina_localizada
652                 , "NOMBRE_RUTINA": nombre_rutina_localizada
653                 , "NUMERO_ORDEN_RUTINA_EN_MODULO": numero_orden_rutina_en_modulo
654                 , "RUTINA_TIENE_PARAMETROS": rutina_tiene_parametros
655                 , "STRING_PARAMETROS_RUTINA": string_parametros
656                 , "STRING_TIPO_DATO_Y_VARIABLES_LOCALES": string_tipo_dato_parametros_y_variables_locales
657                 }
658
659     lista_dicc_datos_rutinas.append(dicc_temp)
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

Una vez la lista **lista_dicc_datos_rutinas** generada, y ya fuera del bucle **bucle_rutinas** se pasa a crear la lista **lista_resultado_funcion** que sirve de base para crear el dataframe que se fusiona con el dataframe **df_bbdd_codigo_param** (parámetro de la presente rutina).

La lista **lista_resultado_funcion** tiene la misma longitud que el dataframe **df_bbdd_codigo_param**.

Los pasos a seguir son:

PASO	DESCRIPCIÓN
1	Se localiza el primer y último índice del dataframe df_bbdd_codigo_param
2	Mediante bucle entre este 1er y último índice (en adelante bucle_1), se localiza que diccionario de la lista lista_dicc_datos_rutinas tiene el el índice de la iteración sobre bucle_1 situado entre sus keys INDICE_INICIO y INDICE_FIN y se agregan los otros datos del diccionario a la lista lista_resultado_funcion. Se calcula tambien el número de linea de código dentro de la rutina / función. Los índices de bucle_1 que no coincidan con lista_dicc_datos_rutinas se agregan los datos a None

1

```
675 primer_indice_df_bbdd_codigo_param = df_bbdd_codigo_param["INDICE"].min()
676 ultimo_indice_df_bbdd_codigo_param = df_bbdd_codigo_param["INDICE"].max()
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

1

opcion = LISTA Rutinas

2

```
681     for indice_df in range(primer_indice_df_bbdd_codigo_param, ultimo_indice_df_bbdd_codigo_param + 1, 1):
682
683         check_rutina = 0
684         for dicc in lista_dicc_datos_rutinas:
685
686             indice_ini = dicc["INDICE_INICIO"]
687             indice_fin = dicc["INDICE_FIN"]
688
689             numero_linea_codigo_rutina = 0
690             if indice_df >= indice_ini and indice_df <= indice_fin:
691
692                 check_rutina = 1
693
694                 tipo_rutina = dicc["TIPO_RUTINA"]
695                 tipo_declaracion = dicc["TIPO_DECLARACION_RUTINA"]
696                 nombre_rutina = dicc["NOMBRE_RUTINA"]
697                 numero_orden_rutina_en_modulo = dicc["NUMERO_ORDEN_RUTINA_EN_MODULO"]
698                 rutina_tiene_parametros = dicc["RUTINA_TIENE_PARAMETROS"]
699                 string_parametros_rutina = dicc["STRING_PARAMETROS_RUTINA"]
700                 string_tipo_dato_parametros_y_variables_locales = dicc["STRING_TIPO_DATO_Y_VARIABLES_LOCALES"]
701
702             #se crean los numeros de linea de codigo dentro de la rutina / funcion
703             cont = 0
704
705             for cont in range(indice_ini, indice_df + 1, 1):
706                 numero_linea_codigo_rutina = cont - indice_ini + 1
707
708
709             #se agregan los datos a lista_resultado_funcion
710             lista_resultado_funcion.append([indice_df
711                                           , tipo_rutina
712                                           , tipo_declaracion
713                                           , nombre_rutina
714                                           , numero_linea_codigo_rutina
715                                           , numero_orden_rutina_en_modulo
716                                           , string_parametros_rutina
717                                           , string_tipo_dato_parametros_y_variables_locales
718                                           ])
719
720
721             #si indice_df no sale lista_dicc_datos_rutinas se añade con valores todos nulos
722             if check_rutina == 0:
723                 lista_resultado_funcion.append([indice_df, None, None, None, None, None, None, None])
```

2

opcion = LISTA_VARIABLES_PUBLICAS

La opción permite crear la lista **lista_resultado_funcion** donde se listan todas las líneas de código del dataframe **df_bbdd_codigo_param** asociadas a la declaración de variables publicas (sean normales o definidas por el usuario).

Es lista de listas donde cada una de sus sublistas se compone de:

ITEMS SUBLISTAS
Tipo de módulo
Nombre de módulo
Indice de la línea de código dentro del dataframe <code>df_bbdd_codigo_param</code>
Identificador del tipo de variable pública (NO_ES_CONSTANTE o SI_ES_CONSTANTE para las que son normales) o flag identificativo único que aplica a todas las líneas de código asociadas a una misma variable pública (para las definidas por el usuario)
Línea de código

Los pasos a seguir para el cálculo de la lista **lista_resultado_funcion** son:

AJUSTE	DESCRIPCIÓN
1	Se extrae del dataframe <code>df_bbdd_codigo_param</code> un dataframe <code>df_variables_publicas</code> que contiene todas las líneas de código que no están asociadas a rutinas / funciones, que no son encabezados de módulo y que las líneas de código no estén vacías. Llegados a este paso, en la rutina <code>del_1_proceso_access_import</code> ya se realizó la llamada a la función <code>func_access_df_bbdd_codigo_calculos</code> (opción <code>LISTA_ROUTINAS</code>) y por lo tanto el dataframe <code>df_bbdd_codigo_param</code> ya tiene la columna <code>NOMBRE_ROUTINA</code> creada e informada.
2	En base al dataframe extraído <code>df_variables_publicas</code> , se crea la lista <code>lista_modulos_con_variables_publicas</code> de los módulos que susceptibles de contener variables públicas.
3	Se crea la lista <code>lista_variables_publicas</code> y mediante bucle sobre la lista <code>lista_modulos_con_variables_publicas</code> (solo si no está vacía), se calculan las listas de listas <code>lista_variables_publicas_normales_modulo</code> y <code>lista_variables_publicas_definidas_por_usuario_modulo</code> donde cada ítem de las mismas se agregan a la lista <code>lista_variables_publicas</code> . Las listas <code>lista_variables_publicas_normales_modulo</code> y <code>lista_variables_publicas_definidas_por_usuario_modulo</code> se calculan mediante la función <code>func_access_bbdd_codigo_calculos_varios</code> (módulo <code>APP_3_BACK_END_MS_ACCESS</code>) y ambas tienen la misma estructura (ver cuadro anterior).
4	Se crea la lista lista_resultado_funcion en base a la lista <code>lista_variables_publicas</code> y se completan los huecos de índices de línea para que tenga la misma longitud y los mismos índices de línea que el dataframe <code>df_bbdd_codigo_param</code> .

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

2

opcion = LISTA_VARIABLES_PUBLICAS

Antes de empezar a explicar los pasos a seguir desde el código de la función, estos se ilustran a continuación con el ejemplo siguiente:

Modulo_PRUEBA_1

```
00 Option Compare Database
01 Option Explicit
02 Option Base 1
03
04 'VARIABLES PUBLICAS
05 Public Public_Version As String
06
07 Global Public_Var_1 As String, Public_Var_2 As String, _
08 Public_Array(1 To 10, 2) As Variant, Public_Var_3 As Date, _
09 Public_Var_4 As Variant
10
11 Public Const Public_Const_1 = "aqui hay comas , y signos = ", _
12     Public_Const_2 = "valor duplicado", Public_Const_3 = 5, _
13     Public_Const_4 = "valor duplicado"
14
15 Global Const Public_Const_5 = "@1", _
16 Public_Const_6 = 8, _
17 Public_Const_7 = "@2", _
18 Public_Const_8 = 5
19
20 Type UserType_Prueba_1 'variable publica definida por el usuario de p
21     SubVar_1 As String
22     SubVar_2 As Long
23 End Type
24
25 Enum Enum_Prueba_1
26     SubVar_1 'sub-variable de otra variable publica definida por el us
27     SubVar_2 = 1
28     SubVar_3
29 End Enum
30
31
32 Sub ActivarPanelNavegacion()
33
34 On Error Resume Next
35
36 DoCmd.LockNavigationPane (False)
37 DoCmd.SelectObject acTable, "MSysObjects", True
38 DoCmd.RunCommand acCmdWindowUnhide
39 MsgBox "bla bla bla"
40
41 End Sub
```



Variables públicas normales



Variables públicas definidas por el usuario

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

2

opcion = LISTA VARIABLES PUBLICAS

1

```
747 df_variables_publicas = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["ES_ENCABEZADO_MODULO"] == "NO") & (df_bbdd_codigo_param["NOMBRE Rutina"].isnull() == True) &
748                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
749                                     ["TIPO_MODULO", "NOMBRE_MODULO", "INDICE", "NOMBRE_RUTINA", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
750
751 df_variables_publicas.reset_index(drop = True, inplace = True)
```



	TIPO_MODULO	NOMBRE_MODULO	INDICE	NOMBRE_RUTINA	CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0	Estandar	Modulo_PRUEBA_1	5	None	Public Public_Version As String
1	Estandar	Modulo_PRUEBA_1	7	None	Global Public_Var_1 As String, Public_Var_2 As...
2	Estandar	Modulo_PRUEBA_1	8	None	Public_Array(1 To 10, 2) As Variant, Public_Va...
18	Estandar	Modulo_PRUEBA_1	28	None	SubVar_3
19	Estandar	Modulo_PRUEBA_1	29	None	End Enum

2

```
755 df_temp = df_variables_publicas.copy()
756 df_temp.drop_duplicates(subset = ["TIPO_MODULO", "NOMBRE_MODULO"], keep = "last", inplace = True)
757 df_temp.reset_index(drop = True, inplace = True)
758
759 lista_modulos_con_variables_publicas = [[df_temp.iloc[ind, df_temp.columns.get_loc("TIPO_MODULO")]
760                                     , df_temp.iloc[ind, df_temp.columns.get_loc("NOMBRE_MODULO")]]
761                                     for ind in df_temp.index]
```



```
lista_modulos_con_variables_publicas [['Estandar', 'Modulo_PRUEBA_1']]
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

2

opcion = LISTA VARIABLES PUBLICAS

3

```
766 lista_variables_publicas = []
767 if len(lista_modulos_con_variables_publicas) != 0:
768
769     for tipo_modulo, nombre_modulo in lista_modulos_con_variables_publicas:
```

```
772         primer_indice_df_bbdd_codigo_param_modulo = df_variables_publicas[(df_variables_publicas["TIPO_MODULO"] == tipo_modulo) & (df_variables_publicas["NOMBRE_MODULO"] == nombre_modulo)]["INDICE"].min()
773         ultimo_indice_df_bbdd_codigo_param_modulo = df_variables_publicas[(df_variables_publicas["TIPO_MODULO"] == tipo_modulo) & (df_variables_publicas["NOMBRE_MODULO"] == nombre_modulo)]["INDICE"].max()
```

```
primer_indice_df_bbdd_codigo_param_modulo    5
ultimo_indice_df_bbdd_codigo_param_modulo    29
```

```
793 # VARIABLES PUBLICAS NORMALES
794 lista_variables_publicas_normales_modulo = func_access_bbdd_codigo_calculos_varios("LISTA_DECLARACIONES_VARIABLES_PUBLICAS_NORMALES"
795                                           , indice_param_ini = primer_indice_df_bbdd_codigo_param_modulo
796                                           , indice_param_fin = ultimo_indice_df_bbdd_codigo_param_modulo
797                                           , df_bbdd_codigo_param = df_variables_publicas
798                                           )
799
800 if len(lista_variables_publicas_normales_modulo) != 0:
801
802     for item in lista_variables_publicas_normales_modulo:
803         lista_variables_publicas.append(item)
```

```
lista_variables_publicas_normales_modulo
-----
[['Estandar', 'Modulo_PRUEBA_1', 5, 'NO_ES_CONSTANTE', 'Public Public_Version As String'],
['Estandar', 'Modulo_PRUEBA_1', 7, 'NO_ES_CONSTANTE', 'Global Public_Var_1 As String, Public_Var_2 As String,'],
['Estandar', 'Modulo_PRUEBA_1', 8, 'NO_ES_CONSTANTE', 'Public_Array(1 To 10, 2) As Variant, Public_Var_3 As Date,'],
['Estandar', 'Modulo_PRUEBA_1', 9, 'NO_ES_CONSTANTE', 'Public_Var_4 As Variant'],
['Estandar', 'Modulo_PRUEBA_1', 11, 'SI_ES_CONSTANTE', 'Public Const Public_Const_1 = "aquí hay comas , y signos = ",'],
['Estandar', 'Modulo_PRUEBA_1', 12, 'SI_ES_CONSTANTE', 'Public_const_2 = "valor duplicado", Public_const_3 = 5,'],
['Estandar', 'Modulo_PRUEBA_1', 13, 'SI_ES_CONSTANTE', 'Public_const_4 = "valor duplicado"'],
['Estandar', 'Modulo_PRUEBA_1', 15, 'SI_ES_CONSTANTE', 'Global Const Public_const_5 = "@1",'],
['Estandar', 'Modulo_PRUEBA_1', 16, 'SI_ES_CONSTANTE', 'Public_const_6 = 8,'],
['Estandar', 'Modulo_PRUEBA_1', 17, 'SI_ES_CONSTANTE', 'Public_const_7 = "@2",'],
['Estandar', 'Modulo_PRUEBA_1', 18, 'SI_ES_CONSTANTE', 'Public_const_8 = 5']]
```


ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

2

opcion = LISTA VARIABLES PUBLICAS

4

```
823 primer_indice_df_bbdd_codigo_param = df_bbdd_codigo_param["INDICE"].min()
824 ultimo_indice_df_bbdd_codigo_param = df_bbdd_codigo_param["INDICE"].max()
825
826 lista_variables_publicas_indices = [indice_linea for tipo_modulo, nombre_modulo, indice_linea, tipo_variable, linea_codigo in lista_variables_publicas]
827
828 lista_resultado_funcion = lista_variables_publicas.copy()
829
830 #se completan los huecos con los indices de codigo que no estan en lista_variables_publicas_indices
831 for indice_df in range(primer_indice_df_bbdd_codigo_param, ultimo_indice_df_bbdd_codigo_param + 1, 1):
832
833     if not indice_df in lista_variables_publicas_indices:
834         lista_resultado_funcion.append([tipo_modulo, nombre_modulo, indice_df, "NO", None])
```



```
[['Estandar', 'Modulo_PRUEBA_1', 5, 'NO_ES_CONSTANT', 'Public Public_Version As String'],
['Estandar', 'Modulo_PRUEBA_1', 7, 'NO_ES_CONSTANT', 'Global Public Var_1 As String, Public_Var_2 As String,'],
['Estandar', 'Modulo_PRUEBA_1', 8, 'NO_ES_CONSTANT', 'Public Array(1 To 10, 2) As Variant, Public_Var_3 As Date,'],
['Estandar', 'Modulo_PRUEBA_1', 9, 'NO_ES_CONSTANT', 'Public_Var_4 As Variant'],
['Estandar', 'Modulo_PRUEBA_1', 11, 'SI_ES_CONSTANT', 'Public Const Public_Const_1 = "aquí hay comas , y signos = ",'],
['Estandar', 'Modulo_PRUEBA_1', 12, 'SI_ES_CONSTANT', 'Public_const_2 = "valor duplicado", Public_const_3 = 5,'],
['Estandar', 'Modulo_PRUEBA_1', 13, 'SI_ES_CONSTANT', 'Public_const_4 = "valor duplicado"'],
['Estandar', 'Modulo_PRUEBA_1', 15, 'SI_ES_CONSTANT', 'Global Const Public_const_5 = "@!",'],
['Estandar', 'Modulo_PRUEBA_1', 16, 'SI_ES_CONSTANT', 'Public_const_6 = 8,'],
['Estandar', 'Modulo_PRUEBA_1', 17, 'SI_ES_CONSTANT', 'Public_const_7 = "@2",'],
['Estandar', 'Modulo_PRUEBA_1', 18, 'SI_ES_CONSTANT', 'Public_const_8 = 5'],
['Estandar', 'Modulo_PRUEBA_1', 20, 'DEFINIDA POR USUARIO_1', 'Type UserType_Prueba_1'],
['Estandar', 'Modulo_PRUEBA_1', 21, 'DEFINIDA POR USUARIO_1', 'SubVar_1 As String'],
['Estandar', 'Modulo_PRUEBA_1', 22, 'DEFINIDA POR USUARIO_1', 'SubVar_2 As Long'],
['Estandar', 'Modulo_PRUEBA_1', 23, 'DEFINIDA POR USUARIO_1', 'End Type'],
['Estandar', 'Modulo_PRUEBA_1', 25, 'DEFINIDA POR USUARIO_2', 'Enum Enum_Prueba_1'],
['Estandar', 'Modulo_PRUEBA_1', 26, 'DEFINIDA POR USUARIO_2', 'SubVar_1'],
['Estandar', 'Modulo_PRUEBA_1', 27, 'DEFINIDA POR USUARIO_2', 'SubVar_2 = 1'],
['Estandar', 'Modulo_PRUEBA_1', 28, 'DEFINIDA POR USUARIO_2', 'SubVar_3'],
['Estandar', 'Modulo_PRUEBA_1', 29, 'DEFINIDA POR USUARIO_2', 'End Enum'],
['Estandar', 'Modulo_PRUEBA_1', 0, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 1, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 2, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 3, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 4, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 6, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 10, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 14, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 19, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 24, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 30, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 31, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 32, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 33, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 34, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 35, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 36, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 37, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 38, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 39, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 40, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 41, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 42, 'NO', None]]
```



Variables públicas normales



Variables públicas definidas por el usuario



Huecos vacíos completados

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

La opción permite preparar una **lista de diccionarios que lista una a una todas las variables públicas** normales y las que están definidas por el usuario (estas al declararse en varias líneas se unifican en 1 sola concatenando sus sub-variables con su tipo de dato o valor)

La estructura de los diccionarios que componen esta lista es **común a todos los objetos MS ACCESS** (tablas / vínculos, variables publicas VBA y rutinas / funciones VBA) por lo que algunos items no aplican aquí y se establecen a None.

KEY	DESCRIPCIÓN	APLICA
TIPO_OBJETO	TABLA_LOCAL, VINCULO_ODBC, VINCULO_OTRO, RUTINAS_VBA, VARIABLES_VBA (son las subkeys_3 del diccionario dicc_control_versiones_tipo_objeto asociado MS_ACCESS)	X
TIPO_MODULO	Formulario, UserForm VBA, módulo estandar VBA	X
NOMBRE_MODULO	Nombre del módulo	X
NOMBRE_OBJETO	Nombre del objeto	X
TIPO_RUTINA	Rutina o función	
TIPO_DECLARACION_RUTINA	Pública o privada	
PARAMETROS_RUTINA	Concatenación de los parámetros de la rutina VBA con sus tipos de datos	
TIPO_VARIABLE_PUBLICA	Variable pública normal o variable pública definida por el usuario	X
TIPO_DATO_VARIABLE_PUBLICA	Tipo de dato para las variables públicas normales que no son constantes	X
VARIABLE_PUBLICA_ES_CONSTANTE	Indica si la variable pública es constante o no	X
VARIABLE_PUBLICA_CONSTANTE_VALOR	Valor de la constante para las variables públicas normales que son constantes	X
VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES	Concatenación de las sub-variables asociadas a una variable pública definida por el usuario	X
DF_CODIGO	Dataframe con el código del objeto (sentencia CREATE TABLE para las tablas, connecting string para los vinculos, código VBA para el resto)	(*)

(*) el código a nivel de variables va por tipo y nombre de módulo VBA, se calcula directamente en la rutina def_proceso_access_1_import

ANEXO_08

FUNCIÓN: `func_access_df_bbdd_codigo_calculos`

3

`opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS`

La opción requiere el uso del parámetro `kwargs` `lista_calculos_variables_publicas`. A continuación, se explica brevemente cual es su origen y de que forma llega la función como parámetro.

En la rutina `def_proceso_access_import_1`, ver **ANEXO_01**, se calcula una lista de variables públicas, mediante la llamada a la función `func_access_df_bbdd_codigo_calculos` (opción `LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS`), ver **ANEXO_08**.

```
2405 lista_calculos_variables_publicas = func_access_bbdd_codigo_calculos("LISTA_VARIABLES_PUBLICAS", df_bbdd_codigo_param = df_bbdd_codigo)
```

Esta lista calculada `lista_calculos_variables_publicas` aplica para todos los tipos de variables públicas (sean normales o definidas por el usuario) y permite disponer de todas las líneas de código relacionadas con la declaración de variables públicas. Es lista de lista donde cada una de sus sublistas tiene la estructura siguiente:

ITEMS SUBLISTAS
Tipo de módulo
Nombre de módulo
Indice de la linea de código dentro del dataframe <code>df_bbdd_codigo_param</code> (parámetro de la función)
Indicador de si la linea de código es de declaración de constante o no (para las variables públicas normales) o flag unico identificador para varias lineas de bódigo (para las variables públicas definidas por el usuario)
Linea de código

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

Usando el ejemplo del bloque 1 del presente anexo (LISTA_VARIABLES_PUBLICAS) y que se usa también para ilustrar la opción LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS detallada en este bloque 3 (en la página siguiente se recuerda el ejemplo):

```
[['Estandar', 'Modulo_PRUEBA_1', 5, 'NO_ES_CONSTANTE', 'Public Public_Version As String'],
['Estandar', 'Modulo_PRUEBA_1', 7, 'NO_ES_CONSTANTE', 'Global Public Var_1 As String, Public Var_2 As String,'],
['Estandar', 'Modulo_PRUEBA_1', 8, 'NO_ES_CONSTANTE', 'Public Array(1 To 10, 2) As Variant, Public Var_3 As Date,'],
['Estandar', 'Modulo_PRUEBA_1', 9, 'NO_ES_CONSTANTE', 'Public Var_4 As Variant'],
['Estandar', 'Modulo_PRUEBA_1', 11, 'SI_ES_CONSTANTE', 'Public Const Public_Const_1 = "aquí hay comas , y signos = ",'],
['Estandar', 'Modulo_PRUEBA_1', 12, 'SI_ES_CONSTANTE', 'Public Const_2 = "valor duplicado", Public_Const_3 = 5,'],
['Estandar', 'Modulo_PRUEBA_1', 13, 'SI_ES_CONSTANTE', 'Public_Const_4 = "valor duplicado"',],
['Estandar', 'Modulo_PRUEBA_1', 15, 'SI_ES_CONSTANTE', 'Global Const Public_Const_5 = "@1",'],
['Estandar', 'Modulo_PRUEBA_1', 16, 'SI_ES_CONSTANTE', 'Public_Const_6 = 8,'],
['Estandar', 'Modulo_PRUEBA_1', 17, 'SI_ES_CONSTANTE', 'Public_Const_7 = "@2",'],
['Estandar', 'Modulo_PRUEBA_1', 18, 'SI_ES_CONSTANTE', 'Public_Const_8 = 5'],
['Estandar', 'Modulo_PRUEBA_1', 20, 'DEFINIDA POR USUARIO_1', 'Type UserType_Prueba_1'],
['Estandar', 'Modulo_PRUEBA_1', 21, 'DEFINIDA POR USUARIO_1', 'SubVar_1 As String'],
['Estandar', 'Modulo_PRUEBA_1', 22, 'DEFINIDA POR USUARIO_1', 'SubVar_2 As Long'],
['Estandar', 'Modulo_PRUEBA_1', 23, 'DEFINIDA POR USUARIO_1', 'End Type'],
['Estandar', 'Modulo_PRUEBA_1', 25, 'DEFINIDA POR USUARIO_2', 'Enum Enum_Prueba_1'],
['Estandar', 'Modulo_PRUEBA_1', 26, 'DEFINIDA POR USUARIO_2', 'SubVar_1'],
['Estandar', 'Modulo_PRUEBA_1', 27, 'DEFINIDA POR USUARIO_2', 'SubVar_2 = 1'],
['Estandar', 'Modulo_PRUEBA_1', 28, 'DEFINIDA POR USUARIO_2', 'SubVar_3'],
['Estandar', 'Modulo_PRUEBA_1', 29, 'DEFINIDA POR USUARIO_2', 'End Enum'],
['Estandar', 'Modulo_PRUEBA_1', 0, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 1, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 2, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 3, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 4, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 6, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 10, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 14, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 19, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 24, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 30, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 31, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 32, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 33, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 34, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 35, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 36, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 37, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 38, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 39, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 40, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 41, 'NO', None],
['Estandar', 'Modulo_PRUEBA_1', 42, 'NO', None]]
```



Variables públicas normales



Variables públicas definidas por el usuario



Huecos vacíos completados para obtener la **misma longitud** y mismos índices de líneas de código que el dataframe df_bbdd_codigo_param

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

Antes de empezar a explicar los pasos a seguir desde el código de la función, estos se ilustran a continuación con el ejemplo siguiente:

Modulo_PRUEBA_1

```
00 Option Compare Database
01 Option Explicit
02 Option Base 1
03
04 'VARIABLES PUBLICAS
05 Public Public_Version As String
06
07 Global Public_Var_1 As String, Public_Var_2 As String, _
08 Public_Array(1 To 10, 2) As Variant, Public_Var_3 As Date, _
09 Public_Var_4 As Variant
10
11 Public Const Public_Const_1 = "aquí hay comas , y signos = ", _
12     Public_Const_2 = "valor duplicado", Public_Const_3 = 5, _
13     Public_Const_4 = "valor duplicado"
14
15 Global Const Public_Const_5 = "@1", _
16 Public_Const_6 = 8, _
17 Public_Const_7 = "@2", _
18 Public_Const_8 = 5
19
20 Type UserType_Prueba_1 'variable publica definida por el usuario de prueba
21     SubVar_1 As String
22     SubVar_2 As Long
23 End Type
24
25 Enum Enum_Prueba_1
26     SubVar_1 'sub-variable de otra variable publica definida por el usuario
27     SubVar_2 = 1
28     SubVar_3
29 End Enum
30
31
32 Sub ActivarPanelNavegacion()
33
34 On Error Resume Next
35
36 DoCmd.LockNavigationPane (False)
37 DoCmd.SelectObject acTable, "MSysObjects", True
38 DoCmd.RunCommand acCmdWindowUnhide
39 MsgBox "bla bla bla"
40
41 End Sub
```



Variables públicas normales



Variables públicas definidas por el usuario

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

Antes de pasar explicar el cálculo de la lista de diccionarios, resultado de la función se requieren unos **ajustes previos**.

AJUSTE	DESCRIPCIÓN
lista_variables_publicas_normales_declar_ini_ord_desc	Fusiona en una lista de listas los valores de las subkey_3 (las que empiezan por LISTA_DECLARACION_INICIO) asociadas a la key_1 (VARIABLE_PUBLICA_NORMAL) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS). Cada uno de los items de la lista resultante se trimea y se le agrega un espacio en blanco al final (es por si el usuario ha configurado mal el diccionario). La lista se re-ordena por la longitud de sus items de mayor a menor
lista_variable_definida_por_usuario_ini	Se recupera la lista almacenada en la subkey_3 (LISTA_DECLARACION_INICIO) asociadas a la key_1 (VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS) donde para cada uno de sus items se trimean y se les agrega un espacio en blanco al final (es por si el usuario ha configurado mal el diccionario).

```
891 #se fusionan todos los inicios de declaracion en una lista donde cada item es el inicio de declaracion ordenado longitud mayor a menor)
892 #(esta lista lista_variables_publicas_normales_declar_ini_ord_desc, que se usa tambien en la opcion DF_COLUMNS_VARIABLES_PUBLICAS
893 #de la presente funcion, no requiere aqui crear tambien una lista de sublistas, con el inicio de declaracion es suficiente porque
894 #se usa tan solo para hacer replace del item por nada)
895 lista_variables_publicas_normales_declar_ini_ord_desc = [item.strip() + " " for key_2 in dicc_objetos_vba["VARIABLE_PUBLICA_NORMAL"].keys()
896 | | | | | | | | | | for item in dicc_objetos_vba["VARIABLE_PUBLICA_NORMAL"][key_2] if "LISTA_DECLARACION_INICIO" in key_2]
897
898 lista_variables_publicas_normales_declar_ini_ord_desc.sort(key = lambda x: len(x[1]), reverse = True)
899
900
901 #se crean la lista lista_variable_definida_por_usuario_ini que recupera la lista almacenada en subkey_3 (LISTA_DECLARACION_INICIO)
902 #asociada a la key_1 (VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO) del diccionario dicc_objetos_vba cada uno de sus items se trime y se
903 #le agrega un espacio en blanco al final (es por si el usuario ha configurado mal el diccionario)
904 lista_variable_definida_por_usuario_ini = [item.strip() + " " for item in dicc_objetos_vba["VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO"]["LISTA_DECLARACION_INICIO"]]
```



```
lista_variables_publicas_normales_declar_ini_ord_desc    ['Public ', 'Global ', 'Public Const ', 'Global Const ', 'Const ']
lista_variable_definida_por_usuario_ini                  ['Type ', 'Enum ']
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

Realizados los ajustes previos, se pasa a detallar los cálculos de la función. Estos se realizan en 2 bloques según el tipo de variable pública (normal o definida por el usuario). Dentro de cada bloque, se detallan, a continuación, los pasos a seguir:

TIPO VARIABLE	BLOQUE	DESCRIPCIÓN
VARIABLES PUBLICAS NORMALES	1	Se crea la lista <code>lista_modulos_con_variables_publicas_normales</code> en base al parámetro <code>lista_calculos_variables_publicas</code> filtrando solo para las variables públicas normales, que lista todos los módulos con este tipo de variables públicas (a la cual se le quitan los duplicados).
	2	Mediante bucle (en adelante <code>bucle_1</code>) sobre la lista <code>lista_modulos_con_variables_publicas_normales</code> , se crea la lista <code>lista_variables_publicas_lineas_modulo</code> que lista todas las líneas de código con su flag correspondiente (<code>NO_ES_CONSTANTE</code> o <code>SI_ES_CONSTANTE</code>).
	3	Mediante bucle (en adelante <code>bucle_2</code>), siempre dentro de <code>bucle_1</code> , sobre los items de la lista <code>lista_variables_publicas_lineas_modulo</code> se hace replace de los inicios de declaración (se usa la lista <code>lista_variables_publicas_normales_declar_ini_ord_desc</code> , calculada en los ajustes previos) y se listan las variables públicas una a una (aportando información complementaria como el nombre de la variable, su tipo de dato o en su defecto su valor si es constante) en la lista <code>lista_variables_publicas_1_a_1</code> , mediante la función <code>func_access_bbdd_codigo_calculos_varios</code> (opción <code>LISTA_VARIABLES_PUBLICAS_NO_ES_CONSTANTE</code> o <code>LISTA_VARIABLES_PUBLICAS_SI_ES_CONSTANTE</code> en función del tipo de variable de la iteración sobre <code>bucle_2</code>).
	4	Mediante bucle sobre la lista <code>lista_variables_publicas_1_a_1</code> , siempre dentro de <code>bucle_1</code> y <code>bucle_2</code> , se crea diccionario temporal en cada iteración que se agrega a la lista de diccionarios <code>lista_resultado_funcion</code> .
VARIABLES DEFINIDAS POR USUARIO	5	Se crea la lista <code>lista_modulos_con_variables_publicas_definidas_usuario</code> en base al parámetro <code>lista_calculos_variables_publicas</code> filtrando solo para las variables públicas definidas por el usuario, que lista todos los módulos con este tipo de variables públicas (a la cual se le quitan los duplicados).
	6	Mediante bucle (en adelante <code>bucle_1</code>) sobre la lista <code>lista_modulos_con_variables_publicas_definidas_usuario</code> , se crea la lista <code>lista_variables_publicas_definidas_usuario_modulo</code> que lista todos los flags dentro del módulo correspondientes a este tipo de variable pública (<code>DEFINIDA_PO_USUARIO_+contador</code>).
	7	Mediante bucle (en adelante <code>bucle_2</code>) sobre la lista <code>lista_variables_publicas_definidas_usuario_modulo</code> , se crea la lista <code>lista_lineas_flag_variable_publica</code> que toda las líneas de código asociadas al flag de la variable pública de la iteración sobre <code>bucle_2</code> , esta lista se re-ordena por índice de línea de menor a mayor. Se crea la lista <code>lista_variables_publicas_1_a_1</code> , mediante la función <code>func_access_bbdd_codigo_calculos_varios</code> , donde se lista una a una todas las variables públicas definidas por el usuario (aportando información complementaria como el nombre de la variable, su tipo de dato y sus sub-variables concatenadas para cada una de las cuales se informa de su tipo de dato y en su defecto de su valor).
	8	Mediante bucle sobre la lista <code>lista_variables_publicas_1_a_1</code> , siempre dentro de <code>bucle_1</code> y <code>bucle_2</code> , se crea diccionario temporal en cada iteración que se agrega a la lista de diccionarios <code>lista_resultado_funcion</code> .

Los cálculos se hacen **por separado según el tipo de variable pública**. Las que son normales, el cálculo se hace línea a línea. Las que están definidas por el usuario, requieren agrupar en el mismo flag identificador único de la variable pública todas las líneas de código asociadas.

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA DICC OBJETOS VARIABLES PUBLICAS



```
912 lista_modulos_con_variables_publicas_normales = [[tipo_modulo, nombre_modulo] for tipo_modulo, nombre_modulo, indice_linea, flag_variable_publica, linea_codigo in lista_calculos_variables_publicas
913 | | | | | if flag_variable_publica in ["NO_ES_CONSTANTE", "SI_ES_CONSTANTE"]]
914
915 lista_modulos_con_variables_publicas_normales = [sublista for i, sublista in enumerate(lista_modulos_con_variables_publicas_normales) if sublista not in lista_modulos_con_variables_publicas_normales[:i]]
```

```
lista_modulos_con_variables_publicas_normales [['Estandar', 'Modulo_PRUEBA_1']]
```

2

```
920     for tipo_modulo_iteracion, nombre_modulo_iteracion in lista_modulos_con_variables_publicas_normales:
921
922         #se recuperan el flag de tipo de variable y las lineas de codigo asociadas a este tipo de variable publica y para el modulo de la iteracion
923         lista_variables_publicas_lineas_modulo = [[flag_variable_publica, linea_codigo]
924                                                    for tipo_modulo, nombre_modulo, indice_linea, flag_variable_publica, linea_codigo in lista_calculos_variables_publicas
925                                                    if tipo_modulo == tipo_modulo_iteracion and nombre_modulo == nombre_modulo_iteracion and
926                                                    flag_variable_publica in ["NO ES CONSTANTE", "SI ES CONSTANTE"]]
```

```
lista_variables_publicas_lineas_modulo

[['NO_ES_CONSTANTE', 'Public Public_Version As String'],
 ['NO_ES_CONSTANTE', 'Global Public_Var_1 As String, Public_Var_2 As String,'],
 ['NO_ES_CONSTANTE', 'Public Array(1 To 10, 2) As Variant, Public_Var_3 As Date,'],
 ['NO_ES_CONSTANTE', 'Public_Var_4 As Variant'],
 ['SI_ES_CONSTANTE', 'Public Const Public_Const_1 = "aquí hay comas , y signos = ",'],
 ['SI_ES_CONSTANTE', 'Public_const_2 = "valor duplicado", Public_const_3 = 5,'],
 ['SI_ES_CONSTANTE', 'Public_const_4 = "valor duplicado"'],
 ['SI_ES_CONSTANTE', 'Global Const Public_const_2 = "@1",'],
 ['SI_ES_CONSTANTE', 'Public_const_3 = 8,'],
 ['SI_ES_CONSTANTE', 'Public_const_4 = "@2",'],
 ['SI_ES_CONSTANTE', 'Public_const_5 = 5']]
```


ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

3

```
934     for flag_variable_publica, linea_codigo in lista_variables_publicas_lineas_modulo:
935
936         #se quitan los inicios de declaracion segun el tipo de variable publica
937         for item_replace in lista_variables_publicas_normales_declar_ini_ord_desc:
938             if item_replace in linea_codigo:
939                 linea_codigo = linea_codigo.replace(item_replace, "")
940
941
942         #se calcula la lista de variables publicas 1 a 1 mediante la funcion func_access_bbdd_codigo_calculos_varios
943         opcion_funcion = "LISTA_VARIABLES_PUBLICAS_NO_ES_CONSTANTE" if flag_variable_publica == "NO_ES_CONSTANTE" else "LISTA_VARIABLES_PUBLICAS_SI_ES_CONSTANTE"
944         lista_variables_publicas_1_a_1 = func_access_bbdd_codigo_calculos_varios(opcion_funcion, linea_codigo = linea_codigo)
```

flag_variable_publica NO_ES_CONSTANTE
linea_codigo Global Public_Var_1 As String, Public_Var_2 As String,
linea_codigo (tras replace) Public_Var_1 As String, Public_Var_2 As String,
lista_variables_publicas_1_a_1 [['Public_Var_1', 'VARIABLE PUBLICA NORMAL', 'String', 'NO', None, None], ['Public_Var_2', 'VARIABLE PUBLICA NORMAL', 'String', 'NO', None, None]]

Item 1

Item 2

flag_variable_publica SI_ES_CONSTANTE
linea_codigo Public Const Public_Const_1 = "aquí hay comas , y signos = ",
linea_codigo (tras replace) Public_Const_1 = "aquí hay comas , y signos = ",
lista_variables_publicas_1_a_1 [['Public_Const_1', 'VARIABLE PUBLICA NORMAL', None, 'SI', 'aquí hay comas , y signos = ', None]]

Se muestran aquí tan solo los ejemplos de los índices de línea 7 y 11

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

4

```
948 for item_variable_publica in lista_variables_publicas_1_a_1:
949
950     variable_publica_nombre = item_variable_publica[0]
951     variable_publica_tipo = item_variable_publica[1]
952     variable_publica_tipo_dato = item_variable_publica[2]
953     variable_publica_es_constante = item_variable_publica[3]
954     variable_publica_es_constante_valor = item_variable_publica[4]
955     variable_publica_definida_usuario_subvariables = item_variable_publica[5]
956
957     dicc_temp = {"TIPO_OBJETO": "VARIABLES_VBA"
958                 , "TIPO_MODULO": tipo_modulo_iteracion
959                 , "NOMBRE_MODULO": nombre_modulo_iteracion
960                 , "NOMBRE_OBJETO": variable_publica_nombre
961                 , "TIPO Rutina": None #key que no aplica
962                 , "TIPO_DECLARACION_Rutina": None #key que no aplica
963                 , "PARAMETROS_Rutina": None #key que no aplica
964                 , "TIPO_VARIABLE_PUBLICA": variable_publica_tipo
965                 , "TIPO_DATO_VARIABLE_PUBLICA": variable_publica_tipo_dato
966                 , "VARIABLE_PUBLICA_ES_CONSTANTE": variable_publica_es_constante
967                 , "VARIABLE_PUBLICA_CONSTANTE_VALOR": variable_publica_es_constante_valor
968                 , "VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES": variable_publica_definida_usuario_subvariables
969                 , "DF_CODIGO": None #key que no aplica por ahora (se asig
970                 , "CONNECTING_STRING_VINCULOS": None #key que no aplica
971                 }
972
973     lista_resultado_funcion.append(dicc_temp)
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

4

```
ITERACIÓN lista_variables_publicas_1_a_1 1
Item ['Public_Var_1', 'VARIABLE PUBLICA NORMAL', 'String', 'NO', None, None]
-----
TIPO_OBJETO          VARIABLES_VBA
TIPO_MODULO          Estandar
NOMBRE_MODULO        Modulo_PRUEBA_1
NOMBRE_OBJETO        Public_Var_1
TIPO Rutina          None
TIPO_DECLARACION_Rutina None
PARAMETROS_Rutina    None
TIPO_VARIABLE_PUBLICA VARIABLE PUBLICA NORMAL
TIPO_DATO_VARIABLE_PUBLICA String
VARIABLE_PUBLICA_ES_CONSTANTE NO
VARIABLE_PUBLICA_CONSTANTE_VALOR None
VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES None
DF_CODIGO            None
CONNECTING_STRING_VINCULOS None

ITERACIÓN lista_variables_publicas_1_a_1 2
Item ['Public_Var_2', 'VARIABLE PUBLICA NORMAL', 'String', 'NO', None, None]
-----
TIPO_OBJETO          VARIABLES_VBA
TIPO_MODULO          Estandar
NOMBRE_MODULO        Modulo_PRUEBA_1
NOMBRE_OBJETO        Public_Var_2
TIPO Rutina          None
TIPO_DECLARACION_Rutina None
PARAMETROS_Rutina    None
TIPO_VARIABLE_PUBLICA VARIABLE PUBLICA NORMAL
TIPO_DATO_VARIABLE_PUBLICA String
VARIABLE_PUBLICA_ES_CONSTANTE NO
VARIABLE_PUBLICA_CONSTANTE_VALOR None
VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES None
DF_CODIGO            None
CONNECTING_STRING_VINCULOS None
```

```
ITERACIÓN lista_variables_publicas_1_a_1 1
Item ['Public_Const_1', 'VARIABLE PUBLICA NORMAL', None, 'SI', 'aquí hay comas , y signos = ', None]
-----
TIPO_OBJETO          VARIABLES_VBA
TIPO_MODULO          Estandar
NOMBRE_MODULO        Modulo_PRUEBA_1
NOMBRE_OBJETO        Public_Const_1
TIPO Rutina          None
TIPO_DECLARACION_Rutina None
PARAMETROS_Rutina    None
TIPO_VARIABLE_PUBLICA VARIABLE PUBLICA NORMAL
TIPO_DATO_VARIABLE_PUBLICA None
VARIABLE_PUBLICA_ES_CONSTANTE SI
VARIABLE_PUBLICA_CONSTANTE_VALOR aquí hay comas , y signos =
VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES None
DF_CODIGO            None
CONNECTING_STRING_VINCULOS None
```

Se muestran aquí tan solo los ejemplos de los índices de línea 7 y 11

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

5

```
986 lista_modulos_con_variables_publicas_definidas_usuario = [[tipo_modulo, nombre_modulo] for tipo_modulo, nombre_modulo, indice_linea, flag_variable_publica, linea_codigo in lista_calculos_variables_publicas
987 if "DEFINIDA_POR_USUARIO" in flag_variable_publica]
988
989 lista_modulos_con_variables_publicas_definidas_usuario = [sublista for i, sublista in enumerate(lista_modulos_con_variables_publicas_normales)
990 if sublista not in lista_modulos_con_variables_publicas_normales[:i]]
```



```
lista_modulos_con_variables_publicas_definidas_usuario [['Estandar', 'Modulo_PRUEBA_1']]
```

6

```
995 for tipo_modulo_iteracion, nombre_modulo_iteracion in lista_modulos_con_variables_publicas_definidas_usuario:
996
997     #se recuperan los distintos flags de este tipo de variable publica dentro del modulo de la iteracion donde se quitan los duplicados
998     lista_variables_publicas_definidas_usuario_modulo = [flag_variable_publica
999 for tipo_modulo, nombre_modulo, indice_linea, flag_variable_publica, linea_codigo in lista_calculos_variables_publicas
1000 if tipo_modulo == tipo_modulo_iteracion and nombre_modulo == nombre_modulo_iteracion and "DEFINIDA_POR_USUARIO" in flag_variable_publica]
1001
1002 lista_variables_publicas_definidas_usuario_modulo = list(dict.fromkeys(lista_variables_publicas_definidas_usuario_modulo))
```



```
nombre_modulo_iteracion      Modulo_PRUEBA_1
lista_variables_publicas_definidas_usuario_modulo ['DEFINIDA_POR_USUARIO_1', 'DEFINIDA_POR_USUARIO_2']
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

7

```
1008     for flag_variable_publica_modulo in lista_variables_publicas_definidas_usuario_modulo:
1009
1010         lista_lineas_flag_variable_publica = [[indice_linea, linea_codigo]
1011                                             for tipo_modulo, nombre_modulo, indice_linea, flag_variable_publica, linea_codigo in lista_calculos_variables_publicas
1012                                             if tipo_modulo == tipo_modulo_iteracion and nombre_modulo == nombre_modulo_iteracion and flag_variable_publica == flag_variable_publica_modulo]
1013
1014         lista_lineas_flag_variable_publica = sorted(lista_lineas_flag_variable_publica, key = lambda x: (x[0]))
1015
1016         lista_variables_publicas_1_a_1 = func_access_bbdd_codigo_calculos_varios("LISTA_VARIABLES_PUBLICAS_DEFINIDAS_POR_USUARIO"
1017                                     , lista_variable_definida_por_usuario_ini = lista_variable_definida_por_usuario_ini
1018                                     , lista_lineas_flag_variable_publica = lista_lineas_flag_variable_publica)
```

flag_variable_publica_modulo DEFINIDA_POR_USUARIO_1
lista_lineas_flag_variable_publica [[20, 'Type UserType_Prueba_1'], [21, 'SubVar_1 As String'], [22, 'SubVar_2 As Long'], [23, 'End Type']]
lista_variables_publicas_1_a_1 [['UserType_Prueba_1', 'VARIABLE PUBLICA DEFINIDA POR USUARIO', 'Type', '-', None, 'SubVar_1 (String), SubVar_2 (Long)']]

flag_variable_publica_modulo DEFINIDA_POR_USUARIO_2
lista_lineas_flag_variable_publica [[25, 'Enum Enum_Prueba_1'], [26, 'SubVar_1'], [27, 'SubVar_2 = 1'], [28, 'SubVar_3'], [29, 'End Enum']]
lista_variables_publicas_1_a_1 [['Enum_Prueba_1', 'VARIABLE PUBLICA DEFINIDA POR USUARIO', 'Enum', '-', None, 'SubVar_1, SubVar_2 (1), SubVar_3']]

8

```
1022     for item_variable_publica in lista_variables_publicas_1_a_1:
1023
1024         variable_publica_nombre = item_variable_publica[0]
1025         variable_publica_tipo = item_variable_publica[1]
1026         variable_publica_tipo_dato = item_variable_publica[2]
1027         variable_publica_es_constante = item_variable_publica[3]
1028         variable_publica_es_constante_valor = item_variable_publica[4]
1029         variable_publica_definida_usuario_subvariables = item_variable_publica[5]
1030
1031         dicc_temp = {"TIPO_OBJETO": "VARIABLES_VBA"
1032                   , "TIPO_MODULO": tipo_modulo_iteracion
1033                   , "NOMBRE_MODULO": nombre_modulo_iteracion
1034                   , "NOMBRE_OBJETO": variable_publica_nombre
1035                   , "TIPO Rutina": None
1036                   , "TIPO_DECLARACION_Rutina": None
1037                   , "PARAMETROS_Rutina": None
1038                   , "TIPO_VARIABLE_PUBLICA": variable_publica_tipo
1039                   , "TIPO_DATO_VARIABLE_PUBLICA": variable_publica_tipo_dato
1040                   , "VARIABLE_PUBLICA_ES_CONSTANTE": variable_publica_es_constante
1041                   , "VARIABLE_PUBLICA_CONSTANTE_VALOR": variable_publica_es_constante_valor
1042                   , "VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES": variable_publica_definida_usuario_subvariables
1043                   , "DF_CODIGO": None
1044                   , "CONNECTING_STRING_VINCULOS": None
1045                   }
1046
1047         lista_resultado_funcion.append(dicc_temp)
```

ANEXO_08

FUNCIÓN: func_access_df_bbdd_codigo_calculos

3

opcion = LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS

8

```
flag_variable_publica_modulo      DEFINIDA_POR_USUARIO_1
lista_lineas_flag_variable_publica [[20, 'Type UserType_Prueba_1'], [21, 'SubVar_1 As String'], [22, 'SubVar_2 As Long'], [23, 'End Type']]
lista_variables_publicas_1_a_1    [['UserType_Prueba_1', 'VARIABLE PUBLICA DEFINIDA POR USUARIO', 'Type', '-', None, 'SubVar_1 (String), SubVar_2 (Long)']]
```

Valores KEYS diccionario - ITERACIÓN 1

```
['UserType_Prueba_1', 'VARIABLE PUBLICA DEFINIDA POR USUARIO', 'Type', '-', None, 'SubVar_1 (String), SubVar_2 (Long)']
TIPO_OBJETO      VARIABLES_VBA
TIPO_MODULO      Estandar
NOMBRE_MODULO    Modulo_PRUEBA_1
NOMBRE_OBJETO    UserType_Prueba_1
TIPO Rutina      None
TIPO_DECLARACION_RUTINA  None
PARAMETROS_RUTINA      None
TIPO_VARIABLE_PUBLICA  VARIABLE PUBLICA DEFINIDA POR USUARIO
TIPO_DATO_VARIABLE_PUBLICA  Type
VARIABLE_PUBLICA_ES_CONSTANTE -
VARIABLE_PUBLICA_CONSTANTE_VALOR  None
VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES  SubVar_1 (String), SubVar_2 (Long)
DF_CODIGO      None
CONNECTING_STRING_VINCULOS  None
```

```
flag_variable_publica_modulo      DEFINIDA_POR_USUARIO_2
lista_lineas_flag_variable_publica [[25, 'Enum Enum_Prueba_1'], [26, 'SubVar_1'], [27, 'SubVar_2 = 1'], [28, 'SubVar_3'], [29, 'End Enum']]
lista_variables_publicas_1_a_1    [['Enum_Prueba_1', 'VARIABLE PUBLICA DEFINIDA POR USUARIO', 'Enum', '-', None, 'SubVar_1, SubVar_2 (1), SubVar_3']]
```

Valores KEYS diccionario - ITERACIÓN 1

```
['Enum_Prueba_1', 'VARIABLE PUBLICA DEFINIDA POR USUARIO', 'Enum', '-', None, 'SubVar_1, SubVar_2 (1), SubVar_3']
TIPO_OBJETO      VARIABLES_VBA
TIPO_MODULO      Estandar
NOMBRE_MODULO    Modulo_PRUEBA_1
NOMBRE_OBJETO    Enum_Prueba_1
TIPO Rutina      None
TIPO_DECLARACION_RUTINA  None
PARAMETROS_RUTINA      None
TIPO_VARIABLE_PUBLICA  VARIABLE PUBLICA DEFINIDA POR USUARIO
TIPO_DATO_VARIABLE_PUBLICA  Enum
VARIABLE_PUBLICA_ES_CONSTANTE -
TIPO_VARIABLE_PUBLICA  VARIABLE PUBLICA DEFINIDA POR USUARIO
TIPO_DATO_VARIABLE_PUBLICA  Enum
VARIABLE_PUBLICA_ES_CONSTANTE -
VARIABLE_PUBLICA_ES_CONSTANTE -
VARIABLE_PUBLICA_CONSTANTE_VALOR  None
VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO_SUB_VARIABLES  SubVar_1, SubVar_2 (1), SubVar_3
DF_CODIGO      None
CONNECTING_STRING_VINCULOS  None
```

ANEXO_09

FUNCIÓN

func_access_bbdd_codigo_calculos_varios

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

La función **func_access_bbdd_codigo_calculos_varios** permite realizar cálculos varios relacionados con tratamiento de string mediante la librería **re** y sobretodo de tratamiento de variables públicas VBA y variables locales de rutinas / funciones VBA. Estos cálculos complementan en gran medida la función **func_access_bbdd_codigo_calculos** (ver **ANEXO_08**) pero se han encapsulado en una función aparte para aligerar el código de la 1era porque los cálculos requieren muchos pasos intermedios.

Permite 8 tipos de cálculos, según el parámetro **opcion** seleccionado:

BLOQUE	OPCION	DESCRIPCIÓN
1	QUITAR_ENCAPSULADO_ENTRE_COMILLAS_DOBLAS	Crea un string donde se quita todo lo encapsulado entre comillas dobles dentro de un string
2	QUITAR_ENCAPSULADO_ENTRE_PARENTESIS	Crea un string donde se quita todo lo encapsulado entre parentesis dentro de un string
3	LISTA_DECLARACIONES_VARIABLES_LOCALES	Crea una lista que contiene las lineas de código asociadas a declaraciones de variables locales de rutinas / funciones VBA
4	LISTA_DECLARACIONES_VARIABLES_PUBLICAS_NORMALES	Crea una lista de lista de variables públicas normales donde cada sublista (*) contiene: --> tipo de módulo --> nombre de módulo --> indice de linea dentro del dataframe de código --> indicador de si es constante o no --> la linea de código VBA
5	LISTA_DECLARACIONES_VARIABLES_PUBLICAS_DEFINIDAS_USUARIO	Crea una lista de lista de variables públicas definidas por el usuario donde cada sublista (*) contiene: --> tipo de módulo --> nombre de módulo --> indice de linea dentro del dataframe de código --> flag identificativo (que aplica a todas las lineas de código correspondientes a la declaración de la misma variable) --> la linea de código VBA
6	LISTA_VARIABLES_PUBLICAS_NO_ES_CONSTANTE	A partir de una linea de código, lista las variables públicas normales que NO son constantes (las que vienen seguidas de " As "). En una misma linea se pueden declara varias variables públicas, la opcion de la función genera una lista de listas donde cada sublista contiene los siguientes items (**): --> nombre variable pública --> tipo variable pública --> tipo dato variable pública --> es constante --> valor de la constante --> subvariables de variable definidas por el usuario
7	LISTA_VARIABLES_PUBLICAS_SI_ES_CONSTANTE	Es lo mismo (**) que la opción LISTA_VARIABLES_PUBLICAS_NO_ES_CONSTANTE pero aplicado a variables públicas que SI son constantes
8	LISTA_VARIABLES_PUBLICAS_DEFINIDAS_POR_USUARIO	Unifica todas las lineas de código asociadas a una variable pública definida por el usuario (se declaran en varias lineas) y prepara una lista de lista en el mismo formato que las variables publicas normales (las que se generan con la opciones LISTA_VARIABLES_PUBLICAS_NO_ES_CONSTANTE y LISTA_VARIABLES_PUBLICAS_SI_ES_CONSTANTE)

(*): la estructura de las sublistas es la misma para todos los tipos de variables públicas sean normales o definidas por el usuario
(**): la estructura de las sublistas es la misma para todos los tipos de variables públicas sean normales o definidas por el usuario

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

La función también tiene **parámetros kwargs** que aplican o no según el parámetro fijo seleccionado. Antes de pasar a explicar cada una de las opciones de la función, se especifican cuales son estos parámetros kwargs:

OPCION	PARAMETROS KWARGS						
	string_param	indice_param_ini	indice_param_fin	df_bbdd_codigo_param	linea_codigo	lista_variable_definida_por_usuario_ini	lista_lineas_flag_variable_publica
QUITAR_ENCAPSULADO_ENTRE_COMILLAS_DOBLAS	X						
QUITAR_ENCAPSULADO_ENTRE_PARENTESIS	X						
LISTA_DECLARACIONES_VARIABLES_LOCALES		X	X	X			
LISTA_DECLARACIONES_VARIABLES_PUBLICAS_NORMALES		X	X	X			
LISTA_DECLARACIONES_VARIABLES_PUBLICAS_DEFINIDAS_USUARIO		X	X	X			
LISTA_VARIABLES_PUBLICAS_NO_ES_CONSTANTE					X		
LISTA_VARIABLES_PUBLICAS_SI_ES_CONSTANTE					X		
LISTA_VARIABLES_PUBLICAS_DEFINIDAS_POR_USUARIO					X	X	X

PARAMETRO	DESCRIPCIÓN
string_param	String que contiene el código VBA
indice_param_ini	Indice inicial de la línea de código en el dataframe de código VBA (parámetro df_bbdd_codigo_param), corresponde a la columna INDICE. --> VARIABLES LOCALES RUTINA: es el indice de la línea de código correspondiente a la declaración de la rutina (las líneas que empiezan por "Sub " o " Function") --> VARIABLES PÚBLICAS NORMALES: es el indice de la línea de código correspondiente a la declaración del módulo VBA donde se ubica --> VARIABLES PÚBLICAS DEFINIDAS POR USUARIO: es el indice de la línea de código correspondiente a la declaración del módulo VBA donde se ubica
indice_param_fin	Indice final de la línea de código en el dataframe de código VBA (parámetro df_bbdd_codigo_param), corresponde a la columna INDICE. --> VARIABLES LOCALES RUTINA: es el indice de la línea de código correspondiente a la declaración de fin de la rutina (las líneas que empiezan por "End ") --> VARIABLES PÚBLICAS NORMALES: es el indice de la línea de código correspondiente a la últimas línea de código del módulo VBA donde se ubica --> VARIABLES PÚBLICAS DEFINIDAS POR USUARIO: es el indice de la línea de código correspondiente a la últimas línea de código del módulo VBA donde se ubica
df_bbdd_codigo_param	Es el dataframe de código VBA que se calcula en la rutina def_proceso_access_1_import
linea_codigo	Es una línea de código VBA
lista_variable_definida_por_usuario_ini	Es la lista almacenada en la subkey_2 (LISTA_DECLARACION_INICIO) asociada a la key_1 (VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS) solo que a cada item se trime y se le agrega un espacio en blanco al final (esto se hace por si el usuario configura mal el diccionario mencionado)
lista_lineas_flag_variable_publica	Es la lista de líneas de código asociadas a una misma variable pública definida por el usuario (este tipo de variables se definen en varias líneas). Es lista de lista donde cada sublista contien el indice de la línea de código en el dataframe de código y la línea de código asociada

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

1

opcion = QUITAR_ENCAPSULADO_ENTRE_COMILLAS_DOBLAS

Esta opción permite, mediante la librería **re**, quitar dentro de un string todos los substring encapsulados entre comillas dobles "".

```
1114     if opcion == "QUITAR_ENCAPSULADO_ENTRE_COMILLAS_DOBLAS":
1115         #parametros kwargs usados --> string_param
1116
1117         resultado_funcion = re.compile(r'"([^\"]*)"').sub(' ', string_param)
```

```
& "WHERE ((PROVINCIA = " & xProvincia_Sede & ") AND " & vbCrLf _
```



```
& " " & xProvincia_Sede & " " & vbCrLf _
```

2

opcion = QUITAR_ENCAPSULADO_ENTRE_PARENTESIS

Esta opción permite quitar dentro de un string todos los substring encapsulados entre paréntesis "(" y ")". No se hace con el método **compile** de la librería **re** como en la opción anterior porque no dio los resultados esperados.

Se calcula de forma más manual, usando también la librería **re**.

Se calculan 2 listas temporales que almacenan los índices de posición de los paréntesis dentro del string de apertura "(" y de cierre ")" (**lista_indices_parentesis_apertura** y **lista_indices_parentesis_cierre** respectivamente).

```
1128     lista_indices_parentesis_apertura = list(re.finditer(r"\(", string_param))
1129     lista_indices_parentesis_cierre = list(re.finditer(r"\)", string_param))
1130
1131     lista_indices_parentesis_apertura = [lista_indices_parentesis_apertura[ind].start() for ind, item in enumerate(lista_indices_parentesis_apertura)]
1132     lista_indices_parentesis_cierre = [lista_indices_parentesis_cierre[ind].start() for ind, item in enumerate(lista_indices_parentesis_cierre)]
```

2

opcion = QUITAR ENCAPSULADO ENTRE PARENTESIS

Se crea otra lista (**lista_indices_parentesis_apertura_reversed**) que revierte los ítems de la lista que contiene los paréntesis de apertura.

```
1135 lista_indices_parentesis_apertura_reversed = lista_indices_parentesis_apertura.copy()
1136 lista_indices_parentesis_apertura_reversed.reverse()
```

Mediante bucle sobre los ítems de esta lista **lista_indices_parentesis_apertura_reversed**, se busca el índice de posición del paréntesis de cierre inmediatamente consecutivo en **lista_indices_parentesis_cierre**, pero tiene que ser un índice de posición que no se haya asociado en una iteración anterior a otro paréntesis de apertura. Para ello, se crea antes de iniciar el bucle la lista **lista_exclusion_parentesis_cierre** donde se almacenan en cada iteración del bucle los índices de posición de los paréntesis de cierre localizados.

En cada iteración se crea una tupla que contiene los índices de posición de apertura y cierre y la longitud del substring intercalado entre los 2 (es la diferencia entre los 2 índices de posición) y se agrega a la lista **lista_tuplas_parentesis_replace**.

```
1143 lista_tuplas_parentesis_replace = []
1144 lista_exclusion_parentesis_cierre = []
1145 for item_apertura in lista_indices_parentesis_apertura_reversed:
1146
1147     try:
1148         item_cierre_consecutivo = [item_cierre for item_cierre in lista_indices_parentesis_cierre if item_cierre > item_apertura and item_cierre not in lista_exclusion_parentesis_cierre][0]
1149         lista_exclusion_parentesis_cierre.append(item_cierre_consecutivo)
1150
1151         tupla = (item_apertura, item_cierre_consecutivo, item_cierre_consecutivo - item_apertura)
1152         lista_tuplas_parentesis_replace.append(tupla)
1153
1154     except:#por si el numero de parentesis de apertura y cierre no coincide
1155         pass
```

Cada iteración se encapsula en un bloque **try except** por si el número de paréntesis de apertura en el string difiere del número de los paréntesis de cierre. VBA avisa de ello, que es erróneo, marcando la línea de código en **ROJO** pero aun así deja guardar y cerrar la base de datos. Se establece aquí por si acaso.

2

opcion = QUITAR ENCAPSULADO ENTRE PARENTESIS

Se re-ordena la lista **lista_tuplas_parenthesis_replace** por el 3er item de las tuplas que la componen (la longitud de los substring encapsulados entre paréntesis) de mayor a menor y se realiza el replace en el string mediante bucle sobre esta lista re-ordenada para quitar todo lo incluido entre paréntesis.

```

1158      #se ordena la lista lista_tuplas_parenthesis_replace por longitud del substring (item 2 tupla - item 1 tupla) de mayor a menor
1159      lista_tuplas_parenthesis_replace = sorted(lista_tuplas_parenthesis_replace, key = lambda x: -x[2])
1160
1161      #se hace el replace en el string para quitar lo encapsulado entre parentesis
1162      for tupla in lista_tuplas_parenthesis_replace:
1163          item_replace = "(" + string_param[tupla[0] + 1:tupla[1]] + ")"
1164          string_param = string_param.replace(item_replace, "")
1165
1166
1167      resultado_funcion = string_param

```

If (Asc(xTemp) >= 65) And (Asc(xTemp) <= 90) _
 
 If And _

3

opcion = LISTA DECLARACIONES VARIABLES LOCALES

Permite localizar **dentro de una rutina / función VBA** las líneas de código que corresponden con declaraciones de variables locales (las que empiezan por "Dim " o "Const " y que pueden declararse en varias líneas truncadas al poner un "_" al final de cada línea).

Se distinguen 4 casos para localizar estas líneas de código:

CASO	DESCRIPCIÓN
1	Son las líneas que <u>si</u> empiezan por "Dim " o "Const " y <u>no</u> acaban por "_"
2	Son las líneas que <u>si</u> empiezan por "Dim " o "Const " y <u>si</u> acaban por "_"
3	Son las líneas que <u>no</u> empiezan por "Dim " o "Const " y <u>si</u> acaban por "_"
4	Son las líneas que <u>no</u> empiezan por "Dim " o "Const " y no acaban por "_"

CASO 1**CASO 2****CASO 3****CASO 4**

```

53 Sub Calc_PRUEBA()
54
55 Dim MiConexion As New ADODB.Connection
56
57 Dim MySQL1 As String, MySQL2 As String, _
58 MySQL3 As String, MySQL4 As String
59
60 Dim MySQL5 As String, MySQL6 As String, _
61 MySQL7 As String, MySQL8 As String, _
62 MySQL9 As String, MySQL10 As String
63
64 Const Const_1 = 1, Const_2 = "hola"
65
66 Const Const_3 = 1, Const_4 = "que tal", _
67 Const_5 = 2, Const_6 = "???"
68
69 Const Const_7 = 1, Const_8 = "Estoy harto", _
70 Const_9 = 2, Const_10 = "de este", _
71 Const_11 = 88, Const_12 = "diseño", _
72 Const_13 = 75, Const_14 = "funcional"
73
74 MySQL = "UPDATE [T_00_PRUEBA_01] SET CAMPO_ID = '" & Const_2 & "'" & vbCrLf _
75         & "WHERE CAMPO_DESC = '" & Const_1 & "'"
76
77 MiConexion.Errors (MySQL1)
78
79 DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
80
81 End Sub

```

3

opcion = LISTA DECLARACIONES VARIABLES LOCALES

El resultado de la función es aquí una lista donde cada item es una línea de código de declaración de variable local. Se ejecuta dentro del ámbito de una rutina / función VBA cuando se utiliza la función en la función **func_access_bbdd_codigo_calculos** (ver **ANEXO_08**).

Los parámetros kwargs son:

- **indice_param_ini**
- **indice_param_fin**
- **df_bbdd_codigo_param**

Se han de realizar **ajustes previos** al cálculo de la lista de líneas de código resultado de la función.

AJUSTE	DESCRIPCIÓN
A1	Se fusionan todas las listas almacenadas en las subkeys_2 (LISTA_DECLARACION_INICIO_NO_ES_CONSTANTE y LISTA_DECLARACION_INICIO_SI_ES_CONSTANTE) asociadas a la key:1 (RUTINAS_VARIABLES_LOCALES) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS) y en cada item de la lista se trime y agrega un espacio en blanco al final (por si el usuario ha configurado mal el diccionario. Se convierte la lista en tupla para poder usar el metodo startswith en el filtrado de dataframes por selecciones multiples.
A2	Se listan las lineas de código correspondientes a los 4 tipos de casos mencionados en la página anterior. Se realiza mediante extracciones de dataframes con los requisitos de cada caso sobre el dataframe df_bbdd_codigo_param (parámetro de la función) en los cuales se conserva el indice de la linea de código y la linea de código sin tabulaciones, trimeada y sin comentarios. Se crean con estos dataframes extraidos 4 listas de listas y se ordena cada una por el indice de la linea de código (item 1 de cada una de las sublistas que las componen). --> lista_indices_sino (corresponde al CASO 1 mencionado) --> lista_indices_sisi (corresponde al CASO 2 mencionado) --> lista_indices_nosi (corresponde al CASO 3 mencionado) --> lista_indices_nono (corresponde al CASO 4 mencionado)

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

3

opcion = LISTA DECLARACIONES VARIABLES LOCALES

A1

```
1200 #para variables locales de rutinas se fusionan las listas de inicios de declaracion configurados en el diccionario dicc_objetos_vba
1201 #y se convierte en tupla (si no el metodo startswith no va en busquedas multiples), cada item se trimea y se agrega un espacio en blanco al final
1202 lista_temp = dicc_objetos_vba["RUTINAS_VARIABLES_LOCALES"]["LISTA_DECLARACION_INICIO_NO_ES_CONSTANTE"] + dicc_objetos_vba["RUTINAS_VARIABLES_LOCALES"]["LISTA_DECLARACION_INICIO_SI_ES_CONSTANTE"]
1203 tupla_declaracion_ini_variable_local_rutina = tuple([item.strip() + " " for item in lista_temp])
```



('Dim ', 'Const ')

A2

```
1215 df_sino = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["INDICE"] > indice_param_ini) & (df_bbdd_codigo_param["INDICE"] < indice_param_fin) &
1216 (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_ini_variable_local_rutina)) &
1217 (~df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.endswith(indicador_linea_codigo_vba_truncada_a_linea_siguiente)) &
1218 (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
1219 ["INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
1220
1221 df_sisi = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["INDICE"] > indice_param_ini) & (df_bbdd_codigo_param["INDICE"] < indice_param_fin) &
1222 (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_ini_variable_local_rutina)) &
1223 (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.endswith(indicador_linea_codigo_vba_truncada_a_linea_siguiente)) &
1224 (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
1225 ["INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
1226
1227 df_nosi = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["INDICE"] > indice_param_ini) & (df_bbdd_codigo_param["INDICE"] < indice_param_fin) &
1228 (~df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_ini_variable_local_rutina)) &
1229 (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.endswith(indicador_linea_codigo_vba_truncada_a_linea_siguiente)) &
1230 (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
1231 ["INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
1232
1233 df_nono = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["INDICE"] > indice_param_ini) & (df_bbdd_codigo_param["INDICE"] < indice_param_fin) &
1234 (~df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_ini_variable_local_rutina)) &
1235 (~df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.endswith(indicador_linea_codigo_vba_truncada_a_linea_siguiente)) &
1236 (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
1237 ["INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
```

Los filtros por índices de líneas de código son estrictamente superiores e inferiores. Como el cálculo es a nivel de rutina / función se excluyen de antemano los inicios y finales de declaración de rutinas / funciones.

En los dataframes **df_nosi** y **df_nono**, en estos ajustes previos, pueden salir líneas que no son de declaración de variables locales. Cuando se realiza el cálculo, más adelante, estas líneas se descartan.

3

opcion = LISTA DECLARACIONES VARIABLES LOCALES

A2

```
df_sino
-----
INDICE CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0      53 Dim MiConexion As New ADODB.Connection
1      62 Const Const_1 = 1, Const_2 = "hola"

df_sisi
-----
INDICE CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0      55 Dim MiSQL1 As String, MiSQL2 As String, _
1      58 Dim MiSQL5 As String, MiSQL6 As String, _
2      64 Const Const_3 = 1, Const_4 = "que tal", _
3      67 Const Const_7 = 1, Const_8 = "Estoy harto", _
```

```
df_nosi
-----
INDICE CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0      59 MiSQL7 As String, MiSQL8 As String, _
1      68 Const_9 = 2, Const_10 = "de este", _
2      60 Const_11 = 88, Const_12 = "disco", _
3      72 MiSQL = "UPDATE [T_00_PRUEBA_01] SET CAMPO_ID ..."

df_nono
-----
INDICE CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0      56 MiSQL3 As String, MiSQL4 As String
1      60 MiSQL9 As String, MiSQL10 As String
2      65 Const_5 = 2, Const_6 = "???"
3      70 Const_13 = 75, Const_14 = "funcional"
4      73 & "WHERE CAMPO_DESC = '" & Const_1 & "'"
5      75 MiConexion.Errors (MiSQL1)
6      77 DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acV...
```

Líneas que no afectan

```
1245 lista_indices_sino = [] #las completion list no van cuando el df esta vacio por lo que se cre la lista en varias etapas
1246 lista_indices_sisi = [] #idem
1247 lista_indices_nosi = [] #idem
1248 lista_indices_nono = [] #idem
1249
1250 if len(df_sino) != 0:
1251     lista_indices_sino = [[df_sino.iloc[ind, df_sino.columns.get_loc("INDICE")],
1252                           df_sino.iloc[ind, df_sino.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]] for ind in df_sino.index]
1253
1254 if len(df_sisi) != 0:
1255     lista_indices_sisi = [[df_sisi.iloc[ind, df_sisi.columns.get_loc("INDICE")],
1256                           df_sisi.iloc[ind, df_sisi.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]] for ind in df_sisi.index]
1257
1258 if len(df_nosi) != 0:
1259     lista_indices_nosi = [[df_nosi.iloc[ind, df_nosi.columns.get_loc("INDICE")],
1260                           df_nosi.iloc[ind, df_nosi.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]] for ind in df_nosi.index]
1261
1262 if len(df_nono) != 0:
1263     lista_indices_nono = [[df_nono.iloc[ind, df_nono.columns.get_loc("INDICE")],
1264                           df_nono.iloc[ind, df_nono.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]] for ind in df_nono.index]
```


ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

3

opcion = LISTA DECLARACIONES VARIABLES LOCALES

A2

```
1272 #se re-ordenan las lista por el indice de codigo
1273 lista_indices_sino = sorted(lista_indices_sino, key = lambda x: x[0])
1274 lista_indices_sisi = sorted(lista_indices_sisi, key = lambda x: x[0])
1275 lista_indices_nosi = sorted(lista_indices_nosi, key = lambda x: x[0])
1276 lista_indices_nono = sorted(lista_indices_nono, key = lambda x: x[0])
```



```
lista_indices_sino
-----
[[53, 'Dim MiConexion As New ADODB.Connection'],
[62, 'Const Const_1 = 1, Const_2 = "hola"']]

lista_indices_sisi
-----
[[55, 'Dim MiSQL1 As String, MiSQL2 As String, _'],
[58, 'Dim MiSQL5 As String, MiSQL6 As String, _'],
[64, 'Const Const_3 = 1, Const_4 = "que tal", _'],
[67, 'Const Const_7 = 1, Const_8 = "Estoy harto", _']]

lista_indices_nosi
-----
[[59, 'MiSQL7 As String, MiSQL8 As String, _'],
[68, 'Const_9 = 2, Const_10 = "de este", _'],
[69, 'Const_11 = 88, Const_12 = "diseño", _'],
[72, 'MiSQL = "UPDATE [T_00_PRUEBA_01] SET CAMPO_ID = \' & Const_2 & "\' & vbCrLf _']]

lista_indices_nono
-----
[[56, 'MiSQL3 As String, MiSQL4 As String'],
[60, 'MiSQL9 As String, MiSQL10 As String'],
[65, 'Const_5 = 2, Const_6 = "???"'],
[70, 'Const_13 = 75, Const_14 = "funcional"'],
[73, '& "WHERE CAMPO_DESC = \' & Const_1 & "\''],
[75, 'MiConexion.Errors (MiSQL1)'],
[77, 'DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal']]
```

Items que no afectan

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

3

opcion = LISTA DECLARACIONES VARIABLES LOCALES

Tras realizar los ajustes previos, se inicia el cálculo de la lista **resultado_funcion** con las líneas de código que son declaraciones de variables locales de rutina / función.

Las líneas de código incluidas en la lista **lista_indices_sino** se agregan directamente a la lista **resultado_funcion**.

```
1284         #se agregan directamente los items de lista_indices_sino
1285         #donde se calcula previamente si declaracion de constante o no
1286         for indice_sino, linea_codigo_sino in lista_indices_sino:
1287             resultado_funcion.append(linea_codigo_sino)
```

Las líneas de código incluidas en la lista **lista_indices_sisi** se agregan directamente a la lista **resultado_funcion** (se quita el "_" al final de la línea al agregarse) mediante bucle (en adelante bucle_1) sobre los items de la misma (índice y línea de código, en adelante índice_1 y línea_1 respectivamente).

```
1296         for indice_sisi, linea_codigo_sisi in lista_indices_sisi:
1297
1298             #se agregan los datos a la lista resultado de la funcion (l
1299             #siempre y cuando el resultado del neteo no sea de longitud
1300             if len(linea_codigo_sisi[:-2]) != 0:
1301                 resultado_funcion.append(linea_codigo_sisi[:-2])
```

En cada iteración sobre bucle_1, se inicia un bucle while (en adelante bucle_while), con incrementos de índices de línea + 1:

- a partir del índice de línea inmediatamente consecutivo (en adelante índice_while) a índice_1.
- hasta al 1er índice de línea de la lista **lista_indices_nono** (en adelante índice_fin_bucle_while) inmediatamente consecutivo a índice_1.

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

3

opcion = LISTA DECLARACIONES VARIABLES LOCALES

Se agregan las líneas de código de la lista **lista_indices_nosi** si sus índices de línea son iguales a **indice_while** (se quita el " _" al final de la línea al agregarse).

Al llegar a la última iteración sobre **bucle_while**, si **indice_nono_fin_bucle_while** es igual a **indice_while** se agrega la línea de código de la lista **lista_indices_nono** asociada a **indice_nono_fin_bucle_while**.

```
1306 lista_indices_nosi_iteracion = [[indice_nosi, linea_codigo_nosi] for indice_nosi, linea_codigo_nosi in lista_indices_nosi if indice_nosi > indice_sisi]
1307 lista_indice_nono_despues_iteracion = [[indice_nono, linea_codigo_nono] for indice_nono, linea_codigo_nono in lista_indices_nono if indice_nono > indice_sisi]
1308
1309 primer_indice_nono_despues_iteracion = lista_indice_nono_despues_iteracion[0][0]
1310 linea_codigo_primer_indice_nono_despues_iteracion = lista_indice_nono_despues_iteracion[0][1]
1311
1312 indice_linea_siguiente = indice_sisi + 1
1313 while indice_linea_siguiente <= primer_indice_nono_despues_iteracion:
1314
1315     for indice_nosi, linea_codigo_nosi in lista_indices_nosi_iteracion:
1316
1317         if indice_linea_siguiente == indice_nosi:
1318
1319             #se agregan los datos a la lista resultado de la funcion (la linea de codigo se neteea quitando el " _" final
1320             #siempre y cuando el resultado del neteo no sea de longitud = 0)
1321             if len(linea_codigo_nosi[:-2]) != 0:
1322                 resultado_funcion.append(linea_codigo_nosi[:-2])
1323
1324             break
1325
1326 if indice_linea_siguiente == primer_indice_nono_despues_iteracion:
1327
1328     #se agregan los datos a la lista resultado de la funcion
1329     resultado_funcion.append(linea_codigo_primer_indice_nono_despues_iteracion)
1330     break
1331
1332 indice_linea_siguiente = indice_linea_siguiente + 1
```

3

opcion = LISTA DECLARACIONES VARIABLES LOCALES

```

53 Sub Calc_PRUEBA()
54
55 Dim MiConexion As New ADODB.Connection
56
57 Dim MiSQL1 As String, MiSQL2 As String, _
58 MiSQL3 As String, MiSQL4 As String
59
60 Dim MiSQL5 As String, MiSQL6 As String, _
61 MiSQL7 As String, MiSQL8 As String, _
62 MiSQL9 As String, MiSQL10 As String
63
64 Const Const_1 = 1, Const_2 = "hola"
65
66 Const Const_3 = 1, Const_4 = "que tal", _
67 Const_5 = 2, Const_6 = "???"
68
69 Const Const_7 = 1, Const_8 = "Estoy harto", _
70 Const_9 = 2, Const_10 = "de este", _
71 Const_11 = 88, Const_12 = "diseño", _
72 Const_13 = 75, Const_14 = "funcional"
73
74 MiSQL = "UPDATE [T_00_PRUEBA_01] SET CAMPO_ID = '" & Const_2 & "'" & vbCrLf _
75         & "WHERE CAMPO_DESC = '" & Const_1 & "'"
76
77 MiConexion.Errors (MiSQL1)
78
79 DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
80
81 End Sub

```



resultado_funcion

```

-----
['Dim MiConexion As New ADODB.Connection',
'Const Const_1 = 1, Const_2 = "hola"',
'Dim MiSQL1 As String, MiSQL2 As String,',
'MiSQL3 As String, MiSQL4 As String',
'Dim MiSQL5 As String, MiSQL6 As String,',
'MiSQL7 As String, MiSQL8 As String,',
'MiSQL9 As String, MiSQL10 As String',
'Const Const_3 = 1, Const_4 = "que tal",',
'Const_5 = 2, Const_6 = "???"',
'Const Const_7 = 1, Const_8 = "Estoy harto",',
'Const_9 = 2, Const_10 = "de este",',
'Const_11 = 88, Const_12 = "diseño",',
'Const_13 = 75, Const_14 = "funcional"']

```

4

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS NORMALES

Permite localizar **dentro de un módulo VBA** las líneas de código que corresponden con declaraciones de variables públicas normales (sean constantes o no), es decir las que no son definidas por el usuario (se calculan con otra opción).

Son las líneas que, fuera de rutinas / funciones (llegados a este paso las rutinas / funciones ya están calculadas), empiezan por “Public”, “Global”, “Public Const” etc etc y que pueden declararse en varias líneas truncadas al poner un “_” al final de cada línea).

El proceso de cálculo es muy similar al de la opción LISTA_DECLARACIONES_VARIABLES_LOCALES de la presente función, pero con pequeños matices.

El resultado de la función es aquí una lista de listas donde cada sublista que la compone contiene:

ITEMS SUBLISTAS
Tipo de módulo
Nombre de módulo
Indice de la línea de código dentro del dataframe df_bbdd_codigo_param (parámetro de la función)
Indicador de si la línea de código es de declaración de constante o no
Línea de código

Se ejecuta dentro del ámbito de un módulo VBA cuando se utiliza la función en la función **func_access_bbdd_codigo_calculos** (ver **ANEXO_08**).

Requiere de parámetros kwargs que son:

- **indice_param_ini**
- **indice_param_fin**
- **df_bbdd_codigo_param**

4

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS NORMALES

Se distinguen 4 casos para localizar estas líneas de código:

CASO	DESCRIPCIÓN
1	Son las líneas que <u>si</u> empiezan por "Public", "Global", "Public Const", "Global Const" etc etc y <u>no</u> acaban por "_"
2	Son las líneas que <u>si</u> empiezan por "Public", "Global", "Public Const", "Global Const" etc etc y <u>si</u> acaban por "_"
3	Son las líneas que <u>no</u> empiezan por "Public", "Global", "Public Const", "Global Const" etc etc y <u>si</u> acaban por "_"
4	Son las líneas que <u>no</u> empiezan por "Public", "Global", "Public Const", "Global Const" etc etc y no acaban por "_"

CASO 1**CASO 2****CASO 3****CASO 4**

```

00 Option Compare Database
01 Option Explicit
02 Option Base 1
03
04 'VARIABLES PUBLICAS
05 Public Public_Version As String
06
07 Global Public_Var_1 As String, Public_Var_2 As String, _
08 Public_Array(1 To 10, 2) As Variant, Public_Var_3 As Date, _
09 Public_Var_4 As Variant
10
11 Public Const Public_Const_1 = "aquí hay comas , y signos = "
12
13 Global Const Public_Const_2 = "@1", _
14 Public_Const_3 = 8, _
15 Public_Const_4 = "@2", _
16 Public_Const_5 = 5
17
18 Type UserType_Prueba_1 'variable publica definida por el usuario de prueba
19     SubVar_1 As String
20     SubVar_2 As Long
21 End Type
22
23 Enum Enum_Prueba_1
24     SubVar_1 'sub-variable de otra variable publica definida por el usuario de prueba
25     SubVar_2 = 1
26     SubVar_3
27 End Enum
28
29
30 Sub ActivarPanelNavegacion()
31
32 On Error Resume Next
33
34 DoCmd.LockNavigationPane (False)
35 DoCmd.SelectObject acTable, "MSysObjects", True
36 DoCmd.RunCommand acCmdWindowUnhide
37 MsgBox "fsdfsd"
38
39 End Sub

```

4

AJUSTE	DESCRIPCIÓN
A1	Se fusionan todas las listas almacenadas en las subkeys_2 (las que empiezan por LISTA_DECLARACION_INICIO) asociadas a la key:1 (VARIABLES_PUBLICAS_NORMALES) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS) y la lista resultante contiene sublistas en las cuales se almacenan: --> indicador de si es constante o no (NO_ES_CONSTANTE o SI_ES_CONSTANTE) --> el inicio de declaración de este tipo de variable, trimeado y con un espacio en blanco al final (por si el usuario ha configurado mal el diccionario) La lista resultante se ordena por la longitud del ítem 2 de cada una de sus sublistas de mayor a menor
A2	Se convierte la lista del ajuste A1 (se recupera solo ítem 2 de cada una de sus sublistas) en tupla para poder usar el método startswith en el filtrado de dataframes por selecciones múltiples.
A3	Se listan las líneas de código correspondientes a los 4 tipos de casos mencionados en la página anterior. Se realiza mediante extracciones de dataframes con los requisitos de cada caso sobre el dataframe df_bbdd_codigo_param (parámetro de la función) en los cuales se conserva el tipo y nombre del módulo, el índice de la línea de código y la línea de código sin tabulaciones, trimeada y sin comentarios. Se crean con estos dataframes extraídos 4 listas de listas y se ordena cada una por el tipo y nombre del módulo y por el índice de la línea de código (ítem 1, 2 y 3 de cada una de las sublistas que las componen). --> lista_indices_sino (corresponde al CASO 1 mencionado) --> lista_indices_sisi (corresponde al CASO 2 mencionado) --> lista_indices_nosi (corresponde al CASO 3 mencionado) --> lista_indices_nono (corresponde al CASO 4 mencionado)



ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

4

opcion = LISTA_DECLARACIONES_VARIABLES_PUBLICAS_NORMALES

A2

```
1388 #se convierte la lista lista_variables_publicas_normales_declar_ini_ord_desc (solo para el item 2 de las sublistas) en tupla
1389 #(si no el metodo startswith no va en busquedas multiples en df), cada item se trimea y se agrega un espacio en blanco al final
1390 tupla_declaracion_ini_variable_publica_normal = tuple([item[1] for item in lista_variables_publicas_normales_declar_ini_ord_desc])
```

('Public Const ', 'Global Const ', 'Public ', 'Global ', 'Const ')

A3

```
1401 df_sino = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["NOMBRE Rutina"].isnull() == True) &
1402                                     (df_bbdd_codigo_param["INDICE"] > indice_param_ini) & (df_bbdd_codigo_param["INDICE"] <= indice_param_fin) &
1403                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_ini_variable_publica_normal)) &
1404                                     (~df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.endswith(indicador_linea_codigo_vba_truncada_a_linea_siguiente)) &
1405                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
1406                                     ["TIPO_MODULO", "NOMBRE_MODULO", "INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
1407
1408 df_sisi = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["NOMBRE Rutina"].isnull() == True) &
1409                                     (df_bbdd_codigo_param["INDICE"] > indice_param_ini) & (df_bbdd_codigo_param["INDICE"] <= indice_param_fin) &
1410                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_ini_variable_publica_normal)) &
1411                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.endswith(indicador_linea_codigo_vba_truncada_a_linea_siguiente)) &
1412                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
1413                                     ["TIPO_MODULO", "NOMBRE_MODULO", "INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
1414
1415 df_nosi = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["NOMBRE Rutina"].isnull() == True) &
1416                                     (df_bbdd_codigo_param["INDICE"] > indice_param_ini) & (df_bbdd_codigo_param["INDICE"] <= indice_param_fin) &
1417                                     (~df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_ini_variable_publica_normal)) &
1418                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.endswith(indicador_linea_codigo_vba_truncada_a_linea_siguiente)) &
1419                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
1420                                     ["TIPO_MODULO", "NOMBRE_MODULO", "INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
1421
1422 df_nono = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["NOMBRE Rutina"].isnull() == True) &
1423                                     (df_bbdd_codigo_param["INDICE"] > indice_param_ini) & (df_bbdd_codigo_param["INDICE"] <= indice_param_fin) &
1424                                     (~df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.startswith(tupla_declaracion_ini_variable_publica_normal)) &
1425                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.endswith(indicador_linea_codigo_vba_truncada_a_linea_siguiente)) &
1426                                     (df_bbdd_codigo_param["CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"].str.len() != 0),
1427                                     ["TIPO_MODULO", "NOMBRE_MODULO", "INDICE", "CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS"]])
```

Los filtros por índices de líneas de código, a diferencia de la opción LISTA_DECLARACIONES_VARIABLES_LOCALES, no son estrictamente superiores e inferiores. Como el cálculo es a nivel de módulo, un módulo VBA no necesariamente empieza por un "Option Compare Database" (por ejemplo), puede empezar por la declaración de una variable pública.

En el dataframe **df_nono**, en estos ajustes previos, pueden salir líneas que no son de declaración de variables públicas normales. Cuando se realiza el cálculo, más adelante, estas líneas se descartan.

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

4

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS NORMALES

A3

```
df_sino
-----
TIPO_MODULO  NOMBRE_MODULO  INDICE  CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0  Estandar  Modulo_PRUEBA_1  5  Public Public Version As String
1  Estandar  Modulo_PRUEBA_1  11  Public Const Public_Const_1 = "aqui hay comas ...

df_sisi
-----
TIPO_MODULO  NOMBRE_MODULO  INDICE  CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0  Estandar  Modulo_PRUEBA_1  7  Global Public Var_1 As String, Public Var_2 As...
1  Estandar  Modulo_PRUEBA_1  13  Global Const Public_Const_2 = "@1", _

df_nosi
-----
TIPO_MODULO  NOMBRE_MODULO  INDICE  CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0  Estandar  Modulo_PRUEBA_1  8  Public Array(1 To 10, 2) As Variant, Public Va...
1  Estandar  Modulo_PRUEBA_1  14  Public Const_3 = 8, _
2  Estandar  Modulo_PRUEBA_1  15  Public_Const_4 = "@2", _
```

```
df_nono
-----
TIPO_MODULO  NOMBRE_MODULO  INDICE  CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0  Estandar  Modulo_PRUEBA_1  9  Public Var_4 As Variant
1  Estandar  Modulo_PRUEBA_1  16  Public Const 5 = 5
2  Estandar  Modulo_PRUEBA_1  18  Type UserType_Prueba_1
3  Estandar  Modulo_PRUEBA_1  19  SubVar_1 As String
4  Estandar  Modulo_PRUEBA_1  20  SubVar_2 As Long
5  Estandar  Modulo_PRUEBA_1  21  End Type
6  Estandar  Modulo_PRUEBA_1  23  Enum Enum_Prueba_1
7  Estandar  Modulo_PRUEBA_1  24  SubVar_1
8  Estandar  Modulo_PRUEBA_1  25  SubVar_2 = 1
9  Estandar  Modulo_PRUEBA_1  26  SubVar_3
10 Estandar  Modulo_PRUEBA_1  27  End Enum
```

Líneas que no afectan

```
1435 lista_indices_sino = []#las completion list no van cuando el df esta vacio por lo que se cre la lista en varias etapas
1436 lista_indices_sisi = []#idem
1437 lista_indices_nosi = []#idem
1438 lista_indices_nono = []#idem
1439
1440 if len(df_sino) != 0:
1441     lista_indices_sino = [[df_sino.iloc[ind, df_sino.columns.get_loc("TIPO_MODULO")]
1442                          , df_sino.iloc[ind, df_sino.columns.get_loc("NOMBRE_MODULO")]
1443                          , df_sino.iloc[ind, df_sino.columns.get_loc("INDICE")]
1444                          , df_sino.iloc[ind, df_sino.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]] for ind in df_sino.index]
1445
1446 if len(df_sisi) != 0:
1447     lista_indices_sisi = [[df_sisi.iloc[ind, df_sisi.columns.get_loc("TIPO_MODULO")]
1448                          , df_sisi.iloc[ind, df_sisi.columns.get_loc("NOMBRE_MODULO")]
1449                          , df_sisi.iloc[ind, df_sisi.columns.get_loc("INDICE")]
1450                          , df_sisi.iloc[ind, df_sisi.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]] for ind in df_sisi.index]
1451
1452 if len(df_nosi) != 0:
1453     lista_indices_nosi = [[df_nosi.iloc[ind, df_nosi.columns.get_loc("TIPO_MODULO")]
1454                          , df_nosi.iloc[ind, df_nosi.columns.get_loc("NOMBRE_MODULO")]
1455                          , df_nosi.iloc[ind, df_nosi.columns.get_loc("INDICE")]
1456                          , df_nosi.iloc[ind, df_nosi.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]] for ind in df_nosi.index]
1457
1458 if len(df_nono) != 0:
1459     lista_indices_nono = [[df_nono.iloc[ind, df_nono.columns.get_loc("TIPO_MODULO")]
1460                          , df_nono.iloc[ind, df_nono.columns.get_loc("NOMBRE_MODULO")]
1461                          , df_nono.iloc[ind, df_nono.columns.get_loc("INDICE")]
1462                          , df_nono.iloc[ind, df_nono.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]] for ind in df_nono.index]
```

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

4

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS NORMALES

A3

```
1471 lista_indices_sino = sorted(lista_indices_sino, key = lambda x: (x[0], x[1], x[2]))
1472 lista_indices_sisi = sorted(lista_indices_sisi, key = lambda x: (x[0], x[1], x[2]))
1473 lista_indices_nosi = sorted(lista_indices_nosi, key = lambda x: (x[0], x[1], x[2]))
1474 lista_indices_nono = sorted(lista_indices_nono, key = lambda x: (x[0], x[1], x[2]))
```



```
lista_indices_sino
-----
[['Estandar', 'Modulo_PRUEBA_1', 5, 'Public Public Version As String'],
 ['Estandar', 'Modulo_PRUEBA_1', 11, 'Public Const Public_Const_1 = "aquí hay comas , y signos = ""']]

lista_indices_sisi
-----
[['Estandar', 'Modulo_PRUEBA_1', 7, 'Global Public Var_1 As String, Public Var_2 As String, _'],
 ['Estandar', 'Modulo_PRUEBA_1', 13, 'Global Const Public_Const_2 = "@1", _']]

lista_indices_nosi
-----
[['Estandar', 'Modulo_PRUEBA_1', 8, 'Public Array(1 To 10, 2) As Variant, Public Var_3 As Date, _'],
 ['Estandar', 'Modulo_PRUEBA_1', 14, 'Public Const_3 = 8, _'],
 ['Estandar', 'Modulo_PRUEBA_1', 15, 'Public Const_4 = "@2", _']]

lista_indices_nono
-----
[['Estandar', 'Modulo_PRUEBA_1', 9, 'Public Var_4 As Variant'],
 ['Estandar', 'Modulo_PRUEBA_1', 16, 'Public Const_5 = 5'],
 ['Estandar', 'Modulo_PRUEBA_1', 18, 'Type UserType_Prueba_1'],
 ['Estandar', 'Modulo_PRUEBA_1', 19, 'SubVar_1 As String'],
 ['Estandar', 'Modulo_PRUEBA_1', 20, 'SubVar_2 As Long'],
 ['Estandar', 'Modulo_PRUEBA_1', 21, 'End Type'],
 ['Estandar', 'Modulo_PRUEBA_1', 23, 'Enum Enum_Prueba_1'],
 ['Estandar', 'Modulo_PRUEBA_1', 24, 'SubVar_1'],
 ['Estandar', 'Modulo_PRUEBA_1', 25, 'SubVar_2 = 1'],
 ['Estandar', 'Modulo_PRUEBA_1', 26, 'SubVar_3'],
 ['Estandar', 'Modulo_PRUEBA_1', 27, 'End Enum']]
```

Líneas que no afectan

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

4

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS NORMALES

Tras realizar los ajustes previos, se inicia el cálculo de la lista **resultado_funcion**.

Las sublistas incluidas en la lista **lista_indices_sino** se agregan directamente a la lista **resultado_funcion**.

Usando la lista **lista_variables_publicas_normales_declar_ini_ord_desc** (calculado en el ajuste A1) se calcula si la línea de código es de declaración de constante o no.

Las sublistas incluidas en la lista **lista_indices_sisi** se agregan directamente a la lista **resultado_funcion** (se quita el " _" al final de la línea de código al agregarse) mediante bucle (en adelante bucle_1) sobre los items de la misma (tipo y nombre de módulo, índice y línea de código, en adelante índice_1 y línea_1).

Al igual a lo comentado en el párrafo anterior, usando la lista **lista_variables_publicas_normales_declar_ini_ord_desc** (calculado en el ajuste A1) se calcula si la línea de código es de declaración de constante o no.

```
1483 for tipo_modulo_sino, nombre_modulo_sino, indice_sino, linea_codigo_sino in lista_indices_sino:
1484
1485     #se calcula si la linea de declaracion de constante o no mediante la lista
1486     #lista_variables_publicas_normales_declar_ini_ord_desc, calculada en los ajustes
1487     es_constante = None
1488     for item_config in lista_variables_publicas_normales_declar_ini_ord_desc:
1489         es_config = item_config[0]
1490         declar_ini_config = item_config[1]
1491
1492         if declar_ini_config in linea_codigo_sino:
1493             es_constante = es_config
1494             break
1495
1496     #se agregan los datos de la iteracion a la lista resultado de la funcion
1497     resultado_funcion.append([tipo_modulo_sino
1498                             , nombre_modulo_sino
1499                             , indice_sino
1500                             , es_constante
1501                             , linea_codigo_sino])
```

```
1509 for tipo_modulo_sisi, nombre_modulo_sisi, indice_sisi, linea_codigo_sisi in lista_indices_sisi:
1510
1511     #se calcula si la linea de declaracion de constante o no mediante la lista
1512     #lista_variables_publicas_normales_declar_ini_ord_desc, calculada en los ajustes
1513     es_constante = None
1514     for item_config in lista_variables_publicas_normales_declar_ini_ord_desc:
1515         es_config = item_config[0]
1516         declar_ini_config = item_config[1]
1517
1518         if declar_ini_config in linea_codigo_sisi:
1519             es_constante = es_config
1520             break
1521
1522
1523     #se agregan los datos a la lista resultado de la funcion (la linea de codigo se neteea quita
1524     #siempre y cuando el resultado del neteo no sea de longitud = 0)
1525     if len(linea_codigo_sisi[:-2]) != 0:
1526
1527         resultado_funcion.append([tipo_modulo_sisi
1528                                 , nombre_modulo_sisi
1529                                 , indice_sisi
1530                                 , es_constante
1531                                 , linea_codigo_sisi[:-2]])
```

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

4

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS NORMALES

En cada iteración sobre bucle_1, se inicia un bucle while (en adelante bucle_while), con incrementos de índices de línea + 1:

- a partir del índice de línea inmediatamente consecutivo (en adelante indice_while) a indice_1.
- hasta al 1er índice de línea de la lista **lista_indices_nono** (en adelante indice_fin_bucle_while) inmediatamente consecutivo a indice_1.

Se agregan las sublistas de la lista **lista_indices_nosi** si sus índices de línea son iguales a indice_while (se quita el "_" al final de la línea al agregarse).

Al llegar a la última iteración sobre bucle_while, si indice_nono_fin_bucle_while es igual a indice_while se agrega la sublista de la lista **lista_indices_nono** asociada a indice_nono_fin_bucle_while.

```
1538 lista_indices_nosi_iteracion = [[tipo_modulo_nosi
1539                                , nombre_modulo_nosi
1540                                , indice_nosi
1541                                , linea_codigo_nosi]
1542                                for tipo_modulo_nosi, nombre_modulo_nosi, indice_nosi, linea_codigo_nosi in lista_indices_nosi
1543                                if indice_nosi > indice_sisi]
1544
1545 lista_indice_nono_despues_iteracion = [[tipo_modulo_nono
1546                                         , nombre_modulo_nono
1547                                         , indice_nono
1548                                         , linea_codigo_nono]
1549                                         for tipo_modulo_nono, nombre_modulo_nono, indice_nono, linea_codigo_nono in lista_indices_nono
1550                                         if indice_nono > indice_sisi]
1551
1552
1553 tipo_modulo_nono_despues_iteracion = lista_indice_nono_despues_iteracion[0][0]
1554 nombre_modulo_nono_despues_iteracion = lista_indice_nono_despues_iteracion[0][1]
1555 primer_indice_nono_despues_iteracion = lista_indice_nono_despues_iteracion[0][2]
1556 linea_codigo_primer_indice_nono_despues_iteracion = lista_indice_nono_despues_iteracion[0][3]
```

4

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS NORMALES

Tanto para las sublistas de **lista_indices_nosi** y **lista_indices_nono** que se agregan a la lista **resultado_funcion** no es necesario realizar el cálculo de si es constante o no porque recoge el valor de esta variable local calculada en la iteración sobre **bucle_1** y se ha de replicar para todas las sublistas que se agregan a **resultado_funcion** en **bucle_while**.

```

1558     indice_linea_siguiente = indice_sisi + 1
1559     while indice_linea_siguiente <= primer_indice_nono_despues_iteracion:
1560
1561         for tipo_modulo_nosi, nombre_modulo_nosi, indice_nosi, linea_codigo_nosi in lista_indices_nosi_iteracion:
1562
1563             if indice_linea_siguiente == indice_nosi:
1564
1565                 #se agregan los datos a la lista resultado de la funcion (la linea de codigo se neteea quitando el
1566                 #siempre y cuando el resultado del neteo no sea de longitud = 0)
1567                 if len(linea_codigo_nosi[:-2]) != 0:
1568
1569                     resultado_funcion.append([tipo_modulo_nosi
1570                                             , nombre_modulo_nosi
1571                                             , indice_nosi
1572                                             , es_constante
1573                                             , linea_codigo_nosi[:-2]])
1574
1575                     break
1576
1577             if indice_linea_siguiente == primer_indice_nono_despues_iteracion:
1578
1579                 #se agregan los datos de la iteracion a la lista resultado de la funcion
1580                 resultado_funcion.append([tipo_modulo_nono_despues_iteracion
1581                                         , nombre_modulo_nono_despues_iteracion
1582                                         , primer_indice_nono_despues_iteracion
1583                                         , es_constante
1584                                         , linea_codigo_primer_indice_nono_despues_iteracion])
1585
1586                 break
1587
1588     indice_linea_siguiente = indice_linea_siguiente + 1

```

4

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS NORMALES

```

00 Option Compare Database
01 Option Explicit
02 Option Base 1
03
04 'Variables globales
05 Public Public_Version As String
06
07 Global Public Var_1 As String, Public Var_2 As String, _
08 Public Array(1 To 10, 2) As Variant, Public Var_3 As Date, _
09 Public Var_4 As Variant
10
11 Public Const Public_Const_1 = "aqui hay comas , y signos = "
12
13 Global Const Public_Const_2 = "@1", _
14 Public_Const_3 = 8, _
15 Public_Const_4 = "@2", _
16 Public_Const_5 = 5
17
18 Type UserType_Prueba_1 'variable publica definida por el usuario de prueba
19     SubVar_1 As String
20     SubVar_2 As Long
21 End Type
22
23 Enum Enum_Prueba_1
24     SubVar_1 'sub-variable de otra variable publica definida por el usuario de prueba
25     SubVar_2 = 1
26     SubVar_3
27 End Enum
28
29
30 Sub ActivarPanelNavegacion()
31
32 On Error Resume Next
33
34 DoCmd.LockNavigationPane (False)
35 DoCmd.SelectObject acTable, "MSysObjects", True
36 DoCmd.RunCommand acCmdWindowUnhide
37 MsgBox "fsdfsdf"
38
39 End Sub

```



```

resultado_funcion
[['Estandar', 'Modulo_PRUEBA_1', 5, 'NO_ES_CONSTANTE', 'Public Public_Version As String'],
['Estandar', 'Modulo_PRUEBA_1', 11, 'SI_ES_CONSTANTE', 'Public Const Public_Const_1 = "aqui hay comas , y signos = "'],
['Estandar', 'Modulo_PRUEBA_1', 7, 'NO_ES_CONSTANTE', 'Global Public Var_1 As String, Public Var_2 As String,'],
['Estandar', 'Modulo_PRUEBA_1', 8, 'NO_ES_CONSTANTE', 'Public Array(1 To 10, 2) As Variant, Public Var_3 As Date,'],
['Estandar', 'Modulo_PRUEBA_1', 9, 'NO_ES_CONSTANTE', 'Public Var_4 As Variant']]
[['Estandar', 'Modulo_PRUEBA_1', 13, 'SI_ES_CONSTANTE', 'Global Const Public_Const_2 = "@1",'],
['Estandar', 'Modulo_PRUEBA_1', 14, 'SI_ES_CONSTANTE', 'Public_Const_3 = 8,'],
['Estandar', 'Modulo_PRUEBA_1', 15, 'SI_ES_CONSTANTE', 'Public_Const_4 = "@2",'],
['Estandar', 'Modulo_PRUEBA_1', 16, 'SI_ES_CONSTANTE', 'Public_Const_5 = 5']]

```

5

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Permite localizar **dentro de un módulo VBA** las líneas de código que corresponden con declaraciones de variables públicas definidas por el usuario.

Son las líneas que, fuera de rutinas / funciones (llegados a este paso las rutinas / funciones ya están calculadas), empiezan por “Type “, “Enum “ etc etc y se declaran en varias líneas.

El proceso de cálculo es totalmente distinto al caso de las variables públicas normales.

El resultado de la función es aquí una lista de listas donde cada sublista que la compone contiene:

ITEMS SUBLISTAS
Tipo de módulo
Nombre de módulo
flag unico indentificador de la variable publica definida por el usuario (todas las lineas de codigo que la componen tienen el mismo flag)
Indicador de si la linea de código es de declaración de constante o no
Linea de código

Se ejecuta dentro del ámbito de un módulo VBA cuando se utiliza la función en la función **func_access_bbdd_codigo_calculos** (ver **ANEXO_08**).

Requiere de parámetros kwargs que son:

- **indice_param_ini**
- **indice_param_fin**
- **df_bbdd_codigo_param**

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

5

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS DEFINIDAS POR USUARIO

```
00 Option Compare Database
01 Option Explicit
02 Option Base 1
03
04 'VARIABLES PUBLICAS
05 Public Public_Version As String
06
07 Global Public_Var_1 As String, Public_Var_2 As String, _
08 Public_Array(1 To 10, 2) As Variant, Public_Var_3 As Date, _
09 Public_Var_4 As Variant
10
11 Public Const Public_Const_1 = "aqui hay comas , y signos = "
12
13 Global Const Public_Const_2 = "@1", _
14 Public_Const_3 = 8, _
15 Public_Const_4 = "@2", _
16 Public_Const_5 = 5
17
18 Type UserType_Prueba_1 'variable publica definida por el usuario de prueba
19     SubVar_1 As String
20     SubVar_2 As Long
21 End Type
22
23 Enum Enum_Prueba_1
24     SubVar_1 'sub-variable de otra variable publica definida por el usuario de prueba
25     SubVar_2 = 1
26     SubVar_3
27 End Enum
28
29
30 Sub ActivarPanelNavegacion()
31
32 On Error Resume Next
33
34 DoCmd.LockNavigationPane (False)
35 DoCmd.SelectObject acTable, "MSysObjects", True
36 DoCmd.RunCommand acCmdWindowUnhide
37 MsgBox "fsdfsdf"
38
39 End Sub
```


ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

5

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Se han de realizar ajustes previos al cálculo de la lista de líneas de código resultado de la función.

AJUSTE	DESCRIPCIÓN
A1	Se convierten en tuplas las listas almacenadas en las subkeys_2 (LISTA_DECLARACION_INICIO y LISTA_DECLARACION_FIN) asociadas a la key:1 (VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS). Cada item de la 1era tupla (inicios de declaración), se trime y se agrega un espacio en blanco al final (por si el usuario ha configurado mal el diccionario). La conversión en tuplas es para poder usar el metodo startswith en el filtrado de dataframes por selecciones multiples.
A2	Se listan las líneas de código correspondientes a lkos inicios y finales de declaración de este tipo de variables públicas mediante extracciones de dataframes sobre el dataframe df_bbdd_codigo_param (parámetro de la función) en los cuales se conserva el tan solo el indice de la linea de código. Se crean con estos dataframes extraidos 2 listas que se ordenan.

A1

```
1630 tupla_variable_definida_por_usuario_ini = tuple([item.strip() + " " for item in dicc_objetos_vba["VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO"]["LISTA_DECLARACION_INICIO"]])
1631 tupla_variable_definida_por_usuario_fin = tuple([item.strip() for item in dicc_objetos_vba["VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO"]["LISTA_DECLARACION_FIN"]])
```



('Type ', 'Enum ')

('End', 'End Type', 'End Enum')

A2

```
1637 df_temp_1 = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["NOMBRE Rutina"].isnull() == True) &
1638                                     (df_bbdd_codigo_param["INDICE"] >= indice_param_ini) & (df_bbdd_codigo_param["INDICE"] <= indice_param_fin) &
1639                                     (df_bbdd_codigo_param["CODIGO SIN TAB TRIMEADO SIN COMENTARIOS"].str.startswith(tupla_variable_definida_por_usuario_ini)) &
1640                                     (df_bbdd_codigo_param["CODIGO SIN TAB TRIMEADO SIN COMENTARIOS"].str.len() != 0),
1641                                     ["INDICE"]])#aqui es startswith
1642
1643 df_temp_2 = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["NOMBRE Rutina"].isnull() == True) &
1644                                     (df_bbdd_codigo_param["INDICE"] >= indice_param_ini) & (df_bbdd_codigo_param["INDICE"] <= indice_param_fin) &
1645                                     (df_bbdd_codigo_param["CODIGO SIN TAB TRIMEADO SIN COMENTARIOS"].isin(tupla_variable_definida_por_usuario_fin)) &
1646                                     (df_bbdd_codigo_param["CODIGO SIN TAB TRIMEADO SIN COMENTARIOS"].str.len() != 0),
1647                                     ["INDICE"]])#aqui es isin
```

Los filtros por índices de líneas de código, al igual que la opción LISTA_DECLARACIONES_VARIABLES_PUBLICAS_NORMALES, no son estrictamente superiores e inferiores. Como el cálculo es a nivel de módulo, un módulo VBA no necesariamente empieza por un "Option Compare Database " (por ejemplo), puede empezar por la declaración de una variable pública.

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

5

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS DEFINIDAS POR USUARIO

A2

df_temp_1	

INDICE	
0	18
1	23

df_temp_2	

INDICE	
0	21
1	27

```
1654 lista_variables_publicas_definidas_por_usuario_indices_ini = []#las completion list no van cuando el df esta vacio por lo que se cre la lista en va
1655 lista_variables_publicas_definidas_por_usuario_indices_fin = []#idem
1656
1657 lista_variables_publicas_definidas_por_usuario_indices_ini = [df_temp_1.iloc[ind, df_temp_1.columns.get_loc("INDICE")] for ind in df_temp_1.index]
1658 lista_variables_publicas_definidas_por_usuario_indices_fin = [df_temp_2.iloc[ind, df_temp_2.columns.get_loc("INDICE")] for ind in df_temp_2.index]
1659
1660
1661 #se ordenan las listas por el indice de la linea de codigo
1662 lista_variables_publicas_definidas_por_usuario_indices_ini = sorted(lista_variables_publicas_definidas_por_usuario_indices_ini)
1663 lista_variables_publicas_definidas_por_usuario_indices_fin = sorted(lista_variables_publicas_definidas_por_usuario_indices_fin)
```



```
lista_variables_publicas_definidas_por_usuario_indices_ini
-----
[18, 23]

lista_variables_publicas_definidas_por_usuario_indices_fin
-----
[21, 27]
```

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

5

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Tras realizar los ajustes previos, se inicia el cálculo de la lista **resultado_funcion**.

Mediante bucle (en adelante bucle_1) sobre la lista **lista_variables_publicas_definidas_por_usuario_indices_ini**, para cada índice de línea de código de la iteración (en adelante índice_1) se busca en la lista **lista_variables_publicas_definidas_por_usuario_indices_fin** el índice de fin de declaración inmediatamente consecutivo a índice_1 (en adelante índice_2).

Se calcula el flag de identificador único de la variable pública definida por el usuario de la iteración sobre bucle_1.

Se extrae del dataframe **df_bbdd_codigo_param** (parámetro de la función), un nuevo dataframe donde los índices de las líneas de código están situados entre índice_1 y índice_2 (inclusive porque el cálculo de este tipo de variable pública, al igual que las variables públicas normales es a nivel de módulo VBA y un módulo puede empezar directamente con la declaración de una variable pública).

El filtro excluye también las líneas de código asociadas a rutinas / funciones (llegados a este paso ya están calculadas).

Los datos extraídos de los dataframes son el tipo y nombre del módulo, el índice de la línea de código y la línea de código sin tabulaciones, trimeada y sin comentarios.

```
1676     for ind_ini, indice_ini in enumerate(lista_variables_publicas_definidas_por_usuario_indices_ini):
1677
1678         #se calcula el indice de codigo correspondiente a fin de declaracion de variable publica definida por el usuario
1679         #inmediatamente posterior al indice de inicio de declaracion
1680         indice_fin = [indice_fin for indice_fin in lista_variables_publicas_definidas_por_usuario_indices_fin if indice_fin > indice_ini][0]
1681
1682
1683         #se calcula el flag unico dentro de cada modula para cada variable publica definida por el usuario
1684         flag_variable_publica_definida_usuario = "DEFINIDA_POR_USUARIO_" + str(ind_ini + 1)
1685
1686
1687         #se extrae del parametro df_variables_publicas los indices y lineas de codigo entre los indices de inicio y fin de declaracion
1688         df_temp = (df_bbdd_codigo_param.loc[(df_bbdd_codigo_param["NOMBRE Rutina"].isnull() == True) &
1689                                             (df_bbdd_codigo_param["INDICE"] >= indice_ini) & (df_bbdd_codigo_param["INDICE"] <= indice_fin),
1690                                             ["TIPO_MODULO", "NOMBRE_MODULO", "INDICE", "CODIGO SIN TAB TRIMEADO SIN COMENTARIOS"]])
```

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

5

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Una vez extraído el dataframe entre los índices índice_1 e índice_2 se ejecuta otro bucle sobre los índices del mismo y en cada iteración se agrega la sublista (con el tipo y nombre de módulo, el índice de la línea de código, el flag identificador único de la variable pública de la iteración sobre bucle_1 y la línea de código sin tabulaciones, trimeada y sin comentarios).a la lista **resultado_funcion**.

```
1696     for ind in df_temp.index:
1697
1698         resultado_funcion.append([df_temp.iloc[ind, df_temp.columns.get_loc("TIPO_MODULO")]
1699                                   , df_temp.iloc[ind, df_temp.columns.get_loc("NOMBRE_MODULO")]
1700                                   , df_temp.iloc[ind, df_temp.columns.get_loc("INDICE")]
1701                                   , flag_variable_publica_definida_usuario
1702                                   , df_temp.iloc[ind, df_temp.columns.get_loc("CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS")]])
```

5

opcion = LISTA DECLARACIONES VARIABLES PUBLICAS DEFINIDAS POR USUARIO

```

00 Option Compare Database
01 Option Explicit
02 Option Base 1
03
04 'VARIABLES PUBLICAS
05 Public Public_Version As String
06
07 Global Public_Var_1 As String, Public_Var_2 As String, _
08 Public_Array(1 To 10, 2) As Variant, Public_Var_3 As Date, _
09 Public_Var_4 As Variant
10
11 Public Const Public_Const_1 = "aqui hay comas , y signos = "
12
13 Global Const Public_Const_2 = "@1", _
14 Public_Const_3 = 8, _
15 Public_Const_4 = "@2", _
16 Public_Const_5 = 5
17
18 Type UserType_Prueba_1 variable publica definida por el usuario de prueba
19     SubVar_1 As String
20     SubVar_2 As Long
21 End Type
22
23 Enum Enum_Prueba_1
24     SubVar_1 'sub-variable de otra variable publica definida por el usuario de prueba
25     SubVar_2 = 1
26     SubVar_3
27 End Enum
28
29
30 Sub ActivarPanelNavegacion()
31
32 On Error Resume Next
33
34 DoCmd.LockNavigationPane (False)
35 DoCmd.SelectObject acTable, "MSysObjects", True
36 DoCmd.RunCommand acCmdWindowUnhide
37 MsgBox "fsdfsdf"
38
39 End Sub

```

```

ITERACIÓN lista_variables_publicas_definidas_por_usuario indices_ini      1
-----
indice_ini                        18
indice_fin                        21
flag_variable_publica_definida_usuario  DEFINIDA_POR_USUARIO_1

df_temp:

TIPO_MODULO  NOMBRE_MODULO  INDICE  CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0  Estandar  Modulo_PRUEBA_1  18                                     Type UserType_Prueba_1
1  Estandar  Modulo_PRUEBA_1  19                                     SubVar_1 As String
2  Estandar  Modulo_PRUEBA_1  20                                     SubVar_2 As Long
3  Estandar  Modulo_PRUEBA_1  21                                     End Type
-----
ITERACIÓN df_temp --> 1
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 18, 'DEFINIDA_POR_USUARIO_1', 'Type UserType_Prueba_1']
ITERACIÓN df_temp --> 2
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 19, 'DEFINIDA_POR_USUARIO_1', 'SubVar_1 As String']
ITERACIÓN df_temp --> 3
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 20, 'DEFINIDA_POR_USUARIO_1', 'SubVar_2 As Long']
ITERACIÓN df_temp --> 4
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 21, 'DEFINIDA_POR_USUARIO_1', 'End Type']
*****

```

```

ITERACIÓN lista_variables_publicas_definidas_por_usuario indices_ini      2
-----
indice_ini                        23
indice_fin                        27
flag_variable_publica_definida_usuario  DEFINIDA_POR_USUARIO_2

df_temp:

TIPO_MODULO  NOMBRE_MODULO  INDICE  CODIGO_SIN_TAB_TRIMEADO_SIN_COMENTARIOS
0  Estandar  Modulo_PRUEBA_1  23                                     Enum Enum_Prueba_1
1  Estandar  Modulo_PRUEBA_1  24                                     SubVar_1
2  Estandar  Modulo_PRUEBA_1  25                                     SubVar_2 = 1
3  Estandar  Modulo_PRUEBA_1  26                                     SubVar_3
4  Estandar  Modulo_PRUEBA_1  27                                     End Enum
-----
ITERACIÓN df_temp --> 1
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 23, 'DEFINIDA_POR_USUARIO_2', 'Enum Enum_Prueba_1']
ITERACIÓN df_temp --> 2
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 24, 'DEFINIDA_POR_USUARIO_2', 'SubVar_1']
ITERACIÓN df_temp --> 3
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 25, 'DEFINIDA_POR_USUARIO_2', 'SubVar_2 = 1']
ITERACIÓN df_temp --> 4
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 26, 'DEFINIDA_POR_USUARIO_2', 'SubVar_3']
ITERACIÓN df_temp --> 5
Sublista que se agrega a resultado_funcion+ ['Estandar', 'Modulo_PRUEBA_1', 27, 'DEFINIDA_POR_USUARIO_2', 'End Enum']
*****

```

6

opcion = LISTA VARIABLES PUBLICAS NO ES CONSTANTE

Permite desde **una línea de código** listar todas las variables públicas normales declaradas que NO son constantes (son las que vienen seguidas de “ As “) y aportar información sobre el tipo de dato de cada una.

En una misma línea de código se pueden declarar varias variables, separándolas por comas “,”.

El resultado de la función es aquí una lista de listas donde cada sublista que la compone contiene:

ITEMS SUBLISTAS	APLICA
Nombre de la variable pública	X
Tipo variable pública. Es el valor subkey_2 (STRING_PARA_LISTADO_EXCEL) asociada a key_1 (VARIABLE_PUBLICA_NORMAL) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS)	X
Tipo dato de la variable pública	X
Es constante?	
Valor de la constante (en caso de serlo)	
Sub-variables asociadas a la variable pública definida por el usuario	

Algunos ítems no se calculan porque la estructura de estas sublistas es la misma para todos los tipos de variables públicas (ver opciones LISTA_VARIABLES_PUBLICAS_SI_ES_CONSTANTE y LISTA_VARIABLES_PUBLICAS_DEFINIDAS_POR_USUARIO de esta misma función). Requiere como único parámetro kwargs la **línea_codigo**.

La función se ejecuta tras ya haber ejecutado la opción LISTA_DECLARACIONES_VARIABLES_PUBLICAS_NORMALES de la presente función donde para la cual se comenta en páginas anteriores que la línea de código se **netea** del inicio de declaración (los “Public”, “Global “, “Public Const “, “Global Const” etc etc).

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

6

opcion = LISTA VARIABLES PUBLICAS NO ES CONSTANTE

Los pasos a seguir para recuperar, desde la línea de código, el nombre de la(s) variable(s) y su(s) tipo(s) de dato son:

PASO	DESCRIPCIÓN
1	Se localiza si en la línea de código hay parentesis de apertura "(" y de cierre ")". En caso de que haya ambos, se crean listas temporales para almacenar los indices de posición de estos parentesis de apertura (<code>lista_indices_parentesis_apertura</code>) y cierre (<code>lista_indices_parentesis_cierre</code>). Mediante el metodo zip, se crea una lista de tuplas (<code>lista_tuplas_indices_parentesis</code>) donde cada tupla contiene la información del de cada indice de posición de los parentesis de apertura y de los de cierre inmediatamente posteriores (no puede haber en en la declaración de este tipo de variables públicas parentesis anidados los unos dentro de los otros).
2	Se crea lista temporal que almacena los indices de posición de las comas "," (<code>lista_indices_comas</code>) dentro de la línea de código y usando esta lista se crea otra lista temporal (<code>lista_indices_comas_replace</code>) que almacena todos los indices de posición de las comas que se situen entre los indices de posición de alguna tupla de la lista <code>lista_tuplas_indices_parentesis</code>
3	En caso de que la lista <code>lista_indices_comas_replace</code> no este vacia se reemplazan en la línea de código las comas asociadas por el carácter @ (@ es un carácter prohibido en la declaración de variables públicas en VBA por lo que no hay riesgo de alter ni el nombre ni el tipo de dato de una variable pública)
4	Se plitea la línea de código por la coma y se inicia bucle sobre los items de la lista resultante donde lo 1ero que se hace es hacer el replace de @ por la coma (por si se ejecuto el paso 3) y así recuperar el vlror original de la línea de código
5	Antes de splitear por " As " cada item de la lista resultante del paso 5, se realiza otro ajuste en caso de que haya arrays de más de una dimensión declarados y consiste en permutar lo encapsulado entre los parentesis "(" y ")" (que viene en VBA inmediatamente despues del nombre de la variable pública) despues del tipo de dato. Para ello, se crea la variable temporal <code>string_tipo_dato_arrays</code> para almacenar lo que que viene encapsulado entre parentesis
6	Se splitea cada item de la lista resultante, tras el spliteo por la coma del paso 5, por " As ". El 1er item de la lista resultante es el nombre de la variable a la cual se hace replace del valor de <code>string_tipo_dato_arrays</code> por nada, y el 2ndo item es el tipo de dato al cual se le concatena el valor de <code>string_tipo_dato_arrays</code>
7	Se agregan los datos a la lista <code>resultado_funcion</code> mediante sublista. La sublista contiene el nombre de la variable, el tipo de la misma (viene pre-definido por el diccionario <code>dicc_objetos_vba</code>), su tipo de dato, el indicador si es constante o no (en este caso es siempre NO), el valor de la constante (aquí no aplica por lo que se establece a None) y las sub-variables para las variables públicas definidas por el usuario (aquí no aplica por lo que se establece a None)

Tal como se ha comentado en la página anterior, cuando en una misma línea de código se declaran varias variables públicas, estas se separan con una coma ",". **No se trata tan solo de splitear por la coma** y con la lista resultante splitear cada uno de sus ítems por " As " para recuperar el nombre de la variable y su tipo de dato.

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

6

opcion = LISTA VARIABLES PUBLICAS NO ES CONSTANTE

Cuando en la línea de código se declaran **arrays** de más de una dimensión (son las variables que se declaran seguidas de paréntesis “(“ y “)” y dentro de estos hay una coma “,”), si se splitease en estos casos directamente por la coma generaría información errónea.

```
Public Public_Array(1 To 10, 2) As Variant
```

Split por coma

```
['Public_Array_1(1 To 10', ' 2) As Variant']
```

Item 1

Item 2

INCORRECTO

Para estos casos requiere realizar unos ajustes previos (son los pasos 1 a 3 marcados en **AMARILLO** en la página anterior).

Se ilustran los pasos a seguir con el ejemplo siguiente:

```
Public Public_Array_1(1 To 10, 2) As Variant, Public_Var_1 As Long, Public_Array(1 To 5, 3) As String
```



Se comenta en páginas anteriores que la línea de código ya viene en el parámetro kwargs de la función **neteado** del inicio de declaración por lo que la línea de código es:

```
Public_Array_1(1 To 10, 2) As Variant, Public_Var_1 As Long, Public_Array(1 To 5, 3) As String
```


ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

6

opcion = LISTA VARIABLES PUBLICAS NO ES CONSTANTE

1

```
1744 lista_indices_parentesis_apertura = list(re.finditer(r"\(", linea_codigo))
1745 lista_indices_parentesis_cierre= list(re.finditer(r"\)", linea_codigo))
1746
1747 if len(lista_indices_parentesis_apertura) != 0 and len(lista_indices_parentesis_apertura) == len(lista_indices_parentesis_cierre):
1748
1749     #en caso de que las 2 listas no son vacias y son de la misma longitud, mediante el metodo zip, se crea una lista de tuplas donde
1750     #cada tupla indica en el item 1 el indice de apertura y en el item 2 el indice de cierre
1751     lista_indices_parentesis_apertura = [lista_indices_parentesis_apertura[ind].start() for ind, item in enumerate(lista_indices_parentesis_apertura)]
1752     lista_indices_parentesis_cierre = [lista_indices_parentesis_cierre[ind].start() for ind, item in enumerate(lista_indices_parentesis_cierre)]
1753
1754
1755     #se crea la lista de tuplas con los indices de posicion de de los parentesis de apertura y cierre
1756     lista_tuplas_indices_parentesis = list(zip(lista_indices_parentesis_apertura, lista_indices_parentesis_cierre))
```



```
lista_indices_parentesis_apertura    [14, 73]
lista_indices_parentesis_cierre      [25, 83]
lista_tuplas_indices_parentesis      [(14, 25), (73, 83)]
```

2

```
1760 lista_indices_comas = list(re.finditer(",", linea_codigo))
1761 lista_indices_comas = [lista_indices_comas[ind].start() for ind, item in enumerate(lista_indices_comas)]
1762
1763 lista_indices_comas_replace = [indice_coma for indice_coma in lista_indices_comas
1764                               for tupla_indices in lista_tuplas_indices_parentesis
1765                               if indice_coma >= tupla_indices[0] and indice_coma <= tupla_indices[1]]
```



```
lista_indices_comas                [22, 37, 59, 80]
lista_indices_comas_replace         [22, 80] → (73, 83)
                                     → (14, 25)
```

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

6

opcion = LISTA VARIABLES PUBLICAS NO ES CONSTANTE

3

```
1768 #se realiza el replace de la coma por @
1769 linea_codigo = "".join([caracter if ind not in lista_indices_comas_replace else "@" for ind, caracter in enumerate(linea_codigo)])
```



```
linea_codigo ajustada      Public_Array_1(1 To 10@ 2) As Variant, Public_Var_1 As Long, Public_Array(1 To 5@ 3) As String
```

4

```
1776 linea_codigo_spliteada_por_coma = linea_codigo.split(",")
1777
1778 for item_spliteado_por_coma in linea_codigo_spliteada_por_coma:
1779
1780     if len(item_spliteado_por_coma) != 0:
1781
1782         #se hace tambien el replace de @ por la coma
1783         item_spliteado_por_coma = item_spliteado_por_coma.replace("@", ",")
```



```
linea_codigo_spliteada_por_coma      ['Public_Array_1(1 To 10@ 2) As Variant', ' Public_Var_1 As Long', ' Public_Array(1 To 5@ 3) As String']
```



```
item_spliteado_por_coma      Public_Array_1(1 To 10@ 2) As Variant
item_spliteado_por_coma (tras replace @)      Public_Array_1(1 To 10, 2) As Variant

item_spliteado_por_coma      Public_Var_1 As Long
item_spliteado_por_coma (tras replace @)      Public_Var_1 As Long

item_spliteado_por_coma      Public_Array(1 To 5@ 3) As String
item_spliteado_por_coma (tras replace @)      Public_Array(1 To 5, 3) As String
```

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

6

opcion = LISTA VARIABLES PUBLICAS NO ES CONSTANTE

5

```
1787 lista_indices_parentesis_apertura = list(re.finditer(r"\\(", item_spliteado_por_coma))
1788 lista_indices_parentesis_cierre= list(re.finditer(r"\\)", item_spliteado_por_coma))
1789
1790
1791 #se almacena lo incluido entre los parentesis "(" y ")" en el string string_tipo_dato_arrays
1792 #para poder concatenarlo al tipo de dato tras splitear item_spliteado_por_coma por " As " (mas adelante)
1793 string_tipo_dato_arrays = ""
1794 if len(lista_indices_parentesis_apertura) != 0 and len(lista_indices_parentesis_apertura) == len(lista_indices_parentesis_cierre):
1795
1796     indice_parentesis_apertura = [lista_indices_parentesis_apertura[ind].start() for ind, item in enumerate(lista_indices_parentesis_apertura)][0]
1797     indice_parentesis_cierre = [lista_indices_parentesis_cierre[ind].start() for ind, item in enumerate(lista_indices_parentesis_cierre)][0]
1798
1799     string_tipo_dato_arrays = item_spliteado_por_coma[indice_parentesis_apertura:indice_parentesis_cierre + 1]
```



item_spliteado_por_coma	Public_Array_1(1 To 10, 2) As Variant
string_tipo_dato_arrays	(1 To 10, 2)
item_spliteado_por_coma	Public_Var_1 As Long
string_tipo_dato_arrays	
item_spliteado_por_coma	Public_Array(1 To 5, 3) As String
string_tipo_dato_arrays	(1 To 5, 3)

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

6

opcion = LISTA VARIABLES PUBLICAS NO ES CONSTANTE

6

```
1805 item_spliteado_por_coma_y_por_as = item_spliteado_por_coma.split(indicador_variable_vba_declaracion_tipo_dato)
1806
1807 variable_publica_nombre = item_spliteado_por_coma_y_por_as[0].replace(string_tipo_dato_arrays, "").strip()
1808
1809 variable_publica_tipo_dato = (item_spliteado_por_coma_y_por_as[1] + string_tipo_dato_arrays.replace("()", ""))
1810 if len(item_spliteado_por_coma_y_por_as) == 2 else string_tipo_dato_arrays.replace("()", "")
```



item_spliteado_por_coma	Public Array_1(1 To 10, 2) As Variant
item_spliteado_por_coma_y_por_as	['Public_Array_1(1 To 10, 2)', 'Variant']
variable_publica_nombre	Public Array_1
variable_publica_tipo_dato	Variant(1 To 10, 2)
item_spliteado_por_coma	Public Var_1 As Long
item_spliteado_por_coma_y_por_as	['Public_Var_1', 'Long']
variable_publica_nombre	Public Var_1
variable_publica_tipo_dato	Long
item_spliteado_por_coma	Public Array_1(1 To 5, 3) As String
item_spliteado_por_coma_y_por_as	['Public_Array_1(1 To 5, 3)', 'String']
variable_publica_nombre	Public Array_1
variable_publica_tipo_dato	String(1 To 5, 3)

Este ajuste se realiza para que en el diagnostico de dependencias se pueda localizar las variables públicas normales del tipo arrays de más de 1 dimensión en el código VBA de rutinas y funciones. Si no se hiciese este ajuste el nombre de las 2 variables del ejemplo seria el de su definición como array **Public_Array_1(1 to 10, 2)** y **Public_Array_1(1 to 5, 3)** que no es como saldrían en caso de uso en rutinas / funciones VBA (saldrían por ejemplo de forma estática para el 1er array como **Public_Array_1(4, 1) =** o de forma dinámica en un bucle **Public_Array_1(i, 2) =**).

ANEXO_09

FUNCIÓN: func_access_bbdd_codigo_calculos_varios

6

opcion = LISTA VARIABLES PUBLICAS NO ES CONSTANTE

7

```
1813 #se agregan los datos a la lista resultado de la funcion
1814 variable_publica_tipo = dicc_objetos_vba["VARIABLE_PUBLICA_NORMAL"]["STRING_PARA_LISTADO_EXCEL"]
1815 variable_publica_es_constante = "NO"
1816 variable_publica_es_constante_valor = None
1817 variable_publica_definida_usuario_subvariables = None
1818
1819 resultado_funcion.append([variable_publica_nombre
1820                          , variable_publica_tipo
1821                          , variable_publica_tipo_dato
1822                          , variable_publica_es_constante
1823                          , variable_publica_es_constante_valor
1824                          , variable_publica_definida_usuario_subvariables])
```

7

opcion = LISTA VARIABLES PUBLICAS SI ES CONSTANTE

Permite desde **una línea de código** listar todas las variables públicas normales declaradas que SI son constantes (son las que vienen seguidas de “ = ”) y aportar información sobre el valor de la constante de cada una.

En una misma línea de código se pueden declarar varias variables, separándolas por comas “,”.

El resultado de la función es aquí una lista de listas donde cada sublista que la compone contiene:

ITEMS SUBLISTAS	APLICA
Nombre de la variable pública	X
Tipo variable pública. Es el valor subkey_2 (STRING_PARA_LISTADO_EXCEL) asociada a key_1 (VARIABLE_PUBLICA_NORMAL) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS)	X
Tipo dato de la variable pública	
Es constante?	X
Valor de la constante (en caso de serlo)	X
Sub-variables asociadas a la variable pública definida por el usuario	

Algunos ítems no se calculan porque la estructura de estas sublistas es la misma para todos los tipos de variables públicas (ver opciones LISTA_VARIABLES_PUBLICAS_NO_ES_CONSTANTE y LISTA_VARIABLES_PUBLICAS_DEFINIDAS_POR_USUARIO de esta misma función). Requiere como único parámetro kwargs la **línea_codigo**.

La función se ejecuta tras ya haber ejecutado la opción LISTA_DECLARACIONES_VARIABLES_PUBLICAS_NORMALES de la presente función donde para la cual se comenta en páginas anteriores que la línea de código se **netea** del inicio de declaración (los “Public”, “Global “, “Public Const “, “Global Const” etc etc).

7

opcion = LISTA VARIABLES PUBLICAS SI ES CONSTANTE

Los pasos a seguir para recuperar, desde la línea de código, el nombre de la(s) variable(s) y su(s) tipo(s) de dato son:

PASO	DESCRIPCIÓN
1	Se localiza si en la línea hay valores de constantes de tipo string (los que vienen encapsulados entre comillas dobles ""). En caso de que los haya, se crea la lista temporal lista_constantes_string para que almacenar sus valores. Usando esta lista se crea la lista lista_constantes_string_con_valor_replace donde cada item es una sublista que contiene los items de lista_constantes_string a los cuales se les asocia un valor para hacer un replace temporal (este valor de repla
2	Se hace el replace en la línea de código del 1er item de cada sublista de la lista lista_constantes_string_con_valor_replace por el 2do item. El replace se hace al 1er valor que encuentra (leyendo la línea de código de izquierda a derecha y no a todos los valores de constantes string porque algunas variables podrian tener el mismo valor (ver ejemplo)
3	Se plitea la línea de código por la coma y se inicia bucle sobre los items de la lista resultante donde cada uno se splitea por el signo "=". El 1er item de la lista resultante es el nombre de la constante y el 2do item su valor. AL valor de la constante calculado en el paso 3 se inicia un bucle sobre la lista lista_constantes_string_con_valor_replace para realizar el replace del 2do item de cada sublista por el 1ero y así se recupera el código original
4	Si el valor de la constante es un string que empieza por una comilla simple, se realiza un pequeño ajuste agregando al principio de su valor otra comilla simple. Esto para de cara a exportar a Excel se visualice correctamente el string (en Excel cuando una celda empieza por una comilla simple no se ve en pantalla, para que se vea hay que tener 2 comillas simples)
5	Se agregan los datos a la lista resultado_funcion mediante sublista. La sublista contiene el nombre de la variable, el tipo de la misma (viene pre-definido por el diccionario dicc_objetos_vba), su tipo de dato (aquí no aplica por lo que se establece a None), el indicador si es constante o no (en este caso es siempre SI), el valor de la constante y las sub-variables para las variables públicas definidas por el usuario (aquí no aplica por lo que se establece a None)

Tal como se ha comentado en la página anterior, cuando en una misma línea de código se declaran varias variables públicas, estas se separan con una coma ",". **No se trata tan solo de splitear por la coma** y con la lista resultante splitear cada uno de sus ítems por "=" para recuperar el nombre de la variable y el valor de la constante.

7

opcion = LISTA VARIABLES PUBLICAS SI ES CONSTANTE

Cuando en la línea de código se declaran constantes de tipo string donde dentro del valor hay comas “,” y/o signos “=”, si se splitease en estos casos directamente por la coma (y posteriormente por el signo igual para recuperar el valor de la constante) generaría información errónea.



Para estos casos requiere realizar unos ajustes previos (son los pasos 1 y 2 marcados en **AMARILLO** en la página anterior).

Se ilustran los pasos a seguir con el ejemplo siguiente:

```
Public Const Public_Const_1 = "aquí hay comas , y signos = ", Public_const_2 = "valor duplicado", Public_const_3 = 5, Public_const_4 = "valor duplicado"
```



Se comenta en páginas anteriores que la línea de código ya viene en el parámetro kwargs de la función **neteado** del inicio de declaración por lo que la línea de código es:

```
Public_Const_1 = "aquí hay comas , y signos = ", Public_const_2 = "valor duplicado", Public_const_3 = 5, Public_const_4 = "valor duplicado"
```


ANEXO_09

FUNCIÓN: func_access_df_bbdd_codigo_calculos_varios

7

opcion = LISTA VARIABLES PUBLICAS SI ES CONSTANTE

1

```
1866 lista_constantes_string = re.findall(r"\"([^\"]*)\"", linea_codigo)
1867 lista_constantes_string_con_valor_replace = [[item, "@" + str(ind + 1)] for ind, item in enumerate(lista_constantes_string)]
```



```
lista_constantes_string          ['aqui hay comas , y signos = ', 'valor duplicado', 'valor duplicado']
lista_constantes_string_con_valor_replace  [['aqui hay comas , y signos = ', '@1'], ['valor duplicado', '@2'], ['valor duplicado', '@3']]
```

2

```
1872 for valor_constante_string, valor_replace in lista_constantes_string_con_valor_replace:
1873     linea_codigo = linea_codigo.replace(valor_constante_string, valor_replace, 1)
```



```
línea código (tras el replace)      Public Const Public_Const_1 = @1, Public_const_2 = @2, Public_const_3 = 5, Public_const_4 = @3
```

3

```
1880 linea_codigo_spliteada_por_coma = linea_codigo.split(",")
1881
1882 for item_spliteado_por_coma in linea_codigo_spliteada_por_coma:
1883
1884     if len(item_spliteado_por_coma) != 0:
1885
1886         linea_codigo_spliteada_por_signo_igual = item_spliteado_por_coma.split(indicador_variable_signo_igual)
1887
1888         #se calculan los datos de la variable publica
1889         variable_publica_nombre = linea_codigo_spliteada_por_signo_igual[0].strip()
1890         variable_publica_es_constante_valor = linea_codigo_spliteada_por_signo_igual[1].replace("\'", "'").strip() if len(linea_codigo_spliteada_por_signo_igual) == 2 else None
1891
1892
1893         #se hace el replace de los valores string de constantes para recuperar el valor original
1894         for item_original, item_replace in lista_constantes_string_con_valor_replace:
1895             if item_replace in variable_publica_es_constante_valor:
1896                 variable_publica_es_constante_valor = variable_publica_es_constante_valor.replace(item_replace, item_original)
1897             break
```

7

opcion = LISTA VARIABLES PUBLICAS SI ES CONSTANTE

3

```

linea_codigo_spliteada_por_coma      ['Public Const Public_Const_1 = @1', ' Public_const_2 = @2', ' Public_const_3 = 5', ' Public_const_4 = @3']
linea_codigo_spliteada_por_signo_igual ['Public Const Public_Const_1', '@1']
variable_publica_nombre               Public Const Public_Const_1
variable_publica_es_constante_valor   aqui hay comas , y signos =

linea_codigo_spliteada_por_signo_igual [' Public_const_2', '@2']
variable_publica_nombre               Public_const_2
variable_publica_es_constante_valor   valor duplicado

linea_codigo_spliteada_por_signo_igual [' Public_const_3', '5']
variable_publica_nombre               Public_const_3
variable_publica_es_constante_valor   5

linea_codigo_spliteada_por_signo_igual [' Public_const_4', '@3']
variable_publica_nombre               Public_const_4
variable_publica_es_constante_valor   valor duplicado

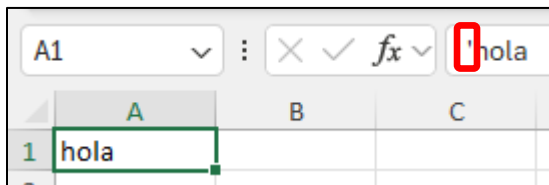
```

4

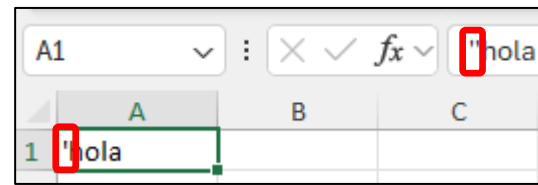
```

1900      #se agrega una comilla simple al principio del valor de la constante si este valor empieza por una comilla simple
1901      #esto es pq en Excel cuando una celda empieza por una comilla simple visualmente no la muestra en pantalla
1902      #(a pesar de que si sale en la barra de formulas)
1903      variable_publica_es_constante_valor = "" + variable_publica_es_constante_valor if variable_publica_es_constante_valor[0:1] == "" else variable_publica_es_constante_valor

```



En la celda aparece como que no empieza por comilla simple, pero en la barra de formulas si aparece



En la celda aparece como que empieza por comilla simple porque en la barra de formulas empieza con 2 comillas simples

ANEXO_09

FUNCIÓN: func_access_df_bbdd_codigo_calculos_varios

7

opcion = LISTA VARIABLES PUBLICAS SI ES CONSTANTE

5

```
1907 variable_publica_tipo = dicc_objetos_vba["VARIABLE_PUBLICA_NORMAL"]["STRING_PARA_LISTADO_EXCEL"]
1908 variable_publica_tipo_dato = None
1909 variable_publica_es_constante = "SI"
1910 variable_publica_definida_usuario_subvariables = None
1911
1912 resultado_funcion.append([variable_publica_nombre
1913                          , variable_publica_tipo
1914                          , variable_publica_tipo_dato
1915                          , variable_publica_es_constante
1916                          , variable_publica_es_constante_valor
1917                          , variable_publica_definida_usuario_subvariables])
```

8

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Permite desde **un conjunto de índices de líneas de código** asociados a una variable pública definida por el usuario (son variables públicas que se declaran en varias líneas) unificar los datos y prepararlos en el mismo formato que las variables públicas normales que se calculan con las opciones LISTA_VARIABLES_PUBLICAS_NO_ES_CONSTANTE y LISTA_VARIABLES_PUBLICAS_SI_ES_CONSTANTE de la presente función.

El resultado de la función es una lista de listas donde cada sublista tiene una estructura común con el resto de las variables públicas normales (al ser comunes a todos los tipos de variables algunos ítems no aplican según el tipo de variable). Estas sublistas, en el caso presente, se componen:

ITEMS SUBLISTAS	APLICA
Nombre de la variable pública	X
Tipo variable pública. Es el valor subkey_2 (STRING_PARA_LISTADO_EXCEL) asociada a key_1 (VARIABLE_PUBLICA_NORMAL) del diccionario dicc_objetos_vba (módulo APP_3_BACK_END_MS_ACCESS)	X
Tipo dato de la variable pública	X
Es constante?	
Valor de la constante (en caso de serlo)	
Sub-variables asociadas a la variable pública definida por el usuario	X

Requiere como parámetros kwargs:

- **lista_variable_definida_por_usuario_ini**
- **lista_lineas_flag_variable_publica**

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

En referencia a los **parámetros kwargs**, enunciados en la página anterior, se explica brevemente a continuación, a modo recordatorio del contenido de otros anexos del presente documento, cuales son sus orígenes para así entender en que formato llegan al usarlos en la presente función y opción que se detalla en este Anexo, antes de proceder a explicar el código de la función.

La función y opción (`LISTA_VARIABLES_PUBLICAS_DEFINIDAS_POR_USUARIO`), que se explica en el presente Anexo, viene como complemento de la función `func_access_df_bbdd_codigo_calculos` (opción `LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS`), ver **ANEXO_08**, para aligerar su código y mejorar su legibilidad.

Esta última se usa en la rutina `def_proceso_access_import_1`, ver **ANEXO_01**.

```
2488 lista_dicc_objetos_variables_publicas = func_access_bbdd_codigo_calculos("LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS", lista_calculos_variables_publicas = lista_calculos_variables_publicas)
```

El parámetro kwargs (`lista_calculos_variables_publicas`) que se usa se calcula también en la rutina `def_proceso_access_import_1`, en un bloque de código anterior, mediante la llamada a la misma función `func_access_df_bbdd_codigo_calculos` pero con la aplicación de otra opción (`LISTA_VARIABLES_PUBLICAS`).

```
2405 lista_calculos_variables_publicas = func_access_bbdd_codigo_calculos("LISTA_VARIABLES_PUBLICAS", df_bbdd_codigo_param = df_bbdd_codigo)
```

La lista generada `lista_calculos_variables_publicas` es una lista de listas donde cada sublista que la compone tiene la estructura siguiente:

ITEMS SUBLISTAS
Tipo de módulo
Nombre de módulo
Indice de la línea de código dentro del dataframe <code>df_bbdd_codigo_param</code> (parámetro de la función)
Indicador de si la línea de código es de declaración de constante o no (para las variables públicas normales) o flag unico identificador para varias líneas de código (para las variables públicas definidas por el usuario)
Línea de código

8

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Antes de proceder a explicar el código de la función, es necesario explicar cuales son los orígenes de los **parámetros kwargs** que se mencionan en la página anterior para así entender con que datos llegan al usarlos en la presente función y opción que se detalla en este anexo.

Para ello, se usa el ejemplo siguiente (es también el que se usa más adelante cuando se explica el funcionamiento de la función).

Modulo_PRUEBA_1

```

00 Option Compare Database
01 Option Explicit
02 Option Base 1
03
04 'VARIABLES PUBLICAS
05 Public Public_Version As String
06
07 Global Public_Var_1 As String, Public_Var_2 As String, _
08 Public_Array(1 To 10, 2) As Variant, Public_Var_3 As Date, _
09 Public_Var_4 As Variant
10
11 Public Const Public_Const_1 = "aquí hay comas , y signos = "
12
13 Global Const Public_Const_2 = "@1", _
14 Public_Const_3 = 8, _
15 Public_Const_4 = "@2", _
16 Public_Const_5 = 5
17
18 Type UserType_Prueba_1 'variable publica definida por el usu
19     SubVar_1 As String
20     SubVar_2 As Long
21 End Type
22
23 Enum Enum_Prueba_1
24     SubVar_1 'sub-variable de otra variable publica definida
25     SubVar_2 = 1
26     SubVar_3
27 End Enum
28
29
30 Sub ActivarPanelNavegacion()
31
32 On Error Resume Next
33
34 DoCmd.LockNavigationPane (False)
35 DoCmd.SelectObject acTable, "MSysObjects", True
36 DoCmd.RunCommand acCmdWindowUnhide
37 MsgBox "fsdfsdf"
38
39 End Sub

```



Variables públicas normales



Variables públicas definidas por el usuario

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

En referencia a los **parámetros kwargs** mencionados, se explica brevemente a continuación, a modo recordatorio del contenido de otros anexos del presente documento, cuales son sus orígenes para así entender en que formato llegan al usarlos en la presente función y opción que se detalla en este Anexo.

En la rutina **def_proceso_access_import_1**, ver **ANEXO_01**, se calcula una lista de variables públicas, mediante la llamada a la función **func_access_df_bbdd_codigo_calculos** (opción **LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS**), ver **ANEXO_08**.

```
2404 lista_calculos_variables_publicas = func_access_bbdd_codigo_calculos("LISTA_VARIABLES_PUBLICAS", df_bbdd_codigo_param = df_bbdd_codigo)
```

Esta lista calculada **lista_calculos_variables_publicas** aplica para todos los tipos de variables públicas (sean normales o definidas por el usuario) y permite disponer de todas las líneas de código relacionadas con la declaración de variables públicas. Es lista de lista donde cada una de sus sublistas tiene la estructura siguiente:

ITEMS SUBLISTAS
Tipo de módulo
Nombre de módulo
Indice de la línea de código dentro del dataframe df_bbdd_codigo_param (parámetro de la función)
Indicador de si la línea de código es de declaración de constante o no (para las variables públicas normales) o flag unico identificador para varias lineas de bódigo (para las variables públicas definidas por el usuario)
Línea de código

Usando el ejemplo:

```
[ 'Estandar', 'Modulo_PRUEBA_1', 5, 'NO_ES_CONSTANTE', 'Public Public_Version As String'],
[ 'Estandar', 'Modulo_PRUEBA_1', 11, 'SI_ES_CONSTANTE', 'Public Const Public_Const_1 = "aqui hay comas , y signos = "',
[ 'Estandar', 'Modulo_PRUEBA_1', 7, 'NO_ES_CONSTANTE', 'Global Public Var_1 As String, Public Var_2 As String,'],
[ 'Estandar', 'Modulo_PRUEBA_1', 8, 'NO_ES_CONSTANTE', 'Public Array(1 To 10, 2) As Variant, Public_Var_3 As Date,'],
[ 'Estandar', 'Modulo_PRUEBA_1', 9, 'NO_ES_CONSTANTE', 'Public_Var_4 As Variant'],
[ 'Estandar', 'Modulo_PRUEBA_1', 13, 'SI_ES_CONSTANTE', 'Global Const Public_Const_2 = "@1",'],
[ 'Estandar', 'Modulo_PRUEBA_1', 14, 'SI_ES_CONSTANTE', 'Public_Const_3 = 8,'],
[ 'Estandar', 'Modulo_PRUEBA_1', 15, 'SI_ES_CONSTANTE', 'Public_Const_4 = "@2",'],
[ 'Estandar', 'Modulo_PRUEBA_1', 16, 'SI_ES_CONSTANTE', 'Public_Const_5 = 5'],
```

```
[ 'Estandar', 'Modulo_PRUEBA_1', 18, 'DEFINIDA_POR_USUARIO_1', 'Type UserType_Prueba_1'],
[ 'Estandar', 'Modulo_PRUEBA_1', 19, 'DEFINIDA_POR_USUARIO_1', 'SubVar_1 As String'],
[ 'Estandar', 'Modulo_PRUEBA_1', 20, 'DEFINIDA_POR_USUARIO_1', 'SubVar_2 As Long'],
[ 'Estandar', 'Modulo_PRUEBA_1', 21, 'DEFINIDA_POR_USUARIO_1', 'End Type'],
[ 'Estandar', 'Modulo_PRUEBA_1', 23, 'DEFINIDA_POR_USUARIO_2', 'Enum Enum_Prueba_1'],
[ 'Estandar', 'Modulo_PRUEBA_1', 24, 'DEFINIDA_POR_USUARIO_2', 'SubVar_1'],
[ 'Estandar', 'Modulo_PRUEBA_1', 25, 'DEFINIDA_POR_USUARIO_2', 'SubVar_2 = 1'],
[ 'Estandar', 'Modulo_PRUEBA_1', 26, 'DEFINIDA_POR_USUARIO_2', 'SubVar_3'],
[ 'Estandar', 'Modulo_PRUEBA_1', 27, 'DEFINIDA_POR_USUARIO_2', 'End Enum'],
```

ANEXO_09

FUNCIÓN: func_access_df_bbdd_codigo_calculos_varios

8

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Esta lista calculada `lista_calculos_variables_publicas`, más adelante en la rutina `def_proceso_access_import_1`, se usa como parámetro kwargs de la función `func_access_df_bbdd_codigo_calculos` (opción `LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS`).

```
2487 lista_dicc_objetos_variables_publicas = func_access_bbdd_codigo_calculos("LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS", lista_calculos_variables_publicas = lista_calculos_variables_publicas)
```

En dicha función y opción, su uso para las variables públicas definidas por el usuario, es el siguiente:

```
986 lista_modulos_con_variables_publicas_definidas_usuario = [[tipo_modulo, nombre_modulo] for tipo_modulo, nombre_modulo, indice_linea, flag_variable_publica, linea_codigo in lista_calculos_variables_publicas
987 if "DEFINIDA_POR_USUARIO" in flag_variable_publica]
988
989 lista_modulos_con_variables_publicas_definidas_usuario = [sublista for i, sublista in enumerate(lista_modulos_con_variables_publicas_normales)
990 if sublista not in lista_modulos_con_variables_publicas_normales[:i]]
```



```
lista_modulos_con_variables_publicas_definidas_usuario
[['Estandar', 'Modulo_PRUEBA_1']]
```

```
995 for tipo_modulo_iteracion, nombre_modulo_iteracion in lista_modulos_con_variables_publicas_definidas_usuario:
996
997     #se recuperan los distintos flags de este tipo de variable publica dentro del modulo de la iteracion donde se quitan los duplicados
998     lista_variables_publicas_definidas_usuario_modulo = [flag_variable_publica
999 for tipo_modulo, nombre_modulo, indice_linea, flag_variable_publica, linea_codigo in lista_calculos_variables_publicas
1000 if tipo_modulo == tipo_modulo_iteracion and nombre_modulo == nombre_modulo_iteracion and "DEFINIDA_POR_USUARIO" in flag_variable_publica]
1001
1002 lista_variables_publicas_definidas_usuario_modulo = list(dict.fromkeys(lista_variables_publicas_definidas_usuario_modulo))
```



```
lista_variables_publicas_definidas_usuario_modulo
['DEFINIDA_POR_USUARIO_1', 'DEFINIDA_POR_USUARIO_2']
```


ANEXO_09

FUNCIÓN: func_access_df_bbdd_codigo_calculos_varios

8

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Función y opción que se explica en el presente Anexo

```
1008 for flag_variable_publica_modulo in lista_variables_publicas_definidas_usuario_modulo
1009
1010     lista_lineas_flag_variable_publica = [[indice_linea, linea_codigo]
1011                                           for tipo_modulo, nombre_modulo, indice_linea, flag_variable_publica, linea_codigo in lista_calculos_variables_publicas
1012                                           if tipo_modulo == tipo_modulo_iteracion and nombre_modulo == nombre_modulo_iteracion and flag_variable_publica == flag_variable_publica_modulo]
1013
1014     lista_lineas_flag_variable_publica = sorted(lista_lineas_flag_variable_publica, key = lambda x: (x[0]))
1015
1016     lista_variables_publicas_1_a_1 = func_access_bbdd_codigo_calculos_varios("LISTA_VARIABLES_PUBLICAS_DEFINIDAS_POR_USUARIO"
1017                                                                           , lista_variable_definida_por_usuario_ini = lista_variable_definida_por_usuario_ini
1018                                                                           , lista_lineas_flag_variable_publica = lista_lineas_flag_variable_publica)
```

DEFINIDA_POR_USUARIO_1

lista_lineas_flag_variable_publica
[[18, 'Type UserType_Prueba_1'], [19, 'SubVar_1 As String'], [20, 'SubVar_2 As Long'], [21, 'End Type']]

DEFINIDA_POR_USUARIO_2

lista_lineas_flag_variable_publica
[[23, 'Enum Enum_Prueba_1'], [24, 'SubVar_1'], [25, 'SubVar_2 = 1'], [26, 'SubVar_3'], [27, 'End Enum']]

La lista **lista_lineas_flag_variable_publica**, parámetro kwargs de la función y opción del presente anexo, como lo ilustra el ejemplo usado es una lista de listas que funciona para una sola variable pública definida por el usuario a la vez y cada una de las sublistas que la componen están ordenadas por el índice de código de menor a mayor e informa de las líneas de códigos asociadas.

El otro parámetro kwargs que usa la función y opción del presente Anexo (**lista_variable_definida_por_usuario_ini**) se define en la función **func_access_df_bbdd_codigo_calculos** (opción LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS) como ajuste previo al cálculo de la misma.

```
2487 lista_dicc_objetos_variables_publicas = func_access_bbdd_codigo_calculos("LISTA_DICC_OBJETOS_VARIABLES_PUBLICAS", lista_calculos_variables_publicas = lista_calculos_variables_publicas)
```

```
904 lista_variable_definida_por_usuario_ini = [item.strip() + " " for item in dicc_objetos_vba["VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO"]["LISTA_DECLARACION_INICIO"]]
```

['Type ', 'Enum ']

8

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Explicado ya el origen de los parámetros kwargs de la presente función, se pasa a detallar su funcionamiento.

El 1er paso es extraer del parámetro **lista_lineas_flag_variable_publica** la línea de código correspondiente a la 1era sublista para poder calcular el nombre y tipo de dato de la variable pública. Se crea también una lista **lista_lineas_flag_variable_publica_para_subvariables** que contiene todas las sublistas salvo la 1era (inicio de declaración) y la última (fin de declaración) para poder disponer de las líneas de código asociadas a las sub-variables.

```
1939 primer_item_lista_lineas_flag_variable_publica = lista_lineas_flag_variable_publica[0][1]
1940 lista_lineas_flag_variable_publica_para_subvariables = lista_lineas_flag_variable_publica[1:-1]
```

```
DEFINIDA_POR_USUARIO_1
lista_lineas_flag_variable_publica
[[18, 'Type UserType Prueba 1'], [19, 'SubVar 1 As String'], [20, 'SubVar 2 As Long'], [21, 'End Type']]
```

```
DEFINIDA_POR_USUARIO_2
lista_lineas_flag_variable_publica
[[23, 'Enum Enum Prueba 1'], [24, 'SubVar_1'], [25, 'SubVar_2 = 1'], [26, 'SubVar_3'], [27, 'End Enum']]
```

Para calcular el nombre de la variable y su tipo de dato se usa la 1era sublista del parámetro **lista_lineas_flag_variable_publica**. El parámetro **lista_variable_definida_por_usuario_ini** se usa para determinar el tipo de dato usado en la línea de código ("Type", "Enum", etc etc) y hacer el replace en la misma para obtener, previo trimeo, el nombre de la variable pública.

```
1945 variable_publica_nombre = None
1946 variable_publica_tipo_dato = None
1947
1948 for item_config in lista_variable_definida_por_usuario_ini:
1949
1950     if item_config in primer_item_lista_lineas_flag_variable_publica:
1951         variable_publica_nombre = primer_item_lista_lineas_flag_variable_publica.replace(item_config, "").strip()
1952         variable_publica_tipo_dato = item_config.strip()
1953         break
```



```
variable_publica_nombre UserType_Prueba_1
variable_publica_tipo_dato Type
```

```
variable_publica_nombre Enum_Prueba_1
variable_publica_tipo_dato Enum
```

8

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

El siguiente paso es concatenar las sub-variables que componen la variable pública poniendo para cada una de sus sub-variables, a continuación del nombre de las mismas, entre paréntesis su tipo de dato o su valor si uno de los 2 esta informado en el código VBA.

Se realiza mediante bucle sobre la lista **lista_lineas_flag_variable_publica_para_subvariables** spliteando cada item de la iteración por “As” y por el signo “=” para quedarse con la lista de las 2 listas resultantes que tiene mayor longitud y dentro de esta lista seleccionada se recupera el último item de la misma que se pone entre paréntesis (si esta informado) y se asocia al nombre de la sub-variable.

```

1959 for ind, item in enumerate(lista_lineas_flag_variable_publica_para_subvariables):
1960
1961     #tan solo se extrae la linea de codigo (2ndo item de cada sublista de lista_lineas_flag_variable_publica_para_subvariables)
1962     linea_codigo = item[1]
1963
1964
1965     #se splitea la linea de codigo por " As " y por "=" por separado
1966     #las 2 listas no pueden tener conjuntamente longitud = 2 por lo que el valor entre parentesis
1967     #a continuacion del nombre de la subvariable viene del item 2 de la lista que tenga longitud 2
1968     #(en caso que las 2 listas tengan longitud = 1 se coje el 1er item de la lista spliteada por " As "
1969     #es el mismo que el de la lista spliteada por "=" )
1970     linea_codigo_spliteada_por_as = linea_codigo.split(indicador_variable_vba_declaracion_tipo_dato)
1971     linea_codigo_spliteada_por_signo_igual = linea_codigo.split(indicador_variable_signo_igual)
1972
1973     sub_variable_iteracion = ""
1974     if len(linea_codigo_spliteada_por_as) == 1 and len(linea_codigo_spliteada_por_signo_igual) == 1:
1975         sub_variable_iteracion = linea_codigo_spliteada_por_as[0].strip()
1976
1977     elif len(linea_codigo_spliteada_por_as) == 2:
1978         sub_variable_iteracion = linea_codigo_spliteada_por_as[0].strip() + " (" + linea_codigo_spliteada_por_as[1].strip() + ")"
1979
1980     elif len(linea_codigo_spliteada_por_signo_igual) == 2:
1981         sub_variable_iteracion = linea_codigo_spliteada_por_signo_igual[0].strip() + " (" + linea_codigo_spliteada_por_signo_igual[1].strip() + ")"
1982
1983     variable_publica_definida_usuario_subvariables = sub_variable_iteracion if ind == 0 else variable_publica_definida_usuario_subvariables + ", " + sub_variable_iteracion

```

```

variable_publica_nombre      UserType_Prueba_1

[19, 'SubVar_1 As String']
*****
linea_codigo_spliteada_por_as      ['SubVar_1', 'String']
linea_codigo_spliteada_por_signo_igual  ['SubVar_1 As String']
variable_publica_definida_usuario_subvariables  SubVar_1 (String)

[20, 'SubVar_2 As Long']
*****
linea_codigo_spliteada_por_as      ['SubVar_2', 'Long']
linea_codigo_spliteada_por_signo_igual  ['SubVar_2 As Long']
variable_publica_definida_usuario_subvariables  SubVar_1 (String), SubVar_2 (Long)

```

```

variable_publica_nombre      Enum_Prueba_1

[24, 'SubVar_1']
*****
linea_codigo_spliteada_por_as      ['SubVar_1']
linea_codigo_spliteada_por_signo_igual  ['SubVar_1']
variable_publica_definida_usuario_subvariables  SubVar_1

[25, 'SubVar_2 = 1']
*****
linea_codigo_spliteada_por_as      ['SubVar_2 = 1']
linea_codigo_spliteada_por_signo_igual  ['SubVar_2', '1']
variable_publica_definida_usuario_subvariables  SubVar_1, SubVar_2 (1)

[26, 'SubVar_3']
*****
linea_codigo_spliteada_por_as      ['SubVar_3']
linea_codigo_spliteada_por_signo_igual  ['SubVar_3']
variable_publica_definida_usuario_subvariables  SubVar_1, SubVar_2 (1), SubVar_3

```

ANEXO_09

FUNCIÓN: func_access_df_bbdd_codigo_calculos_varios

8

opcion = LISTA VARIABLES PUBLICAS DEFINIDAS POR USUARIO

Por último, se agregan los datos calculados en forma de sublista a la lista **resultado_funcion**.

```
1987     variable_publica_tipo = dicc_objetos_vba["VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO"]["STRING_PARA_LISTADO_EXCEL"]
1988     variable_publica_es_constante = "-"
1989     variable_publica_es_constante_valor = None
1990
1991     resultado_funcion.append([variable_publica_nombre
1992                             , variable_publica_tipo
1993                             , variable_publica_tipo_dato
1994                             , variable_publica_es_constante
1995                             , variable_publica_es_constante_valor
1996                             , variable_publica_definida_usuario_subvariables])
```

```
['UserType_Prueba_1', 'VARIABLE PUBLICA DEFINIDA POR USUARIO', 'Type', '-', None, 'SubVar_1 (String), SubVar_2 (Long)']
```

```
['Enum_Prueba_1', 'VARIABLE PUBLICA DEFINIDA POR USUARIO', 'Enum', '-', None, 'SubVar_1, SubVar_2 (1), SubVar_3']
```

ANEXO_10

FUNCIÓN

func_diagnostico_access_tablas_codigo_preservar

La función **`func_diagnostico_access_tablas_codigo_preservar`** permite preservar en una línea de código tan solo lo necesario para poder realizar la búsqueda del uso de tablas o vínculos dentro del código VBA.

Tiene por parámetros:

- Opción `PRESERVAR_SI` o `PRESERVAR_NO`
- la línea de código

Cuando se aplica la función en la rutina **`def_proceso_access_1_import`** (módulo `APP_3_BACK_END_MS_ACCESS`), se pasa como parámetro la línea de código en mayúsculas, para evitar casos de case sensitive en la búsqueda de dependencias, a la cual se le quitan todos los comentarios para limitar la búsqueda de dependencias a código activo.

Esto se hace porque las tablas o vínculos cuando se usan en código VBA suelen venir encapsuladas en string dentro de comillas dobles y la función elimina del código todo lo que no este encapsulado dentro de comillas dobles, pero con excepciones.

Cuando se usan tablas o vínculos en código VBA suele ser en 2 contextos:

- Dentro de sentencias SQL
- Precedidos de una instrucción VBA de manipulación de tabla

En el diccionario **`dicc_diagnostico_tablas_access_donde_buscar`** (módulo `APP_3_BACK_END_MS_ACCESS`) se listan todos los tipos de string que se conservan en el código acorde a los 2 contextos mencionados.

ANEXO_10

FUNCIÓN: func_diagnostico_access_tablas_codigo_preservar

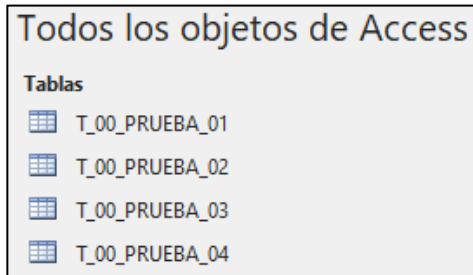
En el diccionario **dicc_diagnostico_tablas_access_donde_buscar** (módulo APP_3_BACK_END_MS_ACCESS) se listan todos los tipos de string que se conservan en el código acorde a los 2 contextos mencionados.

```
143 dicc_diagnostico_tablas_access_donde_buscar = {
144     "LISTA_TEXTOS_SIMILARES_SENTENCIAS_SQL":
145     [ "SELECT", "FROM", "LEFT JOIN", "RIGHT JOIN", "INNER JOIN", "GROUP BY", "ON", "SET", "WHERE", "HAVING",
146       "CREATE TABLE", "DELETE", "DROP TABLE", "INSERT INTO", "UPDATE" ]
147     #es una lista de instrucciones SQL para preservar el codigo incluido entre comillas dobles "" y que se parezcan a sentencias SQL
148     #(es indiferente poner en mayusculas ominusculas, el chequeo se hace poniendo tanto el codigo VBA de las rutinas como los items de la
149     #lista de esta key en mayusculas)
150
151     , "LISTA_TEXTOS_INSTRUCCIONES_VBA_MANIP_TABLAS":
152     [ r"OpenRecordset(", r"DoCmd.OpenTable(acTable,", "DoCmd.CloseacTable,",
153       r"DoCmd.TransferSpreadsheet(acImport, acSpreadsheetTypeExcel12,")
154     #es una lista de instrucciones VBA de uso de tablas / vinculos, si se localizan en la linea de codigo se preserva
155     #lo incluido entre comillas dobles "" inmediatamente posterior a la instruccion VBA
```

ANEXO_10

FUNCIÓN: func_diagnostico_access_tablas_codigo_preservar

Antes de empezar a explicar la función, es conveniente explicar el porque se ha de preservar tan solo una parte del código VBA y no su totalidad con un ejemplo con 4 tablas y una rutina VBA.



Nombres de tablas usados en sentencias SQL

Nombre de tablas usados en sentencias VBA de manipulación de tablas

Nombre de tablas usados en declaraciones de valor de variables

Nombre de variables (o parámetros) con nombre igual al de tablas

```
1 Sub Calc_PRUEBA(xOpcion As String, xValor_1 As String, xValor_2 As String)
2
3 Dim MiConexion As New ADODB.Connection
4 Dim MiRecordset1 As New ADODB.Recordset
5 Dim MiSQL As String
6 Dim T_00_PRUEBA_01 As String
7 Dim xTemp As String
8
9 Set MiConexion = CurrentProject.Connection
10
11 T_00_PRUEBA_01 = "AAA"
12
13 xTemp = "T_00_PRUEBA_02"
14
15 If xOpcion = "OPCION 1" Then
16     MiSQL = "UPDATE [T_00_PRUEBA_03] SET Campo_ID = '" & xValor_1 & "'"
17
18 ElseIf xOpcion = "OPCION 2" Then
19     MiSQL = "UPDATE [" & xTemp & "] SET Campo_ID = '" & xValor_2 & "'"
20
21 End If
22
23 MiConexion.Execute (MiSQL)
24
25 DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
26
27 MiConexion.Close
28 Set MiConexion = Nothing
29
30 End Sub
```


ANEXO_10

FUNCIÓN: func_diagnostico_access_tablas_codigo_preservar

Si no se realizaran estos ajustes, el diagnostico de dependencias diría que las 4 tablas se usan en la rutina VBA Calc_PRUEBA del ejemplo, cuando en realidad tan solo las tablas T_00_PRUEBA_02, T_00_PRUEBA_03 y T_00_PRUEBA_04 se usan.

- **T_00_PRUEBA_01.** No se usa porque el nombre de la tabla es en realidad el nombre de una variable local.
- **T_00_PRUEBA_02.** Se usa porque el nombre de la tabla corresponde al valor de una variable local que luego se usa en una sentencia SQL.
- **T_00_PRUEBA_03.** Se usa porque el nombre de la tabla se usa en una sentencia SQL.
- **T_00_PRUEBA_04.** Se usa porque el nombre de la tabla esta encapsulada en un string entre comillas dobles el cual viene precedido de una sentencia VBA de manipulación de tablas.

La función **func_diagnostico_access_tablas_codigo_preservar** permite:

- Descartar la tabla **T_00_PRUEBA_01**.
- Localizar como dependencias de uso las tablas **T_00_PRUEBA_03** y **T_00_PRUEBA_04**.

No permite directamente localizar la tabla **T_00_PRUEBA_02** como dependencia, pero si ayudar al usuario a determinar si el considera que se trata de una dependencia de uso. Esto se realiza en el Excel de diagnostico de dependencias (hoja "TABLAS (CHECK MANUAL)").

1	TIPO_TABLA	NOMBRE_TABLA	TIPO_MODULO	NOMBRE_MODULO	NOMBRE Rutina	NUMERO LINEA CODIGO	CODIGO LINEA
2	TABLA_LOCAL	T_01_ProveedComerc	Estandar	Mod_00_Config	ActivarPanelNavegacion	5	xTabla = "T_01_ProveedComerc"
3	TABLA_LOCAL	T_Extraccion_Actual	Estandar	Mod_00_Config	Config_CalculosUpDateSQL	10	xTabla = "T_Extraccion_Actual"
4	TABLA_LOCAL	T_Extraccion_Actual	Estandar	Mod_00_Config	Config_CalculosUpDateSQL_Estado	48	xTabla2 = "T_Extraccion_Actual"

ANEXO_10

FUNCIÓN: func_diagnostico_access_tablas_codigo_preservar

Los casos relativos a las tablas T_00_PRUEBA_01, T_00_PRUEBA_03 y T_00_PRUEBA_04 se realizan mediante el parámetro **opcion** igual a PRESERVAR_SI. El caso de la tabla T_00_PRUEBA_02 se realiza mediante el parámetro **opcion** igual a PRESERVAR_NO.

La función realiza los pasos siguientes:

PASO	DESCRIPCIÓN
1	Para la línea de código se agrega a una lista temporal todo lo encapsulado entre comillas dobles (en adelante lista_1)
2	Se localizan los items de lista_1 que son similares a sentencias SQL y se agregan a otra lista temporal (en adelante lista_2)
3	Se localizan los items de lista_1 que vienen en código VBA precedidos por instrucciones VBA de manipulación de tablas y se agregan a otra lista temporal (en adelante lista_3)
4	Se fusionan las listas lista_2 y lista_3 en otra lista temporal (en adelante lista_4). Los items de lista_1 que no se encuentran en lista_4 pasan a almacenarse en otra lista temporal (en adelante lista_5)
5	El resultado de la función para la opción PRESERVAR_Si es la concatenación en un mismo string de los items de lista_4. Para la opción PRESERVAR_NO, es lo mismo pero con lista_5

ANEXO_10

FUNCIÓN: func_diagnostico_access_tablas_codigo_preservar

1

Localización de lo encapsulado entre comillas dobles

Se realiza mediante la librería Python `re` y se crea la lista temporal `lista_string_entre_comillas_dobles` (en adelante `lista_1`).

```
2019 lista_string_entre_comillas_dobles = re.findall(r'"([^\"]*)"', linea_codigo)
2020 lista_string_entre_comillas_dobles = [i for i in lista_string_entre_comillas_dobles if len(i) != 0] if isinstance(lista_string_entre_comillas_dobles, list) else []
```

En el ejemplo mencionado en páginas anteriores:

```
CODIGO: T_00_PRUEBA_01 = "AAA"
LISTA: ['AAA']
*****
CODIGO: xTemp = "T_00_PRUEBA_02"
LISTA: ['T_00_PRUEBA_02']
*****
CODIGO: If xOpcion = "OPCION_1" Then
LISTA: ['OPCION_1']
*****
CODIGO: MiSQL = "UPDATE [T_00_PRUEBA_03] SET Campo_ID = '" & xValor_1 & "'"
LISTA: ["UPDATE [T_00_PRUEBA_03] SET Campo_ID = '", "'"]
*****
CODIGO: ElseIf xOpcion = "OPCION_2" Then
LISTA: ['OPCION_2']
*****
CODIGO: MiSQL = "UPDATE ['" & xTemp & "] SET Campo_ID = '" & xValor_2 & "'"
LISTA: ['UPDATE [' , "]" SET Campo_ID = '", "'"]
*****
CODIGO: DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
LISTA: ['T_00_PRUEBA_04']
*****
```

2

Localización ítems de lista_string_entre_comillas_dobles similares a entencias SQL

Se crea la lista temporal **lista_SI_preservar_codigo_sql** (en adelante lista_2) mediante bucle sobre lista_1 (en adelante bucle_1).

Para cada iteración sobre bucle_1 se realiza otro bucle (en adelante bucle_2) sobre la lista almacenada en la key LISTA_TEXTOS_SIMILARES_SENTENCIAS_SQL del diccionario **dicc_diagnostico_tablas_access_donde_buscar**.

Para evitar casos de case sensitive, tanto el ítem de la iteración sobre bucle_1 (en adelante ítem_1) como el ítem de la iteración sobre bucle_2 (en adelante ítem_2) se ponen en mayúsculas y se trimean.

El matcheo entre ítem_1 y ítem_2 se puede realizar de 2 formas según como este configurado ítem_2:

- **Es una palabra sola.** El matcheo se realiza aquí por la búsqueda de la palabra suelta (excluyendo los casos donde la palabra esta incluida en otra). Para ello, se usa la librería Python re.
- **Son varias palabras** (con espacios en blanco intermedios). El matcheo se realiza por una búsqueda tipo "contiene".

Para localizar que caso toca, se splitea ítem_2 por el espacio en blanco y con la longitud de la lista resultante se determina que caso aplica.

ANEXO_10

FUNCIÓN: func_diagnostico_access_tablas_codigo_preservar

2

Localización ítems de lista_string_entre_comillas_dobles similares a sentencias SQL

En el diccionario **dicc_diagnostico_tablas_access_donde_buscar**, estos casos son:

```
[ 'SELECT', 'FROM', 'LEFT JOIN', 'RIGHT JOIN', 'INNER JOIN', 'GROUP BY', 'ON', 'SET', 'WHERE', 'HAVING', 'CREATE TABLE', 'DELETE', 'DROP TABLE', 'INSERT INTO', 'UPDATE' ]
*****
UNA SOLA PALABRA:
SELECT
FROM
ON
SET
WHERE
HAVING
DELETE
UPDATE
*****
VARIAS PALABRAS:
LEFT JOIN
RIGHT JOIN
INNER JOIN
GROUP BY
CREATE TABLE
DROP TABLE
INSERT INTO
```

```
2025 lista_SI_preservar_codigo_sql = []
2026 for item_codigo in lista_string_entre_comillas_dobles:
2027
2028     item_codigo_mayusc_trimeado = item_codigo.upper().strip()
2029
2030     for item_config in dicc_diagnostico_tablas_access_donde_buscar["LISTA_TEXTOS_SIMILARES_SENTENCIAS_SQL"]:
```

2

Localización ítems de lista_string_entre_comillas_dobles similares a sentencias SQL

```

2037 item_config_mayusc_trimeado = item_config.upper().strip()
2038 item_config_split_espacio_blanco_intermedio = item_config_mayusc_trimeado.split(" ")
2039
2040 check_item_config = 0
2041 if len(item_config_split_espacio_blanco_intermedio) == 1:
2042
2043     check_si_item_config_es_palabra_sola = fr'(?<![\w_-]){re.escape(item_config_mayusc_trimeado)}(?![\w-])'
2044     check_si_item_config_es_palabra_sola = bool(re.search(check_si_item_config_es_palabra_sola, item_codigo_mayusc_trimeado))
2045
2046     if check_si_item_config_es_palabra_sola:
2047         check_item_config = 1
2048
2049     else:
2050         if item_config_mayusc_trimeado in item_codigo_mayusc_trimeado:
2051             check_item_config = 1
2052
2053     if check_item_config == 1:
2054         lista_SI_preservar_codigo_sql.append(item_codigo)
2055         break

```

En el ejemplo mencionado en páginas anteriores, la lista **lista_SI_preservar_codigo_sql** queda así:

```

CODIGO      MiSQL = "UPDATE [T_00_PRUEBA_03] SET Campo_ID = '' & xValor_1 & ''"
LISTA      ["UPDATE [T_00_PRUEBA_03] SET Campo_ID = '"]
*****
CODIGO      MiSQL = "UPDATE [' & xTemp & '] SET Campo_ID = '' & xValor_2 & ''"
LISTA      ['UPDATE [' , "]" SET Campo_ID = '"]
*****

```

3 Localización ítems de lista_string_entre_comillas_dobles similares a instrucciones VBA de manipulación de tablas

Como pasos previos, se crean las listas temporales.

LISTA	DESCRIPCIÓN
lista_indices_localiz_comillas_dobles	es una lista de tuplas donde cada tupla contiene el índice de inicio y el de fin de lo incluido entre comillas dobles "" (metodo zip)
lista_indices_fin_instruccion_vba	es una lista que contiene el índice de fin de la instruccion VBA de todos los items de la lista contenida en la key LISTA_TEXTOS_INSTRUCCIONES_VBA_MANIP_TABLAS del diccionario dicc_diagnostico_tablas_access_donde_buscar

El primer ajuste para ambas listas es poner la línea de código (parámetro de la función).

```
2070 linea_codigo_mayusc = linea_codigo.upper()
```

lista_indices_localiz_comillas_dobles

Se listan todos los índices en la línea de código donde aparecen comillas dobles en la lista temporal **lista_indices_localiz_comillas_dobles_temp_2**.

```
2073 lista_indices_localiz_comillas_dobles_temp_1 = list(re.finditer("\"", linea_codigo_mayusc))
2074 lista_indices_localiz_comillas_dobles_temp_2 = [lista_indices_localiz_comillas_dobles_temp_1[ind].start() for ind, item in enumerate(lista_indices_localiz_comillas_dobles_temp_1)]
```

Los ítems de esta lista que corresponden con índices de posición de apertura de comillas dobles corresponden a los índices de lista (aquí me refiero a los índices dentro de la lista temporal mencionada no a los índices dentro de la línea de código) de **número par** (los índices de lista empiezan 0 no por 1). Los índices de posición de cierre de comillas dobles son los índices de lista de **número impar**.

3

Localización ítems de lista_string_entre_comillas_dobles similares a sentencias VBA de manipulación de tablas

lista_indices_localiz_comillas_dobles

```

2076 lista_indices_localiz_comillas_dobles_temp_3_1 = [item for ind, item in enumerate(lista_indices_localiz_comillas_dobles_temp_2) if int(ind / 2) == ind / 2]
2077 lista_indices_localiz_comillas_dobles_temp_3_2 = [item for ind, item in enumerate(lista_indices_localiz_comillas_dobles_temp_2) if int(ind / 2) != ind / 2]
2078 lista_tuplas_indices_localiz_comillas_dobles = list(zip(lista_indices_localiz_comillas_dobles_temp_3_1, lista_indices_localiz_comillas_dobles_temp_3_2))

```

En el ejemplo mencionado en páginas anteriores:

```

CODIGO T_00_PRUEBA_01 = "AAA"
lista_indices_localiz_comillas_dobles_temp_2 [17, 21]
lista_indices_localiz_comillas_dobles_temp_3_1 [17]
lista_indices_localiz_comillas_dobles_temp_3_2 [21]
lista_tuplas_indices_localiz_comillas_dobles [(17, 21)]
*****
CODIGO xTemp = "T_00_PRUEBA_02"
lista_indices_localiz_comillas_dobles_temp_2 [8, 23]
lista_indices_localiz_comillas_dobles_temp_3_1 [8]
lista_indices_localiz_comillas_dobles_temp_3_2 [23]
lista_tuplas_indices_localiz_comillas_dobles [(8, 23)]
*****
CODIGO If xOpcion = "OPCION_1" Then
lista_indices_localiz_comillas_dobles_temp_2 [13, 22]
lista_indices_localiz_comillas_dobles_temp_3_1 [13]
lista_indices_localiz_comillas_dobles_temp_3_2 [22]
lista_tuplas_indices_localiz_comillas_dobles [(13, 22)]
*****
CODIGO MiSQL = "UPDATE [T_00_PRUEBA_03] SET Campo_ID = '" & xValor_1 & "'"
lista_indices_localiz_comillas_dobles_temp_2 [9, 50, 65, 67]
lista_indices_localiz_comillas_dobles_temp_3_1 [9, 65]
lista_indices_localiz_comillas_dobles_temp_3_2 [50, 67]
lista_tuplas_indices_localiz_comillas_dobles [(9, 50), (65, 67)]
*****

```

```

CODIGO ElseIf xOpcion = "OPCION_2" Then
lista_indices_localiz_comillas_dobles_temp_2 [17, 26]
lista_indices_localiz_comillas_dobles_temp_3_1 [17]
lista_indices_localiz_comillas_dobles_temp_3_2 [26]
lista_tuplas_indices_localiz_comillas_dobles [(17, 26)]
*****
CODIGO MiSQL = "UPDATE [" & xTemp & "] SET Campo_ID = '" & xValor_2 & "'"
lista_indices_localiz_comillas_dobles_temp_2 [9, 18, 30, 49, 64, 66]
lista_indices_localiz_comillas_dobles_temp_3_1 [9, 30, 64]
lista_indices_localiz_comillas_dobles_temp_3_2 [18, 49, 66]
lista_tuplas_indices_localiz_comillas_dobles [(9, 18), (30, 49), (64, 66)]
*****
CODIGO DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
lista_indices_localiz_comillas_dobles_temp_2 [25, 40]
lista_indices_localiz_comillas_dobles_temp_3_1 [25]
lista_indices_localiz_comillas_dobles_temp_3_2 [40]
lista_tuplas_indices_localiz_comillas_dobles [(25, 40)]
*****

```


3 Localización ítems de lista_string_entre_comillas_dobles similares a instrucciones VBA de manipulación de tablas

lista_indices_fin_instruccion_vba

Dentro de la línea de código, mediante bucle sobre la lista almacenada en la key LISTA_TEXTOS_INSTRUCCIONES_VBA_MANIP_TABLAS del diccionario **dicc_diagnostico_tablas_access_donde_buscar**, se buscan las instrucciones VBA de manipulación de tablas y más concretamente los índices de posición dentro de la línea de código de **fin de instrucción** de las mismas y se agregan a la lista temporal **lista_indices_fin_instruccion_vba**.

```

2087 lista_indices_fin_instruccion_vba = []
2088 for item_config in dicc_diagnostico_tablas_access_donde_buscar["LISTA_TEXTOS_INSTRUCCIONES_VBA_MANIP_TABLAS"]:
2089
2090     item_config_mayusc_trimeado = item_config.upper().strip()
2091
2092     lista_indices_fin_instruccion_vba_temp = list(re.finditer(item_config_mayusc_trimeado, linea_codigo_mayusc))
2093
2094     if len(lista_indices_fin_instruccion_vba_temp) != 0:
2095         for ind, item in enumerate(lista_indices_fin_instruccion_vba_temp):
2096             lista_indices_fin_instruccion_vba.append(lista_indices_fin_instruccion_vba_temp[ind].end())
2097
2098     del lista_indices_fin_instruccion_vba_temp
2099
2100 lista_indices_fin_instruccion_vba = sorted(lista_indices_fin_instruccion_vba)

```

En el ejemplo mencionado en páginas anteriores:

```

CODIGO    DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
LISTA     [24]

```

3

Localización ítems de `lista_string_entre_comillas_dobles` similares a instrucciones VBA de manipulación de tablas

lista_indices_fin_instruccion_vba

Mediante bucle sobre la lista `lista_indices_fin_instruccion_vba`, se localiza que tupla de la lista `lista_indices_localiz_comillas_dobles` tiene su 1er ítem (es el índice de posición en la línea de código de apertura de la comilla doble) inmediatamente posterior al valor del ítem de la iteración sobre `lista_indices_localiz_comillas_dobles`.

Para que el texto encapsulado entre comillas dobles pueda ser considerado con instrucción VBA de manipulación de tablas el valor del 1er ítem de la tupla localizada ha de ser inmediatamente posterior al `indice_fin_instruccion_vba` de la iteración del bucle sobre `lista_indices_fin_instruccion_vba`.

Se agrega a la lista temporal `lista_SI_preservar_codigo_instruccion_vba`, el string entre los valores del 1ero y 2ndo ítem de la tupla localizada, al cual se le quitan las comillas dobles.

```

2104 lista_SI_preservar_codigo_instruccion_vba = []
2105 for indice_fin_instruccion_vba in lista_indices_fin_instruccion_vba:
2106     tupla_indices = [tupla for tupla in lista_tuplas_indices_localiz_comillas_dobles if tupla[0] > indice_fin_instruccion_vba][0]
2107
2108     indice_ini_entre_comillas_dobles = tupla_indices[0]
2109     indice_fin_entre_comillas_dobles = tupla_indices[1]
2110
2111     #para que el texto encapsulado entre comillas dobles pueda ser considerado con instruccion VBA de manipulacion de tablas
2112     #el indice_ini_entre_comillas_dobles ha de ser inmediatamente posterior al indice_fin_instruccion_vba de la iteracion
2113     # del bucle sobre lista_indices_fin_instruccion_vba
2114     #se agrega el string entre indice_ini_entre_comillas_dobles y indice_fin_entre_comillas_dobles, al cual se le quitan las comillas dobles "
2115     if indice_ini_entre_comillas_dobles == indice_fin_instruccion_vba + 1:
2116         codigo_preservar = linea_codigo[indice_ini_entre_comillas_dobles:indice_fin_entre_comillas_dobles].replace('"', '')
2117         lista_SI_preservar_codigo_instruccion_vba.append(codigo_preservar)

```

En el ejemplo mencionado en páginas anteriores:

```

CODIGO      DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
indice_fin_instruccion_vba      24
tupla_indices      (25, 40)
codigo_preservar      T_00_PRUEBA_04
lista_SI_preservar_codigo_instruccion_vba      ['T_00_PRUEBA_04']

```

4

Listas donde preservar y no preservar el código

Se fusionan las listas temporales `lista_SI_preservar_codigo_sql` y `lista_SI_preservar_codigo_instruccion_vba` para obtener la lista donde preservar el código.

Los ítems de la lista `lista_string_entre_comillas_dobles` que no estén en la lista fusionada pasan a crear la lista donde no preservar el código.

```
2124 lista_SI_preservar_codigo = lista_SI_preservar_codigo_sql + lista_SI_preservar_codigo_instruccion_vba
2125 lista_NO_preservar_codigo = [item for item in lista_string_entre_comillas_dobles if item not in lista_SI_preservar_codigo]
```

5

Resultado de la función

La opción **PRESERVAR_SI** devuelve la concatenación de la lista temporal `lista_SI_preservar_codigo`. La opción **PRESERVAR_NO** devuelve lo mismo con la lista `lista_NO_preservar_codigo`.

```
2128 #resultado de la funcion
2129 return "".join(lista_SI_preservar_codigo) if opcion == "PRESERVAR_SI" else "".join(lista_NO_preservar_codigo)
```

5

Resultado de la función

Tomando el ejemplo mencionado en páginas anteriores, la rutina VBA Calc_Prueba devolvería con las opciones PRESERVAR_SI y PRESERVAR_NO:

```

1 Sub Calc_PRUEBA(xOpcion As String, xValor_1 As String, xValor_2 As String)
2
3 Dim MiConexion As New ADODB.Connection
4 Dim MiRecordset1 As New ADODB.Recordset
5 Dim MiSQL As String
6 Dim T_00_PRUEBA_01 As String
7 Dim xTemp As String
8
9 Set MiConexion = CurrentProject.Connection
10
11 T_00_PRUEBA_01 = "AAA"
12
13 xTemp = "T_00_PRUEBA_02"
14
15 If xOpcion = "OPCION_1" Then
16     MiSQL = "UPDATE [T_00_PRUEBA_03] SET Campo_ID = '" & xValor_1 & "'"
17
18 ElseIf xOpcion = "OPCION_2" Then
19     MiSQL = "UPDATE [" & xTemp & "] SET Campo_ID = '" & xValor_2 & "'"
20
21 End If
22
23 MiConexion.Execute (MiSQL)
24
25 DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
26
27 MiConexion.Close
28 Set MiConexion = Nothing
29
30 End Sub

```

PRESERVAR_SI:

```

UPDATE [T_00_PRUEBA_03] SET Campo_ID = '
UPDATE [T_00_PRUEBA_04] SET Campo_ID = '

```

PRESERVAR_NO:

```

AAA
T_00_PRUEBA_02
OPCION_1
'
OPCION_2
'

```

Al realizar el diagnostico de dependencias, el app diría que tan solo las tablas T_00_PRUEBA_3 y T_00_PRUEBA_4 se usan en la rutina VBA. El check manual que el usuario ha de realizar en el Excel final de dependencias informaría de que la tabla T_00_PRUEBA_02 se ha localizado en la rutina VBA pero no sabe si se considera un uso de tabla o no.

ANEXO_11

RUTINA

def_proceso_access_2_diagnostico

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

Esta rutina permite realizar los cálculos necesarios para realizar el diagnostico de dependencias de objetos en una base de datos MS ACCESS (por defecto es la BBDD_01) y realizar la exportación del resultado final en un fichero Excel.

La rutina tiene 1 solo parámetro:

- **ruta_destino_excel_diagnostico_access**: es la ruta donde se guarda en el output final en fichero Excel.

La rutina nada más empezar se encapsula en un bloque de flujos de errores (**try except**). El bloque **except** permite almacenar en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL), en las subkeys_3 (LISTA_DICC_ERRORES_CALCULO), los sucesivos errores de ejecución, en caso de que los hubiese, lo que permite generar en otra rutina logs en formatos .txt.

El 1er paso consiste en recuperar los datos necesarios del diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) creando la lista de diccionarios **lista_dicc_objetos** y el dataframe **df_bbdd_codigo**. BBDD_01 es por defecto la base de datos en la cual se realiza el diagnostico de dependencias.

```
3026 lista_dicc_objetos = mod_gen.dicc_codigos_bbdd["BBDD_01"]["MS_ACCESS"]["LISTA_DICC_OBJETOS"]
3027 df_bbdd_codigo = mod_gen.dicc_codigos_bbdd["BBDD_01"]["MS_ACCESS"]["DF_CODIGO_CALCULADO_TRAS_IMPORT"]
```

Los siguientes pasos son:

PASO	DESCRIPCIÓN
1	Ajuste previo calculando la lista lista_dicc_rutinas_codigo_donde_buscar que sirve de base para localizar que objetos de la lista lista_dicc_objetos tienen dependencias o no con otros objetos
2	Calculo del diagnostico de dependencias mediante macheo entre los items de lista_dicc_objetos y los items de lista_dicc_rutinas_codigo_donde_buscar y creación de listas temporales que almacenen estas dependencias
3	Conversión en dataframes de las listas temporales creadas en el paso 2
4	Exportación a Excel, usando la plantilla integrada dentro del app, de los dataframes mencionados en el paso 3

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

1

AJUSTE PREVIO

Como paso previo al cálculo del diagnostico de dependencias, se crea la lista temporal **lista_dicc_rutinas_codigo_donde_buscar** que en la que permite localizar que objetos de **lista_dicc_objetos** se usan en los códigos VBA de otros objetos.

El cálculo de dependencias se realiza mediante iteraciones sobre estas 2 listas temporales porque es mucho más rápido y efectivo que realizarlo sobre dataframes.

```
3053 df_temp = (df_bbdd_codigo.loc[(df_bbdd_codigo["NOMBRE Rutina"].isnull() == False) & (df_bbdd_codigo["CODIGO SIN TAB TRIMEADO SIN COMENTARIOS"].str.len() != 0),
3054                               lista_headers_df_diagnostico_donde_buscar_campos_usados])
3055
3056 df_temp.reset_index(drop = True, inplace = True)
```

La lista **lista_dicc_rutinas_codigo_donde_buscar** se construye en base a **lista_dicc_objetos** para cada objeto (todos son rutinas o funciones VBA) se crea un diccionario con datos basados en columnas del dataframe **df_bbdd_codigo** el cual se filtra creando un nuevo dataframe donde la columna **NOMBRE_RUTINA** no esta vacía.

```
3059 lista_codigo_para_tablas_y_vinculos_temp = df_temp[lista_headers_df_diagnostico_donde_buscar_tablas].values.tolist()
3060 lista_codigo_para_rutinas_y_variables_temp = df_temp[lista_headers_df_diagnostico_donde_buscar_rutinas_y_variables].values.tolist()
3061 lista_codigo_para_variables_definidas_por_usuario_temp = df_temp[lista_headers_df_diagnostico_donde_buscar_variables_definidas_por_usuario].values.tolist()
```

Este dataframe temporal se convierte en diversas listas.

```
191 lista_headers_df_diagnostico_donde_buscar_tablas = ["TIPO_MODULO", "NOMBRE_MODULO", "NOMBRE_RUTINA", "NUMERO LINEA CODIGO_RUTINA", "CODIGO", "CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI",
192                                                    "CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO"]
193
194 lista_headers_df_diagnostico_donde_buscar_rutinas_y_variables = ["TIPO_MODULO", "NOMBRE_MODULO", "NOMBRE_RUTINA", "CODIGO_DIAGNOSTICO_RUTINAS_Y_VARIABLES"]
195 lista_headers_df_diagnostico_donde_buscar_variables_definidas_por_usuario = ["TIPO_MODULO", "NOMBRE_MODULO", "NOMBRE_RUTINA", "TIPO_DATO_PARAMETROS_Y_VARIABLES_LOCALES_RUTINA"]
```

En base a estas listas, mediante bucle sobre **lista_dicc_objetos** (en adelante bucle_1), y exclusivamente para tipos de objeto **RUTINAS_VBA**, se agregan diccionarios para cada objeto a **lista_dicc_rutinas_codigo_donde_buscar**.

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

1

AJUSTE PREVIO

```

3066 lista_dicc_rutinas_codigo_donde_buscar = []
3067 for dicc in lista_dicc_objetos:
3068 
3069     if dicc["TIPO_OBJETO"] == "RUTINAS_VBA":
3070 
3071         tipo_modulo = dicc["TIPO_MODULO"]
3072         nombre_modulo = dicc["NOMBRE_MODULO"]
3073         nombre_rutina = dicc["NOMBRE_OBJETO"]

```

La estructura de los diccionarios que componen esta lista es:

KEY 1	KEY 2	DESCRIPCIÓN CONTENIDO LISTAS	INDICES LISTAS	UTILIDAD
TIPO_MODULO				
NOMBRE_MODULO				
NOMBRE_RUTINA				
LISTA_CODIGO	DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI	Línea de código donde se ha preservado tan solo los string encapsulados entre comillas dobles que parecen a sentencias SQL o vienen a continuación de instrucciones VBA de manipulación de tablas (son los los datos de la columna CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI del dataframe df_bbdd_codigo)	0	Son las líneas de código en las que se busca para cada tabla / vínculo de la lista lista_dicc_objetos si se usa en el script de la rutina / función VBA (key 1 del presente diccionario)
	DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO	Tipo módulo	0	Sirve para calcular el check manual que el usuario ha de realizar en el fichero Excel de diagnostico cuando no se ha localizado para cada objeto de lista_dicc_objetos dependencia con la rutina / función VBA (key 1 del presente diccionario)
		Nombre módulo	1	
		Nombre rutina	2	
		Número línea código en la rutina	3	
		Línea de código original	4	
		Línea de código donde se ha preservado tan solo lo que parece a sentencias SQL o instrucciones de manipulación de tablas (son los los datos de la columna CODIGO_DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO del dataframe df_bbdd_codigo)	5	
	DIAGNOSTICO_RUTINAS_Y_VARIABLES	Línea de código donde se quita todo lo encapsulado entre comillas dobles y por lo tanto se conserva tan solo lo que son instrucciones VBA (son los los datos de la columna CODIGO_DIAGNOSTICO_RUTINAS_Y_VARIABLES del dataframe df_bbdd_codigo)	0	Son las líneas de código en las que se busca para cada rutina / función VBA de la lista lista_dicc_objetos si se usa en el script de la rutina / función VBA (key 1 del presente diccionario)
	DIAGNOSTICO_VARIABLES_DEFINIDAS_POR_USUARIO	Son los tipos de dato de los parametros concatenados con los tipos de datos de las variables locales de la rutina / función VBA (son los los datos de la columna TIPO_DATO_PARAMETROS_Y_VARIABLES_LOCALES_RUTINA del dataframe df_bbdd_codigo)	0	Son las líneas de código en las que se busca para cada variable pública definida por el usuario de la lista lista_dicc_objetos si se usa en el script de la rutina / función VBA (key 1 del presente diccionario)

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

1

AJUSTE PREVIO

SUBKEYS 2 - DIAGNOSTICO TABLAS Y VINCULOS PRESERVAR SI y DIAGNOSTICO TABLAS Y VINCULOS PRESERVAR NO

Dentro de la iteración sobre bucle_1, las listas que se almacenan en estas subkeys_2 se construyen mediante bucle sobre la lista temporal **lista_codigo_para_tablas_y_vinculos_temp** donde hay macheo para el tipo y nombre del módulo y el nombre de la rutina con los de la iteración sobre bucle_1.

```
3059 lista_codigo_para_tablas_y_vinculos_temp = df_temp[lista_headers_df_diagnostico_donde_buscar_tablas].values.tolist()

191 lista_headers_df_diagnostico_donde_buscar_tablas = ["TIPO_MODULO", "NOMBRE_MODULO", "NOMBRE Rutina", "NUMERO LINEA CODIGO Rutina", "CODIGO", "CODIGO DIAGNOSTICO TABLAS Y VINCULOS PRESERVAR SI",
192 "CODIGO DIAGNOSTICO TABLAS Y VINCULOS PRESERVAR NO"]

3085 lista_codigo_para_tablas_y_vinculos_preservar_si = []
3086 lista_codigo_para_tablas_y_vinculos_preservar_no = []
3087 for tipo_modulo_codigo, nombre_modulo_codigo, nombre_rutina_codigo, numero_linea_codigo_rutina, codigo_original, codigo_preservar_si, codigo_preservar_no in lista_codigo_para_tablas_y_vinculos_temp:
3088
3089     if tipo_modulo == tipo_modulo_codigo and nombre_modulo == nombre_modulo_codigo and nombre_rutina == nombre_rutina_codigo:
3090
3091         lista_codigo_para_tablas_y_vinculos_preservar_si.append(codigo_preservar_si)
3092
3093         lista_codigo_para_tablas_y_vinculos_preservar_no.append([
3094             tipo_modulo_codigo,
3095             nombre_modulo_codigo,
3096             nombre_rutina_codigo,
3097             numero_linea_codigo_rutina,
3098             codigo_original,
3099             codigo_preservar_no
3100         ])
```

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

1

AJUSTE PREVIO

SUBKEY 2 - DIAGNOSTICO RUTINAS Y VARIABLES

Dentro de la iteración sobre bucle_1, la lista que se almacena en esta subkey_2 se construye mediante bucle sobre la lista temporal **lista_codigo_para_rutinas_y_variables_temp** donde hay macheo para el tipo y nombre del módulo y el nombre de la rutina con los de la iteración sobre bucle_1.

```
3060 lista_codigo_para_rutinas_y_variables_temp = df_temp[lista_headers_df_diagnostico_donde_buscar_rutinas_y_variables].values.tolist()

194 lista_headers_df_diagnostico_donde_buscar_rutinas_y_variables = ["TIPO_MODULO", "NOMBRE_MODULO", "NOMBRE Rutina", "CODIGO_DIAGNOSTICO_RUTINAS_Y_VARIABLES"]

3104 lista_codigo_para_rutinas_y_variables = []
3105 for tipo_modulo_codigo, nombre_modulo_codigo, nombre_rutina_codigo, codigo_linea in lista_codigo_para_rutinas_y_variables_temp:
3106
3107     if tipo_modulo == tipo_modulo_codigo and nombre_modulo == nombre_modulo_codigo and nombre_rutina == nombre_rutina_codigo:
3108         lista_codigo_para_rutinas_y_variables.append(codigo_linea)
```

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

1

AJUSTE PREVIO

SUBKEY 2 - DIAGNOSTICO VARIABLES DEFINIDAS POR USUARIO

Dentro de la iteración sobre bucle_1, la lista que se almacena en estas subkeys_2 se construyen mediante bucle sobre la lista temporal **lista_codigo_para_variables_definidas_por_usuario_temp** donde hay macheo para el tipo y nombre del módulo y el nombre de la rutina con los de la iteración sobre bucle_1.

```
2650 lista_codigo_para_variables_definidas_por_usuario_temp = df_temp[lista_headers_df_diagnostico_donde_buscar_variables_definidas_por_usuario].values.tolist()

195 lista_headers_df_diagnostico_donde_buscar_variables_definidas_por_usuario = ["TIPO_MODULO", "NOMBRE_MODULO", "NOMBRE Rutina", "TIPO_DATO_PARAMETROS_Y_VARIABLES_LOCALES_Rutina"]

3112 lista_codigo_para_variables_definidas_por_usuario = []
3113 for tipo_modulo_codigo, nombre_modulo_codigo, nombre_rutina_codigo, codigo_linea in lista_codigo_para_variables_definidas_por_usuario_temp:
3114
3115     if tipo_modulo == tipo_modulo_codigo and nombre_modulo == nombre_modulo_codigo and nombre_rutina == nombre_rutina_codigo:
3116         lista_codigo_para_variables_definidas_por_usuario.append(codigo_linea)
```

Se crea el diccionario de la rutina / función VBA donde buscar y se agrega a **lista_dicc_rutinas_codigo_donde_buscar**

```
3120 dicc_temp = {"TIPO_MODULO": tipo_modulo
3121             , "NOMBRE_MODULO": nombre_modulo
3122             , "NOMBRE_Rutina": nombre_rutina
3123             , "LISTA_CODIGO":
3124                 {"DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI": lista_codigo_para_tablas_y_vinculos_preservar_si
3125                 , "DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO": lista_codigo_para_tablas_y_vinculos_preservar_no
3126                 , "DIAGNOSTICO_RUTINAS_Y_VARIABLES": lista_codigo_para_rutinas_y_variables
3127                 , "DIAGNOSTICO_VARIABLES_DEFINIDAS_POR_USUARIO": lista_codigo_para_variables_definidas_por_usuario
3128                 }
3129             }
3130
3131 lista_dicc_rutinas_codigo_donde_buscar.append(dicc_temp)
```

2

CÁLCULO DIAGNOSTICO DEPENDENCIAS

Se calculan las listas temporales siguientes (son listas de listas):

- lista_diagnostico_dependencias
- lista_diagnostico_sin_dependencias
- lista_diagnostico_tablas_check_manual

LISTA	ITEMS SUBLISTAS
lista_diagnostico_dependencias	Tipo objeto
	Tipo módulo
	Nombre módulo
	Nombre objeto
	Tipo módulo asociado a la rutina / función VBA donde se usa el objeto
	Nombre módulo asociado a la rutina / función VBA donde se usa el objeto
	Rutina / función VBA donde se usa el objeto
lista_diagnostico_sin_dependencias	Tipo objeto
	Tipo módulo
	Nombre módulo
	Nombre objeto
lista_diagnostico_tablas_check_manual	Tipo objeto
	Tipo módulo
	Nombre módulo
	Nombre objeto
	Rutina / función VBA a chequear
	Número de la línea de código de la rutina / función VBA a chequear
	Línea de código de la rutina / función VBA a chequear

2

CÁLCULO DIAGNOSTICO DEPENDENCIAS

Listas lista_diagnostico_dependencias y lista_diagnostico_sin_dependencias

Mediante bucle sobre la lista **lista_dicc_objetos** (en adelante bucle_1), para cada uno de sus objetos se extraen los datos necesarios para el cálculo del diccionario de la iteración. El **nombre de objeto se pone en mayúscula**, es para evitar casos de case sensitive en la búsqueda de dependencias.

```

3152 lista_diagnostico_dependencias = []
3153 lista_diagnostico_sin_dependencias = []
3154
3155 for dicc_objetos_a_buscar in lista_dicc_objetos:
3156
3157     tipo_objeto_a_buscar = dicc_objetos_a_buscar["TIPO_OBJETO"]
3158     tipo_modulo_a_buscar = dicc_objetos_a_buscar["TIPO_MODULO"]
3159     nombre_modulo_a_buscar = dicc_objetos_a_buscar["NOMBRE_MODULO"]
3160     nombre_objeto_a_buscar = dicc_objetos_a_buscar["NOMBRE_OBJETO"]
3161     tipo_variable_publica_a_buscar = dicc_objetos_a_buscar["TIPO_VARIABLE_PUBLICA"]
3162
3163     nombre_objeto_mayusc_a_buscar = nombre_objeto_a_buscar.upper()

```

Siempre dentro de bucle_1, se localiza, según el tipo de objeto, que subkey_2 asociada a la key_1 (LISTA_OBJETOS) de la lista **lista_dicc_rutinas_codigo_donde_buscar** se ha de tomar en cuenta.

TIPO OBJETO	TIPO VARIABLE PÚBLICA	SUBKEY_2A TOMAR EN CUENTA
TABLA_LOCAL	-	DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI
VINCULO_ODBC	-	DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI
VINCULO_OTRO	-	DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI
RUTINAS_VBA	-	DIAGNOSTICO_RUTINAS_Y_VARIABLES
VARIABLES_VBA	NORMAL	DIAGNOSTICO_RUTINAS_Y_VARIABLES
	DEFINIDA POR EL USUARIO	DIAGNOSTICO_VARIABLES_DEFINIDAS_POR_USUARIO

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

2

CÁLCULO DIAGNOSTICO DEPENDENCIAS

Listas `lista_diagnostico_dependencias` y `lista_diagnostico_sin_dependencias`

```
3166 #se localiza cual es la lista de lineas de codigo a usar como referencia segun el tipo de objeto
3167 if tipo_objeto_a_buscar in ["TABLA_LOCAL", "VINCULO_ODBC", "VINCULO_OTRO"]:
3168     key_codigo_donde_buscar = "DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_SI"
3169
3170 elif tipo_objeto_a_buscar == "RUTINAS_VBA":
3171     key_codigo_donde_buscar = "DIAGNOSTICO_RUTINAS_Y_VARIABLES"
3172
3173
3174 elif tipo_objeto_a_buscar == "VARIABLES_VBA":
3175
3176     if tipo_variable_publica_a_buscar == dicc_objetos_vba["VARIABLE_PUBLICA_NORMAL"]["STRING_PARA_LISTADO_EXCEL"]:
3177         key_codigo_donde_buscar = "DIAGNOSTICO_RUTINAS_Y_VARIABLES"
3178
3179     elif tipo_variable_publica_a_buscar == dicc_objetos_vba["VARIABLE_PUBLICA_DEFINIDA_POR_USUARIO"]["STRING_PARA_LISTADO_EXCEL"]:
3180         key_codigo_donde_buscar = "DIAGNOSTICO_VARIABLES_DEFINIDAS_POR_USUARIO"
```

Siempre dentro de `bucle_1`, se crea otro bucle sobre la lista `lista_dicc_rutinas_codigo_donde_buscar` (en adelante `bucle_2`) donde en cada iteración se extraen el tipo y nombre del módulo, el nombre de la rutina y la lista de código VBA asociada a la `subkey_2` mencionada en el párrafo anterior.

Antes de iniciar `bucle_2` se define la variable **`check_objeto`** que es la que permite para cada objeto de la iteración sobre `bucle_1` cuando no se localiza ninguna dependencia con rutinas / funciones VBA, tras finalizar `bucle_2`, determinar si se ha de agregar a la lista `lista_diagnostico_sin_dependencias`.

```
3183 check_objeto = 0
3184 for dicc_rutinas_codigo_donde_buscar in lista_dicc_rutinas_codigo_donde_buscar:
3185
3186     tipo_modulo_donde_buscar = dicc_rutinas_codigo_donde_buscar["TIPO_MODULO"]
3187     nombre_modulo_donde_buscar = dicc_rutinas_codigo_donde_buscar["NOMBRE_MODULO"]
3188     nombre_rutina_donde_buscar = dicc_rutinas_codigo_donde_buscar["NOMBRE_RUTINA"]
3189     lista_codigo_donde_buscar = dicc_rutinas_codigo_donde_buscar["LISTA_CODIGO"][key_codigo_donde_buscar]
```

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

2

CÁLCULO DIAGNOSTICO DEPENDENCIAS

Listas lista_diagnostico_dependencias y lista_diagnostico_sin_dependencias

En la iteración sobre bucle_2, se excluye la rutina VBA que corresponde con la rutina VBA de la iteración sobre bucle_1 para evitar localizar una dependencia de un objeto. **Esta exclusión solo afecta a las rutinas** no a las funciones porque en VBA el resultado de las funciones usa el nombre de las mismas dentro del código para declararlas.

```
3194 if not (tipo_objeto_a_buscar == "RUTINAS_VBA" and tipo_modulo_a_buscar == tipo_modulo_donde_buscar and nombre_modulo_a_buscar == nombre_modulo_donde_buscar and
3195         nombre_objeto_a_buscar == nombre_rutina_donde_buscar):
```

Dentro de la iteración sobre bucle_2, se crea otro bucle (en adelante bucle_3) sobre el item **lista_codigo_donde_buscar** de la iteración sobre bucle_2.

Antes de iniciar bucle_3 se define la variable **check_codigo** que es la que permite para cada rutina / función VBA de la iteración sobre bucle_2 cuando se localiza el objeto de la iteración sobre bucle_1 en alguna línea de código de la rutina / función VBA de la iteración sobre bucle_2 determinar si se ha de agregar a la lista **lista_diagnostico_dependencias**.

El macheo entre el objeto de la iteración sobre bucle_1 (puesto en mayúsculas) y la línea de código ajustada de la iteración sobre bucle_3 (ella también esta puesta en mayúscula a modo recordatorio) se hace mediante el uso de la librería re donde se busca que el objeto **aparezca en una palabra de la línea de código no que forme parte de una palabra más larga** (que podría ser otro objeto en bucle_1) y así se evita localizar dependencias erróneas.

En caso de macheo se rompe el bucle_3 y se informan las variables **check_objeto** y **check_codigo** iguales cada una a 1.

CÁLCULO DIAGNOSTICO DEPENDENCIAS

Listas lista_diagnostico_dependencias y lista_diagnostico_sin_dependencias

```

3197         check_codigo = 0
3198         for linea_codigo in lista_codigo_donde_buscar:
3199
3200             if linea_codigo != None:
3201                 if nombre_objeto_mayusc_a_buscar in linea_codigo:
3202
3203                     #se considera que el objeto se usa en otra rutina si nombre_objeto_mayusc esta incluido en la linea de codigo d
3204                     #la lista lista_codigo_donde_buscar y que ademas nombre_objeto_mayusc sea una palabra suelta no un substring in
3205                     #salvo que este seguido de un parentesis de apertura "(" (casos de llamadas de rutinas)
3206                     # ejemplo: rutina Sub_Calc en las lineas de codigo "Call Sub_Calc()" "Call Sub_Calc_1()" se consideraria que Su
3207                     #se usa en la 1era linea pero no en la 2nda)
3208                     check_si_nombre_objeto_es_palabra_sola = fr'(?<![\w-]){re.escape(nombre_objeto_mayusc_a_buscar)}(?![\w-])'
3209                     check_si_nombre_objeto_es_palabra_sola = bool(re.search(check_si_nombre_objeto_es_palabra_sola, linea_codigo))
3210
3211                     check_objeto = 1 if check_si_nombre_objeto_es_palabra_sola == True else 0
3212                     check_codigo = 1 if check_si_nombre_objeto_es_palabra_sola == True else 0
3213
3214                     break

```

Si se localiza cacheo, siempre dentro de bucle_2, se agrega tanto el objeto de la iteración sobre bucle_1 como la rutina / función VBA de la iteración sobre bucle_2 en la lista **lista_diagnostico_dependencias**.

```

3217         if check_codigo == 1:
3218             lista_diagnostico_dependencias.append([tipo_objeto_a_buscar
3219                                                     , tipo_modulo_a_buscar
3220                                                     , nombre_modulo_a_buscar
3221                                                     , nombre_objeto_a_buscar
3222                                                     , tipo_modulo_donde_buscar
3223                                                     , nombre_modulo_donde_buscar
3224                                                     , nombre_rutina_donde_buscar])

```


2

CÁLCULO DIAGNOSTICO DEPENDENCIAS

Listas **lista_diagnostico_dependencias** y **lista_diagnostico_sin_dependencias**

Tras finalizar bucle_2 si no se han localizado ninguna dependencia para el objeto de la iteración sobre bucle_1 se agrega este último a la lista **lista_diagnostico_sin_dependencias**.

```
3227         if check_objeto == 0:
3228             lista_diagnostico_sin_dependencias.append([tipo_objeto_a_buscar
3229                                                         , tipo_modulo_a_buscar
3230                                                         , nombre_modulo_a_buscar
3231                                                         , nombre_objeto_a_buscar])
```

Lista **lista_diagnostico_tablas_check_manual**

Para aquellas tablas / vínculos, para las cuales en las rutinas / funciones VBA no se han localizado dependencias se busca si se usan en código VBA, encapsuladas entre comillas dobles, en sentencias VBA que no parezcan a sentencias SQL ni quevengan precedidas de instrucciones de manipulación de tablas.

Son los casos donde a un parámetro de la rutina / función VBA o a una variable local se le asigna como valor el nombre de una tabla / vínculo y en otra línea de código se usa este parámetro o variable local en sentencias VBA que parecen a sentencias SQL o vienen precedidas de instrucciones de manipulación de tablas.

2

CÁLCULO DIAGNOSTICO DEPENDENCIAS**Lista lista diagnostico tablas check manual**

```

1 Sub Calc_PRUEBA(xOpcion As String, xValor_1 As String, xValor_2 As String)
2
3 Dim MiConexion As New ADODB.Connection
4 Dim MiRecordset1 As New ADODB.Recordset
5 Dim MiSQL As String
6 Dim T_00_PRUEBA_01 As String
7 Dim xTemp As String
8
9 Set MiConexion = CurrentProject.Connection
10
11 T_00_PRUEBA_01 = "AAA"
12
13 xTemp = "T_00_PRUEBA_02"
14
15 If xOpcion = "OPCION_1" Then
16     MiSQL = "UPDATE [T_00_PRUEBA_03] SET Campo_ID = '" & xValor_1 & "'"
17
18 ElseIf xOpcion = "OPCION_2" Then
19     MiSQL = "UPDATE [" & xTemp & "] SET Campo_ID = '" & xValor_2 & "'"
20
21 End If
22
23 MiConexion.Execute (MiSQL)
24 DoCmd.OpenTable acTable, "T_00_PRUEBA_04", acViewNormal
25
26
27 MiConexion.Close
28 Set MiConexion = Nothing
29
30 End Sub

```

Esta lista es la que sirve para informar en el fichero Excel de diagnostico de dependencias la hoja TABLAS (CHECK MANUAL).

A	B	C	D	E	F	G
TIPO_TABLA	NOMBRE_TABLA	TIPO_MODULO	NOMBRE_MODULO	NOMBRE_RUTINA	NUMERO_LINEA_CODIGO	CODIGO_LINEA
TABLA_LOCAL	T_01_ProveedComerc	Estandar	Mod_00_Config	ActivarPanelNavegacion	5	xTabla = "T_01_ProveedComerc"
TABLA_LOCAL	T_Extraccion_Actual	Estandar	Mod_00_Config	Config_CalculosUpDateSQL	49	xTabla = "T_Extraccion_Actual"

CÁLCULO DIAGNOSTICO DEPENDENCIAS**Lista lista_diagnostico_tablas_check_manual**

Mediante bucle sobre la lista **lista_dicc_objetos** (en adelante bucle_1) y solo para tipos de objeto que son tablas o vínculos, en cada iteración sobre bucle_1 se inicia otro bucle sobre **lista_dicc_rutinas_codigo_donde_buscar** (en adelante bucle_2) en el que para cada objeto de la iteración sobre bucle_1 y cada rutina / función VBA de la iteración sobre bucle_2 se comprueba si ya se localizo dependencia en la lista **lista_diagnostico_dependencias**.

```

3238 lista_diagnostico_tablas_check_manual = []
3239 for dicc_objetos_a_buscar in lista_dicc_objetos:
3240
3241     tipo_objeto_a_buscar = dicc_objetos_a_buscar["TIPO_OBJETO"]
3242
3243     #se busca tan solo para las tablas / vinculos
3244     if tipo_objeto_a_buscar in ["TABLA_LOCAL", "VINCULO_ODBC", "VINCULO_OTRO"]:
3245
3246         nombre_objeto_a_buscar = dicc_objetos_a_buscar["NOMBRE_OBJETO"]
3247         nombre_objeto_mayusc_a_buscar = nombre_objeto_a_buscar.upper()
3248
3249         #mediante bucle sobre lista_dicc_rutinas_codigo_donde_buscar se recupera la lista almacenada en la subkey_2
3250         #DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO
3251         for dicc_rutinas_codigo_donde_buscar in lista_dicc_rutinas_codigo_donde_buscar:
3252
3253             tipo_modulo_donde_buscar_check = dicc_rutinas_codigo_donde_buscar["TIPO_MODULO"]
3254             nombre_modulo_donde_buscar_check = dicc_rutinas_codigo_donde_buscar["NOMBRE_MODULO"]
3255             nombre_rutina_donde_buscar_donde_buscar_check = dicc_rutinas_codigo_donde_buscar["NOMBRE Rutina"]
3256             lista_codigo_donde_buscar_check = dicc_rutinas_codigo_donde_buscar["LISTA_CODIGO"]["DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO"]
3257
3258             #se chequea si para la tabla de la iteracion (nombre_objeto_a_buscar) no se localizo anteriormente una dependencia
3259             #con la rutina nombre_rutina_donde_buscar_donde_buscar_check
3260             #en caso de que NO sea el caso se prsigue con el matcho con la lista contenida en la subkey_2 DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO
3261             check_si_dependencia_localizada = 0
3262             for tipo_objeto_depend, tipo_modulo_depend, nombre_modulo_depend, nombre_objeto_depend, tipo_modulo_depend, nombre_modulo_depend, nombre_rutina_depend in lista_diagnostico_dependencias:
3263
3264                 if (tipo_objeto_a_buscar == tipo_objeto_depend and nombre_objeto_a_buscar == nombre_objeto_depend and tipo_modulo_donde_buscar_check == tipo_modulo_depend and
3265                     nombre_modulo_donde_buscar_check == nombre_modulo_depend and nombre_rutina_donde_buscar_donde_buscar_check == nombre_rutina_depend):
3266
3267                     check_si_dependencia_localizada = 1
3268                     break

```

CÁLCULO DIAGNOSTICO DEPENDENCIAS**Lista lista_diagnostico_tablas_check_manual**

En caso no localizar dependencias (se hace mediante la variable **check_si_dependencia_localizada**), tras finalizar bucle_3, siempre dentro de bucle_2, se inicia otro bucle (en adelante bucle_4) sobre la lista **lista_codigo_donde_buscar_check** (es la lista que se almacena en la subkey_2 - DIAGNOSTICO_TABLAS_Y_VINCULOS_PRESERVAR_NO asociada a la key_1 - LISTA_CODIGOS de los diccionarios que componen la lista **lista_dicc_rutinas_codigo_donde_buscar**).

En el bucle_4, el matcheo entre el objeto de la iteración sobre bucle_1 (puesto en mayúsculas) y la línea de código ajustada de la iteración sobre bucle_4 (ella también esta puesta en mayúscula a modo recordatorio) se hace mediante el uso de la librería re donde se busca que el objeto **aparezca en una palabra de la línea de código no que forme parte de una palabra más larga** (que podría ser otro objeto en bucle_1) y así se crear check manuales por el usuario erróneos.

```

3270 if check_si_dependencia_localizada == 0:
3271
3272     #mediante bucle sobre la lista lista_codigo_donde_buscar_check se realizan los matcheos de si se encuentra exactamente el nombre de la tabla
3273     #(como palabra sola y no como substring dentro de otra palabra mas larga) se agrega a la lista lista_diagnostico_tablas_check_manual la info
3274     #necesaria para determinar si se ha de considerar o no usade tabla en la rutina correspondiente
3275     for tipo_modulo_check, nombre_modulo_check, nombre_rutina_check, numero_linea_codigo_rutina_check, codigo_linea_original_check, string_concat_check in lista_codigo_donde_buscar_check:
3276
3277         if nombre_objeto_mayusc_a_buscar in string_concat_check:
3278
3279             check_manual_si_nombre_objeto_no_en_otra_palabra = fr'(?![\w-]){re.escape(nombre_objeto_mayusc_a_buscar)}(?![\w-])'
3280             check_manual_si_nombre_objeto_no_en_otra_palabra = bool(re.search(check_manual_si_nombre_objeto_no_en_otra_palabra, string_concat_check))
3281
3282             if check_manual_si_nombre_objeto_no_en_otra_palabra == True:
3283                 lista_diagnostico_tablas_check_manual.append([tipo_objeto_a_buscar
3284                     , nombre_objeto_a_buscar
3285                     , tipo_modulo_check
3286                     , nombre_modulo_check
3287                     , nombre_rutina_check
3288                     , numero_linea_codigo_rutina_check
3289                     , codigo_linea_original_check])

```

3

CONVERSIÓN EN DATAFRAMES DE LAS LISTAS TEMPORALES

Las listas temporales comentadas en páginas anteriores (**lista_diagnostico_dependencias**, **lista_diagnostico_sin_dependencias** y **lista_diagnostico_tablas_check_manual**) se convierten en dataframes, se re-ordenan. Estos dataframes son los que se exportan en el fichero Excel en las hojas “DEPENDENCIAS”, “SIN DEPENDENCIAS” y “TABLAS (CHECK MANUAL)”.

```

3299 #se crean los df df_diagnostico_dependencias y df_diagnostico_sin_dependencias para exportar a excel
3300 df_diagnostico_dependencias = pd.DataFrame(lista_diagnostico_dependencias, columns = lista_headers_df_dependencias)
3301 df_diagnostico_dependencias = (df_diagnostico_dependencias[lista_headers_df_dependencias].sort_values([i for i in lista_headers_df_dependencias]
3302 , ascending = [True for i in lista_headers_df_dependencias]))
3303
3304 df_diagnostico_sin_dependencias = pd.DataFrame(lista_diagnostico_sin_dependencias, columns = lista_headers_df_sin_dependencias)
3305 df_diagnostico_sin_dependencias = (df_diagnostico_sin_dependencias[lista_headers_df_sin_dependencias].sort_values([i for i in lista_headers_df_sin_dependencias]
3306 , ascending = [True for i in lista_headers_df_sin_dependencias]))
3307
3308 #se crean el df df_diagnostico_tablas_check_manual
3309 df_diagnostico_tablas_check_manual = pd.DataFrame(lista_diagnostico_tablas_check_manual, columns = lista_headers_df_check_manual_tablas)
3310 df_diagnostico_tablas_check_manual = (df_diagnostico_tablas_check_manual[lista_headers_df_check_manual_tablas].sort_values([i for i in lista_headers_df_check_manual_tablas]
3311 , ascending = [True for i in lista_headers_df_check_manual_tablas]))
3312

```

Asimismo, se crea la lista **lista_para_df_listado** mediante bucle sobre la lista **lista_dicc_objetos** y sirve de base para crear el dataframe que se exporta en el fichero Excel en la hoja LISTADO.

```

3316 lista_para_df_listado = []
3317 for dicc in lista_dicc_objetos:
3318
3319     tipo_objeto_2 = dicc["TIPO Rutina"] if dicc["TIPO OBJETO"] == "RUTINAS_VBA" else dicc["TIPO VARIABLE PUBLICA"] if dicc["TIPO OBJETO"] == "VARIABLES_VBA" else None
3320
3321     lista_para_df_listado.append([dicc["TIPO OBJETO"]
3322 , dicc["TIPO MODULO"]
3323 , dicc["NOMBRE MODULO"]
3324 , dicc["NOMBRE OBJETO"]
3325 , tipo_objeto_2
3326 , dicc["TIPO DECLARACION Rutina"]
3327 , dicc["PARAMETROS Rutina"]
3328 , dicc["TIPO DATO VARIABLE PUBLICA"]
3329 , dicc["VARIABLE PUBLICA ES CONSTANTE"]
3330 , dicc["VARIABLE PUBLICA CONSTANTE VALOR"]
3331 , dicc["VARIABLE PUBLICA DEFINIDA POR USUARIO SUB VARIABLES"]
3332 , dicc["CONNECTING STRING VINCULOS"]
3333 ])
3334
3335 df_listado_objetos = pd.DataFrame(lista_para_df_listado, columns = lista_headers_df_listado)
3336 df_listado_objetos = df_listado_objetos[lista_headers_df_listado].sort_values(lista_headers_df_listado, ascending = [True for i in lista_headers_df_listado])

```

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

5

EXPORTACIÓN EXCEL

Usando la plantilla Excel integrada en el app se copia la misma en la ruta indicada **ruta_destino_excel_diagnostico_access** (parámetro de la presente rutina que se documenta), se le cambia el nombre y se abre en 2ndo plano (se usa para ello la librería xlwings).

APP_2_GENERAL

```
25 try:
26     #si es un ejecutable
27     ico_app = sys._MEIPASS + r"\ico_app.ico" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\ico_app.ico"
28     ruta_plantilla_control_versiones_xls = sys._MEIPASS + r"\PLANTILLA_CONTROL_VERSIONES.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_CONTROL_VERSIONES.xlsb"
29     ruta_plantilla_diagnostico_access_xls = sys._MEIPASS + r"\PLANTILLA_DIAGNOSTICO_MS_ACCESS.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_DIAGNOSTICO_MS_ACCESS.xlsb"
30     ruta_plantilla_diagnostico_sql_server_xls = sys._MEIPASS + r"\PLANTILLA_DIAGNOSTICO_SQL_SERVER.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_DIAGNOSTICO_SQL_SERVER.xlsb"
31
32 except:
33     #si se ejecuta desde consola
34     ico_app = sys._MEIPASS / "ico_app.ico" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() / "ico_app.ico"
35     ruta_plantilla_control_versiones_xls = sys._MEIPASS / "PLANTILLA_CONTROL_VERSIONES.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() / "PLANTILLA_CONTROL_VERSIONES.xlsb"
36     ruta_plantilla_diagnostico_access_xls = sys._MEIPASS / "PLANTILLA_DIAGNOSTICO_MS_ACCESS.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() / "PLANTILLA_DIAGNOSTICO_MS_ACCESS.xlsb"
37     ruta_plantilla_diagnostico_sql_server_xls = sys._MEIPASS / "PLANTILLA_DIAGNOSTICO_SQL_SERVER.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() / "PLANTILLA_DIAGNOSTICO_SQL_SERVER.xlsb"
38
39     pass
```

APP_3_BACK_END_MS_ACCESS

```
3344 now = str(dt.datetime.now()).replace("-", "").replace(" ", "_").replace(":", "")[0:15]
3345 saveas = str(ruta_destino_excel_diagnostico_access) + r"\DEPENDENCIAS_MS_ACCESS_" + now + ".xlsb"
3346
3347 shutil.copyfile(mod_gen.ruta_plantilla_diagnostico_access_xls, saveas)
3348
3349 app = xw.App(visible = False)
3350 wb = app.books.open(saveas, update_links = False)
```

5

EXPORTACIÓN EXCEL

Se crea un diccionario temporal en el que almacenar todo lo necesario para realizar la exportación a Excel de forma dinámica mediante un bucle.

```

3352     dicc_export_excel = {"LISTADO":
3353                          {"HOJA_EXCEL": "LISTADO"
3354                           , "DF_EXPORT": df_listado_objetos
3355                          }
3356                          , "DEPENDENCIAS":
3357                          {"HOJA_EXCEL": "DEPENDENCIAS"
3358                           , "DF_EXPORT": df_diagnostico_dependencias
3359                          }
3360                          , "SIN_DEPENDENCIAS":
3361                          {"HOJA_EXCEL": "SIN DEPENDENCIAS"
3362                           , "DF_EXPORT": df_diagnostico_sin_dependencias
3363                          }
3364                          , "TABLAS_CHECK_MANUAL":
3365                          {"HOJA_EXCEL": "TABLAS (CHECK MANUAL)"
3366                           , "DF_EXPORT": df_diagnostico_tablas_check_manual
3367                          }
3368     }

```

Mediante bucle sobre este diccionario temporal, se realiza la exportación a Excel y al finalizar este bucle se guarda el Excel.

```

3370     for key in dicc_export_excel.keys():
3371
3372         hoja_excel = dicc_export_excel[key]["HOJA_EXCEL"]
3373         df_export = dicc_export_excel[key]["DF_EXPORT"]
3374
3375         if len(df_export) != 0:
3376             ws = wb.sheets[hoja_excel]
3377             ws.range("A2:FF65000").clear_contents()
3378             ws["A2"].options(pd.DataFrame, header = 0, index = False, expand = "table").value = df_export
3379
3380
3381
3382     #se guarda en el excel, se cierra y se re-abre en 1er plano
3383     wb.save(saveas)
3384     wb.close()
3385     app = xw.App(visible = True)
3386     app.quit()
3387
3388     wb = xw.Book(saveas, update_links = False)

```

ANEXO_11

RUTINA: def_proceso_access_2_diagnostico

Por último, se crea el bloque **except** para almacenar los posibles errores de ejecución de la rutina en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_CALCULO) asociada a la key_1 (PROCESO_02) y subkey_2 (MS_ACCESS).

```
3405 except Exception as Err:
3406
3407     traceback_error = traceback.extract_tb(Err.__traceback__)
3408     modulo_python = os.path.basename(traceback_error[0].filename)
3409     rutina_python = traceback_error[0].name
3410     linea_error = traceback_error[0].lineno
3411
3412     lista_dicc_errores_migracion = mod_gen.dicc_errores_procesos["PROCESO_02"]["MS_ACCESS"]["LISTA_DICC_ERRORES_CALCULO"]
3413
3414     dicc_errores_temp = {"TIPO_BBDD": "MS_ACCESS"
3415                          , "MODULO_PYTHON": modulo_python
3416                          , "RUTINA_PYTHON": rutina_python
3417                          , "LINEA_ERROR": linea_error
3418                          , "ERRORES": str(Err)
3419                          }
3420
3421     if isinstance(lista_dicc_errores_migracion, list):
3422         lista_dicc_errores_migracion.append(dicc_errores_temp)
3423     else:
3424         lista_dicc_errores_migracion = [dicc_errores_temp]
3425
3426     mod_gen.dicc_errores_procesos["PROCESO_02"]["MS_ACCESS"]["LISTA_DICC_ERRORES_CALCULO"] = lista_dicc_errores_migracion
3427     del dicc_errores_temp
3428     del lista_dicc_errores_migracion
3429     pass #es pass para que se localicen todos los posibles errores en el proceso
```


ANEXO_12

RUTINA

def_proceso_sql_server_2_diagnostico

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

Esta rutina permite realizar los cálculos necesarios para realizar el diagnostico de dependencias de objetos en una base de datos SQL SERVER (por defecto es la BBDD_01) y realizar la exportación del resultado final en un fichero Excel.

La rutina tiene 4 parámetros:

PARÁMETRO	DESCRIPCIÓN
opcion	Tiene por valores "Realizar diagnostico" o ""Descargar códigos T-SQL", son los ítems de la lista lista_GUI_diagnostico_combobox_sql_server (módulo APP_2_GENERAL).
servidor_selecc	Es el servidor SQL SERVER seleccionado
lista_bbdd_selecc	Es la lista de bases de datos seleccionadas
ruta_destino	Ruta de la carpeta donde guardar el output según la opción seleccionada

La rutina nada más empezar se encapsula en un bloque de flujos de errores (**try except**). El bloque **except** permite almacenar en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL), en las subkeys_3 (LISTA_DICC_ERRORES_CALCULO), los sucesivos errores de ejecución, en caso de que los hubiese, lo que permite generar en otra rutina logs en formatos .txt.

El 1er paso consiste en recuperar los datos necesarios del diccionario **dicc_codigos_bbdd** (módulo APP_2_GENERAL) creando la lista de diccionarios **lista_dicc_objetos_sql_server** y la lista **lista_sql_server_bbdd_servidor**.

BBDD_01 es por defecto la base de datos en la cual se realiza el diagnostico de dependencias.

```
860 lista_dicc_objetos_sql_server = mod_gen.dicc_codigos_bbdd["BBDD_01"]["SQL_SERVER"]["LISTA_DICC_OBJETOS"]
861 lista_sql_server_bbdd_servidor = mod_gen.dicc_codigos_bbdd["BBDD_01"]["SQL_SERVER"]["LISTA_BBDD_SERVIDOR"]
```

A continuación, se explica la rutina en función del valor del parámetro **opcion**.

1

Opcion = "Realizar diagnostico"

Se crean las listas temporales **lista_sql_server_objetos** y **lista_sql_server_objetos_dependencias** mediante bucle sobre la lista **lista_dicc_objetos_sql_server**. Estas listas son la base sobre la cual se calcula el diagnostico de dependencias, este último se calcula iterando sobre listas porque es mucho más rápido y efectivo que hacerlo sobre dataframes.

LISTA	ITEMS SUBLISTAS
lista_sql_server_objetos	Base de datos del servidor SQL Server seleccionado
	Esquema
	Tipo de objeto (TABLAS, VIEWS, STORED_PROCEDURES, FUNCIONES). Son las subkeys_3 asociadas a la key_1 (SQL_SERVER) y subkey_2 (TIPO_OBJETO) del diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL)
	Nombre del objeto
	Objeto long, es decir nombre del objeto en mayúsculas en formato mi_bbdd.mi_esquema.mi_objeto
	Objeto short, es decir nombre del objeto en mayúsculas en formato mi_esquema.mi_objeto
	Lista donde cada item es una línea de código "limpiada" (donde se pone todo en mayúsculas, se quitan los corchetes "[" y "]", se quitan las tabulaciones, se trimea y se quitan las líneas vacías)
	Lista que contiene sublistas con los objetos dependientes
lista_sql_server_objetos_dependencias	Base de datos del servidor SQL Server seleccionado
	Esquema
	Tipo de objeto (TABLAS, VIEWS, STORED_PROCEDURES, FUNCIONES). Son las subkeys_3 asociadas a la key_1 (SQL_SERVER) y subkey_2 (TIPO_OBJETO) del diccionario dicc_control_versiones_tipo_objeto (módulo APP_2_GENERAL)
	Nombre del objeto
	Objeto short, es decir nombre del objeto en mayúsculas en formato mi_esquema.mi_objeto

El último item de cada una de las sublistas de **lista_sql_server_objetos** (el que está marcado en **AMARILLO**) se establece como lista vacía de inicio, se calcula más adelante.

1

Opcion = "Realizar diagnostico"

```

871 lista_sql_server_objetos = []
872 lista_sql_server_objetos_dependencias = []
873
874 for dicc in lista_dicc_objetos_sql_server:
875
876     bdd = dicc["BBDD"]
877     repositorio = dicc["REPOSITORIO"]
878     tipo_objeto = dicc["TIPO_OBJETO"]
879     nombre_objeto = dicc["NOMBRE_OBJETO"]
880     objeto_long_mayusc = dicc["OBJETO_LONG"].upper()
881     objeto_short_mayusc = dicc["OBJETO_SHORT"].upper()
882     lista_codigo_limpiado = dicc["LISTA_CODIGO_LIMPIADO"]

```



```

887 lista_sql_server_objetos.append([bdd
888     , repositorio
889     , tipo_objeto
890     , nombre_objeto
891     , objeto_long_mayusc
892     , objeto_short_mayusc
893     , lista_codigo_limpiado
894     , [] #sirve para almacenar la lis
895 ])
896
897 lista_sql_server_objetos_dependencias.append([bdd
898     , repositorio
899     , tipo_objeto
900     , nombre_objeto
901     , objeto_short_mayusc
902 ])

```

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

1

Opcion = "Realizar diagnostico"

Una vez creadas estas 2 listas temporales se pasa a calcular el diagnostico de dependencias, los pasos a seguir son:

PASO	DESCRIPCIÓN
1	Se inicia bucle sobre la lista lista_sql_server_objetos (en adelante bucle_1) donde para objeto (en adelante objeto_1) se recupera su código T-SQL "limpiado" en forma de lista
2	Dentro de bucle_1, se crea la lista temporal lista_dependencias_objeto que sirve para almacenar los objetos usados dentro del código T-SQL de cada objeto de la iteración
3	Dentro de bucle_1, se inicia otro bucle (en adelante bucle_2) sobre la lista lista_sql_server_objetos_dependencias
4	Dentro de bucle_2, se excluyen de la búsqueda de objeto_1 que coincidan con los de bucle_2 (en adelante objeto_2) para evitar localizar dependencias de un objeto con su propio código
5	Dentro de bucle_2, se inicia un bucle (en adelante bucle_3) sobre la lista de código T-SQL "limpiado" de cada objeto_1 donde de inicio se excluye el item correspondiente a la linea de declaración del script T-SQL (CREATE o ALTER TABLE para tablas, CREATE o ALTER VIEW para views, CREATE o ALTER PROCEDURE para stored procedure, CREATE o ALTER PROCEDURE para funciones)
6	Dentro de bucle_3, se extralimita la búsqueda cuando el objeto short (en formato mi_esquema.mi_bbdd) de cada objeto_2 esta incluido dentro del item de código T-SQL de la iteración sobre bucle_3 (el adelante item_codigo_bucle_3)
7	Dentro de bucle_3, se busca que el objeto short de objeto_2 aparezca como palabra sola dentro de item_codigo_bucle_3 y no como palabre incluida en otra más larga (que podria corresponder con otro objeto de las bases de datos seleccionadas), se excluye tambien que el objeto short venga precedido de un punto "." (porque significaria que antes aparece el nombre de una base de datos del servidor que no necesariamente es la misma que la base de datos donde el script de objeto_1 esta guardado)
8	Si el macheo por objeto short coincide se agragan los datos de objeto_1 y objeto_2 a la lista lista_dependencias_objeto (ver paso 2)
9	En caso contrario, se pasa a realizar el macheo por objeto long, para ello mediante bucle (en adelante bucle_4) sobre la lista de bases de datos que componen el servidor seleccionado (lista_sql_server_bbdd_servidor), se reconstruye en cada iteración el objeto long en base al objeto short de objeto_2 y se repiten los 6 a 8 adaptados a este objeto long reconstruido (matizando que a diferencia del paso 8, cuando hay macheo por objeto long, se agregan los datos de objeto_1 y los de objeto_2 pero para estos últimos la base datos pasa a ser la de la iteración sobre bucle_4)
10	Siempre dentro de bucle_1, en cada iteración tras finalizar bucle_2 (y sus bucles anidados bucle_3 y bucle_4) se agrega a cada sublista (indice 7) de la lista lista_sql_server_objetos la lista lista_dependencias_objeto (comentada en los pasos 2, 8 y 9)

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

1

Opcion = "Realizar diagnostico"

1

```
927     for indice_lista, item_1 in enumerate(lista_sql_server_objetos):
928
929         bdd = item_1[indice_bdd]
930         repositorio = item_1[indice_repositorio]
931         tipo_objeto = item_1[indice_tipo_objeto]
932         nombre_objeto = item_1[indice_nombre_objeto]
933         objeto_long_mayusc = item_1[indice_objeto_long]
934         objeto_short_mayusc = item_1[indice_objeto_short]
935         lista_codigo = item_1[indice_lista_codigo]
```

2

```
938     lista_dependencias_objeto = []
```

3

```
939     for item_2 in lista_sql_server_objetos_dependencias:
940
941         bdd_dependencias = item_2[indice_dependencias_bdd]
942         repositorio_dependencias = item_2[indice_dependencias_repositorio]
943         tipo_objeto_dependencias = item_2[indice_dependencias_tipo_objeto]
944         nombre_objeto_dependencias = item_2[indice_dependencias_nombre_objeto]
945         objeto_short_mayusc_dependencias = item_2[indice_dependencias_objeto_short]
```

4

```
949     if not (bdd == bdd_dependencias and repositorio == repositorio_dependencias and
950            tipo_objeto == tipo_objeto_dependencias and nombre_objeto == nombre_objeto_dependencias):
```

5

```
953     for linea_codigo in lista_codigo:
954
955         #se excluye de la busqueda la sentencia de declaracion de la rutina / funcion
956         if objeto_long_mayusc not in linea_codigo and objeto_short_mayusc not in linea_codigo:
```

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

1

Opcion = "Realizar diagnostico"

6

```
960 if objeto_short_mayusc_dependencias in linea_codigo:
```

7

```
964 check_si_objeto_short_es_palabra_sola = fr'(?<![\w_-]){re.escape(objeto_short_mayusc_dependencias)}(?![\w_-])'
965 check_si_objeto_short_es_palabra_sola = bool(re.search(check_si_objeto_short_es_palabra_sola, linea_codigo))
966
967 check_objeto_short = 1 if check_si_objeto_short_es_palabra_sola == True else 0
```

8

```
972 if check_objeto_short == 1:
973
974     lista_temp = [bbdd, repositorio_dependencias, tipo_objeto_dependencias, nombre_objeto_dependencias]
975
976     if lista_temp not in lista_dependencias_objeto:
977         lista_dependencias_objeto.append(lista_temp)
```

9

```
982 elif check_objeto_short == 0:
983     for bdd_servidor in lista_sql_server_bdd_servidor:
984         objeto_long_mayusc_bdd_servidor = bdd_servidor.upper() + "." + objeto_short_mayusc_dependencias
985         #se extralimita el macheo cuando se localiza que el objeto short esta en la linea de codigo
986         if objeto_long_mayusc_bdd_servidor in linea_codigo:
987
988             check_si_objeto_long_es_palabra_sola = fr'(?<![\w_-]){re.escape(objeto_long_mayusc_bdd_servidor)}(?![\w_-])'
989             check_si_objeto_long_es_palabra_sola = bool(re.search(check_si_objeto_long_es_palabra_sola, linea_codigo))
990
991             check_objeto_long = 1 if check_si_objeto_long_es_palabra_sola == True else 0
992
993             #en caso de macheo se agregan los datos de la iteracion sobre lista_sql_server_objetos_dependencias
994             #asignandoles la bdd del servidor localizada
995             if check_objeto_long == 1:
996
997                 lista_temp = [bdd_servidor, repositorio_dependencias, tipo_objeto_dependencias, nombre_objeto_dependencias]
998
999                 if lista_temp not in lista_dependencias_objeto:
1000                     lista_dependencias_objeto.append(lista_temp)
```

10

```
1007 lista_sql_server_objetos[indice_lista][indice_lista_dependencias] = lista_dependencias_objeto if len(lista_dependencias_objeto) != 0 else None
```

1

Opcion = “Realizar diagnostico”

Matizar que la búsqueda de dependencias se realiza poniendo los objetos short y long en mayúsculas puesto que las listas de código T-SQL “limpiado” de cada objeto_1 también tienen todos sus ítems en mayúsculas, esto se hace para **evitar casos de case sensitive**.

Una limitación del app (ver apartado 8 del presente documento) es que tan solo localiza dependencias de objetos para bases de datos de un mismo servidor y **no con los objetos de servidores vinculados** por lo que la búsqueda por objeto long expuesta en el paso 9 puede dar resultados erróneos. En el apartado 10 del presente documento, se comenta que es uno de los futuros desarrollos previstos que tengo pensado implementar en el futuro.

Comentados estos 2 puntos, tras finalizar bucle_1, el paso siguiente consiste en crear las listas temporales **lista_sql_server_para_df_dependencias_objetos** y **lista_sql_server_para_df_sin_dependencias_objetos** que sirven de base para construir los dataframes que se exportan en el fichero Excel.

Se crean mediante bucle sobre la lista **lista_sql_server_objetos** donde se recuperan en cada iteración sobre sus sublistas los datos del objeto como la lista de dependencias generada y comentada en páginas anteriores (índice 7 de las sublistas).

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

1

Opcion = "Realizar diagnostico"

```
1011 lista_sql_server_para_df_dependencias_objetos = []
1012 lista_sql_server_para_df_sin_dependencias_objetos = []
1013
1014 for indice_lista, item in enumerate(lista_sql_server_objetos):
1015
1016     bdd = item[indice_bdd]
1017     repositorio = item[indice_repositorio]
1018     tipo_objeto = item[indice_tipo_objeto]
1019     nombre_objeto = item[indice_nombre_objeto]
1020     lista_dependencias_objeto = item[indice_lista_dependencias]
```



```
1022 if bdd in lista_bdd_selecc:
1023
1024     if isinstance(lista_dependencias_objeto, list):
1025         for bdd_dependencias, repositorio_dependencias, tipo_objeto_dependencias, nombre_objeto_dependencias in lista_dependencias_objeto:
1026
1027             if bdd_dependencias in lista_bdd_selecc:
1028                 lista_sql_server_para_df_dependencias_objetos.append([bdd_dependencias
1029                                     , repositorio_dependencias
1030                                     , tipo_objeto_dependencias
1031                                     , nombre_objeto_dependencias
1032                                     , bdd
1033                                     , repositorio
1034                                     , tipo_objeto
1035                                     , nombre_objeto])
1036
1037             else:
1038                 lista_sql_server_para_df_sin_dependencias_objetos.append([bdd
1039                                     , repositorio
1040                                     , tipo_objeto
1041                                     , nombre_objeto])
```

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

1

Opcion = "Realizar diagnostico"

Con todo ya creado, se pasa a exportar los datos a un fichero Excel. Para ello, se crea un diccionario temporal donde poder almacenar todo lo necesario para poder realizar esta exportación de forma dinámica mediante bucle.

Se crea previamente a este diccionario, las listas que sirven para generar los dataframes que se exportan en las hojas "BBDD SELECCIONADAS" y "LISTADO" del fichero Excel mencionado.

```
1047 lista_sql_server_para_df_lista_bbdd = [[servidor_selecc, bbdd] for bbdd in lista_bbdd_selecc]
1048
1049 lista_sql_server_para_df_listado_objetos = [[dicc["BBDD"], dicc["REPOSITORIO"], dicc["TIPO_OBJETO"], dicc["NOMBRE_OBJETO"], dicc["PARAMETROS"]]]
1050 for dicc in lista_dicc_objetos_sql_server if dicc["BBDD"] in lista_bbdd_selecc]
```

```
1052 dicc_objetos_sql_server = {"BBDD_SELECCIONADAS":
1053                             {"HOJA_EXCEL": "BBDD SELECCIONADAS"
1054                               , "LISTA_PARA_DF": lista_sql_server_para_df_lista_bbdd
1055                               , "LISTA_HEADERS": lista_headers_sql_server_diagnostico_df_bbdd_selecc
1056                              }
1057                             , "LISTADO_OBJETOS":
1058                               {"HOJA_EXCEL": "LISTADO"
1059                                , "LISTA_PARA_DF": lista_sql_server_para_df_listado_objetos
1060                                , "LISTA_HEADERS": lista_headers_sql_server_diagnostico_df_listado_objetos
1061                               }
1062                              , "DEPENDENCIAS":
1063                                {"HOJA_EXCEL": "DEPENDENCIAS"
1064                                 , "LISTA_PARA_DF": lista_sql_server_para_df_dependencias_objetos
1065                                 , "LISTA_HEADERS": lista_headers_sql_server_diagnostico_df_dependencias
1066                                }
1067                               , "SIN_DEPENDENCIAS":
1068                                 {"HOJA_EXCEL": "SIN DEPENDENCIAS"
1069                                  , "LISTA_PARA_DF": lista_sql_server_para_df_sin_dependencias_objetos
1070                                  , "LISTA_HEADERS": lista_headers_sql_server_diagnostico_df_sin_dependencias
1071                                 }
1072                                }
1073                               }
1074                               }
1075                               }
```

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

1

Opcion = "Realizar diagnostico"

Usando la plantilla Excel integrada en el app se copia la misma en la ruta indicada **ruta_destino** (parámetro de la presente rutina que se documenta), se le cambia el nombre y se abre en 2do plano (se usa para ello la librería xlwings).

Módulo APP_2_GENERAL

```
26 try:
27     #si es un ejecutable
28     ico_app = sys._MEIPASS + r"\ico_app.ico" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\ico_app.ico"
29     ruta_plantilla_control_versiones_xls = sys._MEIPASS + r"\PLANTILLA_CONTROL_VERSIONES.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_CONTROL_VERSIONES.xlsb"
30     ruta_plantilla_diagnostico_access_xls = sys._MEIPASS + r"\PLANTILLA_DIAGNOSTICO_MS_ACCESS.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_DIAGNOSTICO_MS_ACCESS.xlsb"
31     ruta_plantilla_diagnostico_sql_server_xls = sys._MEIPASS + r"\PLANTILLA_DIAGNOSTICO_SQL_SERVER.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_DIAGNOSTICO_SQL_SERVER.xlsb"
32
33 except:
34     #si se ejecuta desde consola
35     ico_app = sys._MEIPASS + r"\ico_app.ico" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\ico_app.ico"
36     ruta_plantilla_control_versiones_xls = sys._MEIPASS + r"\PLANTILLA_CONTROL_VERSIONES.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_CONTROL_VERSIONES.xlsb"
37     ruta_plantilla_diagnostico_access_xls = sys._MEIPASS + r"\PLANTILLA_DIAGNOSTICO_MS_ACCESS.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_DIAGNOSTICO_MS_ACCESS.xlsb"
38     ruta_plantilla_diagnostico_sql_server_xls = sys._MEIPASS + r"\PLANTILLA_DIAGNOSTICO_SQL_SERVER.xlsb" if getattr(sys, 'frozen', False) else pathlib.Path(__file__).parent.absolute() + r"\PLANTILLA_DIAGNOSTICO_SQL_SERVER.xlsb"
39
40     pass
```

Módulo APP_3_BACK_END_SQL_SERVER

```
1079 now = str(dt.datetime.now()).replace("-", "").replace(" ", "_").replace(":", "")[0:15]
1080 saveas = str(ruta_destino) + r"\DEPENDENCIAS_SQL_SERVER_" + now + ".xlsb"
1081
1082 shutil.copyfile(mod_gen.ruta_plantilla_diagnostico_sql_server_xls, saveas)
1083
1084 app = xw.App(visible = False)
1085 wb = app.books.open(saveas, update_links = False)
```

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

1

Opcion = "Realizar diagnostico"

Mediante bucle sobre este diccionario temporal, se realiza la exportación a Excel y al finalizar este bucle se guarda el Excel.

```
1088     for key in dicc_objetos_sql_server.keys():
1089
1090         hoja_excel = dicc_objetos_sql_server[key]["HOJA_EXCEL"]
1091         lista_para_df = dicc_objetos_sql_server[key]["LISTA_PARA_DF"]
1092         lista_headers = dicc_objetos_sql_server[key]["LISTA_HEADERS"]
1093
1094
1095         if len(lista_para_df) != 0:
1096
1097             df_temp = pd.DataFrame(lista_para_df, columns = lista_headers)
1098             df_temp = df_temp[lista_headers].sort_values(lista_headers, ascending = [True for i in lista_headers])
1099             df_temp.reset_index(drop = True, inplace = True)
1100
1101             ws = wb.sheets[hoja_excel]
1102             ws.range("A2:FF65000").clear_contents()
1103             ws["A2"].options(pd.DataFrame, header = 0, index = False, expand = "table").value = df_temp
1104
1105
1106         #se guarda en el excel, se cierra y se re-abre en 1er plano
1107         wb.save(saveas)
1108         wb.close()
1109         app = xw.App(visible = True)
1110         app.quit()
```

2

Opcion = "Descargar códigos T-SQL"

Los códigos T-SQL de los objetos se descargan en la ruta indicada en el parámetro de la rutina **ruta_destino**. En dicha ruta se crea una subcarpeta a partir con el nombre del parámetro de la rutina **servidor_selecc**.

```

1127 now = "CODIGOS_T_SQL_" + servidor_selecc.replace(r"\", "_") + str(dt.datetime.now()).replace("-", "").replace(" ", "_").replace(":", "")[0:15]
1128 ruta_export = ruta_destino + r"%s" % now
1129 os.makedirs(ruta_export, exist_ok = True)

```

El siguiente paso es crear subcarpetas anidadas las unas dentro de las otros según este orden usando la lista de diccionarios **lista_dicc_objetos_sql_server**, que se calcula al inicio de la presente rutina.

ORDEN	SUBCARPETAS
1	Tantas subcarpetas como hay de bases de datos seleccionadas
2	Dentro de cada subcarpeta por bases de datos, tantas subcarpetas como tiene de esquemas la base de datos
3	Dentro de cada subcarpeta por esquema, tantas subcarpetas como tipos de objetos localizados en los esquemas
4	Dentro de cada subcarpeta por tipo de objeto, se descargan los scripts T-SQL de los objetos en ficheros .sql

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

2

Opcion = "Descargar códigos T-SQL"

1

```
1132 #se crean las subcarpetas por bdd
1133 for bdd in lista_bdd_selecc:
1134     ruta_export_bdd = ruta_export + r"%s" % bdd
1135     os.makedirs(ruta_export_bdd, exist_ok = True)
1136
1137     lista_esquemas = [dicc["REPOSITORIO"] for dicc in lista_dicc_objetos_sql_server if dicc["BBDD"] == bdd]
1138     lista_esquemas = list(dict.fromkeys(lista_esquemas))
```

2

```
1141 #se crean las subcarpetas por esquemas
1142 for esquema in lista_esquemas:
1143     ruta_export_esquema = ruta_export_bdd + r"%s" % esquema
1144     os.makedirs(ruta_export_esquema, exist_ok = True)
1145
1146     lista_tipo_objetos = [dicc["TIPO_OBJETO"] for dicc in lista_dicc_objetos_sql_server if dicc["BBDD"] == bdd and dicc["REPOSITORIO"] == esquema]
1147     lista_tipo_objetos = list(dict.fromkeys(lista_tipo_objetos))
```

3

```
1150 #se crean las subcarpetas por tipo_objeto
1151 for tipo_objeto in lista_tipo_objetos:
1152     ruta_export_tipo_objeto = ruta_export_esquema + r"%s" % tipo_objeto
1153     os.makedirs(ruta_export_tipo_objeto, exist_ok = True)
1154
1155     lista_objetos = [[dicc["NOMBRE_OBJETO"], dicc["DF_CODIGO_ORIGINAL"]]
1156                     for dicc in lista_dicc_objetos_sql_server if dicc["BBDD"] == bdd and dicc["REPOSITORIO"] == esquema and dicc["TIPO_OBJETO"] == tipo_objeto]
```

4

```
1159 #se crean los ficheros .sql
1160 for nombre_objeto, df_codigo in lista_objetos:
1161     nombre_objeto_corr = nombre_objeto + ".sql"
1162     nombre_objeto_corr = ruta_export_tipo_objeto + r"%s" % nombre_objeto_corr
1163
1164     with open(nombre_objeto_corr, 'w') as fich:
1165
1166         #se concatena el encabezado de codigo con el df de codigo (y se reemplaza la bdd en el 1er
1167         df_codigo_corr = pd.concat([df_encabezado_descarga_codigo_objetos, df_codigo])
1168         df_codigo_corr.reset_index(drop = True, inplace = True)
1169
1170         df_codigo_corr.iloc[0, 0] = df_codigo_corr.iloc[0, 0].replace("REEMPLAZA_BBDD", bdd)
1171
1172         for ind in df_codigo_corr.index:
1173             fich.write(df_codigo_corr.iloc[ind, df_codigo_corr.columns.get_loc("CODIGO")] + "\n")
```

ANEXO_12

RUTINA: def_proceso_sql_server_2_diagnostico

Por último, se crea el bloque **Except** para almacenar los posibles errores de ejecución de la rutina en el diccionario **dicc_errores_procesos** (módulo APP_2_GENERAL) en la subkey_3 (LISTA_DICC_ERRORES_CALCULO) asociada a la key_1 (PROCESO_02) y subkey_2 (SQL_SERVER).

```
1179     except Exception as Err:
1180
1181         traceback_error = traceback.extract_tb(Err.__traceback__)
1182         modulo_python = os.path.basename(traceback_error[0].filename)
1183         rutina_python = traceback_error[0].name
1184         linea_error = traceback_error[0].lineno
1185
1186         lista_dicc_errores_migracion = mod_gen.dicc_errores_procesos["PROCESO_03"]["SQL_SERVER"]["LISTA_DICC_ERRORES_CALCULO"]
1187
1188         dicc_errores_temp = {"TIPO_BBDD": "SQL_SERVER"
1189                             , "MODULO_PYTHON": modulo_python
1190                             , "RUTINA_PYTHON": rutina_python
1191                             , "LINEA_ERROR": linea_error
1192                             , "ERRORES": str(Err)
1193                             }
1194
1195         if isinstance(lista_dicc_errores_migracion, list):
1196             lista_dicc_errores_migracion.append(dicc_errores_temp)
1197         else:
1198             lista_dicc_errores_migracion = [dicc_errores_temp]
1199
1200         mod_gen.dicc_errores_procesos["PROCESO_03"]["SQL_SERVER"]["LISTA_DICC_ERRORES_CALCULO"] = lista_dicc_errores_migracion
1201
1202         del dicc_errores_temp
1203         del lista_dicc_errores_migracion
1204         pass #es pass para que se localicen todos los posibles errores en el proceso
```

FIN

Gracias a l@s valientes por haber llegado hasta aquí!